

嵌入式 **Linux** 开发详解

——基于 **EP93XX** 系列 **ARM**

www.emsyschina.com

内容简介

本书全面介绍基于 32 位 ARM 架构嵌入式微处理器的软、硬件系统的高级应用开发，以 Cirrus Logic 的 EP93XX（ARM920T 核）系列微处理器为基础，包含 EP93XX 的体系结构特点、片内资源、软硬件开发平台、嵌入式操作系统移植及应用程序开发等内容，着重强调了基于 EP93XX 系统微处理器的硬件系统设计，嵌入式实时操作系统 eCos 在 EP93XX 系列微处理器上的移植及应用开发，嵌入式 Linux 操作系统及应用程序开发，设备驱动程序的开发等内容，最后以一个完整的系统开发说明嵌入式应用系统设计的一般流程。

全书内容以应用为出发点，内容详尽，可作为 ARM 应用技术开发人员的参考用书，以及高等院校相关专业的师生阅读使用。

www.emsyschina.com

前 言

以嵌入式微处理器和嵌入式操作系统为核心的嵌入式技术是近年来一个新的技术发展方向，目前已在一些领域得到了广泛的应用。在所有的 32 位嵌入式微处理器中，以 ARM 公司的 32 位 IP 核为基础的 ARM 嵌入式微处理器，因其高性能、低功耗、低成本、小体积以及完整的产业链支持，取得了巨大的成功，并占据 32 位嵌入式微处理器的绝大多数市场份额。

目前，国内外有众多的半导体生产商设计生产基于 ARM 核的嵌入式微处理器，其中，Cirrus Logic 的 EP93XX 系列（ARM920T 核）高性能嵌入式 ARM 微处理器以其优化的片上性能，格外引人注目，在诸如工业控制、数字媒体服务器、自动音乐点播机、瘦客户机、机顶盒、POS 终端、生物测量与安全系统以及 GPS 装置等各个应用领域，为用户提供了多种性价比选择。

EP93XX 系列主要包括 EP9301、EP9302、EP9307、EP9312 和 EP9315 等，其中，EP9301/EP9302 是极具价格优势的 ARM9 处理器，它的高性能设计是许多消费类和工业电子产品的理想选择；EP9307 是面向复杂应用方案的低成本硬件平台；而 EP9312/EP9315 是目前集成度最高的片上系统处理器，为下一代消费类和工业电子产品的开发提供了新的思路。

本书以 EP93XX 系列微处理器为基础，详细介绍了系统的软、硬件系统的高级应用开发。

本书的各章节内容安排如下：

第一章 简要介绍目前 ARM 微处理器的类型及主要结构特征，着重介绍了 EP93XX 系列微处理器的特点及应用领域。

第二章 介绍 EP93XX 系列微处理器的结构特点，主要性能、片内外围电路以及应用领域。

第三章 介绍基于 EP93XX 系列的硬件系统设计与调试方法及系统的初始化过程。

第四章 介绍 eCos 嵌入式实时操作系统的基本结构，以及在 EP93XX 系统微处理器的移植过程与应用程序的开发方法。

第五章 介绍嵌入式 Linux 操作系统在 EP93XX 系列微处理器的移植与开发过程，包括嵌入式 Linux 开发环境的建立、Linux 内核裁剪与编译，根文件系统的制作，以及简单应用程序的开发。

第六章 介绍基于 EP93XX 嵌入式 Linux 的深层次开发过程，包括设备驱动程序的开发、网络应用开发、基于 Web 的应用开发，图形应用开发，以及嵌入式数据库的应用开发等。

第七章 介绍一个基于 EP9301 的无线数据终端的应用系统的设计，包括从系统的需求分析，到硬件平台的选择与设计，软件系统的移植与应用开发的全过程。

在本书的编写过程中，得到了中国科学院自动化研究所博士生导师刘国平、郁文生老师的指导和帮助，同时得到了中嵌科技有限公司（www.emsyschina.com）的大力支持，在此谨向他们深表谢意。

本书在编写过程中，引用了参考文献所列论著的有关部分，在此谨向以上作者表示由衷的谢意，同时还参考了一些期刊和网站的最新内容，不能一一列出，也在此向相关作者表示感谢。

由于编者的水平有限，书中的错误在所难免，恳请读者批评指正，E_mail: LJG_express@263.net。

李驹光
于西南科技大学
2006 年 8 月

www.emsyschina.com

目 录

第 1 章 EP93XX 系列微处理器概述	8
1.1 EP93XX 系列嵌入式微处理器	8
1.2 EP9301 的结构特征与应用领域	9
1.3 EP9302 的结构特征与应用领域	14
1.4 EP9307 的结构特征与应用领域	19
1.5 EP9312 的结构特征与应用领域	25
1.6 EP9315 的结构特征与应用领域	31
1.7 EP93XX 系列与其它主要 ARM9 微处理器的性能对比	37
1.8 本章小结	39
第 2 章 EP93XX 系列微处理器的片内资源	40
2.1 EP93XX 系列嵌入式微处理器的片内资源	40
2.2 ARM920T 核与先进高速总线 (AHB, Advanced High-Speed Bus)	41
2.3 Boot ROM 及系统启动	42
2.4 MaverickCrunch™ 数学引擎	46
2.5 系统控制器	47
2.6 向量中断控制器	50
2.7 以太网控制器	53
2.8 DMA 控制器	54
2.9 USB 主控制器	54
2.10 静态存储器控制器	55
2.11 SDRAM 控制器	55
2.12 UART 接口	56
2.13 红外接口	56
2.14 定时器	57
2.15 看门狗定时器	58
2.16 I ² S 控制器	59
2.17 AC'97 控制器	60
2.18 同步串行接口	60
2.19 脉冲宽度调制器	61
2.20 ADC 接口/触摸屏接口	61
2.21 键盘接口	62
2.22 IDE 接口	62
2.23 Raster/LCD 控制器	63
2.24 图像加速器	64

2.25	PCMCIA 接口	64
2.26	GPIO 接口	64
2.27	安全特性.....	64
2.28	本章小结.....	65
第 3 章 基于 EP93XX 系列微处理器的系统设计		66
3.1	嵌入式系统设计的一般原则.....	66
3.2	基于 EP9301/EP9302 的硬件系统设计	66
3.3	基于 EP9307 的硬件系统设计	94
3.4	基于 EP9312/15 的硬件系统设计	105
3.5	本章小结.....	111
第 4 章 RedBoot 与 eCos.....		113
4.1	嵌入式系统的 BootLoader.....	113
4.2	Redboot 简介及常用命令	116
4.3	在基于 EP93XX 的系统上运行 Redboot 及嵌入式 Linux.....	123
4.4	eCos 概述与特性.....	127
4.5	eCos 开发环境的建立.....	130
4.6	eCos 在特定嵌入式系统的移植.....	132
4.7	基于 eCos 的简单应用程序开发.....	139
4.8	本章小节.....	142
第 5 章 嵌入式 Linux 开发基础.....		143
5.1	嵌入式 Linux 概述	143
5.2	嵌入式 Linux 开发环境的建立	145
5.3	嵌入式 Linux 内核的配置与编译	150
5.4	根文件系统及制作.....	155
5.5	网络文件系统（NFS）及配置	157
5.6	嵌入式 Linux 应用程序的开发	160
5.7	嵌入式 Linux 应用程序的调试方法	164
5.8	本章小节.....	170
第 6 章 嵌入式 Linux 的应用开发.....		172
6.1	Linux 设备驱动程序开发	172
6.2	Linux 字符设备驱动的开发	176
6.3	网络应用程序的开发.....	185
6.4	基于串行通信的应用程序开发.....	190
6.5	基于 Web 的应用程序开发	201
6.6	图形应用开发.....	218

6.7 本章小结.....	240
第 7 章 一个完整的应用开发实例.....	241
7.1 系统的需求分析.....	241
7.2 硬件平台的选择与设计.....	243
7.3 软件系统的移植与应用开发.....	243
7.4 本章小结.....	278

www.emsyschina.com

第 1 章 EP93XX 系列微处理器概述

本章简介 EP93XX 系列（ARM920T 核）嵌入式微处理器的一些基本特性、应用领域，使读者对 EP93XX 系列微处理器有一个比较全面的了解。

本章主要包括：

- λ EP93XX 系列嵌入式微处理器简介；
- λ EP9301 的结构特征与应用领域；
- λ EP9302 的结构特征与应用领域；
- λ EP9307 的结构特征与应用领域；
- λ EP9312 的结构特征与应用领域；
- λ EP9315 的结构特征与应用领域；
- λ EP93XX 系列与其它常用基于 ARM9 内核处理器性能对比；

1.1 EP93XX 系列嵌入式微处理器

基于 ARM 架构的 32 位嵌入式微处理器，以其完备的性能，完整的产业链支持，在 32 位嵌入式微处理器应用领域，取得了巨大的成功。

采用 32 位 ARM 微处理器进行系统开发，有助于设计者在完成先进系统功能的同时，兼顾对产品体积、上市周期、开发成本和功耗等多项指标的要求，无疑是当前嵌入式微处理器市场的佼佼者。

在 ARM 嵌入式微处理器内核系列中，以 ARM7 和 ARM9 内核为基础的 ARM 微处理器最为常用，生产厂商众多，性价比高，是嵌入式系统设计的理想选择。其中，基于 ARM7 内核的处理器定位于对价格敏感的应用场合，着眼于代换与升级传统 8 位单片机市场，这些处理器片内外围比较简单，应用于相对较简单的系统。

而基于 ARM9 内核的嵌入式微处理器则以其较高性能、丰富的片内资源，非常适合于比较复杂和高端的应用场合，目前，已有多家半导体生产商设计生成的基于 ARM9 核的嵌入式微处理器进入市场，其中，Cirrus Logic 公司以 EP9301、EP9302、EP9307、EP9312 和 EP9315 为代表 EP93XX 系列微处理器最为引人注目，几乎涵盖了目前主要的工业与商业应用领域。

EP93XX 系列嵌入式微处理器以其最优化的片上性能、极具吸引力的价格定位，正在快速占领日益扩大的消费类、商用和工业产品等嵌入式微处理器市场。

表 1-1 是 EP93XX 系列嵌入式微处理器的主要性能对比。

表 1-1 EP93XX 系列嵌入式微处理器的性能对比

处理器 型号	处理速度 (MHz)	以太网 控制器	PCMCIA 接口	IDE 接口	USB 主口	显示 接口	图像 加速器	触摸屏接口 /ADC
-----------	---------------	------------	--------------	-----------	-----------	----------	-----------	---------------

EP9301	166	有			2			5 通道 ADC
EP9302	200	有			2			5 通道 ADC
EP9307	200	有			3	有	有	8 线触摸屏接口
EP9312	200	有		2 通道	3	有		8 线触摸屏接口
EP9315	200	有	有	2 通道	3	有	有	8 线触摸屏接口

以下我们将详细介绍 EP93XX 系列微处理器的结构特征、片内功能模块及相关应用领域。

1.2 EP9301 的结构特征与应用领域

EP9301 被定位为一款入门级的产品，是现有的最具价格优势的 ARM9 处理器之一，其高度集成的结构和先进的性能设计尤其适合各种消费类电子产品和工业控制应用。

1.2.1 EP9301 的主要特性

EP9301 的主要特性描述如下：

- λ 166MHz 的 ARM920T 处理器：
 - v 16KB 指令 Cache；
 - v 16KB 数据 Cache；
 - v 内嵌 MMU，支持 Linux，WinCE 等嵌入式操作系统；
 - v 66MHz 的系统总线频率；
- λ MaverickKey™ 硬件编程 ID，可用于数字版权管理（DRM）和设计知识产权的保护：
 - v 32 位的唯一 ID；
 - v 128 位随机 ID；
- λ 集成外围接口：
 - v 多达 4 组的 16 位 SDRAM 接口；
 - v 16/8 位 SRAM/FLASH/ROM 接口；
 - v 串行 EEPROM 接口；
 - v 1/10/100Mbps 以太网控制器；
 - v 2 路 UART；
 - v 2 路 USB2.0 全速（12Mbps）主口；
 - v 红外接口，支持慢速、中速和快速模式；
 - v 5 通道 ADC 接口；
 - v 串行外围接口（SPI）；
 - v 2 通道串行音频接口（I²S）；
 - v AC'97 接口；

λ 其它片内功能：

- v 12 个 DMA 通道；
- v 可由软件调整的 RTC；
- v 双 PLL 可控制所有时钟；
- v 看门狗（Watchdog）定时器；
- v 2 个通用 16 位定时器；
- v 1 个通用 32 位定时器；
- v 1 个 40 位调试定时器；
- v 中断控制器；
- v 片内 Boot ROM；
- v 16 个带中断功能的增强型 GPIO（EGPIO）；
- v 8 个标准型的 GPIO；

λ 芯片封装方式：

- v 208 引脚 LQFP 封装，可选择无铅封装；

λ 工作温度范围：

- v 商业级：0°C~+70°C；
- v 工业级：-40°C~+85°C；

1.2.2 EP9301 的内部结构

EP9301 的内部结构如图 1-1 所示：

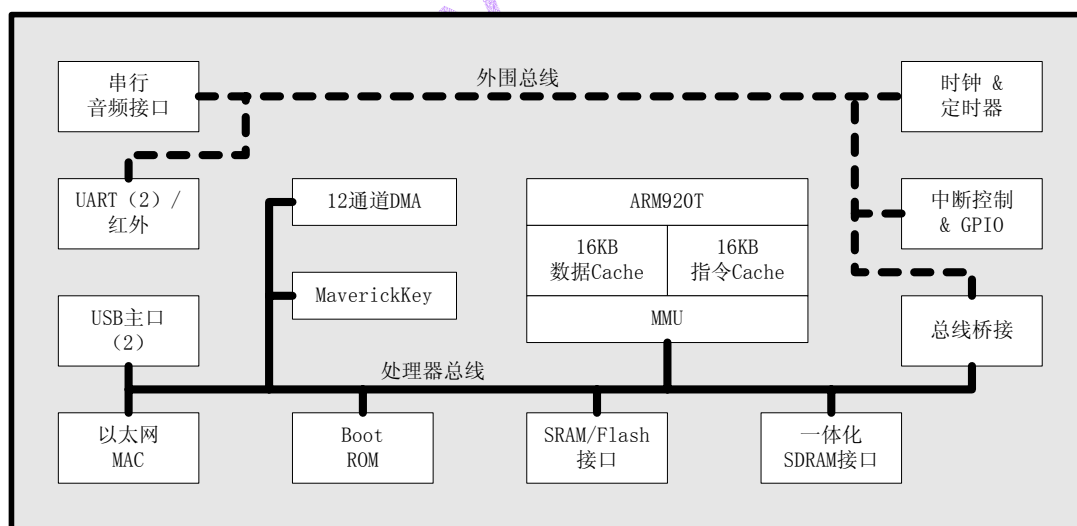


图 1-1 EP9301 的内部结构

1.2.3 EP9301 的应用领域

EP9301 是一款基于 ARM920T 核的片上系统（SOC），其丰富的片内外围设计使之适用于如下领域：

- λ 工业控制；
- λ 数字媒体服务器；
- λ 集成家用媒体网关；
- λ 数字音频自动点唱机；
- λ 流式音频播放器；
- λ 机顶盒；
- λ 瘦客户端设备；
- λ 生物测量与安全系统；
- λ GPS 及车船导航管理系统；
- λ 教育产品；
- λ 工业计算机；
- λ 工业手持设备；
- λ 医疗设备；
- λ 其他嵌入式应用领域。

1.2.4 EP9301 概述

高性能 ARM920T 微处理器核

EP9301 内嵌先进的运行于 166MHz 的 ARM920T 微处理器核，以及支持 Linux、Windows CE 和其他许多嵌入式操作系统的存储器管理单元（MMU）。ARM920T 的 32 位微控制器结构具有 5 级流水线，可以以极低的功耗提供优异的性能，同时，16KB 指令高速缓存和 16KB 数据高速缓存可为现有的程序和数据提供零等待时间，或者以锁定的方式确保对关键指令和数据的无延迟存取。

对于一些程序存储器大小有限制的应用场合，ARM920T 所支持的压缩型 Thumb 指令集有助于空间利用率的提高和程序存储器的最大化使用。

EP9301 的 ARM920T 内核工作电压为 1.8V，输入/输出电压（I/O）为 3.3V，根据不同的运行速度，功耗从 100mW~675mW 不等。

MaverickKey™ 硬件可编程 ID

EP9301（包括 EP93XX 系列）内嵌的 MaverickKey™ 独特的硬件可编程 ID，是解决网上内容和电子商务安全问题的一个可行方案。MaverickKey™ 为 OEM 厂商提供了一种新的方法，为硬件赋予特定硬件 ID，比如 SDMI（安全数字音乐计划）或其他数字版权管理机制使用的 ID。

MaverickKey™ 使用与一个 32 位专用 ID 和一个 128 位的随机 ID 相匹配的方式，实现网

络安全与数字版权保护，以及电子商务的安全。

通用存储器接口（SDRAM、SRAM、ROM 和 Flash）

EP9301 支持统一的存储器寻址模式，所有的存储设备都可以通过一组公共的地址总线 and 数据总线进行访问，存储器的访问通过内部高速处理器总线完成。其中，SRAM 存储控制器支持访问 8/16 位存储设备，可完成 8/16 位的 ROM 和 SRAM 访问；SDRAM 控制器可以支持多达 4 组的 16 位 SDRAM 的访问与控制。

存储器接口的主要特性如下：

- λ 支持 1~4 组 16 位 SDRAM，66MHz 总线频率；
- λ 内部与高速处理器总线相连；
- λ SDRAM、SRAM、ROM 和 Flash 共享数据总线与地址总线；
- λ 支持 NAND Flash 和 NOR Flash；

以太网控制器

EP9301（包括 EP93XX 系列）内部集成了一个高性能 1/10/100Mbps 以太网媒体访问控制器（MAC），通过与支持 MII 接口的外置 PHY（物理层接口）简单连接，系统即可实现高性能、低系统开销的 Internet 访问功能。

以太网控制器的主要特性如下：

- λ 支持 1/10/100Mbps 的传输速率，可用于各种网络应用场合；
- λ 可通过工业标准 MII 接口与片外物理层芯片接口，实现网络连接功能；

串行接口（SPI、I²S 和 AC'97）

EP9301 内嵌的 SPI 接口可配置为主模式和从模式，支持 National Semiconductor[®]，Motorola[®] 和 TI 的传输协议。

AC'97 接口支持一路立体声输入和多通道音频输出的编码解码。I²S 接口支持 24 位立体声音频。

以上端口支持复用功能，I²S 端口可以占用 AC'97 引脚和 SPI 引脚，方式如下：

- λ 正常模式：支持一个 SPI 端口和一个 AC'97 端口；
- λ 基于 SSP 的 I²S 模式：支持一个 SPI 端口和一个 I²S 端口；
- λ 基于 AC'97 的 I²S 模式：支持一个 AC'97 端口和一个 I²S 端口；

注意 I²S 不能同时作为 AC'97 和 SSP 的输出端口。

12 位 A/D 转换器

EP9301 内嵌一个带多路模拟输入开关的 12 位 A/D 转换器，可用于完成常用的 A/D 转换功能。该 A/D 转换器可以用于数据采样、控制等多种场合。

UART 接口

EP9301 带有两个 UART 接口：UART1 和 UART2。除作为通用 UART 使用以外，UART1 还可支持异步 HDLC 全双工发送与接收协议，UART2 还可支持红外发送与接收功能。

2 通道 USB2.0 全速主口

EP9301 内部集成了 2 通道 USB2.0 全速主口，与 OHCI USB2.0 全速规范（12Mbps）完全兼容。USB 主口能够提供与诸如海量存储设备、打印机、USB Hub、音频播放器、数码相机等外置便携式设备的连接与数据通信，最多可以挂接 127 个 USB 设备。USB 主口同时也支持 USB2.0 低速标准，大大扩展了系统的应用领域和设计的灵活性。

支持 EEPROM 的二线接口

EP9301 的二线接口支持与诸如 EEPROM、RTC 等二线接口设备的通信与控制。

支持软件微调的实时时钟（RTC）

RTC 的软件微调功能可以对 32.768KHz 的晶振进行补偿，补偿可以精确到 1.24 秒/月。

PLL 与系统时钟

EP9301 的处理器核与外围模块的工作时钟由一个 14.7456MHz 的晶振提供，片内 RTC 的工作时钟由一个 32.768KHz 的晶振提供。

定时器

看门狗定时器用于确保系统处于正常工作状态，需要定时“喂狗”。

两个 16 位定时器可以作为递减计数器，或产生固定时间间隔的中断，时间间隔的范围可以从 0.03 毫秒到 4.27 秒。

一个 32 位的定时器，可以提供 0.03 微妙到 73.3 小时的精确定时。

一个 40 位的调试定时器，可以提供 1.0 微妙到 12.7 天的精确定时。

中断控制器

EP9301 的中断控制器可以支持多达 54 个中断源产生 IRQ 和 FIQ 请求。IRQ 和 FIQ 均支持硬件优先级编程，以便以最快的速度响应时间紧急的中断事件。

EP9301 的 EGPIO 引脚可编程为高电平、低电平、上升沿、下降沿，以及上升沿和下降沿同时等方式触发中断。

12 通道 DMA 控制器

EP9301 的 DMA 控制器支持 12 个独立的 DMA 通道，其中 10 个通道可用于外围模块与存储器之间的数据传输，另外 2 个通道则专门用于存储器之间的数据传输。

灵活的系统启动方式

EP9301 内嵌 16KB 的 Boot ROM，可以完成标准的系统配置。Boot ROM 支持从 Flash 存储器、SPI 串行 ROM、或通过 UART 下载等方式启动系统。灵活的系统启动方式便于设计易于控制、可现场更新的嵌入式应用系统。

双 LED 驱动

EP9301 使用两个专门的引脚，标识为 GRLED 和 REDLED，用于外接两个发光二极管，指示系统的运行状态。

丰富的 GPIO

EP9301 包含增强型和标准型的 GPIO，其中，16 个增强型的 GPIO（EGPIO）可单独配置为输入、输出和带中断功能的输入引脚；还有 8 个标准型的 GPIO 可单独配置为输入、输出或漏极开路（Open-drain）引脚。

由于一些标准型的 GPIO 引脚可能与片内功能模块复用，因此 GPIO 的实际可用数量与片内功能模块的使用方式有关。

JTAG 硬件调试接口

EP9301 内嵌的 JTAG 调试接口支持 ARM 的 Multi-ICE 及与之兼容的系统调试。

关于 EP9301 各个功能模块更详细的描述，可以参考以后的章节或相关用户手册。

1.3 EP9302 的结构特征与应用领域

EP9302 是一款高性能的 ARM9 微处理器，是现有的最具价格优势的 ARM9 处理器之一，其高度集成的结构和先进的性能设计尤其适合各种消费类电子产品和工业控制应用。

EP9302 的封装方式与引脚定义与 EP9301 完全兼容，它们可以直接互换使用，EP9302 优于 EP9301 的性能主要体现在 EP9302 具有更高的工作频率（200MHz）、更高的系统总线频率（100MHz）和具有 MaverickCrunch™ 数学引擎。

Cirrus Logic 公司推荐用户使用 EP9302 代替 EP9301。

1.3.1 EP9302 的主要特性

EP9302 的主要特性描述如下：

- λ 200MHz 的 ARM920T 处理器：
 - v 16KB 指令 Cache；
 - v 16KB 数据 Cache；
 - v 内嵌 MMU，支持 Linux，WinCE 等嵌入式操作系统；
 - v 100MHz 的系统总线频率；
- λ MaverickCrunch™ 数学引擎：
 - v 支持浮点、整型和信号处理指令；
 - v 针对数字音频的压缩与解压算法进行了优化；
 - v 硬件联锁（Interlocks）装置允许联机编码；
- λ MaverickKey™ 硬件编程 ID，可用于数字版权管理（DRM）和设计知识产权的保护：
 - v 32 位的唯一 ID；
 - v 128 位随机 ID；
- λ 集成外围接口：
 - v 多达 4 组的 16 位 SDRAM 接口；

- v 16/8 位 SRAM/FLASH/ROM 接口;
- v 串行 EEPROM 接口;
- v 1/10/100Mbps 以太网控制器;
- v 2 路 UART;
- v 2 路 USB2.0 全速 (12Mbps) 主口;
- v 红外接口, 支持慢速模式;
- v 5 通道 ADC;
- v 串行外围接口 (SPI);
- v 2 通道串行音频接口 (I²S);
- v AC'97 接口;
- λ 其它片内功能:
 - v 12 个 DMA 通道;
 - v 可由软件调整的 RTC;
 - v 双 PLL 可控制所有时钟;
 - v 看门狗 (Watchdog) 定时器;
 - v 2 个通用 16 位定时器;
 - v 1 个通用 32 位定时器;
 - v 1 个 40 位调试定时器;
 - v 中断控制器;
 - v 片内 Boot ROM;
 - v 16 个带中断功能的增强型 GPIO (EGPIO);
 - v 8 个标准型 GPIO;
- λ 芯片封装方式:
 - v 208 引脚 LQFP 封装, 可选择无铅封装;
- λ 工作温度范围:
 - v 商业级: 0°C~+70°C;
 - v 工业级: -40°C~+85°C;

1.3.2 EP9302 的内部结构

EP9302 的内部结构如图 1-2 所示:

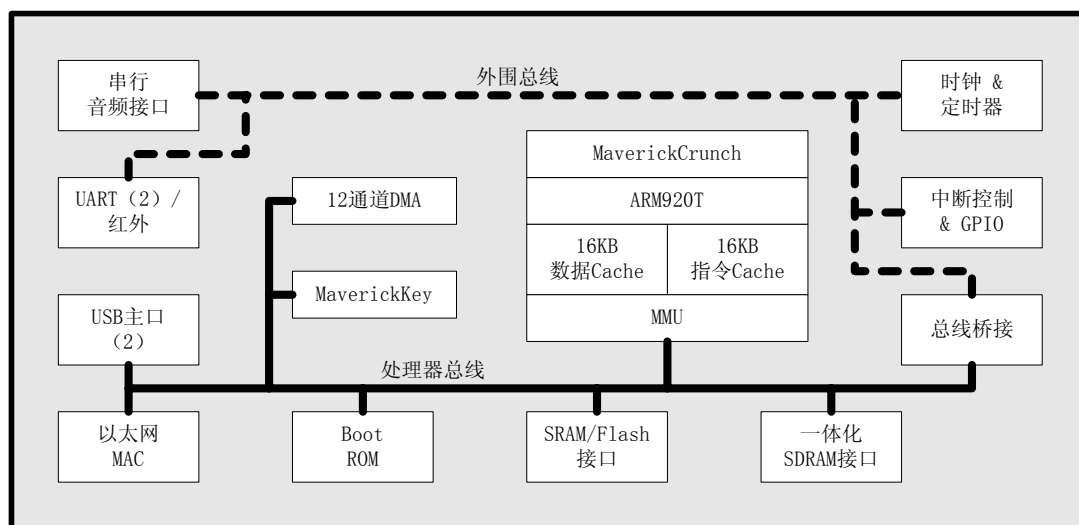


图 1-2 EP9302 的内部结构

1.3.3 EP9302 的应用领域

与 EP9301 类似，EP9302 也是一款基于 ARM920T 核的片上系统（SOC），其丰富的片内外围设计使之适用于如下领域：

- λ 工业控制；
- λ 数字媒体服务器；
- λ 集成家用媒体网关；
- λ 数字音频自动点唱机；
- λ 流式音频播放器；
- λ 机顶盒；
- λ POS（Point-of-sale）终端；
- λ 瘦客户端设备；
- λ 生物测量与安全系统；
- λ GPS 及车船导航管理系统；
- λ 教育产品；
- λ 工业计算机；
- λ 工业手持设备；
- λ 医疗设备；
- λ 其他嵌入式应用领域。

1.3.4 EP9302 概述

高性能 ARM920T 微处理器核

EP9302 内嵌先进的运行于 200MHz（工业条件下推荐运行 184MHz）的 ARM920T 微处理器核，以及支持 Linux、Windows CE 和其他许多嵌入式操作系统的存储器管理单元（MMU）。ARM920T 的 32 位微控制器结构具有 5 级流水线，可以以极低的功耗提供优异的性能，同时，16KB 指令高速缓存和 16KB 数据高速缓存可为现有的程序和数据提供零等待时间，或者以锁定的方式确保对关键指令和数据的无延迟存取。

对于一些程序存储器大小有限制的应用场合，ARM920T 所支持的压缩型 Thumb 指令集有助于空间利用率的提高和程序存储器的最大化使用。

EP9302 的 ARM920T 内核工作电压为 1.8V，输入/输出电压（I/O）为 3.3V，根据不同的运行速度，功耗从 100mW~750mW 不等。

MaverickCrunch™数学引擎

EP9302 内部集成了一个MaverickCrunch™数学协处理器，显著提高了ARM920T的浮点运算、整型运算与信号处理能力，同时还专门针对数字音频的压缩与解压算法进行了优化，因此，当对数字音频进行编码解码、执行工业控制运算以及其它运算密集型应用和数据处理时，该协处理器可使EP9302 具有高速、精确的计算能力。

MaverickKey™硬件可编程ID

EP9302（包括EP93XX系列）内嵌的MaverickKey™独特的硬件可编程ID是解决网上内容和电子商务安全问题的一个可行方案。MaverickKey™为OEM厂商提供了一种新方法，为硬件赋予特定硬件ID，比如SDMI（安全数字音乐计划）或其他数字版权管理机制使用的ID。

MaverickKey™使用与一个 32 位专用ID和一个 128 位的随机ID相匹配的方式，实现网络安全与数字版权保护，以及电子商务的安全。

通用存储器接口（SDRAM、SRAM、ROM 和 Flash）

EP9302 支持统一的存储器寻址模式，所有的存储设备都可以通过一组公共的地址总线 and 数据总线进行访问，存储器的访问通过高速处理器总线完成。其中，SRAM 存储控制器支持访问 8/16 位存储设备，可完成 8/16 位的 ROM 和 SRAM 访问；SDRAM 控制器可以支持多达 4 组的 16 位 SDRAM 的访问与控制。

存储器接口的主要特性如下：

- λ 支持 1~4 组 16 位 SDRAM，100MHz 总线频率；
- λ 内部与高速处理器总线相连；
- λ SDRAM、SRAM、ROM 和 Flash 共享数据总线与地址总线；
- λ 支持 NAND Flash 和 NOR Flash；

以太网控制器

EP9302（包括 EP93XX 系列）内部集成了一个高性能 1/10/100Mbps 以太网媒体访问控

制器 (MAC), 通过与支持 MII 接口的外置 PHY (物理层接口) 简单连接, 系统即可实现高性能、低系统开销的 Internet 访问功能。以太网控制器的主要特性如下:

- λ 支持 1/10/100Mbps 的传输速率, 可用于各种网络应用场合;
- λ 可通过工业标准 MII 接口与片外物理层芯片接口, 实现网络连接;

串行接口 (SPI、I²S和AC'97)

EP9302 内嵌的 SPI 接口可配置为主模式和从模式, 支持 National Semiconductor[®], Motorola[®] 和 TI 的传输协议。

AC'97 接口支持一路立体声输入和多通道音频输出的编码解码。I²S 接口支持 24 位立体声音频。

以上端口支持复用功能, I²S 端口可以占用 AC'97 引脚和 SPI 引脚, 方式如下:

- λ 正常模式: 支持一个 SPI 端口和一个 AC'97 端口;
 - λ 基于 SSP 的 I²S 模式: 支持一个 SPI 端口和一个 I²S 端口;
 - λ 基于 AC'97 的 I²S 模式: 支持一个 AC'97 端口和一个 I²S 端口;
- 注意 I²S 不能同时作为 AC'97 和 SSP 的输出端口。

12 位 A/D 转换器

EP9302 内嵌一个带多路模拟输入开关的 12 位 A/D 转换器, 可用于通常的 A/D 转换。该 A/D 转换器可以用于数据采样、控制等多种场合。

UART 接口

EP9302 带有两个 UART 接口: UART1 和 UART2。除作为通用 UART 以外, UART1 还支持异步 HDLC 全双工发送与接收协议, UART2 还支持红外发送与接收功能。

2 通道 USB2.0 全速主口

EP9302 内部集成了 2 通道 USB2.0 全速主口, 与 OHCI USB2.0 全速规范 (12Mbps) 完全兼容。USB 主口能够提供与诸如海量存储设备、打印机、USB Hub、音频播放器、数码相机等外置便携式设备的连接与数据通信, 最多可以挂接 127 个 USB 设备。USB 主口同时也支持 USB2.0 低速标准, 大大扩展了系统的应用领域和设计的灵活性。

支持 EEPROM 的二线接口

EP9302 的二线接口支持与诸如 EEPROM、RTC 等二线接口设备的通信与控制。

支持软件微调的实时时钟 (RTC)

EP9302 的 RTC 软件微调功能可以对 32.768KHz 的晶振进行补偿, 补偿可以精确到 1.24 秒/月。

PLL 与系统时钟

EP9302 的处理器核与外围模块的工作时钟由一个 14.7456MHz 的晶振提供, RTC 的工作时钟由一个 32.768KHz 的晶振提供。

定时器

EP9302 的看门狗定时器用于确保系统处于正常工作状态, 需要定时“喂狗”。

两个 16 位定时器可以作为递减计数器，或产生固定时间间隔的中断，时间间隔的范围可以从 0.03 毫秒到 4.27 秒。

一个 32 位的定时器，可以提供 0.03 微妙到 73.3 小时的精确定时。

一个 40 位的调试定时器，可以提供 1.0 微妙到 12.7 天的精确定时。

中断控制器

EP9302 的中断控制器可以支持多达 54 个中断源产生 IRQ 和 FIQ 请求。IRQ 和 FIQ 均支持硬件优先级编程，以便以最快的速度响应时间紧急的中断事件。

EP9302 的 EGPIO 引脚可编程为高电平、低电平、上升沿、下降沿，以及上升沿和下降沿同时等方式触发中断。

12 通道 DMA 控制器

EP9302 的 DMA 控制器支持 12 个独立的 DMA 通道，其中 10 个通道可用于外围模块与存储器之间的数据传输，另外 2 个通道则专门用于存储器之间的数据传输。

灵活的系统启动方式

EP9302 内嵌 16KB 的 Boot ROM，可以完成标准的系统配置。Boot ROM 支持从 Flash 存储器、SPI 串行 ROM、或通过 UART 下载等方式启动系统。灵活的系统启动方式便于设计易于控制、可现场更新的嵌入式应用系统。

双 LED 驱动

EP9302 使用两个专门的引脚，标识为 GRLED 和 REDLED，用于外接两个发光二极管，指示系统的运行状态。

丰富的 GPIO

EP9302 包含增强型和标准型的 GPIO，其中，16 个增强型的 GPIO 可单独配置为输入、输出和带中断功能的输入引脚；还有 8 个标准型的 GPIO 可单独配置为输入、输出或漏极开路（Open-drain）引脚。由于一些标准型的 GPIO 引脚可能与片内功能模块复用，因此 GPIO 的实际可用数量与片内功能模块的使用方式有关。

JTAG 硬件调试接口

EP9302 内嵌的 JTAG 调试接口支持 ARM 的 Multi-ICE 及与之兼容的系统调试。

关于 EP9302 各个功能模块更详细的描述，可以参考以后的章节或相关用户手册。

1.4 EP9307 的结构特征与应用领域

EP9307 是一款高集成度的 ARM9 嵌入式微处理器，具有 200MHz 的运行速度和丰富的用户接口功能，是高端游戏机、POS 终端、工业控制、家庭自动化系统和车船导航系统等低成本网络应用的理想选择。

EP9307 特别适合于对图像处理能力、用户接口功能和成本等有严格要求的嵌入式应用场合，在处理能力和系统设计灵活性等方面都极具优势，因此，EP9307 能以更低的成本及

功耗完成包括音/视频处理、工业控制和高速网络接入等应用。

1.4.1 EP9307 的主要特性

EP9307 的主要特性描述如下：

- λ 200MHz 的 ARM920T 处理器：
 - v 16KB 指令 Cache；
 - v 16KB 数据 Cache；
 - v 内嵌 MMU，支持 Linux，WinCE 等嵌入式操作系统；
 - v 100MHz 的系统总线频率；
- λ MaverickCrunchTM数学引擎：
 - v 支持浮点、整型和信号处理指令；
 - v 针对数字音频的压缩与解压算法进行了优化；
 - v 硬件联锁（Interlocks）装置允许联机编码；
- λ MaverickKeyTM硬件编程ID，可用于数字版权管理（DRM）和设计知识产权的保护：
 - v 32 位的唯一 ID；
 - v 128 位随机 ID；
- λ 集成外围接口：
 - v 多达 4 组的 32 位 SDRAM 接口；
 - v 32/16/8 位 SRAM/FLASH/ROM 接口；
 - v 串行 EEPROM 接口；
 - v 1/10/100Mbps 以太网控制器；
 - v 3 路 UART；
 - v 3 路 USB2.0 全速（12Mbps）主口；
 - v 红外接口，支持慢速和快速模式；
 - v 带图像加速器的 LCD/光栅（Raster）接口；
 - v 带 ADC 的触摸屏接口；
 - v 8×8 键盘扫描接口；
 - v SPI 接口；
 - v 多达 6 通道的 I²S 接口；
 - v AC'97 接口；
- λ 其它片内功能：
 - v 12 个 DMA 通道；
 - v 可由软件调整的 RTC；
 - v 双 PLL 可控制所有时钟；
 - v 看门狗（Watchdog）定时器；
 - v 2 个通用 16 位定时器；

- v 1 个通用 32 位定时器;
 - v 1 个 40 位调试定时器;
 - v 中断控制器;
 - v 片内 Boot ROM;
 - v 14 个带中断功能的增强型 GPIO (EGPIO);
 - v 18 个标准型 GPIO;
- λ 芯片封装方式:
 - v 272 引脚 TFBGA 封装, 可选择无铅封装;
- λ 工作温度范围:
 - v 商业级: $0^{\circ}\text{C} \sim +70^{\circ}\text{C}$;
 - v 工业级: $-40^{\circ}\text{C} \sim +85^{\circ}\text{C}$;

1.4.2 EP9307 的内部结构

EP9307 的内部结构如图 1-3 所示:

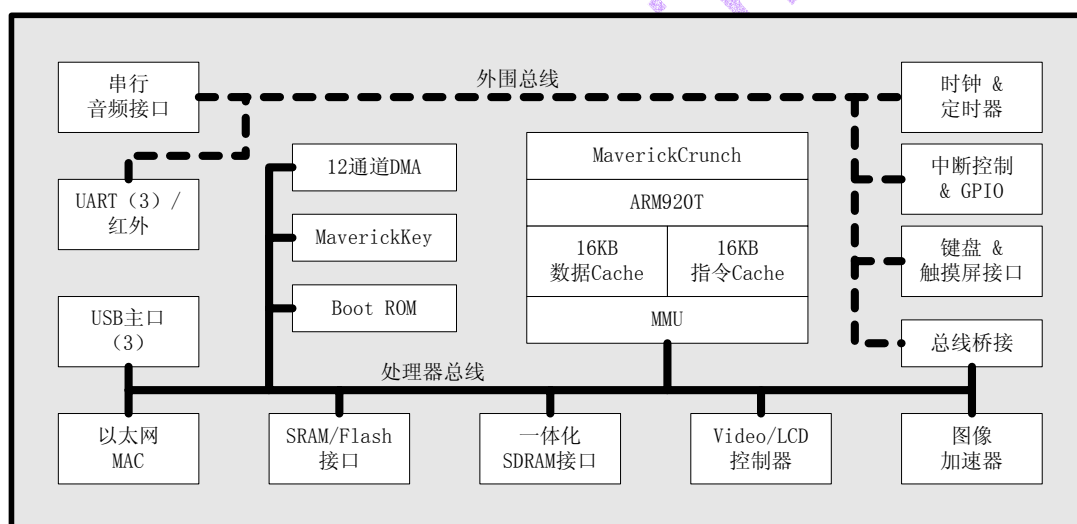


图 1-3 EP9307 的内部结构

1.4.3 EP9307 的应用领域

作为一款高集成度和片内外围丰富的 ARM9 嵌入式微处理器, EP9307 适用于如下领域:

- λ 数字媒体服务器;
- λ 商用与家用的瘦客户端计算机;
- λ 网络收音机;
- λ 网络访问设备;

- λ 工业计算机;
- λ POS 终端;
- λ 专业智能终端;
- λ 测试与测量设备;
- λ GPS 与车船管理系统;
- λ 教育产品;
- λ 工业手持设备;
- λ 医疗设备;
- λ 其他嵌入式应用领域。

1.4.4 EP9307 概述

高性能 ARM920T 微处理器核

EP9307 内嵌先进的运行于 200MHz（工业条件下推荐运行 184MHz）的 ARM920T 微处理器核，以及支持 Linux、Windows CE 和其他许多嵌入式操作系统的存储器管理单元（MMU）。ARM920T 的 32 位微控制器结构具有 5 级流水线，可以以极低的功耗提供优异的性能，同时，16KB 指令高速缓存和 16KB 数据高速缓存可为现有的程序和数据提供零等待时间，或者以锁定的方式确保对关键指令和数据的无延迟存取。

而对于一些程序存储器大小有限制的应用场合，ARM920T 所支持的压缩型 Thumb 指令集有助于空间利用率的提高和程序存储器的最大化使用。

EP9307 的 ARM920T 内核工作电压为 1.8V，输入/输出电压（I/O）为 3.3V，根据不同的运行速度，功耗从 100mW~750mW 不等。

MaverickCrunch™数学引擎

EP9307 内部集成了一个MaverickCrunch™数学协处理器，显著提高了ARM920T的浮点运算、整型运算与信号处理能力，同时还专门针对数字音频的压缩与解压算法进行了优化，因此，当对数字音频进行编码解码、执行工业控制运算以及其它运算密集型计算和数据处理时，该协处理器可使EP9307 具有高速、精确的计算能力。

MaverickKey™硬件可编程ID

EP9307（包括EP93XX系列）内嵌的MaverickKey™独特的硬件可编程ID是解决网上内容和电子商务安全问题的一个可行方案。MaverickKey™为OEM厂商提供了一种新方法，为硬件赋予特定硬件ID，比如SDMI（安全数字音乐计划）或其他数字版权管理机制使用的ID。

MaverickKey™使用与一个 32 位专用ID和一个 128 位的随机ID相匹配的方式，实现网络安全与数字版权保护，以及电子商务的安全。

通用存储器接口（SDRAM、SRAM、ROM 和 Flash）

EP9307 支持统一的存储器寻址模式，所有的存储设备都可以通过一组公共的地址总线 and 数据总线进行访问。除一个专用于 Raster/LCD 刷新引擎的内部端口，存储器的访问通过

高速处理器总线完成。其中,SRAM 存储控制器支持访问 8/16/32 位存储设备,可完成 8/16/32 位的 ROM 和 SRAM 访问;SDRAM 控制器可以支持多达 4 组的 32 位 SDRAM 的访问与控制。

存储器接口的主要特性如下:

- λ 支持 1~4 组 32 位 SDRAM, 100MHz 总线频率;
- λ 一个专用于 Raster/LCD 刷新引擎的内部端口;
- λ SDRAM、SRAM、ROM 和 Flash 共享数据总线与地址总线;
- λ 支持 NAND Flash 和 NOR Flash;

以太网控制器

EP9307 (包括 EP93XX 系列) 内部集成了一个高性能 1/10/100Mbps 以太网媒体访问控制器 (MAC), 通过与支持 MII 接口的外置 PHY (物理层接口) 简单连接, 系统即可实现高性能、低系统开销的 Internet 访问功能。

以太网控制器的主要特性如下:

- λ 支持 1/10/100Mbps 的传输速率, 可用于各种网络应用场合;
- λ 可通过工业标准 MII 接口与片外物理层芯片接口, 实现网络连接;

串行接口 (SPI、I²S 和 AC'97)

EP9307 内嵌的 SPI 接口可配置为主模式和从模式, 支持 National Semiconductor®, Motorola® 和 TI 的传输协议。

AC'97 接口支持一路立体声输入和多通道音频输出的编码解码。I²S 接口支持 24 位立体声音频。

以上端口支持复用功能, I²S 的端口 0 可以占用 AC'97 引脚和 SPI 引脚, I²S 的端口 1 和端口 2 的串行输入/输出引脚与 EGPIIO[4,5,6,13] 复用。三个 I²S 端口共用时钟源。具体使用如下:

- λ 正常模式: 支持一个 SPI 端口和一个 AC'97 端口;
- λ 基于 SSP 的 I²S 模式: 支持一个 SPI 端口和最多三个 I²S 端口;
- λ 基于 AC'97 的 I²S 模式: 支持一个 AC'97 端口和最多三个 I²S 端口;

注意 I²S 不能同时作为 AC'97 和 SSP 的输出端口。

Raster/LCD 接口

EP9307 的 Raster/LCD 接口可以为多种类型的显示器提供数据与信号接口, 主要特性如下:

- λ 支持数字 LCD 与 TFT 的时序与接口信号;
- λ 可编程支持隔行扫描、双扫描彩色和灰度级平板显示器;
- λ 基于 SDRAM 的帧缓存控制方式, 便于提升显示性能;
- λ 支持 4、8、16、24 位像素和 256 个灰度等级;
- λ 硬件支持最大 64×64 像素光标;
- λ 支持 8 位数据接口;

图像加速器

EP9307 内嵌一个硬件图像加速器，可以完成块拷贝、块填充等操作，显著提高系统的图像处理能力。图像加速器独立于微处理器，不会增加系统开销。

带 12 位 A/D 转换器的触摸屏接口

EP9307 内嵌的触摸屏接口可用于数据采样、控制以及与电阻式模拟触摸屏接口等多种场合。主要特性如下：

- λ 支持 4 线、5 线、7 线或 8 线电阻式模拟触摸屏接口；
- λ 未使用的引脚可用于其它功能；
- λ 支持触摸屏中断功能；

键盘接口

EP9307 的键盘接口电路可以扫描 8×8 的矩阵键盘，支持按键中断功能。当系统不使用键盘接口使，所有引脚可以用作 GPIO。

UART 接口

EP9307 带有三个 UART 接口：UART1、UART2 和 UART3。除作为通用 UART 使用以外，UART1 还支持异步 HDLC 全双工发送与接收协议，UART2 还支持红外发送与接收功能，UART3 还支持 HDLC。

灵活的系统启动方式

EP9307 内嵌 16KB 的 Boot ROM，可以完成标准的系统配置。Boot ROM 支持从 Flash 存储器、SPI 串行 ROM、或通过 UART 下载等方式启动系统。灵活的系统启动方式便于设计易于控制、可现场更新的嵌入式应用系统。

3 通道 USB2.0 全速主口

EP9307 内部集成了 3 通道 USB2.0 全速主口，与 OHCI USB2.0 全速规范（12Mbps）完全兼容。USB 主口能够提供与诸如海量存储设备、打印机、USB Hub、音频播放器、数码相机等外置便携式设备的连接与数据通信，最多可以挂接 127 个 USB 设备。USB 主口同时也支持 USB2.0 低速标准，大大扩展了系统的应用领域和设计的灵活性。

支持 EEPROM 的二线接口

EP9307 的二线接口支持与诸如 EEPROM、RTC 等二线接口设备的通信与控制。

支持软件微调的实时时钟（RTC）

EP9307 的 RTC 软件微调功能可以对 32.768KHz 的晶振进行补偿，补偿可以精确到 1.24 秒/月。

PLL 与系统时钟

EP9307 的处理器核与外围模块的工作时钟由一个 14.7456MHz 的晶振提供，RTC 的工作时钟由一个 32.768KHz 的晶振提供。

定时器

EP9307 的看门狗定时器用于确保系统处于正常工作状态，需要定时“喂狗”。

两个 16 位定时器可以作为递减计数器，或产生固定时间间隔的中断，时间间隔的范围可以从 0.03 毫秒到 4.27 秒。

一个 32 位的定时器，可以提供 0.03 微妙到 73.3 小时的精确定时。

一个 40 位的调试定时器，可以提供 1.0 微妙到 12.7 天的精确定时。

中断控制器

EP9307 的中断控制器可以支持多达 62 个中断源产生 IRQ 和 FIQ 请求。IRQ 和 FIQ 均支持硬件优先级编程，以便以最快的速度响应时间紧急的中断事件。

EP9307 的 EGPIO 引脚可编程为高电平、低电平、上升沿、下降沿，以及上升沿和下降沿同时等方式触发中断。

12 通道 DMA 控制器

EP9307 的 DMA 控制器支持 12 个独立的 DMA 通道，其中 10 个通道可用于外围模块与存储器之间的数据传输，另外 2 个通道则专门用于存储器之间的数据传输。

双 LED 驱动

EP9307 使用两个专门的引脚，标识为 GRLED 和 REDLED，用于外接两个发光二极管，指示系统的运行状态。

丰富的 GPIO

EP9307 包含增强型和标准型的 GPIO，其中，14 个增强型的 GPIO 可单独配置为输入、输出和带中断功能的输入引脚；还有 18 个标准型的 GPIO 可单独配置为输入、输出或漏极开路（Open-drain）引脚。由于一些标准型的 GPIO 引脚可能与片内功能模块复用，因此 GPIO 的实际可用数量与片内功能模块的使用方式有关。

JTAG 硬件调试接口

EP9307 内嵌的 JTAG 调试接口支持 ARM 的 Multi-ICE 及与之兼容的系统调试。

关于 EP9307 各个功能模块更详细的描述，可以参考以后的章节或相关用户手册。

1.5 EP9312 的结构特征与应用领域

EP9312 是一款基于 ARM920T 的片上系统（SOC），200MHz 的 ARM920T 微处理器核的高性能以及包括 IDE 接口的丰富的外围电路，使得 EP9312 成为诸如数字媒体服务器、自动点唱机、远程信息控制系统、瘦客户机、机顶盒、POS 终端、工业控制、生物测量与安全系统以及 GPS 设备等应用的理想选择，为下一代消费和工业电子产品的开发提供了新的思路。

1.5.1 EP9312 的主要特性

EP9312 的主要特性描述如下：

- λ 200MHz 的 ARM920T 处理器：
 - v 16KB 指令 Cache；
 - v 16KB 数据 Cache；
 - v 内嵌 MMU，支持 Linux，WinCE 等嵌入式操作系统；
 - v 100MHz 的系统总线频率；
- λ MaverickCrunchTM数学引擎：
 - v 支持浮点、整型和信号处理指令；
 - v 针对数字音频的压缩与解压算法进行了优化；
 - v 硬件联锁（Interlocks）装置允许联机编码；
- λ MaverickKeyTM硬件编程ID，可用于数字版权管理（DRM）和设计知识产权的保护：
 - v 32 位的唯一 ID；
 - v 128 位随机 ID；
- λ 集成外围接口：
 - v 多达 4 组的 32 位 SDRAM 接口；
 - v 32/16/8 位 SRAM/FLASH/ROM 接口；
 - v 串行 EEPROM 接口；
 - v 2 路 EIDE 接口；
 - v 1/10/100Mbps 以太网控制器；
 - v 3 路 UART；
 - v 3 路 USB2.0 全速（12Mbps）主口；
 - v 红外接口，支持慢速、快速模式；
 - v LCD 与光栅（Raster）接口；
 - v 带 ADC 的触摸屏接口；
 - v 8×8 键盘扫描接口；
 - v SPI 接口；
 - v 最多 6 通道的 I²S 接口；
 - v AC'97 接口；
 - v 2 通道 16 位 PWM 输出；
- λ 其它片内功能：
 - v 12 个 DMA 通道；
 - v 可由软件调整的 RTC；
 - v 双 PLL 可控制所有时钟；
 - v 看门狗（Watchdog）定时器；

- v 2 个通用 16 位定时器;
 - v 1 个通用 32 位定时器;
 - v 1 个 40 位调试定时器;
 - v 中断控制器;
 - v 片内 Boot ROM;
 - v 16 个带中断功能的增强型 GPIO (EGPIO);
 - v 49 个标准型 GPIO;
- λ 芯片封装方式:
 - v 352 引脚 PBGA 封装, 可选择无铅封装;
- λ 工作温度范围:
 - v 商业级: $0^{\circ}\text{C} \sim +70^{\circ}\text{C}$;
 - v 工业级: $-40^{\circ}\text{C} \sim +85^{\circ}\text{C}$;

1.5.2 EP9312 的内部结构

EP9312 的内部结构如图 1-4 所示:

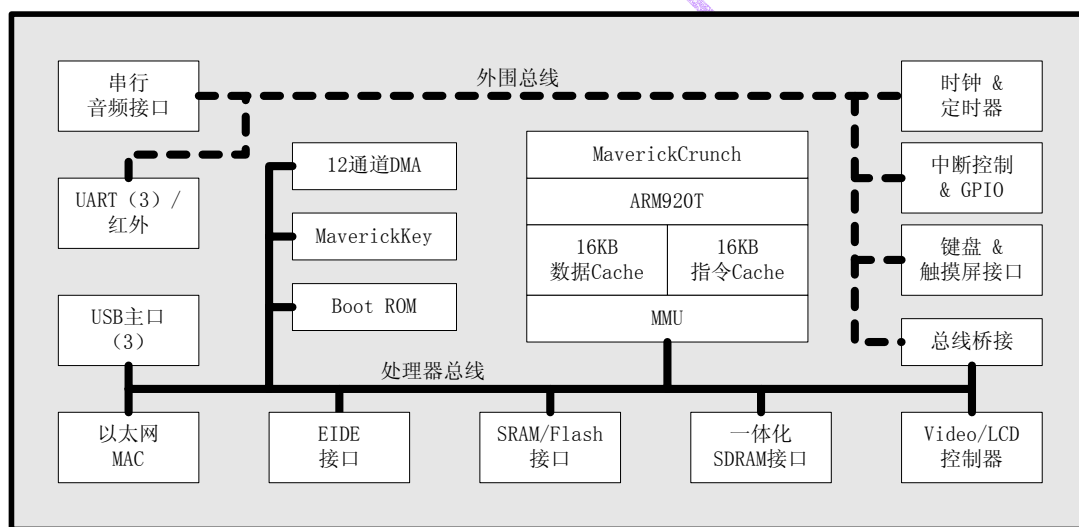


图 1-4 EP9312 的内部结构

1.5.3 EP9312 的应用领域

作为一款高集成度和片内外围丰富的 ARM9 嵌入式微处理器, EP9312 适用于如下领域:

- λ 商用与家用的瘦客户端计算机;
- λ 网络收音机;
- λ 网络访问设备;

- λ 工业计算机;
- λ POS 终端;
- λ 专业智能终端;
- λ 测试与测量设备;
- λ 工业手持设备;
- λ 医疗设备;
- λ 生物测量与安全系统;
- λ 其他嵌入式应用领域。

1.5.4 EP9312 概述

高性能 ARM920T 微处理器核

EP9312 内嵌先进的运行于 200MHz（工业条件下推荐运行 184MHz）的 ARM920T 微处理器核，以及支持 Linux、Windows CE 和其他许多嵌入式操作系统的存储器管理单元（MMU）。ARM920T 的 32 位微控制器结构具有 5 级流水线，可以以极低的功耗提供优异的性能，同时，16KB 指令高速缓存和 16KB 数据高速缓存可为现有的程序和数据提供零等待时间，或者以锁定的方式确保对关键指令和数据的无延迟存取。

对于一些程序存储器大小有限制的应用场合，ARM920T 所支持的压缩型 Thumb 指令集有助于空间利用率的提高和程序存储器的最大化使用。

EP9312 的 ARM920T 内核工作电压为 1.8V，输入/输出电压（I/O）为 3.3V，根据不同的运行速度，功耗从 100mW~750mW 不等。

MaverickCrunch™数学引擎

EP9312 内部集成了一个MaverickCrunch™数学协处理器，显著提高了ARM920T的浮点运算、整型运算与信号处理能力，同时还专门针对数字音频的压缩与解压算法进行了优化，因此，当对数字音频进行编码解码、执行工业控制运算以及其它运算密集型计算和数据处理时，该协处理器可使EP9312 具有高速、精确的计算能力。

MaverickKey™硬件可编程ID

EP9312（包括EP93XX系列）内嵌的MaverickKey™独特的硬件可编程ID是解决网上内容和电子商务安全问题的一个可行方案。MaverickKey™为OEM厂商提供了一种新方法，为硬件赋予特定硬件ID，比如SDMI（安全数字音乐计划）或其他数字版权管理机制使用的ID。

MaverickKey™使用与一个 32 位专用ID和一个 128 位的随机ID相匹配的方式，实现网络安全与数字版权保护，以及电子商务的安全。

通用存储器接口（SDRAM、SRAM、ROM 和 Flash）

EP9312 支持统一的存储器寻址模式，所有的存储设备都可以通过一组公共的地址总线 and 数据总线进行访问。除一个专用于 Raster/LCD 刷新引擎的内部端口，存储器的访问通过高速处理器总线完成。其中，SRAM 存储控制器支持访问 8/16/32 位存储设备，可完成 8/16/32

位的 ROM 和 SRAM 访问；SDRAM 控制器可以支持多达 4 组的 32 位 SDRAM 的访问与控制。存储器接口的主要特性如下：

- λ 支持 1~4 组 32 位 SDRAM，66/100MHz 总线频率；
- λ 一个专用于 Raster/LCD 刷新引擎的内部端口；
- λ SDRAM、SRAM、ROM 和 Flash 共享数据总线与地址总线；
- λ 支持 NAND Flash 和 NOR Flash；

以太网控制器

EP9312（包括 EP93XX 系列）内部集成了一个高性能 1/10/100Mbps 以太网媒体访问控制器（MAC），通过与支持 MII 接口的外置 PHY（物理层接口）简单连接，系统即可实现高性能、低开销的 Internet 访问功能。以太网控制器的主要特性如下：

- λ 支持 1/10/100Mbps 的传输速率，可用于各种网络应用场合；
- λ 可通过工业标准 MII 接口与片外物理层芯片接口，实现网络连接；

IDE 接口

EP9312 的 IDE 接口可以支持与 2 个 ATA/ATAPI 工业标准兼容设备的连接，支持主模式和从模式，其内部的 DMA 控制器支持 Multiword DMA 和 Ultra DMA 模式的数据传输。

串行接口（SPI、I²S 和 AC'97）

EP9312 内嵌的 SPI 接口可配置为主模式和从模式，支持 National Semiconductor[®]，Motorola[®] 和 TI 的传输协议。

AC'97 接口支持一路立体声输入和多通道音频输出的编码解码。I²S 接口支持 24 位立体声音频。

以上端口支持复用功能，I²S 的端口 0 可以占用 AC'97 引脚和 SPI 引脚，I²S 的端口 1 和端口 2 的串行输入/输出引脚与 EGPIIO[4,5,6,13] 复用。三个 I²S 端口共用时钟源。具体使用如下：

- λ 正常模式：支持一个 SPI 端口和一个 AC'97 端口；
 - λ 基于 SSP 的 I²S 模式：支持一个 SPI 端口和最多三个 I²S 端口；
 - λ 基于 AC'97 的 I²S 模式：支持一个 AC'97 端口和最多三个 I²S 端口；
- 注意 I²S 不能同时作为 AC'97 和 SSP 的输出端口。

Raster/LCD 接口

EP9312 的 Raster/LCD 接口可以为多种类型的显示器提供数据与接口信号，主要特性如下：

- λ 支持数字 LCD 与 TFT 的时序与接口信号；
- λ 可编程支持隔行扫描、双扫描彩色和灰度级平板显示器；
- λ 基于 SDRAM 的帧缓存控制方式，便于提升显示性能；
- λ 支持 4、8、16、24 位像素和 256 个灰度等级；
- λ 硬件支持最大 64×64 像素光标；
- λ 支持 8 位数据接口显示器；

带 12 位 A/D 转换器的触摸屏接口

EP9312 内嵌的触摸屏接口可用于数据采样、控制以及与模拟电阻式触摸屏接口等多种场合。主要特性如下：

- λ 支持 4 线、5 线、7 线或 8 线模拟电阻式触摸屏接口；
- λ 未使用的引脚可用于其它功能；
- λ 支持触摸屏中断功能；

键盘接口

EP9312 的键盘接口电路可以扫描 8×8 的矩阵键盘，支持按键中断功能。当系统不使用键盘接口使，所有引脚可以用作 GPIO。

UART 接口

EP9312 带有三个 UART 接口：UART1、UART2 和 UART3。除作为通用 UART 以外，UART1 支持异步 HDLC 全双工发送与接收协议，UART2 支持红外发送与接收功能，UART3 也支持 HDLC。

灵活的系统启动方式

EP9312 内嵌 16KB 的 Boot ROM，可以完成标准的系统配置。Boot ROM 支持从 Flash 存储器、SPI 串行 ROM、或通过 UART 下载等方式启动系统。灵活的系统启动方式便于设计易于控制、可现场更新的嵌入式应用系统。

3 通道 USB2.0 全速主口

EP9312 内部集成了 3 通道 USB2.0 全速主口，与 OHCI USB2.0 全速规范（12Mbps）完全兼容。USB 主口能够提供与诸如海量存储设备、打印机、USB Hub、音频播放器、数码相机等外置便携式设备的连接与数据通信，最多可以挂接 127 个 USB 设备。USB 主口同时也支持 USB2.0 低速标准，大大扩展了系统的应用领域和设计的灵活性。

支持 EEPROM 的二线接口

EP9312 的二线接口支持与诸如 EEPROM、RTC 等二线接口设备的通信与控制。

支持软件微调的实时时钟（RTC）

EP9312 的 RTC 软件微调功能可以对 32.768KHz 的晶振进行补偿，补偿可以精确到 1.24 秒/月。

PLL 与系统时钟

EP9312 的处理器核与外围模块的工作时钟由一个 14.7456MHz 的晶振提供，RTC 的工作时钟由一个 32.768KHz 的晶振提供。

定时器

EP9312 的看门狗定时器用于确保系统处于正常工作状态，需要定时“喂狗”。

两个 16 位定时器可以作为递减计数器，或产生固定时间间隔的中断，时间间隔的范围可以从 0.03 毫秒到 4.27 秒。

一个 32 位的定时器，可以提供 0.03 微妙到 73.3 小时的精确定时。

一个 40 位的调试定时器，可以提供 1.0 微妙到 12.7 天的精确定时。

中断控制器

EP9312 的中断控制器可以支持多达 64 个中断源产生 IRQ 和 FIQ 请求。IRQ 和 FIQ 均支持硬件优先级编程，以便以最快的速度响应时间紧急的中断事件。

EP9312 的 EGPIO 引脚可编程为高电平、低电平、上升沿、下降沿，以及上升沿和下降沿同时触发中断。

12 通道 DMA 控制器

EP9312 的 DMA 控制器支持 12 个独立的 DMA 通道，其中 10 个通道可用于外围模块与存储器之间的数据传输，另外 2 个通道则专门用于存储器之间的数据传输。

双 LED 驱动

EP9312 使用两个专门的引脚，标识为 GRLED 和 REDLED，用于外接两个发光二极管，指示系统的运行状态。

丰富的 GPIO

EP9312 包含增强型和标准型的 GPIO，其中，16 个增强型的 GPIO 可单独配置为输入、输出和带中断功能的输入引脚；还有 49 个标准型的 GPIO 可单独配置为输入、输出或漏极开路（Open-drain）引脚。由于一些标准型的 GPIO 引脚可能与片内功能模块复用，因此 GPIO 的实际可用数量与片内功能模块的使用方式有关。

JTAG 硬件调试接口

EP9312 内嵌的 JTAG 调试接口支持 ARM 的 Multi-ICE 及与之兼容的系统调试。

关于 EP9312 各个功能模块更详细的描述，可以参考以后的章节或相关用户手册。

1.6 EP9315 的结构特征与应用领域

EP9315 是 EP93XX 系列微处理器的旗舰产品，几乎是目前集成度最高的 ARM 处理器。

EP9315 是一款基于 ARM920T 的片上系统，200MHz 的 ARM920T 微处理器核的高性能以及包括 IDE、PCMCIA 接口的丰富的外围电路，使得 EP9315 成为诸如数字媒体服务器、自动点唱机、远程信息控制系统、瘦客户机、机顶盒、POS 终端、工业控制、生物测量与安全系统以及 GPS 设备等应用的理想选择，为下一代消费和工业电子产品的理想选择。

1.6.1 EP9315 的主要特性

EP9315 的主要特性描述如下：

- λ 200MHz 的 ARM920T 处理器；
- v 16KB 指令 Cache；

- v 16KB 数据 Cache;
- v 内嵌 MMU, 支持 Linux, WinCE 等嵌入式操作系统;
- v 100MHz 的系统总线频率;
- λ MaverickCrunchTM数学引擎:
 - v 支持浮点、整型和信号处理指令;
 - v 针对数字音频的压缩与解压算法进行了优化;
 - v 硬件联锁 (Interlocks) 装置允许联机编码;
- λ MaverickKeyTM硬件编程ID, 可用于数字版权管理 (DRM) 和设计知识产权的保护:
 - v 32 位的唯一 ID;
 - v 128 位随机 ID;
- λ 集成外围接口:
 - v 多达 4 组的 32 位 SDRAM 接口;
 - v 32/16/8 位 SRAM/FLASH/ROM 接口;
 - v 串行 EEPROM 接口;
 - v 2 路 EIDE 接口;
 - v 1/10/100Mbps 以太网控制器;
 - v 3 路 UART;
 - v 3 路 USB2.0 全速 (12Mbps) 主口;
 - v 红外接口, 支持慢速和快速模式;
 - v 带图像加速器的 LCD 与光栅 (Raster) 接口;
 - v PCMCIA 接口;
 - v 带 ADC 的触摸屏接口;
 - v 8×8 键盘扫描接口;
 - v SPI 接口;
 - v 最多 6 通道的 I²S 接口;
 - v AC'97 接口;
 - v 2 通道 16 位 PWM 输出;
- λ 其它片内功能:
 - v 12 个 DMA 通道;
 - v 可由软件调整的 RTC;
 - v 双 PLL 可控制所有时钟;
 - v 看门狗 (Watchdog) 定时器;
 - v 2 个通用 16 位定时器;
 - v 1 个通用 32 位定时器;
 - v 1 个 40 位调试定时器;
 - v 中断控制器;
 - v 片内 Boot ROM;

- v 16 个带中断功能的增强型 GPIO;
 - v 49 个标准型 GPIO;
- λ 芯片封装方式:
 - v 352 引脚 PBGA 封装, 可选择无铅封装;
- λ 工作温度范围:
 - v 商业级: $0^{\circ}\text{C} \sim +70^{\circ}\text{C}$;
 - v 工业级: $-40^{\circ}\text{C} \sim +85^{\circ}\text{C}$;

1.6.2 EP9315 的内部结构

EP9315 的内部结构如图 1-5 所示:

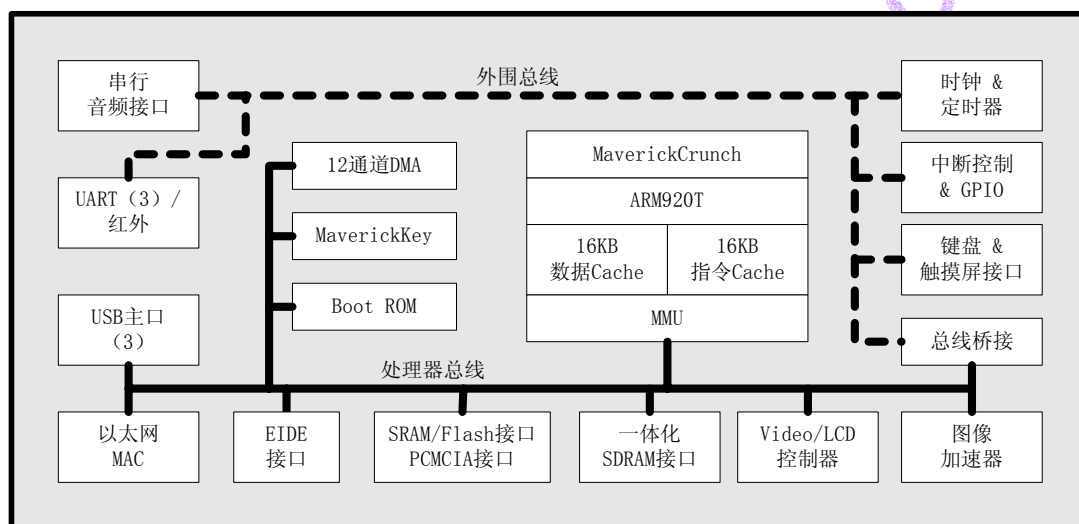


图 1-5 EP9315 的内部结构

1.6.3 EP9315 的应用领域

作为一款高集成度和片内外围丰富的 ARM9 嵌入式微处理器, EP9315 适用于如下领域:

- λ 商用与家用的瘦客户端计算机
- λ 网络收音机
- λ 网络访问设备
- λ 工业计算机
- λ POS 终端
- λ 专业智能终端
- λ 测试与测量设备
- λ 工业手持设备

- λ 医疗设备
- λ 其他嵌入式应用领域。

1.6.4 EP9315 概述

高性能 ARM920T 微处理器核

EP9315 内嵌先进的运行于 200MHz（工业条件下推荐运行 184MHz）的 ARM920T 微处理器核，以及支持 Linux、Windows CE 和其他许多嵌入式操作系统的存储器管理单元（MMU）。ARM920T 的 32 位微控制器结构具有 5 级流水线，可以以极低的功耗提供优异的性能，同时，16KB 指令高速缓存和 16KB 数据高速缓存可为现有的程序和数据提供零等待时间，或者以锁定的方式确保对关键指令和数据的无延迟存取。

对于一些程序存储器大小有限制的应用场合，ARM920T 所支持的压缩型 Thumb 指令集有助于空间利用率的提高和程序存储器的最大化使用。

EP9315 的 ARM920T 内核工作电压为 1.8V，输入/输出电压（I/O）为 3.3V，根据不同的运行速度，功耗从 100mW~750mW 不等。

MaverickCrunch™数学引擎

EP9315 内部集成了一个MaverickCrunch™数学协处理器，显著提高了ARM920T的浮点运算、整型运算与信号处理能力，同时还专门针对数字音频的压缩与解压算法进行了优化，因此，当对数字音频进行编码解码、执行工业控制运算以及其它运算密集型计算和数据处理时，该协处理器可使EP9315 具有高速、精确的计算能力。

MaverickKey™硬件可编程ID

EP9315（包括EP93XX系列）内嵌的MaverickKey™独特的硬件可编程ID是解决网上内容和电子商务安全问题的一个可行方案。MaverickKey™为OEM厂商提供了一种新方法，为硬件赋予特定硬件ID，比如SDMI（安全数字音乐计划）或其他数字版权管理机制使用的ID。

MaverickKey™使用与一个 32 位专用ID和一个 128 位的随机ID相匹配的方式，实现网络安全与数字版权保护，以及电子商务的安全。

通用存储器接口（SDRAM、SRAM、ROM 和 Flash）

EP9315 支持统一的存储器寻址模式，所有的存储设备都可以通过一组公共的地址/数据总线进行访问。除一个专用于 Raster/LCD 刷新引擎的内部端口，存储器的访问通过高速处理器总线完成。其中，SRAM 存储控制器支持访问 8/16/32 位存储设备，可完成 8/16/32 位的 ROM 和 SRAM 访问；SDRAM 控制器可以支持多达 4 组的 32 位 SDRAM 的访问与控制。存储器接口的主要特性如下：

- λ 支持 1~4 组 32 位 SDRAM，66/100MHz 总线频率；
- λ 一个专用于 Raster/LCD 刷新引擎的内部端口
- λ SDRAM、SRAM、ROM 和 Flash 共享数据总线与地址总线；
- λ 支持 NAND Flash 和 NOR Flash；

以太网控制器

EP9315 (包括 EP93XX 系列) 内部集成了一个高性能 1/10/100Mbps 以太网媒体访问控制器 (MAC), 通过与支持 MII 接口的外置 PHY (物理层接口) 简单连接, 系统即可实现高性能、低开销的 Internet 访问功能。以太网控制器的主要特性如下:

- λ 支持 1/10/100Mbps 的传输速率, 可用于各种网络应用场合;
- λ 可通过工业标准 MII 接口与片外物理层芯片接口, 实现网络连接;

IDE 接口

EP9315 的 IDE 接口可以支持与 2 个 ATAPI 工业标准兼容设备的连接, 支持主模式和从模式, 其内部的 DMA 控制器支持 Multiword DMA 和 Ultra DMA 模式的数据传输。

串行接口 (SPI、I²S 和 AC'97)

EP9315 内嵌的 SPI 接口可配置为主模式和从模式, 支持 National Semiconductor[®], Motorola[®] 和 TI 的传输协议。

AC'97 接口支持一路立体声输入和多通道音频输出的编码解码。I²S 接口支持 24 位立体声音频。

以上端口支持复用功能, I²S 的端口 0 可以占用 AC'97 引脚和 SPI 引脚, I²S 的端口 1 和端口 2 的串行输入/输出引脚与 EGPIIO[4,5,6,13] 复用。三个 I²S 端口共用时钟源。具体使用如下:

- λ 正常模式: 支持一个 SPI 端口和一个 AC'97 端口;
 - λ 基于 SSP 的 I²S 模式: 支持一个 SPI 端口和最多三个 I²S 端口;
 - λ 基于 AC'97 的 I²S 模式: 支持一个 AC'97 端口和最多三个 I²S 端口;
- 注意 I²S 不能同时作为 AC'97 和 SSP 的输出端口。

Raster/LCD 接口

EP9315 的 Raster/LCD 接口可以为多种类型的显示器提供数据与接口信号, 主要特性如下:

- λ 支持数字 LCD 与 TFT 的时序与接口信号;
- λ 可编程支持隔行扫描、双扫描彩色和灰度级平板显示器;
- λ 基于 SDRAM 的帧缓存控制方式, 便于提升显示性能;
- λ 支持 4、8、16、24 位像素和 256 个灰度等级;
- λ 硬件支持最大 64×64 像素光标;
- λ 支持 8 位数据接口显示器;

图像加速器

EP9315 内嵌一个硬件图像加速器, 可以完成块拷贝、块填充等操作, 显著提高系统的图像处理能力。图像加速器独立于微处理器, 不会增加系统开销。

带 12 位 A/D 转换器的触摸屏接口

EP9315 内嵌的触摸屏接口可用于数据采集、控制以及与模拟电阻式触摸屏接口等多种场合。主要特性如下:

- λ 支持 4 线、5 线、7 线或 8 线模拟电阻式触摸屏接口；
- λ 未使用的引脚可用于其它功能；
- λ 支持触摸屏中断功能；

键盘接口

EP9315 的键盘接口电路可以扫描 8×8 的矩阵键盘，支持按键中断功能。当系统不使用键盘接口使，所有引脚可以用作 GPIO。

UART 接口

EP9315 带有三个 UART 接口：UART1、UART2 和 UART3。除作为通用 UART 以外，UART1 支持异步 HDLC 全双工发送与接收协议，UART2 支持红外发送与接收功能，UART3 也支持 HDLC。

灵活的系统启动方式

EP9315 内嵌 16KB 的 Boot ROM，可以完成标准的系统配置。Boot ROM 支持从 Flash 存储器、SPI 串行 ROM、或通过 UART 下载等方式启动系统。灵活的系统启动方式便于设计易于控制、可现场更新的嵌入式应用系统。

3 通道 USB2.0 全速主口

EP9315 内部集成了 3 通道 USB2.0 全速主口，与 OHCI USB2.0 全速规范（12Mbps）完全兼容。USB 主口能够提供与诸如海量存储设备、打印机、USB Hub、音频播放器、数码相机等外置便携式设备的连接与数据通信，最多可以挂接 127 个 USB 设备。USB 主口同时也支持 USB2.0 低速标准，大大扩展了系统的应用领域和设计的灵活性。

支持 EEPROM 的二线接口

EP9315 的二线接口支持与诸如 EEPROM、RTC 等二线接口设备的通信与控制。

支持软件微调的实时时钟（RTC）

EP9315 的 RTC 软件微调功能可以对 32.768KHz 的晶振进行补偿，补偿可以精确到 1.24 秒/月。

PLL 与系统时钟

EP9315 的处理器核与外围模块的工作时钟由一个 14.7456MHz 的晶振提供，RTC 的工作时钟由一个 32.768KHz 的晶振提供。

定时器

EP9315 的看门狗定时器用于确保系统处于正常工作状态，需要定时“喂狗”。

两个 16 位定时器可以作为递减计数器，或产生固定时间间隔的中断，时间间隔的范围可以从 0.03 毫秒到 4.27 秒。

一个 32 位的定时器，可以提供 0.03 微妙到 73.3 小时的精确定时。

一个 40 位的调试定时器，可以提供 1.0 微妙到 12.7 天的精确定时。

中断控制器

EP9315 的中断控制器可以支持多达 64 个中断源产生 IRQ 和 FIQ 请求。IRQ 和 FIQ 均支持硬件优先级编程，以便以最快的速度响应时间紧急的中断事件。

EP9315 的 EGPIO 引脚可编程为高电平、低电平、上升沿、下降沿，以及上升沿和下降沿同时触发中断。

12 通道 DMA 控制器

EP9315 的 DMA 控制器支持 12 个独立的 DMA 通道，其中 10 个通道可用于外围模块与存储器之间的数据传输，另外 2 个通道则专门用于存储器之间的数据传输。

双 LED 驱动

EP9315 使用两个专门的引脚，标识为 GRLED 和 REDLED，用于外接两个发光二极管，指示系统的运行状态。

丰富的 GPIO

EP9315 包含增强型和标准型的 GPIO，其中，16 个增强型的 GPIO 可单独配置为输入、输出和带中断功能的输入引脚；还有 49 个标准型的 GPIO 可单独配置为输入、输出或漏极开路（Open-drain）引脚。由于一些标准型的 GPIO 引脚可能与片内功能模块复用，因此 GPIO 的实际可用数量与片内功能模块的使用方式有关。

PCMCIA 接口

EP9315 内嵌一个 PCMCIA 接口，可以支持与 8 位或 16 位的 PCMCIA 设备的连接。

JTAG 硬件调试接口

EP9315 内嵌的 JTAG 调试接口支持 ARM 的 Multi-ICE 及与之兼容的系统调试。

关于 EP9315 各个功能模块更详细的描述，可以参考以后的章节或相关用户手册。

1.7 EP93XX 系列与其它主要 ARM9 微处理器的性能对比

如前文所述，EP93XX 系列高性能嵌入式处理器为消费类电子产品、工业控制及其它通用领域提供了多种性价比选择，几乎每一个嵌入式应用都能从 EP93XX 系列微处理器中找到合适的解决方案。

与其它基于 ARM9 核的嵌入式微处理器相比较，EP93XX 系列无论是在整体性能、片内外围接口，以及系统成本方面，都有比较明显的优势。

表 1-2 为 EP93XX 系列嵌入式微处理器与其它常用 ARM9 微处理器的各项指标对比。

表 1-2 EP93XX 系列与其它常用 ARM9 微处理器的对比

生产商	Cirrus	Cirrus	Cirrus	Cirrus	Cirrus	Samsung	Atmel	Motorola	Intel	TI	Sharp
处理器型号	EP9301	EP9302	EP9307	EP9312	EP9315	S3C2410	AT91RM9200	MC9328MX1	PXA255	OMAP5910	LH7A404
CPU 核	920T	920T	920T	920T	920T	920T	920T	920T	Xscale	925+CS55	922T
运行速度（MHz）	166	200	200	200	200	200	180	200	400	150	200
数学协处理器		有	有	有	有					200MHz	
USB 协议	2.0	2.0	2.0	2.0	2.0	1.1	2.0	1.1	1.1	1.1	1.1
USB 接口	2 个主口	2 个主口	3 个主口	3 个主口	3 个主口	2 主 1 从	2 主 1 从	1 个从口	1 个从口	1 主 1 从	1 个从口
以太网控制器	1/10/100M	1/10/100M	1/10/100M	1/10/100M	1/10/100M		10/100M				
LCD 接口			有	有	有	有		有	有	有	有
触摸屏接口			有	有	有	有		有	有	有	有
图像加速器			有		有						
键盘接口			有	有	有						
AC'97	有	有	有	有	有				有		有
IDE 接口				有	有						
PCMCIA 接口					有				有		
UART 接口	2	2	3	3	3	3	4	2	3	3	3
DMA 通道	12	12	12	12	12	4	20	11	16	15	12
加密 ID	有	有	有	有	有						
温度等级	工业/商业	工业/商业	工业/商业	工业/商业	工业/商业	工业/商业	工业	商业	工业	工业	工业/商业
封装方式	QFP	QFP	BGA	BGA	BGA	BGA	QFP/BGA	BGA	BGA	BGA	BGA
应用领域	通用	通用	通用	通用	通用	PDA，通用	通用	PDA，通用	PDA，3G，蓝牙	3G，通用	通用

1.8 本章小结

本章对 EP93XX 系列嵌入式微处理器的中的 EP9301、EP9302、EP9307、EP9312 和 EP9315 的性能特征、应用领域进行了比较详细的介绍，同时与目前的常见基于 ARM9 核的嵌入式微处理器进行了对比分析，希望读者通过对本章的阅读，能对 EP93XX 系列嵌入式微处理器有一个比较全面的认识，从而为特定嵌入式应用的硬件平台作出合适的选择。

www.emsyschina.com

第 2 章 EP93XX 系列微处理器的片内资源

本章介绍 EP93XX 系列嵌入式微处理器的片内资源，包括其基本特性及功能的简单描述等，使读者对 EP93XX 系列微处理器的片内资源有比较全面的了解。

由于 EP93XX 系列微处理器每款产品的应用领域定位各不相同，其片内资源的配置也有一定的差异，请读者在阅读和进行系统设计时注意区分。

2.1 EP93XX 系列嵌入式微处理器的片内资源

对于一款嵌入式微处理器来说，其应用领域很大程度上是由它的片内资源配置所决定的，同时，系统设计者在进行嵌入式系统设计与开发时，绝大多数情况也是对片内的功能模块进行编程与开发，因此，对 EP93XX 系列嵌入式微处理器的片内资源进行深入、详细的了解和掌握，是系统设计与开发的必然要求。

EP93XX 系列嵌入式微处理器的片内资源，从 EP9301 到 EP9315，具有非常好的一致性，即无论是哪一款处理器，只要是相同功能的外围模块，其寄存器映射、硬件接口设计和软件编程方法，都几乎是完全一致的，设计者只需要掌握其中某一款处理器的外围模块的使用方法，就可以熟练应用整个系列的处理器，甚至不需要做任何修改，这也是 EP93XX 系列微处理器的优势之一。

EP93XX 系列微处理器的主要片内资源有：

- λ ARM920T 核与先进高速总线（AHB，Advanced High-Speed Bus）；
- λ Boot ROM；
- λ MaverickCrunchTM数学引擎（EP9301 不包括该部件）；
- λ 系统控制器；
- λ MaverickKeyTM硬件ID；
- λ 向量中断控制器；
- λ 以太网控制器；
- λ DMA 控制器；
- λ USB 主控制器；
- λ 静态存储器控制器（SMC，Static Memory Controller）；
- λ SDRAM 控制器；
- λ UART 接口；
- λ 红外接口；
- λ PWM（仅限于 EP9307、EP9312、EP9315）；
- λ 定时器、看门狗定时器；

- λ 实时时钟 RTC;
- λ I²S 控制器;
- λ AC'97 控制器;
- λ 同步串行接口 (SSP);
- λ ADC 接口/触摸屏接口 (触摸屏接口仅限于 EP9307、EP9312、EP9315);
- λ IDE 接口 (仅限于 EP9312、EP9315);
- λ Raster/LCD 控制器 (仅限于 EP9307、EP9312、EP9315);
- λ 图像加速器 (仅限于 EP9307、EP9315);
- λ 键盘接口 (仅限于 EP9307、EP9312、EP9315);
- λ PCMCIA 接口 (仅限于 EP9315);
- λ GPIO 接口;

在以上的部件中, 绝大多数是 EP93XX 系列所共有的, 他们的工作原理和寄存器映射地址完全相同, 但也有少部分部件是某些型号的处理器专有的, 请读者注意区分。

2.2 ARM920T 核与先进高速总线 (AHB, Advanced High-Speed Bus)

EP93XX 系列的所有微处理器都是基于 ARM920T 核的, 我们在此对 ARM920T 核的基本特性和相关的总线结构做一个简单的介绍。

ARM920T 核采用哈佛 (Harvard) 总线结构, 包括 ARM9TDMI 核、MMU, 指令及数据 Cache, 支持 32 位 ARM 指令集和 16 位 Thumb 压缩指令集。

ARM920T 内部总线采用先进微控制器总线架构 AMBA (Advanced Micro-controller Bus Architecture), 包括处理器核内部的高速总线 AHB、处理器核外部的低速外围总线 APB, 以及他们之间的桥接。

高速总线 AHB (Advanced High-performance Bus) 连接处理器、MMU、Cache、DMA 控制器及存储器接口部件, 采用一个高速的内部总线时钟同步工作。而低速外围总线 APB (Advanced Peripheral Bus) 则连接诸如 UART、GPIO 等低速的外围设备, 以较低的时钟频率工作, AHB 和 APB 通过一个 AHB/APB 桥连接。

典型的 AMBA 架构如图 2-1 所示。

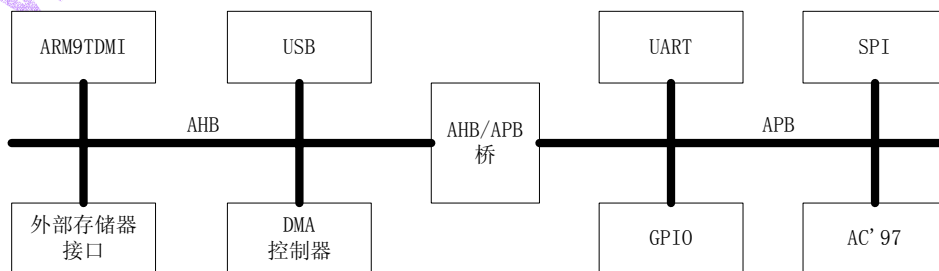


图 2-1 典型的 AMBA 架构

对于 EP93XX 系列嵌入式微处理器的 AMBA 总线连接,读者可以参考第一章处理器的内部结构图,图中的实线表示 AHB,虚线表示 APB。

关于 ARM920T 和 AMBA 更详细的信息,可以参考其它相关资料。

2.3 Boot ROM 及系统启动

2.3.1 Boot ROM 及地址映射

EP93XX 系列嵌入式微处理器都设计了片内的 Boot ROM, Boot ROM 是一个挂接在 AHB 上的 16KB 掩膜 ROM,该 ROM 包括三个代码区,其中低 8KB 为系统的启动代码,随后 4KB 为第一安全代码块,最后 4KB 为第二安全代码块。如果系统以非安全模式启动,则仅访问低 8KB 代码,当系统以安全模式启动时,需要访问其中一个安全代码块。

当系统复位时,ARM920T 从 0x0 地址开始执行代码,EP93XX 系列微处理器可以通过硬件选择,将不同的启动设备映射到 0x0 地址。当选择从内部启动时,Boot ROM 被映射到 0 地址,处理器就开始执行 Boot ROM 中固化的代码,代码执行完毕以后,则将系统的片外存储器映射到 0x0 地址。

EP93XX 系列的 Boot ROM 地址固定为 0x8009_0000,当选择系统从内部启动时,Boot ROM 同时被映射到 0x0 地址。

2.3.2 Boot ROM 的功能描述

EP93XX 系列的 Boot ROM 代码支持以以下方式启动系统:

- λ UART1: 用户代码可以通过 UART1 下载到片内的启动缓冲区并执行。
- λ SPI 串行 ROM: 存放在 SPI 串行 ROM 中的用户代码可以被拷贝到片内的启动缓冲区并执行。
- λ Flash: 直接执行固化在系统 Flash 中的代码,这也是最常用的系统启动方式。

由于受到片内 SRAM 容量的限制,通过前两种方式下载到片内的可执行代码都不能太大(最大一般不超过 2KB),因此,采用这种方式一般都需要再通过红外、USB 或以太网络,以下载运行更复杂的可执行代码,如嵌入式操作系统等。

可执行代码的头标志

EP93XX 的可执行代码必须以 ASCII 字符串“CRUS”或“SURC”作为头标志(HeaderID),从 SPI 串行 ROM 或从 Flash 中读取的代码必须有该头标志。

启动过程

EP93XX 系列微处理器 Boot ROM 的启动过程如下:

- 1、重映射存储器;

- 2、关闭 GRLED，点亮 REDLED；
- 3、禁用看门狗；
- 4、读取由硬件设置的启动状态；
- 5、由外部时钟源建立系统时钟运行；
- 6、根据启动状态设置的存储器宽度，进行如下操作：
 - A、用最小配置对 SDRAM 和 Flash 存储器接口进行初始化，以保持最好的兼容性；
 - B、以同样的方式初始化 SRAM 接口；
 - C、进行最基本的存储器测试；
- 7、根据系统的硬件启动状态配置，决定是否开始串行下载，如果为串行下载，则进行如下操作：
 - A、以 9600 的波特率、8 个数据位、1 个停止位、无校验的方式初始化 UART1；
 - B、然后向 UART1 输出一个字符 “<”，等待接收从 UART1 下载代码；
 - C、从 UART1 接收 2048 字节（2KB）的数据并存储到片内启动缓冲区中（以太网的 MAC 缓冲区）；
 - D、输出一个字符 “>”，表示已成功接收到 2048 字节的数据；
 - E、点亮 GRLED；
 - F、跳转到片内启动缓冲区的程序执行；
- 8、如果不是串行下载方式，就尝试读取 SPI 串行 ROM，并进行如下操作：
 - A、检测串行 ROM 的前 4 个字节是否为“CRUS”或“SURC”的 ASCII 码，校验头标志；
 - B、读取其后 2048 字节（2KB）的数据并存储到片内启动缓冲区中；
 - C、点亮 GRLED；
 - D、跳转到片内启动缓冲区的程序执行；
- 9、如果不能从串行 ROM 启动，就尝试从系统的 Flash（Flash 的基地址+0x0000 处）读取头标志“CRUS”或“SURC”的 ASCII 码并进行校验。每个 FLASH 存储区都会被读取，然后进行如下操作：
 - A、点亮 GRLED；
 - B、跳转到 Flash 基地址加 4 字节的位置读取代码执行；
- 10、如果校验错误，则尝试从系统的 Flash（Flash 的基地址+0x1000 处）读取头标志“CRUS”或“SURC”的 ASCII 码并进行校验。每个 Flash 存储区都会被读取，然后进行如下操作：
 - A、点亮 GRLED；
 - B、跳转到 Flash 的起始地址位置读取代码执行；
- 11、如果校验错误，则尝试从系统的 SDRAM 或 Sflash（0xC000_0000 和 0xF000_0000 处）读取头标志“CRUS”或“SURC”的 ASCII 码并进行校验，然后进行如下操作：
 - A、点亮 GRLED；

- B、 跳转到 0xC000_0004 或 0xF000_0004 处读取代码执行；
- 12、 如果校验错误，则继续尝试从系统的 SDRAM 或 Sflash（0xC000_1000 和 0xF000_1000 处）读取头标志“CRUS”或“SURC”的 ASCII 码并进行校验，然后进行如下操作：
- A、 点亮 GRLED；
- B、 跳转到 0xC000_0000 或 0xF000_0000 处读取代码执行；
- 13、 若以上步骤均未能正常启动系统，则将未初始化的异常向量拷贝低地址的 SDRAM 中，同时 GRLED 闪烁。

尽管 EP93XX 系列微处理器的 Boot ROM 启动过程设计得比较复杂，但几乎考虑到了系统启动所有可能的情况，便于用户设计灵活的启动方法和现场的代码更新。

以上的启动过程可以用图 2—2 所示的流程描述。

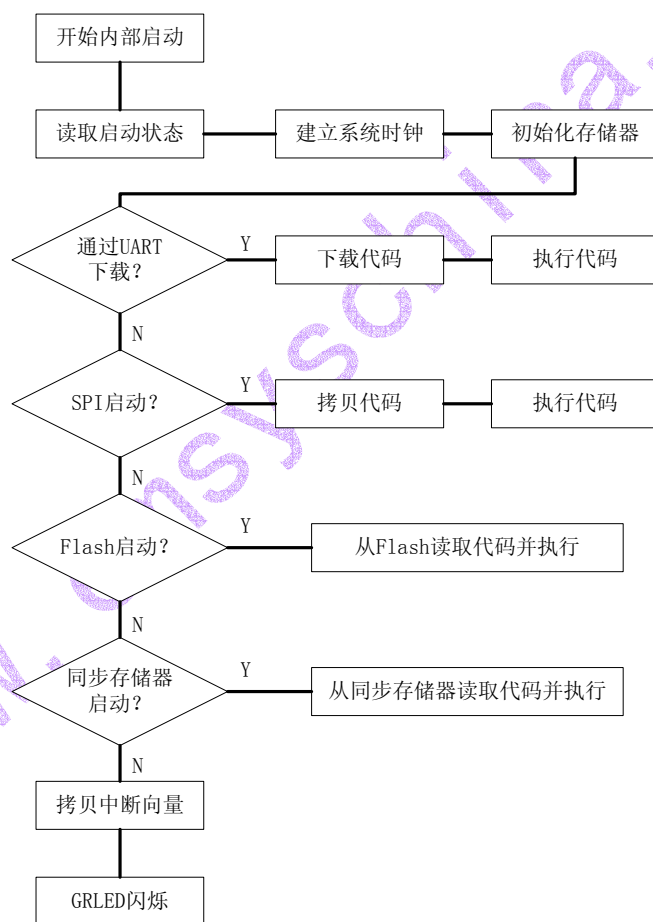


图 2—2 Boot ROM 的启动流程图

2.3.3 系统的启动配置

EP93XX 系列微处理器根据硬件的启动配置决定系统的启动运行状态，所有的配置选择如表 2-1 和表 2-2 所示。

表 2-1 EP9301/EP9302 系列的启动配置选择

EECLK	EEDAT	BOOT1	BOOT0	ASDO	CSn[7:6]	
0	1	0	0	1	00 01	从 SDRAM 控制器所选择的同步存储器外部启动，存储器必须为 SROM 或 SFlash，数据宽度由 CSn[7:6]的上电锁存状态决定： 16 位 SFlash 16 位 SROM
0	1	0	0	0	00 01	从静态存储控制器控制的 nCS0 所选择的存储器外部启动，存储器数据宽度由 CSn[7:6]的上电锁存状态决定： 8 位 SRAM 16 位 SRAM
1	1	0	1	X	01	16 位串行启动
1	1	0	0	X	XX	从片内 ROM 启动，读取 SPI 存储器
1	1	0	0	1	00 01	从片内 ROM 启动。系统存储器数据宽度由 CSn[7:6]的上电锁存状态决定： 8 位 16 位
1	1	0	0	0	00 01	从片内 ROM 启动。系统存储器数据宽度由 CSn[7:6]的上电锁存状态决定： 8 位 16 位

表 2-2 EP9307/EP9312/EP9315 系列的启动配置选择

EECLK	EEDAT	BOOT1	BOOT0	ASDO	CSn[7:6]	
0	1	0	0	1	00 01 10 11	从 SDRAM 控制器所选择的同步存储器外部启动，存储器必须为 SROM 或 SFlash，数据宽度由 CSn[7:6]的上电锁存状态决定： 16 位 SFlash 16 位 SROM 32 位 SFlash 32 位 SROM
						从静态存储控制器控制的 nCS0 所选择的异步存储器外部启动，存储器数据宽度由 CSn[7:6]的上电锁存状态决定：

0	1	0	0	0	00 01 10 11	8 位 SRAM 16 位 SRAM 32 位 SRAM 32 位 SRAM
1	1	0	1	X	01 10 11	16 位串行启动 32 位串行启动 32 位串行启动
1	1	0	0	X	XX	从片内 ROM 启动，读取 SPI 存储器
1	1	0	0	1	00 01 10 11	从片内 ROM 启动。系统同步存储器数据宽度由 CSn[7: 6]的上电锁存状态决定： 16 位 16 位 32 位 32 位
1	1	0	0	0	00 01 10 11	从片内 ROM 启动。系统异步存储器数据宽度由 CSn[7: 6]的上电锁存状态决定： 8 位 16 位 32 位 32 位

在表 2-1 和表 2-2 中，请注意有阴影的启动配置方式为系统设计的常用方式，我们在系统设计与开发中会使用到。

2.4 MaverickCrunch™数学引擎

MaverickCrunch™数学引擎可以加速IEEE-754 格式浮点数学运算和 32/64 位定点数学运算，它包含一个高性能整型乘加器，其运算速度大大高于ARM920T核内嵌的乘加单元。MaverickCrunch™数学引擎可以显著加速数字音频格式的编码解码运算。

MaverickCrunch™数学引擎的主要特性如下：

- λ 支持 IEEE-754 格式单、双精度浮点运算；
- λ 支持 32/64 位整型运算；
- λ 支持加法、乘法和比较运算；
- λ 32 位输入的整型乘加器，支持 72 位加法；
- λ 支持整型移位运算；
- λ 支持浮点数与定点数之间的相互转换；
- λ 内嵌 16 个 64 位寄存器；
- λ 内嵌 4 个 72 位加法器；

MaverickCrunch™数学引擎使得EP93XX（不包括EP9301）系列微处理器在进行数字音频编码解码、工业控制运算以及其它运算密集型计算和数据处理时，具有高速、精确的计

算能力。

关于MaverickCrunch™数学引擎更详细的描述，可以参考EP93XX系列微处理器的相关资料。

2.5 系统控制器

EP93XX 系列微处理器的系统控制器主要包含三部分功能：系统配置管理、时钟控制和电源管理。

2.5.1 系统复位与启动

以下几种情况会引起系统复位：

- λ 上电复位：由 EP93XX 系列微处理器的外部引脚 PRSTn 控制，当该引脚出现低电平信号时，将复位整个芯片的所有部件；
 - λ 用户复位：由 EP93XX 系列微处理器的外部引脚 PSTOn 控制，当该引脚出现低电平信号时，将复位芯片的大多数部件，但 RTC、SDRAM 刷新控制等部件不会受到影响；
 - λ 三键复位：当用户同时按下外接矩阵键盘的三个特定按键时，会产生用户复位。显然，只有支持键盘接口的处理器 EP9307、EP9312 和 EP9315 才具有该功能。
 - λ 软复位与看门狗复位：软件复位和看门狗复位与用户复位相同。
- 当以上复位状态结束时，系统就进入启动状态。

2.5.2 硬件配置控制

EP93XX 系列微处理器的硬件配置控制为系统提供了灵活的启动机制，同时可以选择外接启动存储器的类型和数据宽度等。

硬件配置控制根据芯片部分引脚的启动锁存状态，决定诸如是否使能看门狗定时器、启动模式选择（内部或外部启动）、以及外部启动存储器数据宽度选择等。

EP93XX 系列微处理器可用于启动锁存状态的引脚如下：

- λ CSn[1]: 该引脚下拉禁用看门狗定时器；
- λ CSn[2]: 该引脚下拉禁止看门狗定时器的输出脉冲；
- λ CSn[3]: 该引脚应该上拉；
- λ EECLK: 用于选择内部启动与外部启动；
- λ EEDAT: 该引脚应该上拉；
- λ BOOT[1:0]: 用于选择启动模式；
- λ ASDO: 用于选择同步或异步启动；
- λ CSn[7:6]: 用于选择外部启动存储器数据宽度；

对于这些引脚复位锁存状态所表示的系统启动配置功能，可以参考表 2-1 和表 2-2。

2.5.3 时钟控制

EP93XX 系列微处理器的时钟控制系统采用了非常灵活的方式，产生系统各个部件所需要的时钟信号。

EP93XX 系列微处理器通过一个外部时钟源，产生多达 20 个独立的频率不同的时钟信号，提供给系统的各个部件，不同的时钟信号之间互不影响。

EP93XX 系列微处理器可以外接两个晶振，频率分别为 32.768KHz 和 14.7456MHz，其中，32.768KHz 的晶振主要用于 RTC 的时钟源，14.7456MHz 的晶振为系统的主时钟，为产生系统所需要的高频时钟信号，处理器内部采用了两个锁相环电路（PLL1 和 PLL2）对 14.7456MHz 的频率进行倍乘，达到较高的频率，然后再对这个高频时钟进行分频，产生部件工作所需频率的时钟信号。

锁相环电路 PLL1 的结构如图 2-3 所示，PLL2 的结构与之类似。

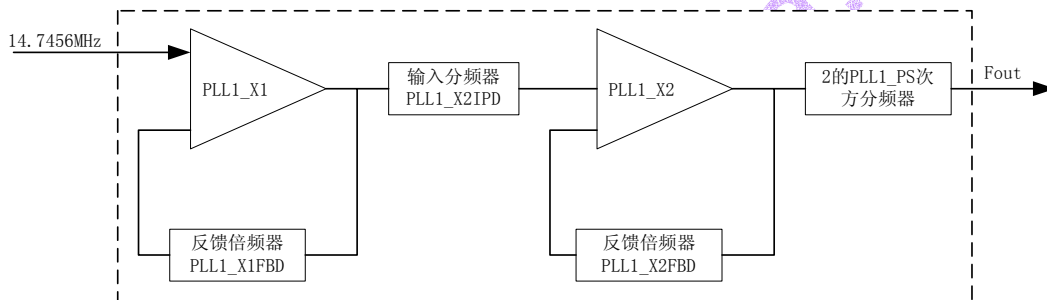


图 2-3 PLL1 的结构图

两个 PLL 均可由软件编程，其输出频率 F_{out} 可由下式求得：

$$F_{out} = 14.7456\text{MHz} \times \frac{(\text{PLL1_X1FBD} + 1) \times (\text{PLL1_X2FBD} + 2)}{(\text{PLL1_X2IPD} + 1) \times 2^{\text{PLL1_PS}}}$$

式中的 PLL1_X1FBD、PLL1_X2FBD、PLL1_X2IPD 和 PLL1_PS 可从系统的时钟速度控制寄存器（ClkSet1，0x8093_0020）中获取。

图 2-3 中 PLL1 各部分的输入输出频率范围如下：

- λ PLL1_X1 要求的输入频率范围为：11.058MHz~166MHz；
 - λ PLL1_X1 的输出频率范围为：294MHz~368MHz；
 - λ PLL1_X2（输入分频器之后）要求的输入频率范围为：12.9MHz~166MHz；
 - λ PLL1_X2（2 的 PLL1_PS 次方分频器之前）的输出频率范围为：290MHz~528MHz；
- PLL2 各部分的输入输出频率的范围与 PLL1 相同。

EP93XX 系列微处理器的各个功能部件工作所需时钟信号的产生如图 2-4 所示。

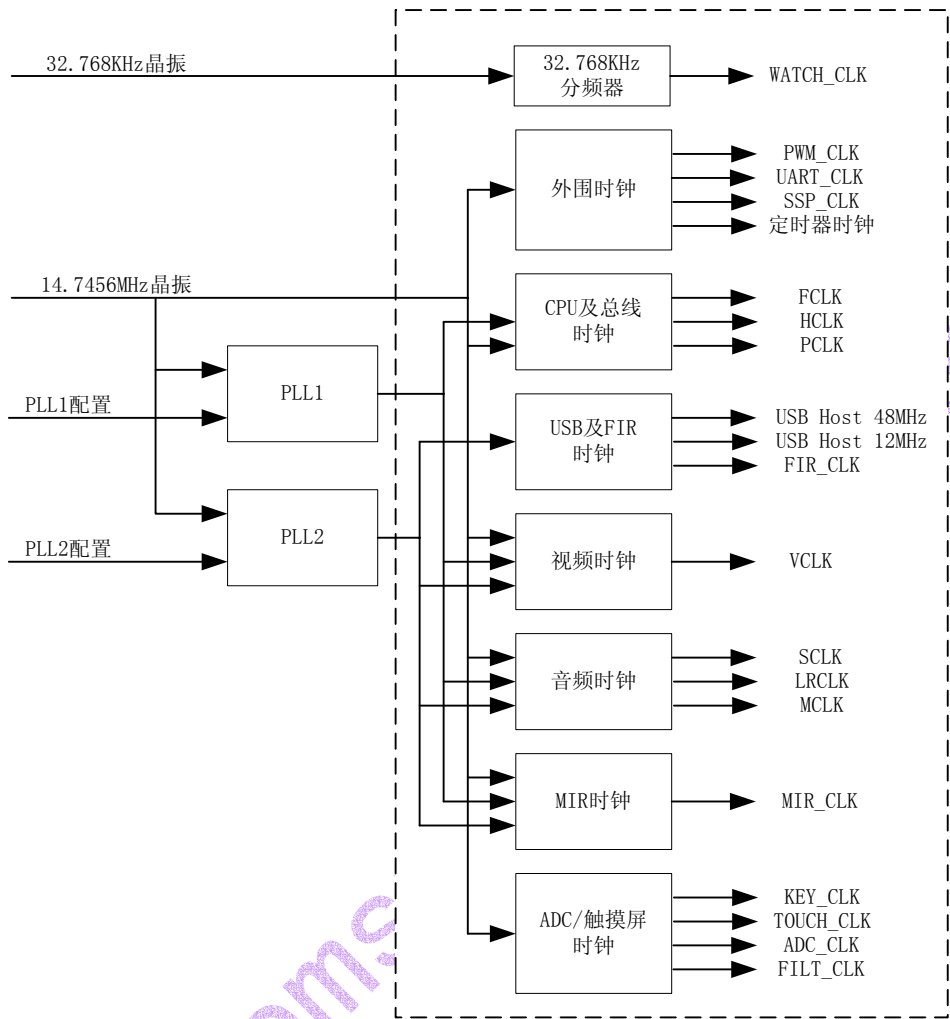


图 2—4 总线与外围部件的时钟信号

2.5.4 电源管理

EP93XX 系列微处理器支持电源管理功能，以降低系统的功耗，处理器支持三种工作模式，并可在三种模式之间进行切换，以保证在完成系统功能的同时，降低系统的整体运行功耗：

- λ 运行模式（Run）：正常工作模式，当系统上电启动时，自动进入该模式；
- λ 暂停模式（Halt）：此时 ARM920T 核停止运行；
- λ 旁路模式（Standby）：仅有 SDRAM 自刷新和 RTC 运行。

关于 EP93XX 系列微处理器系统控制器更详细的内容，可以参考相关用户手册。

2.6 向量中断控制器

EP93XX 系列微处理器采用两个级连的向量中断控制器（VIC，Vectored Interrupt Controller）管理和处理系统的中断。与一般的中断控制器相比较，向量中断控制器具有更快的响应速度，因为它可以直接获取中断服务程序的入口地址信息，而不必通过软件判断的方式。

EP93XX 系列微处理器 VIC 的结构如图 2-5 所示。

每一个 VIC 最多能够支持 32 个中断源，两个 VIC 采用菊花链（Daisy-chain）连接，可以支持最多 64 个中断源。

每一个 VIC 最多可以支持 16 个向量中断和 16 个非向量中断，其中，向量中断只能产生 IRQ 请求。当向量中断或非向量中断产生中断请求时，可以通过读取 VIC 向量地址寄存器的内容，获取中断服务程序的入口地址。读取 VIC 向量地址寄存器也同时通知中断优先级控制硬件，该中断服务程序已在执行，需要屏蔽同级或低级的中断请求，当中断服务程序执行完毕时，需要向 VIC 向量地址寄存器写入一个任意值，通知中断优先级控制硬件当前的中断服务程序已经执行完毕，可以允许同级或低级的中断请求。

EP93XX 系列微处理器的 VIC 支持两种类型的中断请求：

λ FIQ 中断请求，用于完成快速的中断处理；

λ IRQ 中断请求，用于完成一般的中断处理；

所有的中断输入信号均为高电平有效。

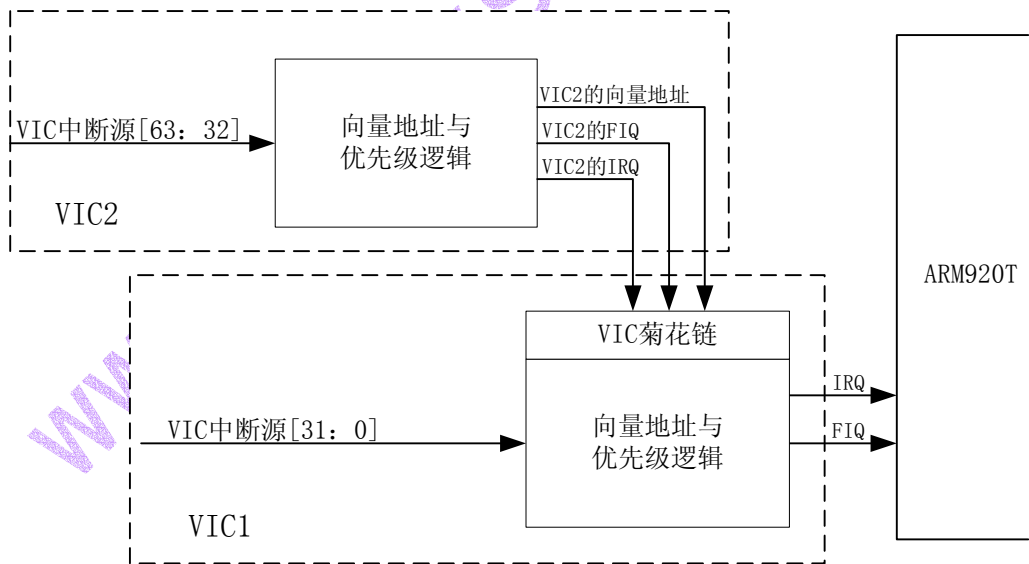


图 2-5 向量中断控制器结构图

2.6.1 中断优先级

由于 ARM9 核总是以较高的优先级处理 FIQ 中断，因此，FIQ 中断具有最高的优先级，然后是向量中断 0 到向量中断 15，非向量 IRQ 中断的优先级最低。

所有的非向量中断都可编程为 FIQ 或 IRQ 中断，但向量中断只能为 IRQ 中断。同时，每一个 VIC 所支持的 32 个中断源中，其中任意的 16 个可以编程设置为向量中断，因为每一个 VIC 最多只能支持 16 个向量中断。

2.6.2 中断源描述

如前所述，EP93XX 系列微处理器采用两个级连的 VIC 管理和处理中断，最多可以支持 64 个中断源，中断源及具体描述如表 2-3 所示。

表 2-3 EP93XX 的中断源及描述

VIC 中断源	中断源名	描 述
0	—	未使用。
1	—	未使用。
2	COMMRX	ARM 调试通信通道接收中断。当该信号为高电平时，表示通信通道接收缓存区有数据等待 ARM 处理器核读取。更详细的内容可以参考 ARM 技术手册。
3	COMMTX	ARM 调试通信通道发送中断。当该信号为高电平时，表示通信通道发送缓存区空。更详细的内容可以参考 ARM 技术手册。
4	TC1OI	定时/计数器 1 (TC1) 下溢出中断。当 TC1 计数到 0 时的下一个下降沿产生该中断请求。
5	TC2OI	定时/计数器 2 (TC2) 下溢出中断。当 TC2 计数到 0 时的下一个下降沿产生该中断请求。
6	AACINTR	音频编码解码器中断请求。
7	DMAM2P0	存储器与外围模块的 DMA 通道 0 传输中断。
8	DMAM2P1	存储器与外围模块的 DMA 通道 1 传输中断。
9	DMAM2P2	存储器与外围模块的 DMA 通道 2 传输中断。
10	DMAM2P3	存储器与外围模块的 DMA 通道 3 传输中断。
11	DMAM2P4	存储器与外围模块的 DMA 通道 4 传输中断。
12	DMAM2P5	存储器与外围模块的 DMA 通道 5 传输中断。
13	DMAM2P6	存储器与外围模块的 DMA 通道 6 传输中断。
14	DMAM2P7	存储器与外围模块的 DMA 通道 7 传输中断。
15	DMAM2P8	存储器与外围模块的 DMA 通道 8 传输中断。
16	DMAM2P9	存储器与外围模块的 DMA 通道 9 传输中断。
17	DMAM2M0	存储器与存储器的 DMA 通道 0 传输中断。
18	DMAM2M1	存储器与存储器的 DMA 通道 1 传输中断。

19	GPIO0INTR	GPIO 中断 0。EP9301/EP9302 未使用
20	GPIO1INTR/ GPIO0INTR	GPIO 中断 1。EP9301/EP9302 为 GPIO 中断 0。
21	GPIO2INTR/ GPIO1INTR	GPIO 中断 2。EP9301/EP9302 为 GPIO 中断 1。
22	GPIO3INTR/ GPIO2INTR	GPIO 中断 3。EP9301/EP9302 为 GPIO 中断 2。
23	UART1RXINTR1	UART1 接收中断。
24	UART1TXINTR1	UART1 发送中断。
25	UART2RXINTR1	UART2 接收中断。
26	UART2TXINTR1	UART2 发送中断。
27	UART3RXINTR1	UART3 接收中断。EP9301/EP9302 未使用。
28	UART3TXINTR1	UART3 发送中断。EP9301/EP9302 未使用。
29	INT_KRY	矩阵键盘中断。EP9301/EP9302 未使用。
30	INT_TOUCH	触摸屏控制器中断。EP9301/EP9302 未使用。
31	—	未使用。
32	INT_EXT[0]	外部中断 0。
33	INT_EXT[1]	外部中断 1。
34	INT_EXT[2]	外部中断 2。EP9301/EP9302 未使用。
35	TINTR	64Hz 滴答中断。该中断由内部 64Hz 时钟信号的上升沿产生。
36	WEINT	看门狗时间到中断。
37	INT_RTC	RTC 中断。
38	INT_IrDA	IrDA 中断。
39	INT_MAC	以太网 MAC 中断。
40	INT_EXT[3]/ INT_EXT[2]	外部中断 3。EP9301/EP9302 为外部中断 2。
41	INT_PROG	光栅可编程中断。EP9301/EP9302 未使用。
42	CLK1Hz	1Hz 时钟中断。EP9301/EP9302 未使用。
43	V_SYNC	视频场同步中断。EP9301/EP9302 未使用。
44	INT_VIDEO_FIFO	光栅视频 FIFO 中断。EP9301/EP9302 未使用。
45	INT_SSP1RX	SSP 接收中断。
46	INT_SSP1TX	SSP 发送中断。
47	GPIO4INTR	GPIO 中断 4。EP9301/EP9302 未使用。
48	GPIO5INTR	GPIO 中断 5。EP9301/EP9302 未使用。
49	GPIO6INTR	GPIO 中断 6。EP9301/EP9302 未使用。
50	GPIO7INTR	GPIO 中断 7。EP9301/EP9302 未使用。
51	TC3OI	定时/计数器 3 (TC3) 下溢出中断。当 TC3 计数到 0 时的下一个下降沿产生该中断请求。
52	INT_UART1	UART1 中断。当 UART1 产生任何中断时，都将触发该中断。
53	SSPINTR	SSP 中断。
54	INT_UART2	UART2 中断。当 UART2 产生任何中断时，都将触发该中断。
55	INT_UART3	UART3 中断。当 UART3 产生任何中断时，都将触发该中断。 EP9301/EP9302 未使用。
56	USHINTR	USB 主口中断。

57	INT_PME	PME 中断。EP9301/EP9302 未使用。
58	INT_DSP	ARM 内核中断。EP9301/EP9302 未使用。
59	GPIOINTR	GPIO 联合中断。
60	SAINTR	I ² S 模块联合中断。
61	—	未使用。
62	—	未使用。
63	—	未使用。

在 EP93XX 系列微处理器中，EP9302/EP9302 的中断源是一致的，EP9307/EP9312/EP9315 的中断源是一致的，但 EP9302/EP9302 和 EP9307/EP9312/EP9315 的中断源之间有少量的不同，请读者注意区分。

关于 EP93XX 系列微处理器向量中断控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.7 以太网控制器

EP93XX 系列微处理器内部集成了一个高性能 1/10/100Mbps 以太网媒体访问控制器（MAC），通过与支持 MII 接口的外置 PHY（物理层接口）简单连接，系统即可实现高性能、低开销的 Internet 访问功能。以太网控制器的主要特性如下：

- λ 支持 1/10/100Mbps 的传输速率，可用于多种网络应用场合；
- λ 可通过工业标准 MII 接口与片外物理层芯片接口，实现网络连接；

EP93XX 系列微处理器的以太网控制器是 AHB 与 MII 之间的逻辑接口，包含本地存储器、DMA 控制器，并支持带流控的全双工操作。以太网控制器的结构如图 2—6 所示。

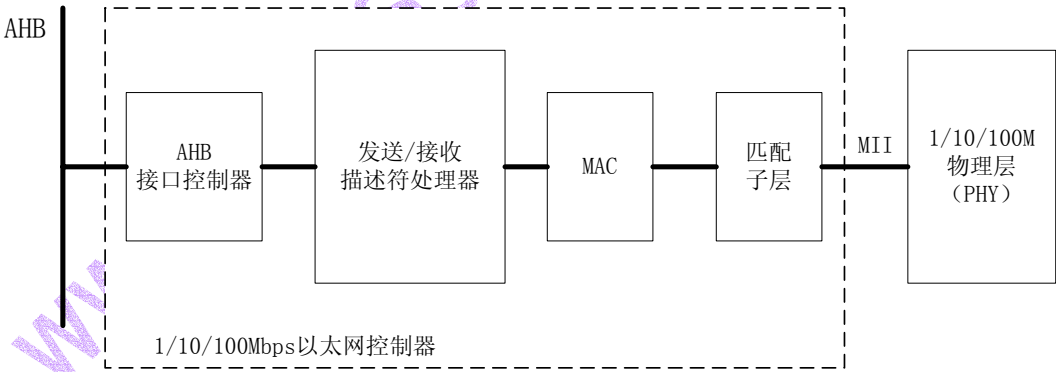


图 2—6 以太网控制器结构图

以太网控制器还包含 544 字的 RAM（约 2KB），用于数据包缓冲区和控制器数据存储，所有的 RAM 都映射到寄存器地址空间，用户可以直接访问。

EP93XX 系列微处理器通过 UART1 下载启动和从 SPI 接口 ROM 启动时所使用的片内启动缓冲区，也是这部分 RAM。

由于以太网控制器的工作原理比较复杂，相应的控制寄存器也较多，在此不作详述。

关于 EP93XX 系列微处理器以太网控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.8 DMA 控制器

EP93XX 系列微处理器的 DMA 控制器可以用于完成存储器与功能模块之间（M2P）、存储器与存储器之间（M2M）的高速流式数据传输。

EP93XX 系列微处理器共支持 12 个 DMA 通道，其中 2 个通道专用于存储器之间的数据传输，其余 10 个通道则完成存储器与功能模块之间的数据传输，具体功能如下：

- λ I²S : 可以包含 3 个发送与 3 个接收 DMA 通道；
- λ AAC: 可以包含 3 个发送与 3 个接收 DMA 通道；
- λ UART1: 可以包含 1 个发送与 1 个接收 DMA 通道；
- λ UART2: 可以包含 1 个发送与 1 个接收 DMA 通道；
- λ UART3: 可以包含 1 个发送与 1 个接收 DMA 通道，EP9301/EP9302 无此项；
- λ IrDA: 可以包含 1 个发送与 1 个接收 DMA 通道；

每一个部件自身都有双向的 DMA 总线，可以支持双向数据的同时传输。存储器之间的 DMA 传输则通过系统 AHB 完成。

关于 EP93XX 系列微处理器 DMA 控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.9 USB 主控制器

EP93XX 系列嵌入式微处理器的 USB 主控制器可以与支持 USB2.0 协议的低速（1.2Mbps）和全速（12Mbps）设备通信。USB 主控制器支持 2 个（或 3 个）Root Hub 接口，与 OpenHCI 规范兼容（1.0a 版本），其主要特性如下：

- λ 兼容 OpenHCI 规范（1.0a 版本）；
- λ 兼容 USB2.0 规范（USB2.0 协议）；
- λ 支持与低速和全速设备的通信；
- λ 支持 2 个（EP9301/EP9302）或 3 个（EP9307/EP9312/EP9315）Root Hub 端口；
- λ 支持主、从方式的 AHB 接口；
- λ 支持 DMA 功能；

关于 EP93XX 系列微处理器 USB 主控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.10 静态存储器控制器

EP93XX 系列微处理器的静态存储器控制器（SMC，Static Memory Controller）提供片外静态存储器与 AHB 系统总线的接口。SMC 最多能同时支持 8 个独立的可配置静态存储器组，每一个存储器组可以挂接如下类型的静态存储器：

- λ SRAM；
- λ ROM；
- λ Flash；
- λ Burst ROM；

每一个存储器组都能支持 8 位、16 位或 32 位数据宽度的存储器（注意 EP9301/EP9302 不支持 32 位数据宽度）。SMC 配置为小端模式。

SMC 的主要特性如下：

- λ 支持存储器选择；
- λ 支持存储器的顺序访问；
- λ 支持产生等待状态；
- λ 支持字节读/写操作；
- λ 支持外部总线接口；

SMC 可以通过读取 WAITn 引脚信号，支持插入等待周期的功能，以匹配与低速外设的读写操作。

关于 EP93XX 系列微处理器静态存储器控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.11 SDRAM 控制器

EP93XX 系列微处理器的 SDRAM 控制器提供多种同步存储器与 AHB 系统总线的高速存储器接口，包括 SDRAM、同步 Flash（SFlash）、同步 ROM（SROM）等。

SDRAM 控制器的主要特性如下：

- λ 最多支持 4 个可独立配置的同步存储器组；
- λ 支持 SROM 操作；
- λ 支持 SFlash 的擦除与编程；
- λ 支持 SDRAM 与控制器之间的 4 倍数据猝发传输；
- λ 支持 16 位、32 数据宽度（注意 EP9301/EP9302 不支持 32 位数据宽度）；

关于 EP93XX 系列微处理器 SDRAM 控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.12 UART 接口

EP93XX 系列微处理器的 UART 模块能够通过 DMA 方式、中断方式或查询方式完成数据的收发，同时还可以采用回环（Loopback）模式完成数据本地收发的测试。

UART 模块主要完成数据的串行发送与接收，以及数据的串—并行转换，CPU 可以通过 APB 总线接口访问 UART 模块的数据寄存器、控制寄存器和状态寄存器。

UART 模块支持数据的缓冲发送与接收，其内部的 FIFO 存储器允许发送或接收各自最多 16 字节的缓存存储。

UART 模块的主要特性包括：

- λ 可编程的波特率发生器，可由 UART 的内部参考输入时钟产生数据发送和接收所需要的时钟信号。
- λ 与工业标准 16C550 型 UART 设备兼容；
- λ 支持 115200 或更高的波特率，与 UART 的参考时钟频率有关；

EP93XX 系列微处理器 UART 功能模块包括三个独立的 UART：UART1、UART2 和 UART3（EP9301/EP9302 不包括 UART3），其中，UART1 不仅可以完成串行数据的发送与接收，还能支持与 9 针 Modem 接口和同步/异步 HDLC 协议的全双工数据传输。

UART2 可以完成串行数据的发送与接收，但不支持 Modem 和 HDLC 接口。UART2 与红外模块配合，可以实现低速红外（SIR，Slow Infrared）接口。

UART3 可以完成串行数据的发送与接收，同时还象 UART1 一样支持 HDLC 接口，但不支持与 Modem 的接口，如果外接数据收发器，UART3 还能支持 RS485 接口通信。

关于 EP93XX 系列微处理器 UART 接口的寄存器描述及更详细的内容，可以参考相关用户手册。

2.13 红外接口

EP93XX 系列微处理器的红外接口实现红外串行通信的物理层，与 IrDA（Infrared Data Association）1.1 版本标准兼容，支持最高速度为 4MBit/s 的数据传输。通过与模拟变换器结合，红外接口可以在红外媒介与 APB 总线之间实现一个完全的接口。

红外模块包含三个独立的编码/解码单元，以实现 IrDA 标准所定义的三种不同的调制与数据编码接口，分别是：

- λ 慢速红外（SIR，Slow Infrared）接口：该接口需要与 UART2 配合，接口的控制与数据传输都需要通过 UART2 的寄存器完成，传输速率最高为 1.152MBit/s；
 - λ 中速红外（MIR，Medium Infrared）接口：该接口独立于 UART，传输速率可以为 0.576MBit/s 或 1.152MBit/s；
 - λ 快速红外（FIR，Fast Infrared）接口：该接口独立于 UART，传输速率可以为 4MBit/s；
- IrDA 慢速红外接口如图 2-7 所示。

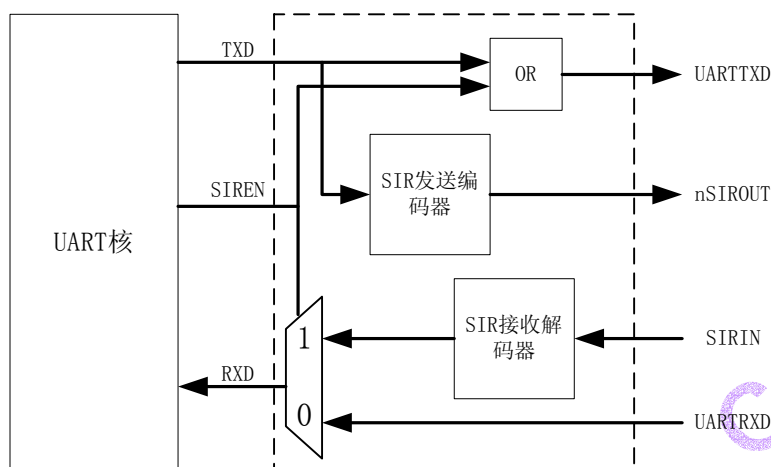


图 2-7 IrDA 慢速红外接口

关于 EP93XX 系列微处理器红外接口的寄存器描述及更详细的内容，可以参考相关用户手册。

2.14 定时器

定时器在系统中用于处理和控制在时间相关的事件，EP93XX 系列微处理器的定时器模块包含两个 16 位定时器、一个 32 位定时器和一个 40 位时间戳（Time Stamp）调试定时器。

定时器的主要特性如下：

- λ 2 个 16 位定时器，支持自由运行模式和预加载模式；
- λ 1 个 32 位定时器，支持自由运行模式和预加载模式；
- λ 1 个 40 位定时器，支持自由运行模式；

2.14.1 16 位和 32 位定时器

两个 16 位定时器记为 TC1 和 TC2，每一个定时器都有一个 16 位可读写的寄存器和控制寄存器，以及一个加载寄存器，当向加载寄存器写入一个数值时，该数值会立即被载入计数单元，并会在下一个有效的时钟沿递减。当计数单元递减到“0”时，可以触发相关中断，而定时器的时钟源和工作模式则由控制寄存器控制。

定时器的时钟源可以是 508KHz 和 2KHz，均与系统 AHB 总线时钟同步。

32 位定时器记为 TC3，有一个 32 位可读写的寄存器和控制寄存器，以及一个加载寄存器，其工作方式与 TC1 和 TC2 完全相同。16 位定时器和 32 位定时器的计数单元都能工作在两种模式：自由运行模式和预加载模式。

自由运行模式：

在自由运行模式下，当 TC1 和 TC2 的计数单元递减到 0（下溢出）时，会自动重新载入 0xFFFF，然后继续递减计数。当 TC3 的计数单元递减到 0 时，会自动重新载入 0xFFFF_FFFF，然后继续递减计数。

预加载模式：

在预加载模式下，当 TC1、TC2 和 TC3 的计数单元递减到 0 时，写入加载寄存器的值会自动重新载入计数单元，然后继续递减计数，显然，该模式可以用于产生一个可编程的周期性中断。

2.14.2 40 位定时器

时间戳调试定时器 TC4 可用于长时间记时功能，该定时器有一个 40 位递增计数单元，其时钟源为 14.7456MHz 时钟经 15 分频后所得到的 983.04KHz 的参考时钟。TC4 的当前时间值可以在任何时候通过先读取低 32 位，然后读取高 8 位的方式获得，时间值除以 983 即可得到一个毫秒时间戳。

TC4 不能触发中断，并仅由一个使能位控制其操作，当使能 TC4 时，它从 0 开始递增计数，计数到最大值 0xFF_FFFF_FFFF 后，又重新从 0 开始计数。当禁用 TC4 时，计数值被清 0。

关于 EP93XX 系列微处理器定时器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.15 看门狗定时器

EP93XX 系列微处理器的看门狗定时器（Watchdog Timer）在系统运行异常的情况下，提供了一种复位系统的机制，该功能允许看门狗在系统运行中止的情况下重新恢复系统。但为防止看门狗在系统运行正常的情况下复位系统，必须定时对看门狗定时器进行复位操作，俗称“喂狗”。

可以通过软件或硬件的方式禁用看门狗定时器。

看门狗定时器由一个 7 位计数单元构成，最高位用于触发 WATCHDOG_RESETh 输出信号，该信号进而触发系统控制模块产生 AMBA 总线复位信号 HRESETn。

看门狗定时器产生 WATCHDOG_RESETh 信号前的时间长度和输出复位脉冲的宽度如下：

- λ 时间长度和输出复位脉冲的宽度 = $64 / \text{WATCHDOG_CLK}$ 的频率，单位为秒；
- λ 对于 256Hz 的 WATCHDOG_CLK，则以上时间为 $64 / 256 = 250\text{ms}$ 。

显然，在系统正常运行的情况下，为防止看门狗定时器产生复位脉冲，需要通过软件周期性的对看门狗定时器的计数单元进行复位。可以通过向相应的看门狗寄存器写入 0x5555 复位计数单元。

当看门狗定时器引起系统复位时，它输出一个 250ms 宽度的复位脉冲，当复位脉冲输出完毕后，计数单元又开始新一轮的计数。当然，也可以通过对芯片 CSn[2]引脚下拉的方式禁止输出复位脉冲，具体可以参考前文的启动硬件配置部分。

可以通过软件向看门狗寄存器写入 0xAA55 禁用看门狗定时器，而写入 0xAAAA 则重新使能看门狗定时器。也可以通过硬件方式，即通过对芯片 CSn[1]引脚下拉，禁用看门狗定时器，具体可以参考前文的启动硬件配置部分。

关于 EP93XX 系列微处理器看门狗定时器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.16 I²S控制器

I²S总线（Inter-IC Sound Bus）是Philips提出的一种串行数字音频总线协议。

EP93XX系列微处理器的I²S控制器用于完成外部I²S编码解码器的ADC/DAC与ARM核之间的流式数据传输，其中，EP9301/EP9302 的I²S控制器由一个独立的立体声发送通道和一个独立的立体声接收通道构成，每一个通道都有一组寄存器，可以通过APB总线或DMA方式访问。而EP9307/EP9312/EP9315 的I²S控制器则由三个独立的立体声发送通道和三个独立的立体声接收通道构成，每一个通道都可以处理一路立体声音频流。同时，每一个通道都有一组寄存器，可以通过APB总线或DMA方式访问。

EP93XX系列微处理器I²S控制器的结构如图 2-8 所示。

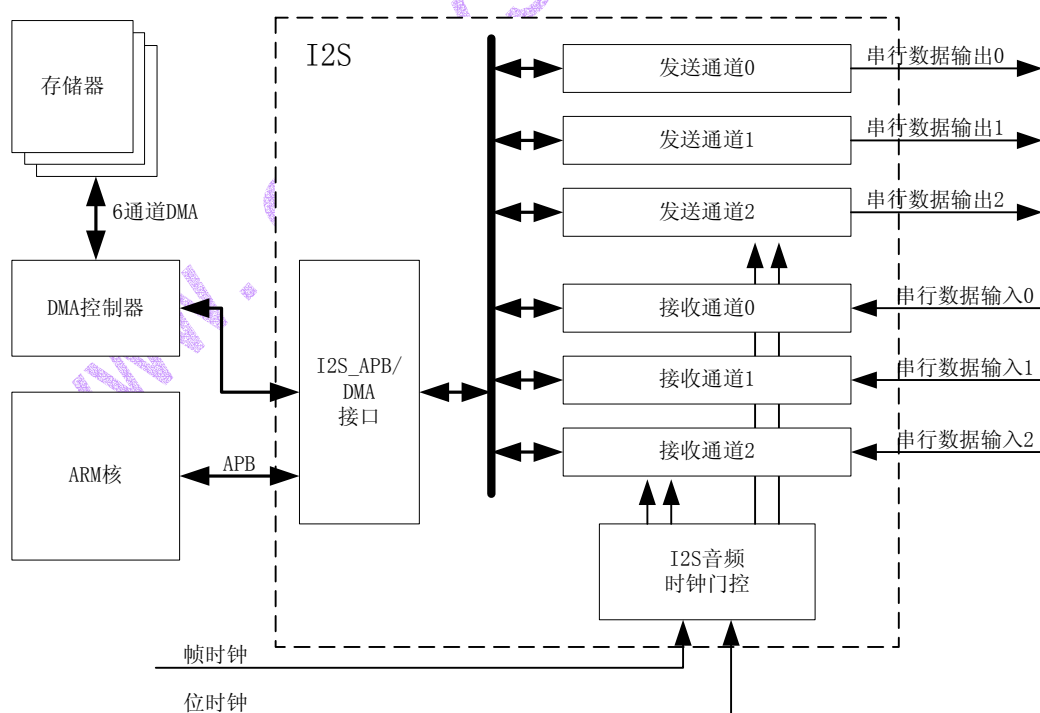


图 2-8 I²S 控制器的结构图

注意：EP9301/EP9302 仅包含一个发送通道和一个接收通道，以及 2 个 DMA 通道。

关于 EP93XX 系列微处理器 I²S 控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.17 AC'97 控制器

EP93XX 系列微处理器的 AC'97 控制器提供外部音频编解码器与 AMBA APB 之间的 5 线串行接口，支持双向、固定速率的串行脉冲编码调制（PCM，Pulse Code Modulation）数据流传输。

AC'97 控制器的主要特性如下：

- λ 完成从外部编解码器接收数据串行到并行的转换；
- λ 完成发送到外部编解码器数据并行到串行的转换；
- λ 完成数据的发送/接收控制并提供状态信息；
- λ 发送和接收的数据可以由内部 FIFO 存储器缓冲，数据可以独立存放在缓冲区中；

关于 EP93XX 系列微处理器 AC'97 控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.18 同步串行接口

EP93XX 系列微处理器的同步串行接口（SSP，Synchronous Serial Port）支持以主模式和从模式与外设的同步串行通信，包括 Motorola SPI、Semiconductor Microwire 或 TI 的同步串行接口设备。

SSP 完成与外设收发数据串行与并行的相互转换，可以通过 CPU 查询或 DMA 方式完成 SSP 的数据传输，发送和接收通道都带有内部 FIFO 存储缓冲区，最多可以独立存放 8 个 16 位的数据。SSP 的主要特性如下：

- λ 支持主模式和从模式操作；
- λ 支持可编程的位时钟；
- λ 独立的发送与接收 FIFO 存储缓冲区，最多可以存放 8 个 16 位的数据；
- λ 支持可编程的 4~16 位数据帧长度；
- λ 可分别屏蔽的发送 FIFO、接收 FIFO 和接收溢出中断；

关于 EP93XX 系列微处理器 SSP 的寄存器描述及更详细的内容，可以参考相关用户手册。

2.19 脉冲宽度调制器

EP93XX 系列微处理器（不包括 EP9301 和 EP9302）的脉冲宽度调制器（PWM, Pulse Width Modulator）支持两路独立可编程的 PWM 输出，其具体特性如下：

- λ 可配置的两路 PWM 输出；
- λ 每一路 PWM 输出都支持独立的输入时钟；
- λ 16 位分辨率；
- λ 支持可编程同步模式；
- λ 支持脉冲宽度、脉冲间隔（频率）和极性编程；

关于 EP93XX 系列微处理器脉冲宽度调制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.20 ADC 接口/触摸屏接口

EP93XX 系列微处理器均带有 12 位的 A/D 转换接口，可以完成通用的 A/D 转换功能及与电阻式触摸屏接口，其中，EP9301 和 EP9302 的 ADC 仅完成通用的 A/D 转换功能，而 EP9307、EP9312 和 EP9315 还能直接支持与电阻式模拟触摸屏的连接。

EP9301、EP9302 的 ADC 模块由一个 12 位的 A/D 转换器和一个多路模拟输入开关构成，支持 5 路模拟电压输入，输入电压范围为 0~3.3V。

EP9307、EP9312 和 EP9315 的触摸屏接口是一个支持 4 线、5 线、7 线和 8 线电阻式模拟触摸屏的硬件引擎，能完成采样、求均值及对 A/D 转换结果的范围检测。硬件引擎同时控制一个模拟开关阵列，以切换不同的扫描模式。

也可以禁用触摸屏的硬件引擎，从而由 CPU 直接控制开关阵列和 A/D 转换器，采用这种方式可以在需要的情况下实现更复杂的软件扫描算法，但硬件引擎能够在无需 CPU 干预的情况下，实现标准接口的所有必要的特性。

触摸屏控制引擎可以在 X 轴和 Y 轴实现如下功能：

- λ 支持对采样点的 4、8、16 或 32 次采样；
- λ 检测采样值是否有较大的偏离；
- λ 对采样值平均；
- λ 检测采样均值是否与实际情况偏离太大；
- λ 检测触摸位置的变化是否足够大，以决定是否需要通知 CPU。

很多触摸屏控制器在触摸屏处于工作状态时，总是持续的将数据发送给 CPU，而 EP93XX 系列微处理器的触摸屏控制器只在检测到发生有意义的变化时，才会中断 CPU，显然，采用这种方式可以大大提高系统效率。

电阻式模拟触摸屏的工作原理如下：触摸屏通常以一层玻璃或硬塑料平板作为基层，表面涂有一层透明氧化金属（透明的导电电阻）导电层，上面再盖有一层外表面硬化处理、光滑

防擦的塑料层，它的内表面也有一层导电涂层，在它们之间有许多细小的（通常小于 1/1000 英寸）透明隔离点把两层导电层隔开绝缘。

当触摸屏幕时，两层导电层在触摸点位置就有了接触，电阻发生变化，在 X 轴和 Y 轴两个方向上产生信号，然后传送触摸屏控制器。触摸屏控制器检测到这一接触并计算出 (X, Y) 的位置，同时作出相应的处理。

关于 EP93XX 系列微处理器触摸屏控制器与触摸屏的连接方法，寄存器描述及更详细的内容，可以参考相关用户手册。

2.21 键盘接口

EP93XX 系列微处理器（不包括 EP9301、EP9302）的键盘接口支持 8 行 8 列多达 64 个常开、单触点按键，用户可以外接使用部分或全部按键。键盘接口主要特性如下：

- λ 最多可以支持 8×8 的常开、单触点按键；
- λ 后台驱动模式可以将电容效应减至最小。
- λ 采用三次连续扫描方式确定按键；
- λ 按键最大机械抖动时间为 20ms；
- λ 典型按键中断时间间隔为 24~40ms；
- λ 支持低功耗唤醒模式；
- λ 支持三键复位功能；

当系统未使用键盘接口时，所有的键盘接口引脚都可用作 GPIO。

关于 EP93XX 系列微处理器键盘接口的寄存器描述及更详细的内容，可以参考相关用户手册。

2.22 IDE 接口

EP9312/EP9315 的 IDE 接口提供了一个与 ATA/ATAPI（Advanced Technology Attachment/Advanced Technology Attachment Packet Interface）兼容设备的工业标准接口。该接口支持：

- λ PIO 模式 4；
- λ Multiword DMA 模式 2；
- λ Ultra DMA 模式 4；

IDE 接口模块使用内部的 DMA 控制器，完成 Multiword DMA 模式和 Ultra DMA 模式的数据传输，数据宽度为 16 位。

关于 EP93XX 系列微处理器 IDE 接口的寄存器描述及更详细的内容，可以参考相关用户手册。

2.23 Raster/LCD 控制器

EP93XX 系列微处理器（不包括 EP9301、EP9302）的 Raster/LCD 控制器可以为多种显示器提供数据信号与控制时序。独立的 DAC 接口可以产生 RGB 模拟信号，支持与模拟 LCD 和 CRT 显示器的接口。控制器同时还可以产生 CCIR656 4: 2: 2 YCrCb 数字视频输出信号，支持与 NTSC 编码器的接口。

Raster/LCD 控制器可以支持的显示类型如表 2-4 所示。

表 2-4 Raster/LCD 控制器可以支持的显示类型

显示类型	分辨率	视频时钟频率 (MHz)	帧缓冲区存储格式	显示数据格式	像素/每个移位时钟	像素移位时钟频率 (MHz)	场扫描频率 (Hz)	备 注
VFD	128×32	2	4bpp	单色	8	0.25	400	
LCD	128×64	2	4bpp	单色	4	0.5	230	并行命令字接口
LCD	256×128	2	4bpp	单色	4	0.5	60	
QVGA TFT LCD	320×234	6.4	8bpp	模拟	1	6.4	80	
QVGA STN LCD	320×240	4	4 位 RGB	4 位 RGB	1	4	50	
HVGA STN LCD	640×240	8	4 位 RGB	4 位 RGB	1	8	50	
VGA DC Plasma	640×400	16	4bpp	单色	4	4	60	
VGA EL	640×480	24	4/8 bpp	灰度级	8	3	75	
VGA STN LCD	640×480	24	8/16 bpp	18 位 RGB	1	24	75	
VGA TFT LCD	640×480	24	8/16/24 bpp	18 位 RGB	1	24	75	
VGA CRT	640×480	32	8/16/24 bpp	模拟	1	NA	85	外部 DAC
SVGA TFT LCD	800×600	40	8/16/24 bpp	18 位 RGB	1	40	80	
SVGA CRT	800×600	50	8/16/24 bpp	模拟	1	NA	85	外部 DAC
XGA CRT	1024×768	75	8/16/24 bpp	模拟	1	NA	80	外部 DAC
SXGA TFT LCD	1280×1024	85	8/16/24 bpp	18/24 位 RGB	1	85	60	24 位
SXGA CRT	1280×1024	110	8/16/24 bpp	模拟	1	NA	70	外部 DAC
HDTV-2 LCD	1280×720	50	8/16/24 bpp	24 位 RGB	1	50	50	24 位

HDTV-2 CRT	1280×720	66	8/16/24 bpp	模拟	1	NA	60	外部 DAC
---------------	----------	----	-------------	----	---	----	----	--------

关于 EP93XX 系列微处理器 Raster/LCD 控制器的寄存器描述及更详细的内容，可以参考相关用户手册。

2.24 图像加速器

EP93XX 系列微处理器的 EP9307 和 EP9315 内嵌硬件图像加速引擎，可以通过完成块拷贝、块填充和硬件画线等操作，增强系统的图像处理能力，图像加速引擎独立工作，不占用 CPU 资源。

关于 EP93XX 系列微处理器图像加速引擎的寄存器描述及更详细的内容，可以参考相关用户手册。

2.25 PCMCIA 接口

EP9315 内嵌单通道的 PCMCIA 接口，支持与 8 位/16 位的 PCMCIA 设备连接。

关于 EP9315 的 PCMCIA 接口的寄存器描述及更详细的内容，可以参考相关用户手册。

2.26 GPIO 接口

EP93XX 系列微处理器的 GPIO 可以分为两种类型：标准 GPIO 和增强型 GPIO，其中增强型 GPIO（EGPIO，Enhanced GPIO）具有产生中断请求的能力。

EP93XX 系列微处理器的 GPIO 模块分为七个端口，分别为端口 A、端口 B、端口 C、端口 E、端口 F、端口 G 和端口 H，其中端口 C、E、G 和 H 为标准 GPIO，端口 A、B 和 F 为增强型 GPIO。

每一个 GPIO 端口都有一个 8 位的数据寄存器和一个 8 位的数据方向寄存器，可以控制最多 8 个独立的引脚。EGPIO 端口还要多一个 8 位的寄存器以控制中断产生。端口的 8 位寄存器都可按位操作，以便对某一个引脚进行单独控制，而其它的引脚不受影响。

关于 EP93XX 系列微处理器 GPIO 模块的寄存器描述及更详细的内容，可以参考相关用户手册。

2.27 安全特性

EP93XX 系列微处理器最显著的特点之一是其独有的安全特性，它采用了多种手段实现网络安全与数字版权保护，以及电子商务的安全。

EP93XX 系列微处理器的主要安全特性如下：

- λ 256 位激光蚀刻 ID，永久支持系统 ID 及密码；
- λ 当芯片处于“锁定”状态时，用于安全特性的启动固件和密码均“不可见”；
- λ 每一个不同的系统的软件，均可以采用独立编码的方式和专用 ID 进行保护；
- λ 可在同一个系统中将多种安全手段配合使用；
- λ 当使能安全特性时，JTAG 功能不可用；
- λ 当使能安全特性时，不支持外部启动模式。

关于 EP93XX 系列微处理器安全特性更详细的内容，可以参考相关用户手册。

2.28 本章小结

本章对 EP93XX 系列嵌入式微处理器片内资源的基本特性及功能进行了简单的描述，希望读者通过对本章的阅读，能对 EP93XX 系列嵌入式微处理器的片内资源有一个比较全面的了解和掌握，但在此不需要过于深入每一个功能模块的具体细节。

由于 EP93XX 系列微处理器每款芯片的应用领域定位各不相同，其片内资源的配置也不尽相同，希望读者在阅读本章后，能够根据自身系统的特定需求，选择出一款合适的嵌入式微处理器。

如果读者有兴趣对 EP93XX 系列微处理的各个功能模块进行详细的了解，可以参考相关用户手册。

第 3 章 基于 EP93XX 系列微处理器的系统设计

本章介绍基于 EP93XX 系列嵌入式微处理器硬件系统设计步骤,实现细节和系统调试方法,通过对本章的阅读和理解,绝大多数的读者应该能够根据自身的需求,设计出特定的应用系统。

本章主要内容:

- 基于 EP9301/EP9302 的系统设计;
- 基于 EP9307 的系统设计;
- 基于 EP9312/EP9315 的系统设计;
- 硬件系统的调试方法。

3.1 嵌入式系统设计的一般原则

嵌入式系统是基于特定应用的专用计算机系统,这就决定了嵌入式系统最大的特点是差异性和专用性,即应用于不同领域的嵌入式设备、产品,无论是从外观、功能、成本和体积上看,都有着很大的差别,同时,嵌入式设备和产品在功能上决不追求大而全,相反,由于受到成本、功耗、体积等因素的限制,绝大多数的嵌入式设备和产品是“够用就好”。

本章以中嵌科技有限公司(www.emsyschina.com)设计生产的基于 EP93XX 系列微处理器的开发板系统(HDB9301/02、HDB9307、HDB9312/15)为原型,详细分析系统的硬件设计步骤、实现细节等。

3.2 基于 EP9301/EP9302 的硬件系统设计

EP9301/EP9302 是最具价格优势的 ARM9 处理器之一,其高度集成的结构和先进的性能设计尤其适合各种消费类电子产品和工业控制应用。

由于 EP9301 和 EP9302 引脚完全兼容,因此其硬件系统的设计完全一致。图 3-1 为 HDB9301/02 开发系统的硬件结构框图。

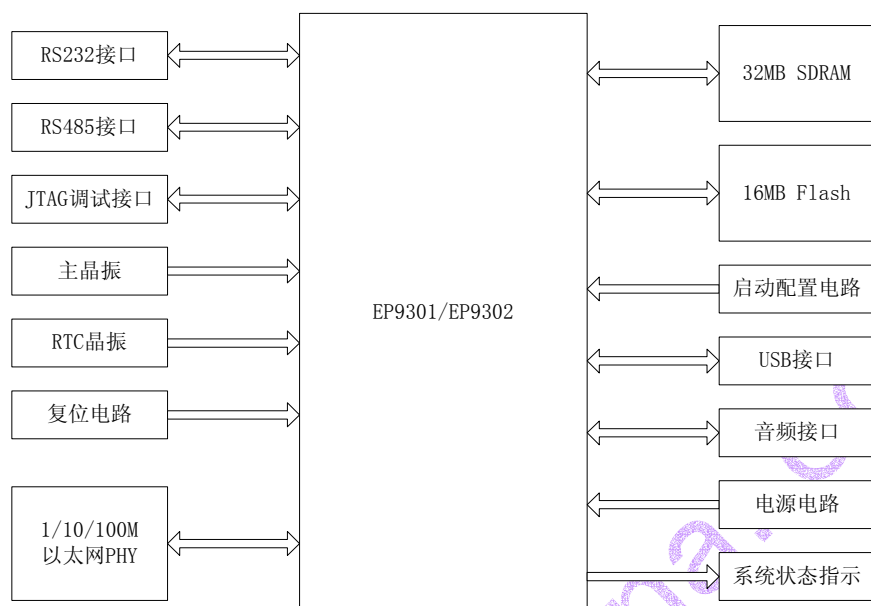


图 3-1 HDB9301/02 开发系统硬件结构框图

各部分基本功能描述如下：

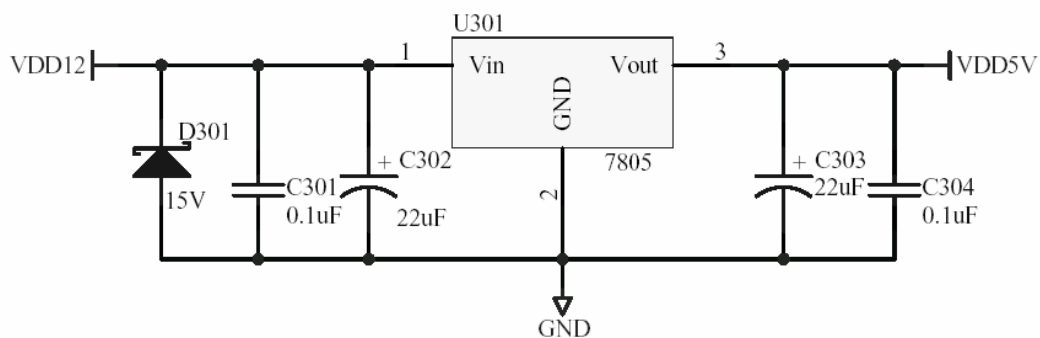
- λ RS232 接口电路用于与其他应用系统的 RS232 通讯；
- λ RS485 接口电路用于与其他应用系统的 RS485 通讯；
- λ 复位电路可完成系统上电复位和在系统工作时用户按键复位；
- λ 电源电路为 5V 到 3.3V 和 5V 到 1.8V 的转换器，用于给芯片内核及外围电路的供电；
- λ JTAG 调试接口可对芯片内部的所有部件进行访问，通过该接口可对系统进行软、硬件调试、编程等；
- λ 主晶振为系统提供工作时钟，通过片内 PLL 电路倍频为微处理器核和其它功能部件的工作时钟；
- λ RTC 晶振专为片内的 RTC 电路提供工作时钟；
- λ Flash 存储器可用于存放用户应用程序、嵌入式操作系统及其他在系统掉电后需要保存的用户数据等；
- λ SDRAM 存储器为系统程序的运行空间，系统及用户数据、堆栈均位于 SDRAM 存储器中；
- λ 1/10/100M 以太网物理层接口为系统提供以太网接入的物理通道，通过该接口，系统可以 1/10/100Mbps 的速率接入以太网；
- λ USB 接口支持与 USB 从设备的数据通信；
- λ 音频接口提供系统的音频输入/输出功能；
- λ 系统状态指示电路由芯片引脚外接的两个 LED 组成，用于指示系统的工作状态。

3.2.1 电源电路

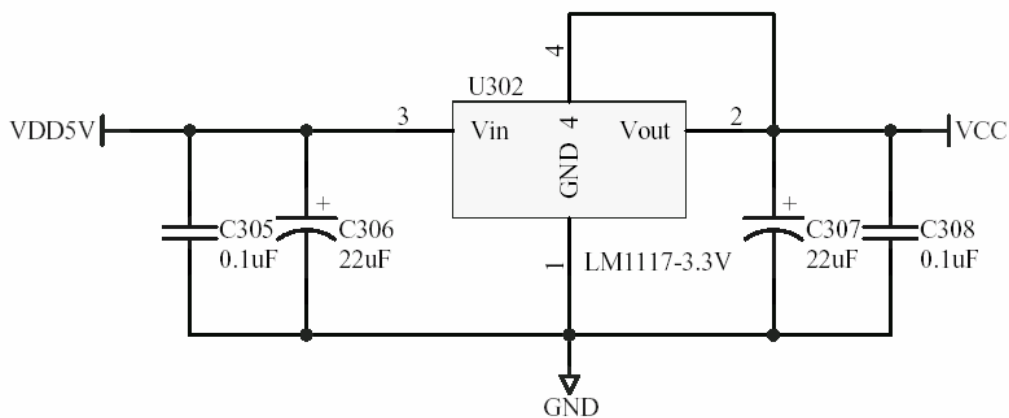
EP93XX 系列微处理器的内核工作电压为 DC 1.8V，但其片内功能模块的工作电压为 DC 3.3V，HDB9301/02 开发系统采用 DC 12V 电源，因此，需要完成 DC12V 到 DC 5V、DC 3.3V 和 DC 1.8V 的电源转换。

有很多型号的电压转换器可完成 DC12V 到 DC 5V、DC 3.3V 和 DC 1.8V 的转换，在此选用 78 系列和 LM117 系列转换器，完成电压的转换。

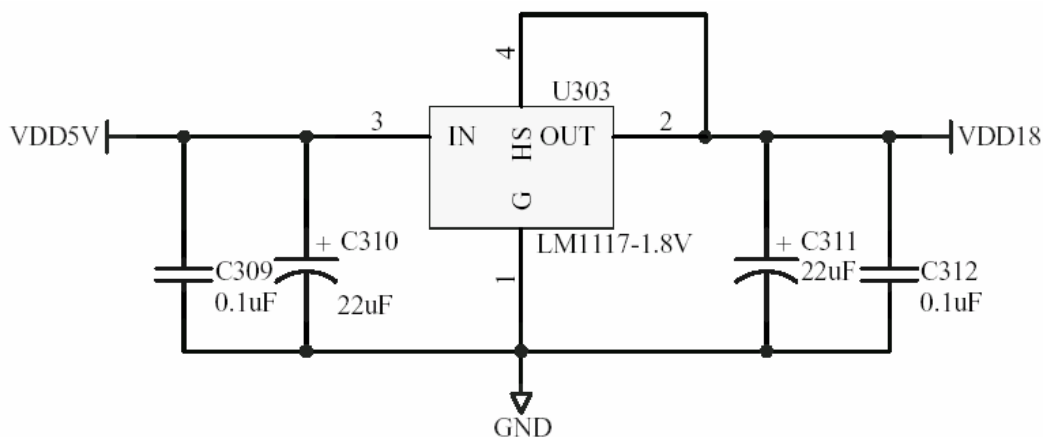
系统电源电路如图 3—2 所示。



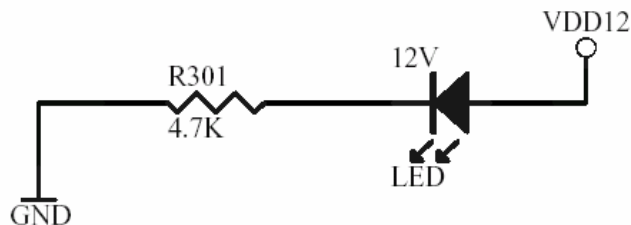
a DC12V 转 DC5V 电路图



b DC5V 转 DC3.3V 电路图



c DC5V 转 DC1.8V 电路图



d DC12V 电源指示

图 3-2 HDB9301/02 开发系统电源电路图

在图 3-2a 中, D301 为 15V 稳压二极管, 用于钳位输入电源电压, 即当输入电源电压高于 DC 15V 时, 电压被 D301 钳位在 15V, 可以防止因输入电压过高对系统造成损坏。

该电路可以在基于 EP93XX 系列微处理器的系统中通用。

3.2.2 晶振电路

EP93XX 系列微处理器都需要两个无源晶振作为系统的工作时钟, 其中主晶振频率为 14.7456MHz, 经芯片内部 PLL 电路倍频后作为 CPU 及总线的工作时钟和片内功能模块的工作时钟。片内的 PLL 电路兼有频率放大和信号提纯的功能, 因此, 系统可以以较低的外部时钟信号获得较高的工作频率, 以降低因高速开关时钟所造成的高频噪声, 这是基于 ARM 微处理器系统常采用的一种手段。

另一个 32.768KHz 的晶振为片内 RTC 电路提供工作时钟。具体可参考图 2-3 和图 2-4。

晶振电路如图 3-3 所示, 该电路可以在基于 EP93XX 系列微处理器的系统中通用。

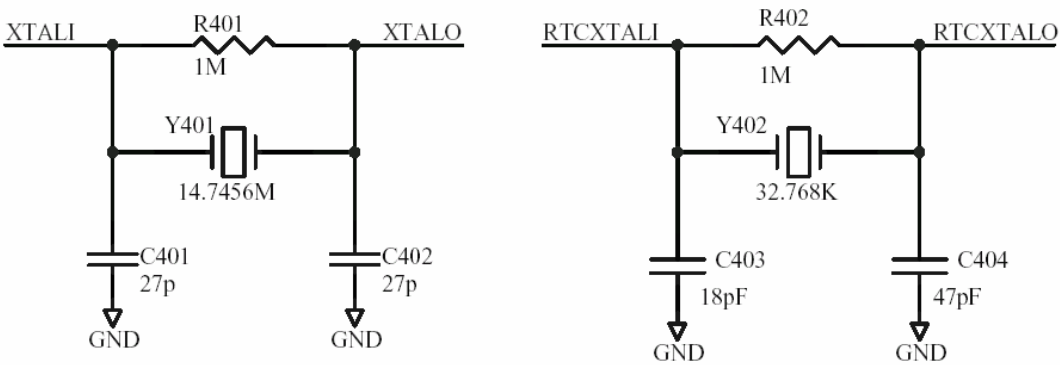


图 3-3 HDB9301/02 开发系统晶振电路图

3.2.3 复位电路

EP93XX 系列微处理器均有两个复位引脚：PRSTn 和 RSTOn，其中，PRSTn 称为上电复位引脚，当该引脚出现低电平时，会对 EP93XX 芯片的所有部件复位。RSTn 称为用户复位引脚，当该引脚出现低电平时，会对 EP93XX 芯片的绝大多数部件复位，但 RTC、SDRAM 刷新控制等不受影响。

为保证系统的可靠复位，设计选用了 MAX708 作为系统的复位控制，MAX708 是一种微处理器电源监控芯片，可同时输出高电平有效和低电平有效的复位信号，复位信号可由电源电压、手动复位输入或由独立的比较器触发。

MAX708 SO8 封装方式的引脚定义如图 3-4，引脚具体说明为表 3-1。

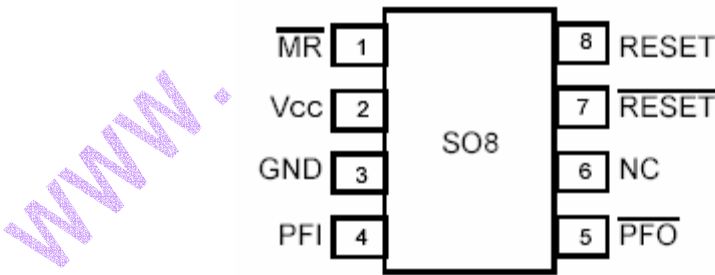


图 3-4 MAX708 的引脚定义图

表 3-1 MAX708 的引脚说明

引 脚	功能描述
/MR	手动复位输入，低电平有效。当/MR 拉至 0.6V 时触发一个复位脉冲，未用时接至 VCC。
VCC	电源输入。

GND	接地。
PFI	电源失效比较器输入，未用时接 GND。
/PFO	电源失效比较器输出，低电平有效。当引脚 PFI 上的电压低于 1.25V 时，/PFO 输出低电平，拉电流。未用时悬空。
NC	内部未连接。
/RESET	低有效复位输出。当 VCC 低于域值电平或/MR 保持为低时，/RESET 输出低电平，在复位条件结束后 /RESET 信号将继续保持 200ms。
RESET	高有效复位输出。当 VCC 低于域值电平或/MR 保持为低时，RESET 输出高电平，在复位条件结束后 RESET 信号将继续保持 200ms。

系统复位电路如图 3-5 所示，该电路可以在基于 EP93XX 系列微处理器的系统中通用。

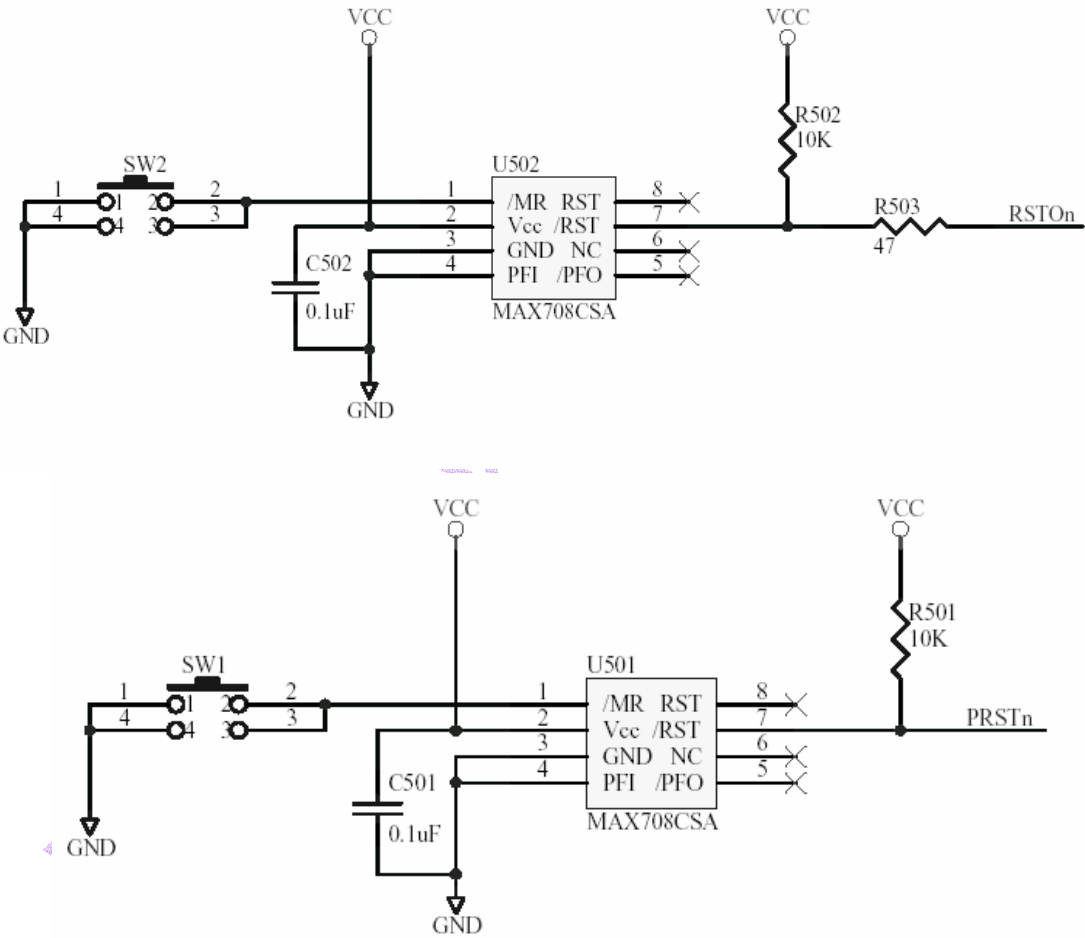


图 3-5 HDB9301/02 开发系统复位电路图

3.2.4 Flash 存储器接口电路

Flash 存储器是一种在系统（In-System）可编程器件，存储的信息在系统掉电后不会丢失。它具有低功耗、大容量、擦写速度快、可整片或分扇区编程（烧写）、擦除等特点，并且可由内部嵌入的算法完成对芯片的操作，因而在各种嵌入式系统中得到了广泛的应用。

作为一种非易失性存储器，Flash 在系统中通常用于存放程序代码、常量表以及一些在系统掉电后需要保存的用户数据等。常用的 Flash 为 8 位或 16 位的数据宽度，编程电压为单 3.3V。主要的生产厂商为 Intel、ATMEL、AMD 等，他们生产的同型器件一般具有相同的电气特性和封装形式，可通用。

本系统中使用的 Flash 存储器为 Intel 生产的 E28F128 J3，其基本特性如下：

E28F128J3 是 Intel 基于 NOR Flash 技术的 3V StrataFlash 高密度 Flash 存储器系列中的一款，支持 8 位/16 位操作模式，单片容量为 16MB（128M 位），按 128 个可擦除的存储块组织，每个存储块的容量为 128KB。

StrataFlash 高密度 Flash 存储器具有存储密度高、读写速度快、价格便宜等优点，可用于代码与数据存储，并可方便升级。

E28F128J3 的引脚分布及信号描述分别如图 3—6 表 3—2 所示。

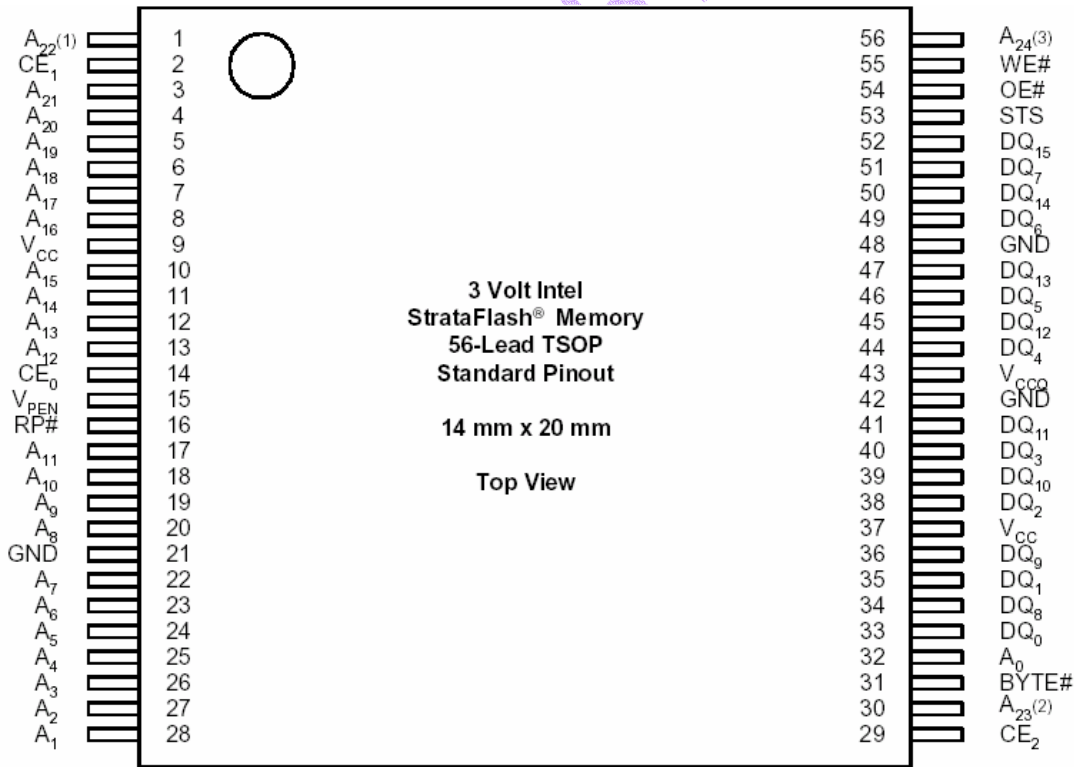


图 3—6 E28F128J3 的引脚分布

表 3-2 E28F128J3 的引脚说明

引 脚	类型	名称与功能描述
A0	输入	字节选择地址：当存储器用作 8 位模式时，该引脚用于选择高字节或低字节，当存储器用作 16 位模式时，该引脚不用。
A[23: 1]	输入	地址总线：在存储器进行读、写操作时作为地址信号。
D[7: 0]	双向	低字节数据总线：在存储器进行读、写操作时作为低 8 位数据总线。
D[15: 8]	双向	高字节数据总线：在存储器进行读、写操作时作为高 8 位数据总线。
CE0, CE1, CE2	输入	芯片使能：该引脚用于控制芯片的工作状态，具体功能参考表 3-3。
RP #	输入	复位/掉电：芯片复位与掉电模式控制引脚；当引脚为高电平时，芯片为正常操作模式，当引脚为低电平时，芯片复位。
OE #	输入	输出使能：低电平有效。
WE #	输入	写使能：低电平有效。
STS	漏极开路输出	状态：用于指示存储器内部状态机状态。当配置为电平模式（默认模式）时，该引脚为 Ready/Busy # 信号；当配置为脉冲模式时，可用于指示编程或擦除完毕。STS 为漏极开路输出引脚，需要用电阻上拉到 VCC。
BYTE #	输入	字节使能：当该引脚为低电平时，存储器工作在 8 位模式，所有的数据均通过 D[7: 0]输入/输出，数据总线 D[15: 8]浮空，此时 A0 用于高低字节选择。当该引脚为高电平时，存储器工作在 16 位模式。
VPEN	输入	擦除/编程/块锁定使能：当该引脚为低电平时，存储器内容不能被改变。
VCC		接电源。
VCCQ		接电源。
GND		接数字地。
NC		未连接。
RFU		系统保留，未使用。

E28F128J3 的芯片使能控制如表 3-3 所示。

表 3-3 E28F128J3 芯片使能控制真值表

模式	CE2	CE1	CE0	工作状态
1	0	0	0	使能
2	0	0	1	禁用
3	0	1	0	禁用
4	0	1	1	禁用
5	1	0	0	使能
6	1	0	1	使能
7	1	1	0	使能
8	1	1	1	禁用

注意：如果系统中只使用一片 Flash 存储器，可以直接将 CE1 和 CE2 接地，仅由 CE0 控制芯片工作状态。

关于 E28F128J3 更具体的内容可以参考相关用户手册，其它类型 Flash 存储器的特性与使用方式与之类似，用户可以根据自己的实际需要选择不同的器件。

以下介绍 EP9301/EP9302 与 E28F128J3 的接口：

ARM 微处理器的体系结构支持 8/16/32 位数据宽度的读、写，对应的可以构建 8/16/32 位的 Flash 存储器系统。32 位的存储器系统具有较高的性能，而 16 位的存储器系统则在成本及功耗方面占有优势，8 位的存储器系统现在已经很少使用。

但由于 EP9301/EP9302 仅引出了低 16 位数据总线，因此只能与支持 8 位或 16 位数据宽度的存储器接口。

图 3-7 为 EP9301/EP9302 与 E28F128J3 的接口电路图，与其它类型 Flash 存储器的接口方法与之类似。

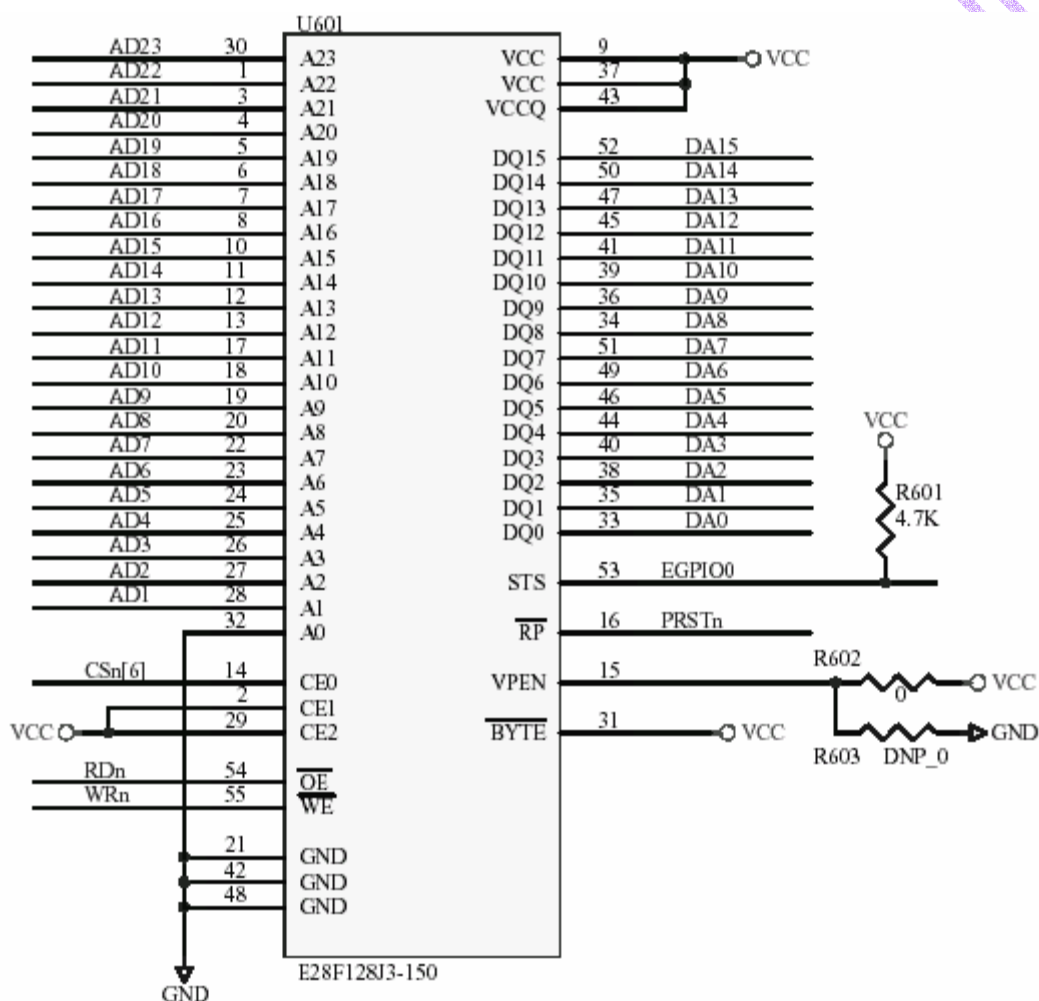


图 3-7 EP9301/EP9302 与 E28F128J3 的接口电路图

在图 3-7 中，E28F128J3 工作在 16 位模式，因此地址总线 A0 接地，A23~A1 与 EP9301/EP9302 的地址总线 A23~A1 相连，同时 BYTE# 引脚需要接高电平。

E28F128J3 的数据总线 D15~D0 与 EP9301/EP9302 的数据总线 D15~D0 相连；

E28F128J3 的 OE# 端与 EP9301/EP9302 的读信号 RDn 相连;

E28F128J3 的 WE# 端与 EP9301/EP9302 的写信号 WRn 相连;

E28F128J3 的 RP# 端与 EP9301/EP9302 的上电复位信号 PRSTn 相连;

E28F128J3 的 STS 端与 EP9301/EP9302 的 EGPIIO 相连, 这样 EP9301/EP9302 就可以以查询或中断方式获得 E28F128J3 的工作状态。注意 STS 为漏极开路输出, 需要用电阻 R601 上拉;

E28F128J3 的 VPEN 端为硬件锁定引脚, 在正常操作状态下, 该引脚必须为高电平, 此时需要焊接电阻 R602, 但 R603 不能焊接; 当需要锁定 E28F128J3 的内容时, 可以将 R603 焊接, VPEN 置为低电平状态, 但此时 R602 不能焊接。

E28F128J3 的芯片使能控制端 CE2、CE1 和 CE0 的接法采用了表 3-3 中的第七种模式, CE2 和 CE1 直接接高电平, CE0 与 EP9301/EP9302 的 CSn[6] 引脚相连。

VCC 为 DC 3.3V, GND 接系统数字地。

3.2.5 SDRAM 接口电路

与 Flash 存储器相比较, SDRAM 不具有掉电保持数据的特性, 但其存取速度大大高于 Flash 存储器, 且具有读、写属性, 因此, SDRAM 在系统中主要用作程序的运行空间, 数据及堆栈区。当系统启动时, CPU 首先从复位地址 0x0 处读取启动代码, 在完成系统的初始化后, 程序代码一般应调入 SDRAM 中运行, 以提高系统的运行速度, 同时, 系统及用户堆栈、运行数据也都放在 SDRAM 中。

SDRAM 具有单位空间存储容量大、价格便宜的优点, 已广泛应用在各种嵌入式系统中。SDRAM 的存储单元可以理解为一个电容, 总是倾向于放电, 为避免数据丢失, 必须定时刷新(充电)。因此, 要在系统中使用 SDRAM, 就要求微处理器具有刷新控制逻辑, 或在系统中另外加入刷新控制逻辑电路。EP93XX 系列微处理器片内都具有独立的 SDRAM 刷新控制逻辑, 可方便地与 SDRAM 接口。

目前常用的 SDRAM 为 16 位的数据宽度, 工作电压一般为 3.3V。主要的生产厂商为 SAMSUNG、HYUNDAI、ISSI 等。他们生产的同型器件一般具有相同的电气特性和封装形式, 可通用。

HDB9301/02 开发系统采用 SAMSUNG 的 K4S561632 作为 SDRAM, 其基本特性如下。

K4S561632 基于 SAMSUNG 的高性能 CMOS 技术, 按 $4M \times 16$ 位 $\times 4$ 组方式组织, 单片存储容量为 32MB, 工作电压为 3.3V, 常见封装为 54 脚 TSOP, 兼容 LVTTTL 接口, 支持自动刷新(Auto-Refresh)和自刷新(Self-Refresh)功能, 16 位数据宽度。

K4S561632 的引脚分布及信号描述分别如图 3-8 和表 3-4 所示。

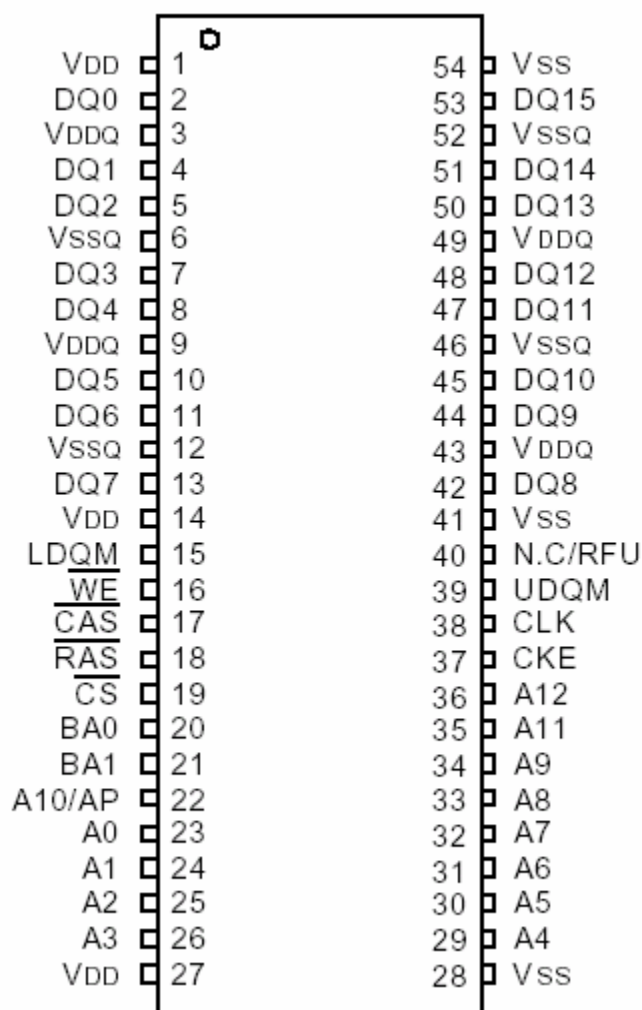


图 3—8 K4S561632 引脚分布

表 3—4 K4S561632 引脚信号描述

引 脚	名 称	描 述
CLK	系统时钟	芯片时钟输入。
CKE	时钟使能	片内时钟信号控制。
/CS	片选	禁止或使能除 CLK、CKE 和 DQM 外的所有输入信号。
BA0, BA1	组地址选择	用于片内 4 个组的选择。
A12~A0	地址总线	行地址: A12~A0, 列地址: A9~A0。
/RAS, /CAS, /WE	行地址锁存 列地址锁存 写使能	行地址锁存、列地址锁存和芯片写操作使能。
LDQM, UDQM	数据 I/O 屏蔽	在读模式下控制输出缓冲; 在写模式下屏蔽输入数据
DQ15~DQ0	数据总线	数据输入输出引脚

VDD/VSS	电源/地	内部电路及输入缓冲电源/地
VDDQ/VSSQ	电源/地	输出缓冲电源/地
NC/RFU	未连接	未连接

关于 K4S561632 更具体的内容可参考相关用户手册。其他类型 SDRAM 的特性与使用方法与之类似，用户可根据自己的实际需要选择不同的器件。

图 3-9 为 EP9301/EP9302 与 K4S561632 的接口电路图。

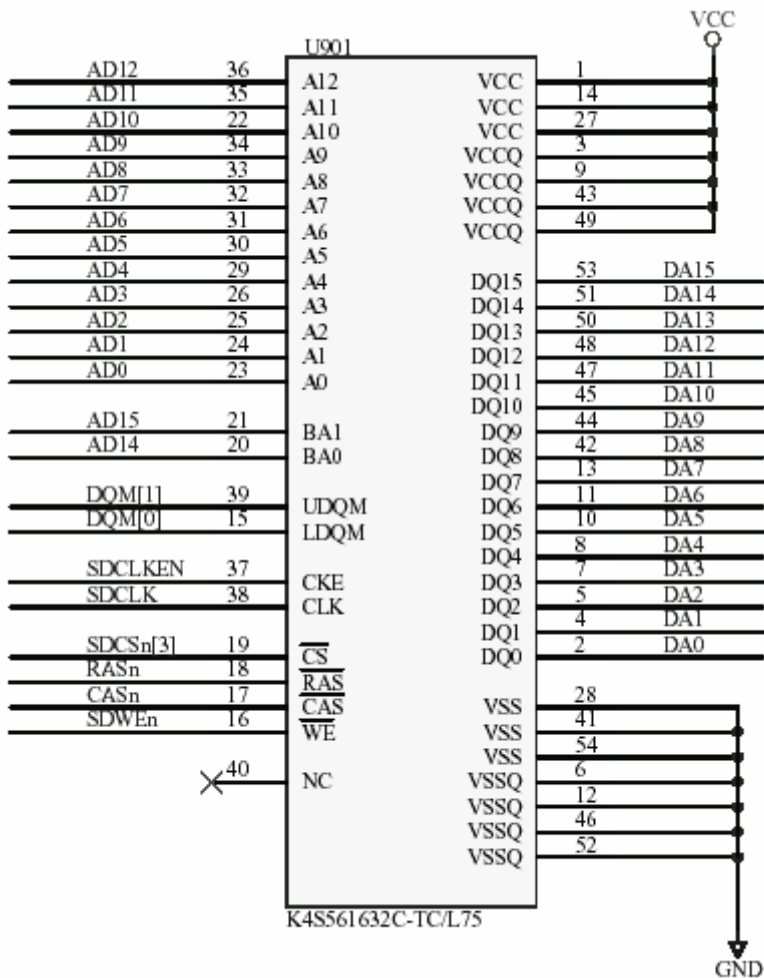


图 3-9 EP9301/EP9302 与 K4S561632 的接口电路图

与 Flash 存储器相比，SDRAM 的控制信号较多，其连接电路也相对复杂。K4S561632 的数据总线 DQ15~DQ0 与 EP9301/EP9302 的数据总线 DA15~DA0 相连；K4S561632 的地址总线 A12~A0 与 EP9301/EP9302 的地址总线 AD12~AD0 相连；K4S561632 的其它引脚分别与 EP9301/EP9302 的对应引脚相连。

3.2.6 启动配置电路

如前文所述，EP93XX 系列微处理器可以通过硬件配置的方式完成灵活的系统启动功能，EP9301 和 EP9302 的硬件启动配置由芯片的 EECLK、EEDAT、BOOT[1: 0]、ASDO 和 CSn[7: 6]引脚的上电初始化状态决定，如图 3-10 所示。

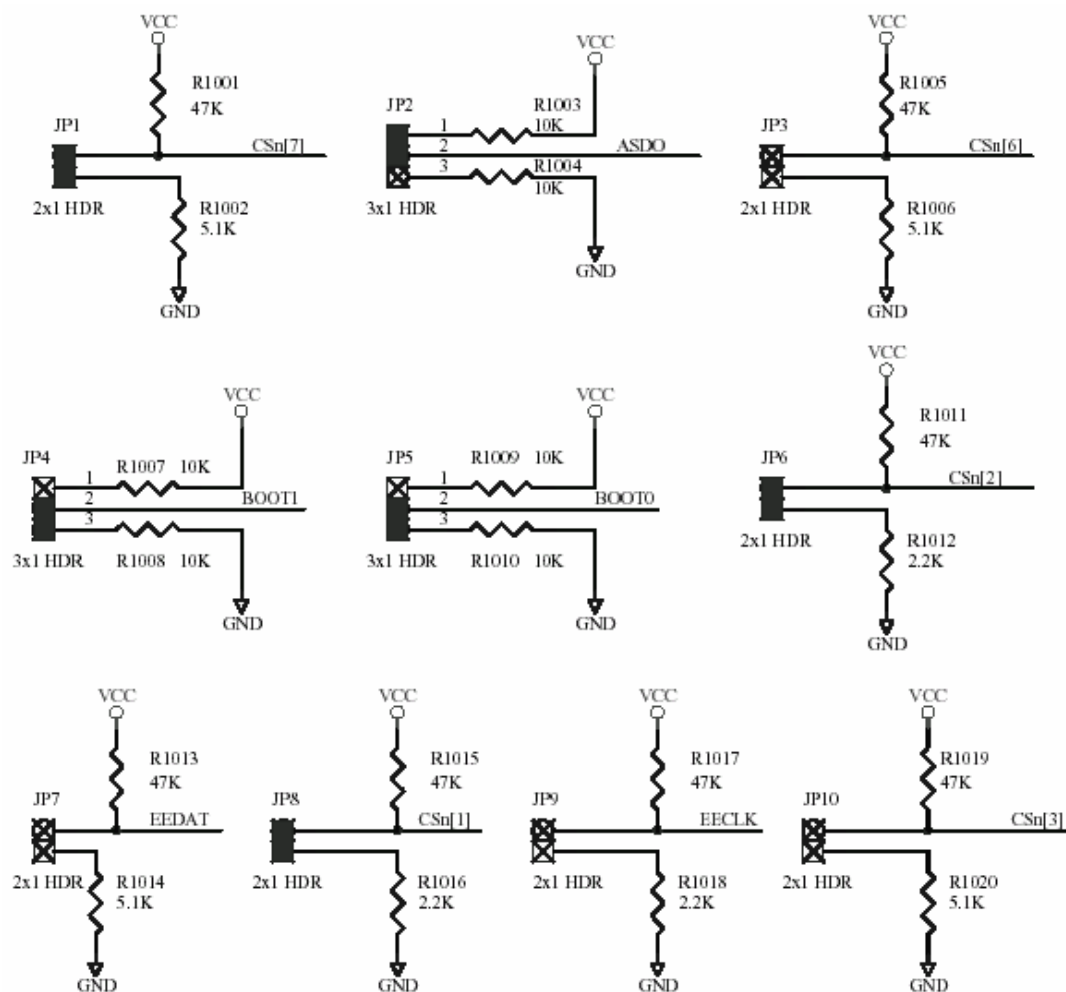


图 3-10 HDB9301/02 开发系统的启动配置

在图 3-10 的启动配置中，各引脚的配置含义可以参考第二章的启动硬件配置部分的相关内容。

图中的阴影部分表示默认的硬件配置状态。

注意图中的 BOOT0 引脚，它的配置决定系统执行串行下载方式或从 Flash 存储器运行代码，我们在进行系统调试时会经常改变该引脚的状态。

3.2.7 RS232 接口电路

EP93XX 系列微处理器带有 2~3 个 UART 接口,可以通过 TTL 到 RS232 的电平转换,实现 RS232 接口。

几乎所有的微控制器、PC 都提供 RS232 接口,使用电子工业协会(EIA)推荐的 RS-232-C 标准,这是一种很常用的串行数据传输总线标准。早期它被应用于计算机和终端通过电话线和 MODEM 进行远距离的数据传输,随着微型计算机和微控制器的发展,不仅远距离,近距离也采用该通信方式。在近距离通信系统中,不再使用电话线和 MODEM,而直接进行端到端的连接。

RS-232-C 标准采用的接口是 9 芯或 25 芯的 D 型插头,以常用的 9 芯 D 型插头为例,引脚定义如表 3-5 所示:

表 3-5 9 芯 D 型插头引脚信号描述

引 脚	名 称	功 能 描 述
1	DCD	数据载波检测
2	RXD	数据接收
3	TXD	数据发送
4	DTR	数据终端准备好
5	GND	地
6	DSR	数据设备准备好
7	RTS	请求发送
8	CTS	清除发送
9	RI	振铃指示

如果仅完成最基本的串行通信功能,只需要 RXD、TXD 和 GND 即可,但由于 RS-232-C 标准所定义的高、低电平信号与 EP93XX 系列微处理器的 LVTTTL 电路所定义的高、低电平信号完全不同,LVTTTL 的标准逻辑“1”对应 2V~3.3V 电平,标准逻辑“0”对应 0V~0.4V 电平,而 RS-232-C 标准采用负逻辑方式,标准逻辑“1”对应-5V~-15V 电平,标准逻辑“0”对应+5V~+15V 电平,显然,两者间要进行通信必须经过信号电平的转换,目前常使用的电平转换电路为 MAX232,其引脚分布如图 3-11 所示。

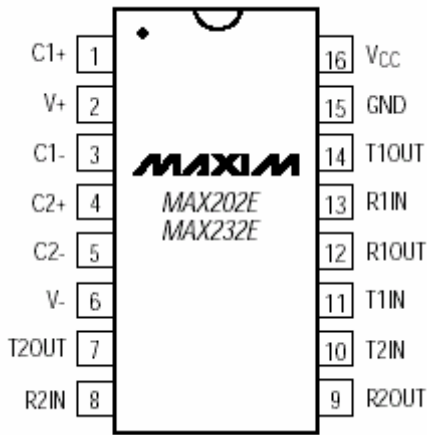


图 3—11 MAX232 引脚分布

关于 MAX232 更具体的内容可参考相关用户手册。

HDB9301/02 开发系统的 RS232 接口电路如图 3—12 所示,该电路可以在基于 EP93XX 系列微处理器的系统中通用。

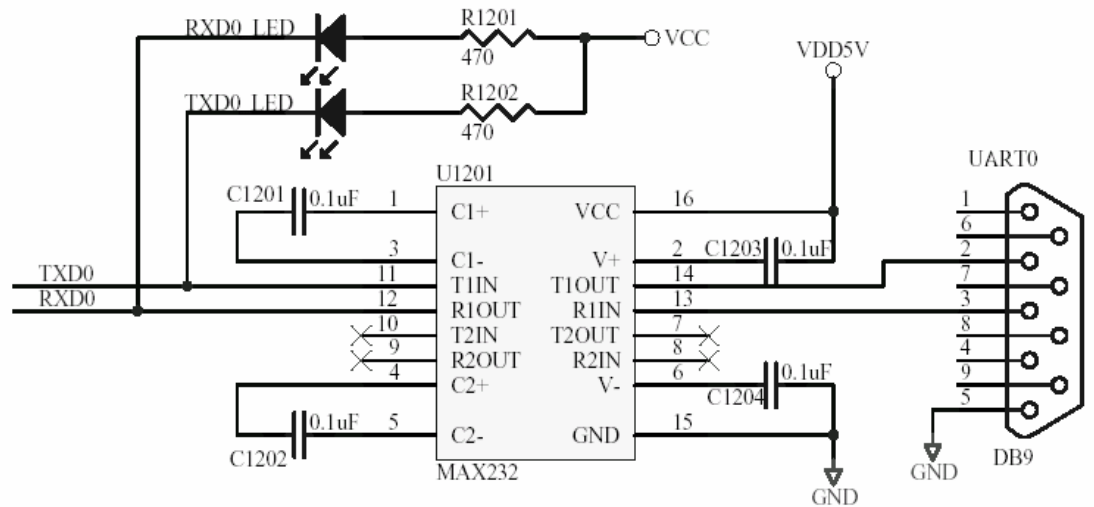


图 3—12 HDB9301/02 开发系统的 RS232 接口电路图

图 3—12 中的 RXD0_LED 和 TXD0_LED 用于发送和接收的状态指示,当有数据通过 UART0 传输时,LED 闪烁,便于用户掌握其工作状态以及进行软、硬件的调试。

3.2.8 RS485 接口电路

RS485 串行接口可以用最少的信号线完成通信任务,能够支持较高的数据传输速率和

较远的传输距离，同时可以方便进行多点互连，是工业控制中经常使用的电气接口。

可以通过对 EP93XX 系列微处理器的 UART 接口进行信号转换，在系统中实现 RS485 接口，信号转换可由 MAX485 完成。

HDB9301/02 开发系统的 RS485 接口由 EP9301/02 的 UART1 经 MAX485 转换完成。

MAX485 是用于支持 RS485/RS422 通信的低功耗收发器，内含一个独立的驱动器和接收器，支持半双工收发，其 DIP/SO 封装的引脚分布和相关功能描述如图 3-13 和表 3-6 所示。

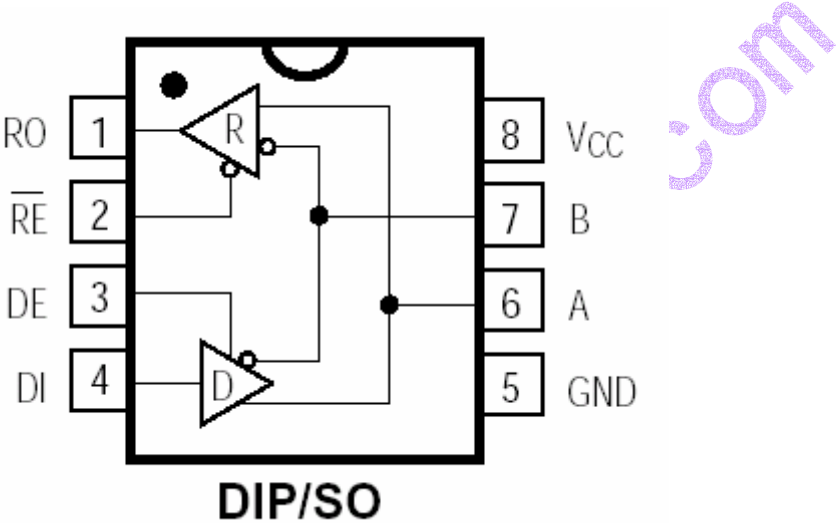


图 3-13 MAX485 的引脚分布

表 3-6 MAX485 引脚功能描述

引脚	名称	功能描述
1	RO	接收器输出：如果 A>B 200mV，RO 输出为高电平，如果 A<B 200mV，RO 输出为低电平。
2	/RE	接收器输出使能：当该引脚为低电平时 RO 使能，当该引脚为高电平时 RO 为高阻状态。
3	DE	驱动器输出使能：当该引脚为高电平时驱动器输出使能，当该引脚为低电平时，驱动器输出为高阻状态。
4	DI	驱动器输入。
5	GND	地。
6	A	收发端 A。
7	B	收发端 B。
8	VCC	电源（4.75~5.25V）。

关于 MAX485 更具体的内容可参考相关用户手册。

HDB9301/02 开发系统的 RS485 接口电路如图 3-14 所示，该电路可以在基于 EP93XX 系列微处理器的系统中通用。

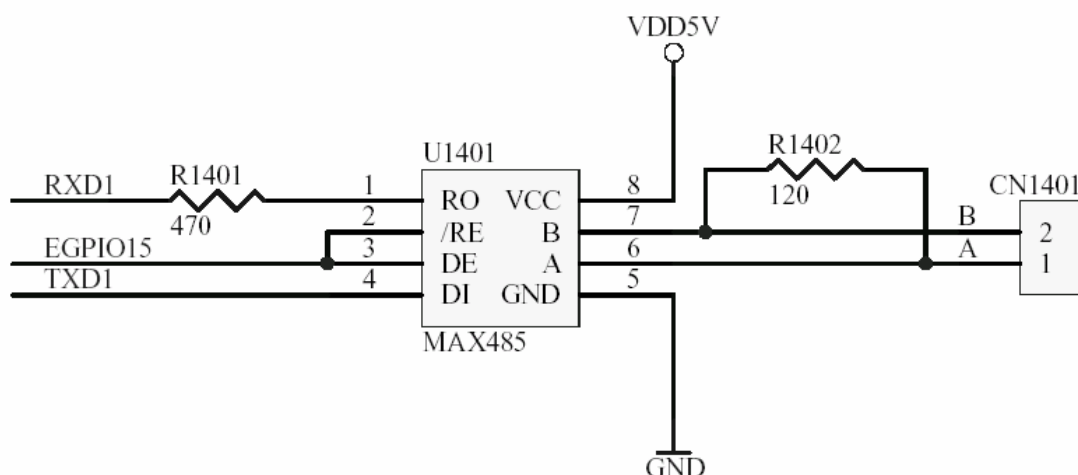


图 3-14 HDB9301/02 开发系统的 RS485 接口电路图

在图 3-14 中，MAX485 的 /RE 和 DE 由 EP9301/02 的 EGPIO15 控制，由于 MAX485 以半双工的方式工作，因此由 EGPIO15 控制其发送与接收，当 EGPIO15 为高电平时，驱动器使能而接收器为高阻态，此时可以发送数据，而当 EGPIO15 为低电平时，接收器使能而驱动器为高阻态，此时可以接收数据。

RS485 是采用平衡式（差分式）传输信号，对同时出现在两条信号线 A 和 B 的干扰有较强的抑制能力，当两条线绞在一起时，对通讯各种分布参数耦合过来的干扰信号则可平均地分配到这两条线上，因此对 RS485 的平衡式传输线路而言，用双绞线可获得较强的抗干扰能力。

3.2.9 USB 接口电路

EP9301/02 片内集成了 2 个 USB2.0 的全速主口，与 OHCI USB2.0 全速规范完全兼容，通过简单接口电路，用户可以构建各种基于 USB 接口的复杂应用。

HDB9301/02 开发系统的 USB 接口电路如图 3-15 所示，该电路可以在基于 EP93XX 系列微处理器的系统中通用。

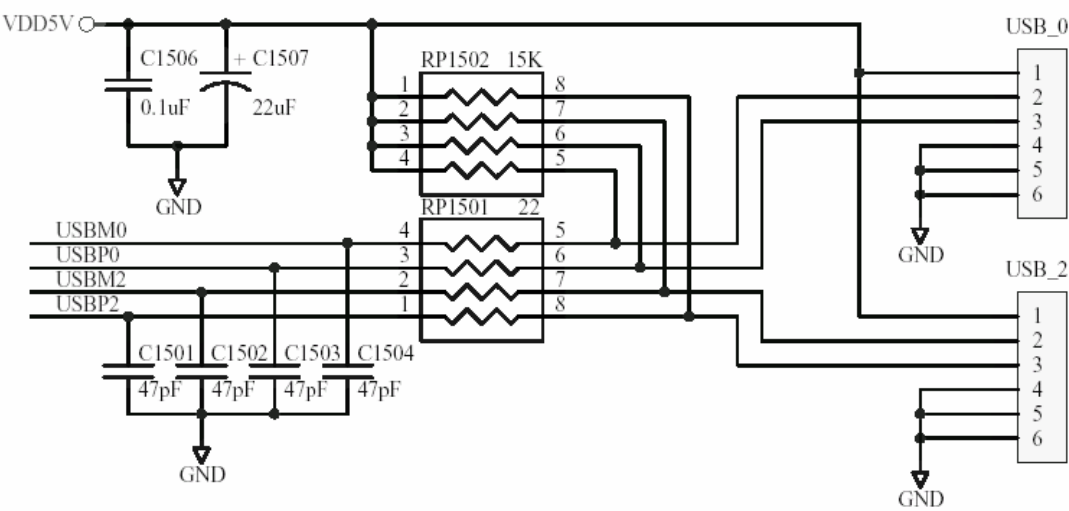


图 3-15 HDB9301/02 开发系统的 USB 接口电路图

EP9307、EP9312 和 EP9315 具有三个 USB 主口，其连接方式与图 3-15 类似。

3.2.10 以太网接口电路

EP93XX 系列微处理器都内嵌 1/10/100Mbps 的以太网 LAN 控制器，用户只需要外接物理层（PHY，Physical Layer）接口芯片，即可实现高效率的以太网接入。

EP93XX 系列微处理器内嵌的以太网 LAN 控制器支持支持媒体独立接口（MII，Media Independent Interface），可以方便的与各种物理层接口芯片连接。HDB9301/02 开发系统采用 Cirrus Logic 的专用 100M 以太网物理层接口芯片 CS8952，实现网络接入。

CS8952 是基于 CMOS 技术的低功耗、高性能 100BASE-X/10BASE-T 物理层接口，支持 MII 接口，可方便的与 10M/100M 以太网控制器接口。CS8952 同时提供 100BASE-FX 光纤接口。

CS8952 的 TQFP 封装引脚分布及引脚功能描述如图 3-16 和表 3-7 所示。

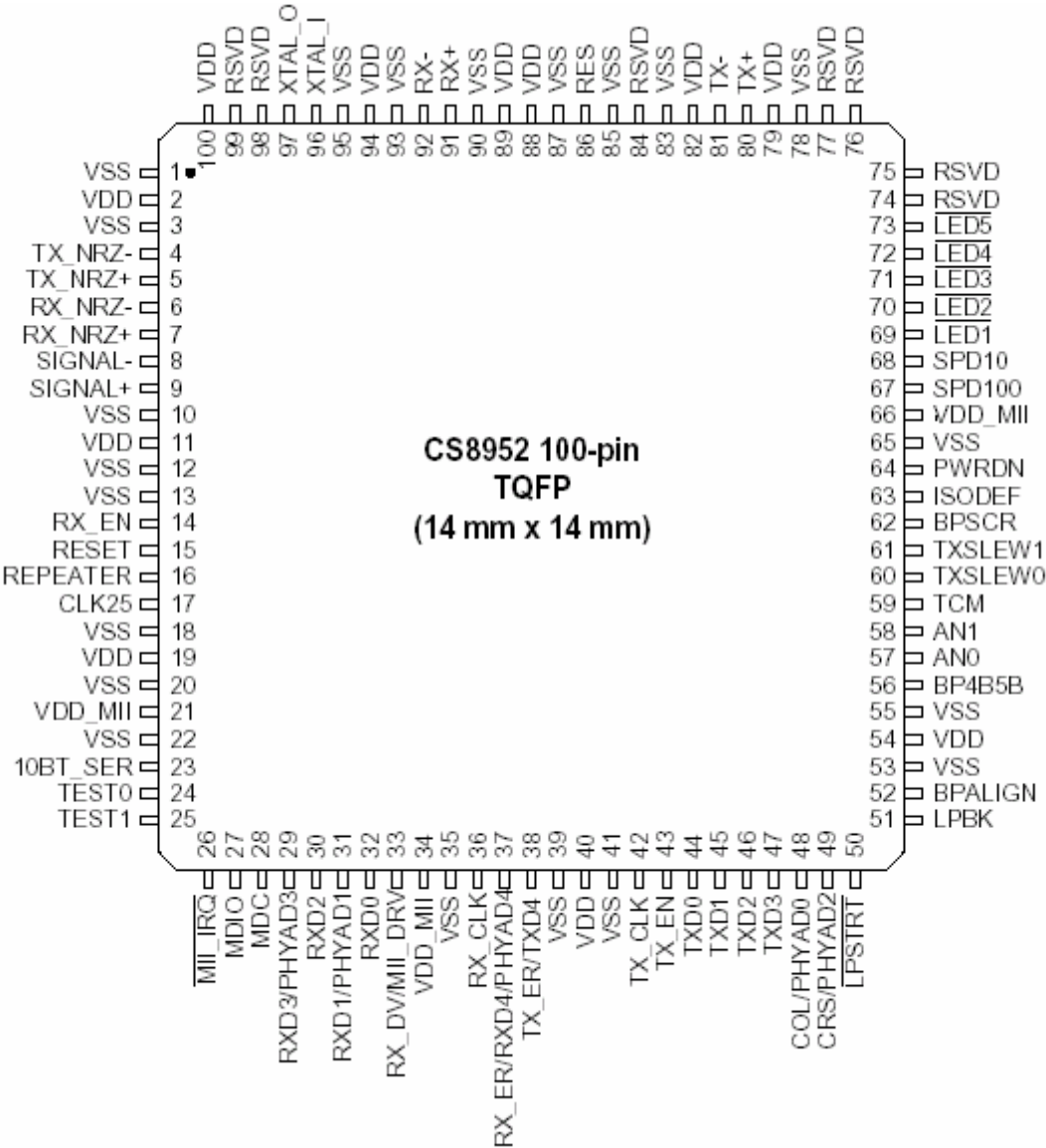


图 3-16 CS8952 引脚分布图

表 3-7 CS8952 引脚功能描述

信号	类型	引脚	功 能 描 述
COL/PHYAD0	输入/三态输出	48	冲突指示/物理层地址 0: 当系统上电或复位时, 锁存该引脚状态作为物理层地址的第 0 位, 该引脚内部弱上拉 (>150K), 可以通过外接 4.7K 上拉或下拉电阻改变锁存状态。系统工作时作为网络冲突指示。
CRS/PHYAD2	输入/三态输出	49	载波检测/物理层地址 2: 当系统上电或复位时, 锁存该引脚状态作为物理层地址的第 2 位, 该引脚内部弱下拉 (>20K), 可以通过外接 4.7K 上拉或下拉电阻改变锁存状态。

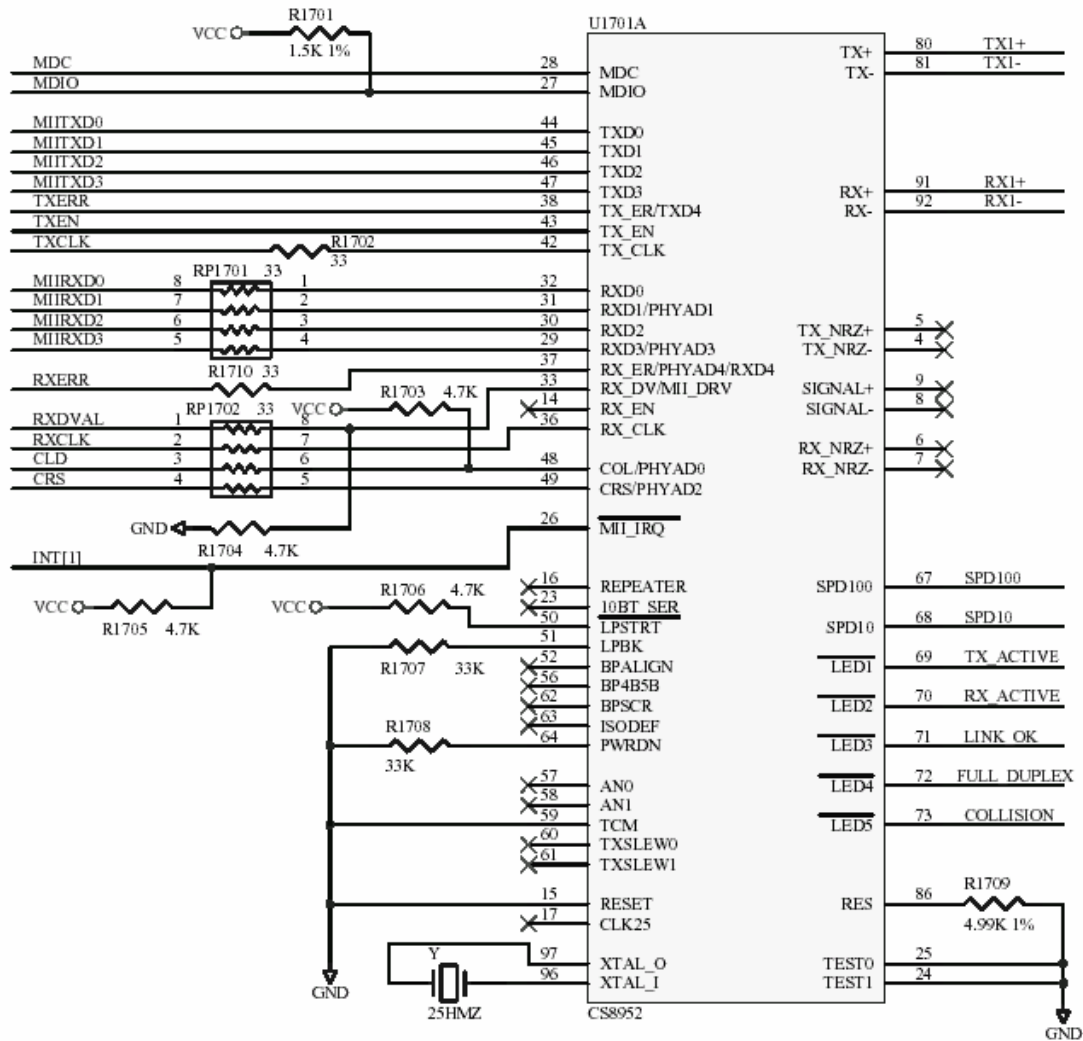
			系统工作时作为载波检测。
MDC	输入	28	管理数据时钟。
MDIO	双向	27	管理数据输入/输出。
/MII_IRQ	漏极开路 输出	26	MII 中断请求信号：漏极开路输出，需外接上拉电阻。
RX_CLK	三态输出	36	接收时钟。
RX_DV /MII_DRV	输入/三 态输出	33	接收数据有效/MII 驱动强度参考：当系统上电或复位时，锁存该引脚状态作为 MII 输出驱动强度参考，决定 MII 输出驱动能力的大小。 系统工作时指示接收数据是否有效。
RX_EN	输入	14	接收使能：当该引脚为高电平，所有接收引脚使能，当该引脚为低电平，所有接收引脚为高阻态。 该引脚内部弱上拉 (>150K)。
RX_ER/RXD4 /PHYAD4	输入/三 态输出	37	接收错误/接收数据 4/物理层地址 4：当系统上电或复位时，锁存该引脚状态作为物理层地址的第 4 位，该引脚内部弱下拉 (>20K)，可以通过外接 4.7K 上拉或下拉电阻改变锁存状态。系统工作时作为接收错误指示，或作为接收数据的第 4 位。
RXD3 /PHYAD3	输入/三 态输出	29	接收数据 3/物理层地址 3：当系统上电或复位时，锁存该引脚状态作为物理层地址的第 3 位，该引脚内部弱下拉 (>20K)，可以通过外接 4.7K 上拉或下拉电阻改变锁存状态。系统工作时作为接收数据的第 3 位。
RXD2	三态输出	30	接收数据的第 2 位。
RXD1 /PHYAD1	输入/三 态输出	31	接收数据 1/物理层地址 1：当系统上电或复位时，锁存该引脚状态作为物理层地址的第 1 位，该引脚内部弱下拉 (>20K)，可以通过外接 4.7K 上拉或下拉电阻改变锁存状态。系统工作时作为接收数据的第 1 位。
RXD0	三态输出	32	接收数据的第 0 位。
TX_CLK	输入/三 态输出	42	发送时钟：其输入/输出模式由 TCM 引脚控制。
TX_EN	输入	43	发送使能。
TX_ER/TXD4	输入	38	发送错误/发送数据 4：系统工作时作为发送错误指示，或作为发送数据的第 4 位。
TXD[3: 0]	输入	47、46、 45、44	发送数据输入。
10BT_SER	输入	23	10Mbps 串行模式选择：该引脚用于选择 10M 串行模式或并行模式，当芯片工作在 100Mbps 模式时，该引脚可忽略。
AN[1: 0]	输入	58、57	自动协议控制：具体含义可参考 CS8952 用户手册
BP4B5B	输入	56	4B5B 编码器旁路控制，该引脚内部弱下拉 (>20K)。
BPALIGN	输入	52	符号对齐旁路控制，该引脚内部弱下拉 (>20K)。
BPSCR	输入	62	扰频器旁路控制，该引脚内部弱下拉 (>20K)。
ISODEF	输入	63	MII 隔离控制，该引脚内部弱下拉 (>20K)。

/LED1	漏极开路输出	69	发送状态 LED: 当有数据发送时, 该引脚为低电平。
/LED2	漏极开路输出	70	接收状态 LED: 当接收到数据时, 该引脚为低电平。
/LED3	漏极开路输出	71	连接好 LED: 当 CS8952 检测到有效连接时, 该引脚为低电平。
/LED4	漏极开路输出	72	半双工/全双工 LED: 在 100M 模式下, 该引脚为低电平表示全双工模式。
/LED5	漏极开路输出	73	冲突 LED: 当检测到数据冲突时, 该引脚为低电平。
LPBK	输入	51	回环 (Loopback) 使能: 当该引脚为高电平, 芯片工作在回环模式。
/LPSTRT	输入	50	低功耗启动控制: 当系统上电或复位时, 若该引脚为低电平, CS8952 进入低功耗状态, 此时大多数电路不工作。该引脚内部弱下拉 (>20K)。
PWRDN	输入	64	掉电模式控制: 当系统上电或复位时, 若该引脚为高电平, CS8952 进入掉电模式, 此时大多数电路不工作。
REPEATER	输入	16	中继器模式选择。
SPD10	输出	68	10Mbps 速度指示: 当 CS8952 工作在 10M 速度时, 该引脚为高电平。
SPD100	输出	67	100Mbps 速度指示: 当 CS8952 工作在 100M 速度时, 该引脚为高电平。
TCM	输入	59	发送时钟模式初始化控制: 该引脚用于控制 TX_CLK 引脚模式。
TEST[1: 0]	输入	24、25	工厂测试: 该引脚仅用于工厂测试, 为保证系统正常工作, 该引脚应该接地。
TXSLEW [1: 0]	输入	61、60	发送回转率控制。
RX+、RX-	差分输入	91、92	数据接收。
TX+、TX-	差分输出	80、81	数据发送。
RX_NRZ+、 RX_NRZ-	差分输入	7、6	光纤数据接收。
TX_NRZ+、 TX_NRZ-	差分输出	5、4	光纤数据发送。
SIGNAL+、 SIGNAL-	差分输入	9、8	光纤信号检测。
CLK25	输出	17	25MHz 时钟输出。
RES	输入	86	参考电阻: 该引脚需外接一个精度为 1% 的 4.99K 的电阻到地。
XTAL_I、 XTAL_O	输入、 输出	96 97	连接 25MHz 晶振。
RSVD		74、75、 76、77、	系统保留: 该引脚接地。

		84、98、99	
VDD_MII	电源	21、34、66	MII 电源。
VDD	电源	2,11,19,40,54,79,82,88,89,94,100	内核电源。
VSS	地	1,3,10,12,13,18,20,22,35,39,41,53,55,65,78,83,85,87,90,93,95	接地。

关于 CS8952 更具体的内容可参考相关用户手册。

HDB9301/02 开发系统的以太网接口电路如图 3—17 所示，该电路可以在基于 EP93XX 系列微处理器的系统中通用。



a CS8952 的主要信号连接图

测试工具对内部节点进行测试。目前大多数比较复杂的器件都支持 JTAG 协议，如 ARM、DSP、FPGA 器件等。JTAG 接口还常用于实现在系统编程（ISP，In-System Programmable）功能，对诸如 FLASH 器件等进行编程。

标准的 JTAG 接口是一个 20 针的双列直插接口，引脚定义如表 3-8 所示。

表 3-8 标准 JTAG 接口定义

引 脚	名 称	描 述
1	VTref	目标系统参考电压，接电源。
2	VCC	接电源。
3	nTRST	测试系统复位信号。
4、6、8、10、12、14、16、18、20	GND	接地。
5	TDI	测试数据串行输入。
7	TMS	测试模式选择。
9	TCK	测试时钟。
11	RTCK	测试时钟返回信号。
13	TDO	测试数据串行输出。
15	nRESET	目标系统复位信号。
17、19	NC	未连接。

EP93XX 系列微处理器内嵌支持 JTAG 调试接口的部件，其接口电路如图 3-18 所示，该电路可以在基于 EP93XX 系列微处理器的系统中通用。

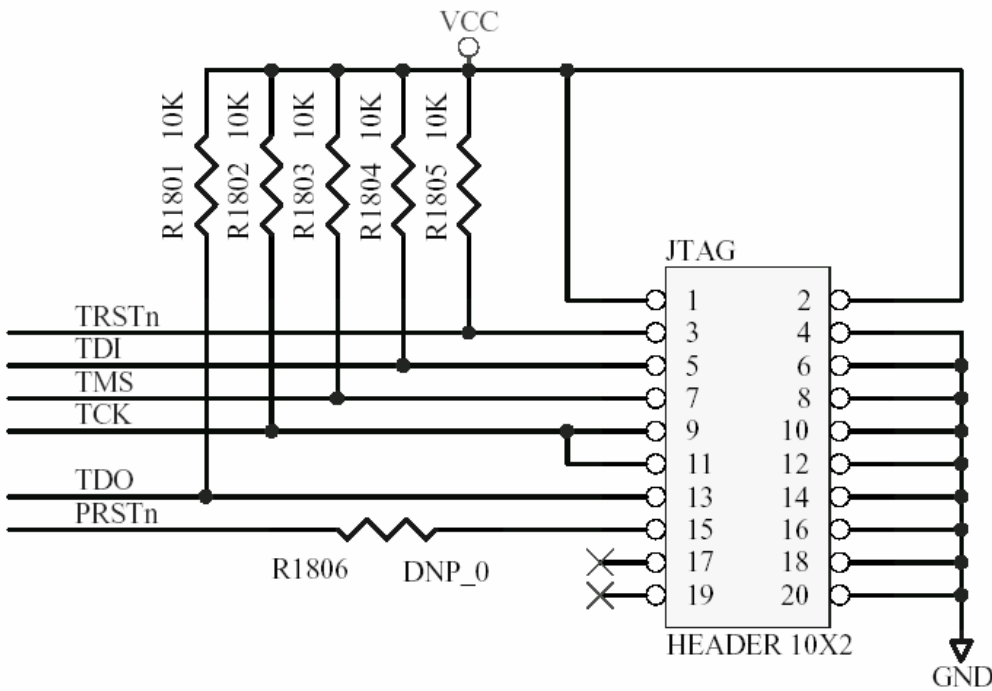


图 3—18 HDB9301/02 开发系统的 JTAG 调试接口电路

3.2.12 音频接口电路

EP93XX系列微处理器均内嵌音频接口电路，支持I²S和AC’97 音频接口，用户只需外接音频编码/解码电路，即可完成系统的音频输入/输出接口。

HDB9301/02 外接 AC’97 接口编码/解码芯片 CS4202，完成系统的音频输入/输出功能。

CS4202 是基于多媒体应用的立体声编码/解码器，兼容 AC’97 2.2 规范，内嵌 20 位立体声 D/A 转换器和 18 位立体声 A/D 转换器，可提供高质量的音频输入/输出。

CS4202 采用 48 脚 TQFP 封装，引脚分布如图 3—19 所示。

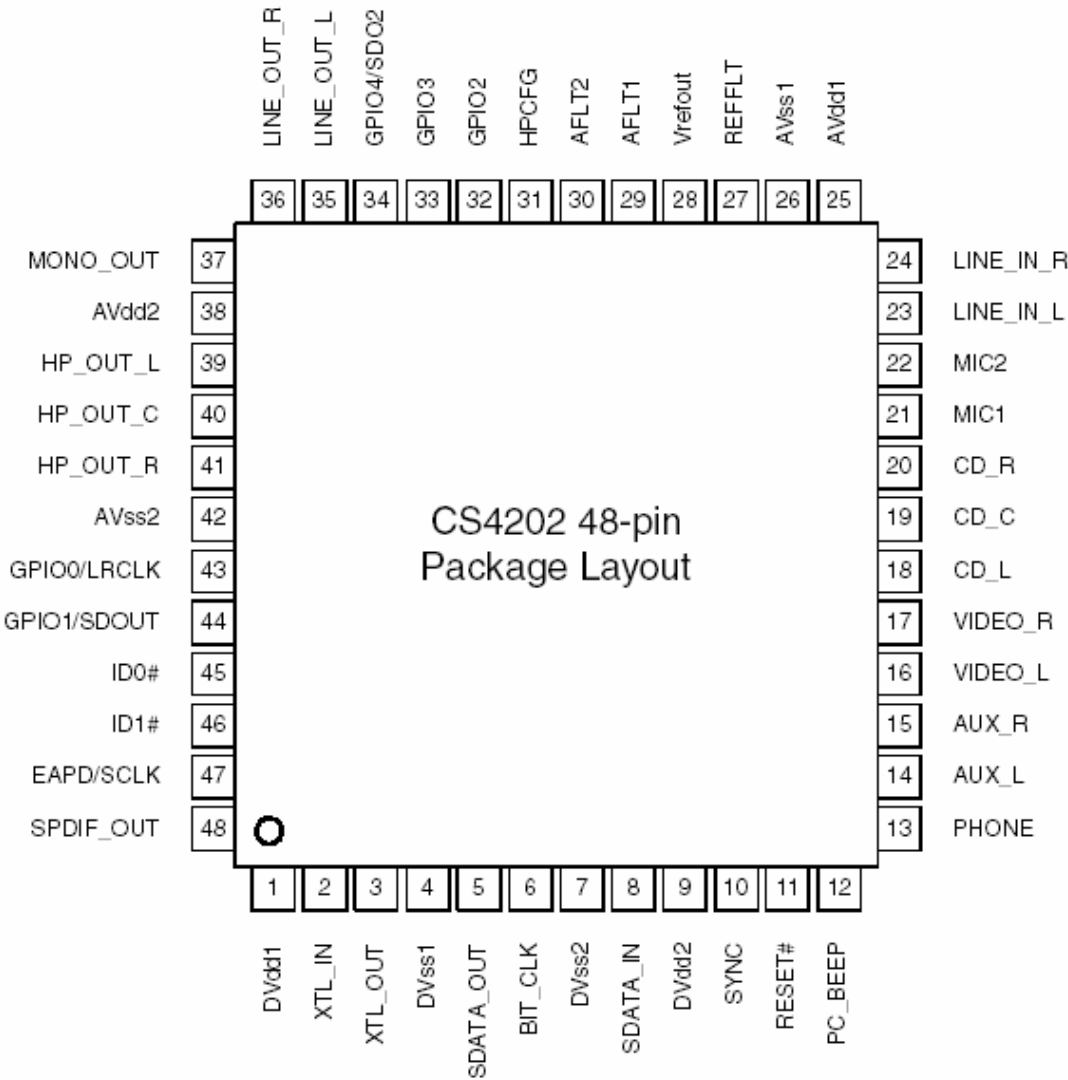
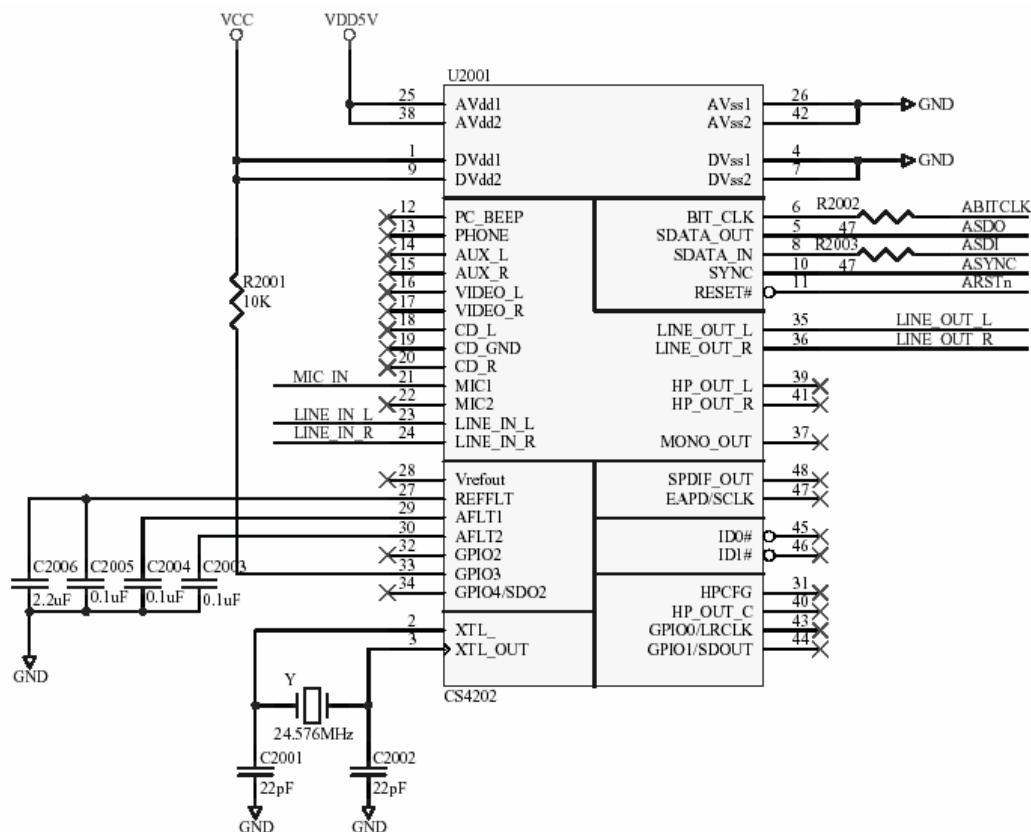


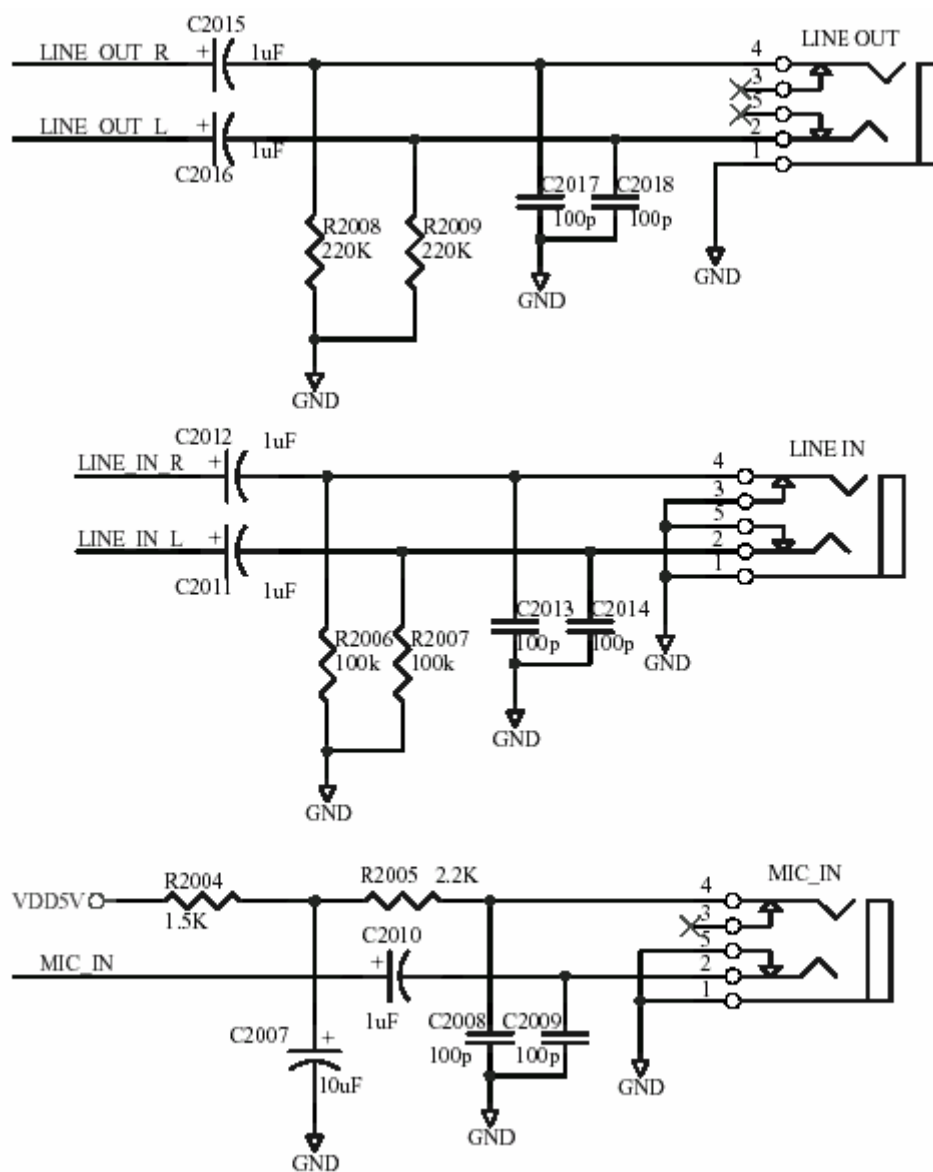
图 3—19 CS4202 引脚分布图

关于 CS4202 更具体的内容可参考相关用户手册。

HDB9301/02 开发系统的音频接口电路如图 3-20 所示，该电路可以在基于 EP93XX 系列微处理器的系统中通用。



a CS4202 的主要信号连接图



b 音频输入/输出接口

图 3-20 HDB9301/02 开发系统的音频接口电路

事实上, CS4202 功能非常强大, 除图 3-20 所示以外, 还可以支持其它音频信号的输入/输出, 用户可以根据自身需求, 做适当的修改。

3.2.13 系统状态指示电路

EP93XX 系列微处理器使用两个 GPIO (RDLED 和 GRLED) 作为系统的状态显示, 这

两个 GPIO 通常外接 LED 显示器，用于指示系统的运行状态，用户可以根据他们的状态，在一定程度上判断系统的工作情况。

RDLED 和 GRLED 常用的连接方式如图 3-21 所示，该电路可以在基于 EP93XX 系列微处理器的系统中通用。

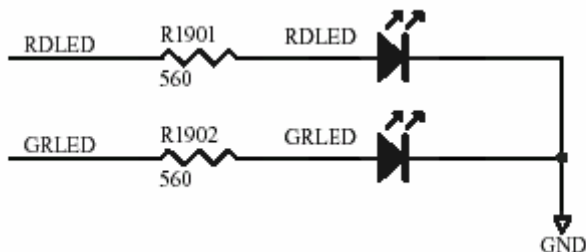


图 3-21 HDB9301/02 开发系统状态指示电路图

以上所描述的内容为基于 EP9301/02 应用系统常用接口电路，参照了 Cirrus Logic 所提供的参考设计，同时根据笔者的开发应用情况，做了一些必要的更改，读者可以以此为基础，根据自身的特定应用需求，做适当的增减，设计出最佳的应用系统。

3.3 基于 EP9307 的硬件系统设计

EP9307 是一款高集成度的 ARM9 嵌入式微处理器，具有 200MHz 的运行速度和丰富的用户接口功能，是工业控制、高端游戏机、POS 终端、家庭自动化系统和车船导航系统等低成本网络应用的理想选择。

EP9307 具有强大的处理能力，丰富的片内资源，同时芯片采用 BGA 封装方式，面积很小，非常适合对性能、功耗和体积有严格要求的系统。

中嵌技术有限公司 (www.emsyschina.com) 基于 EP9307 的开发系统为 HDB9307。图 3-22 为 HDB9307 开发系统的硬件结构框图。

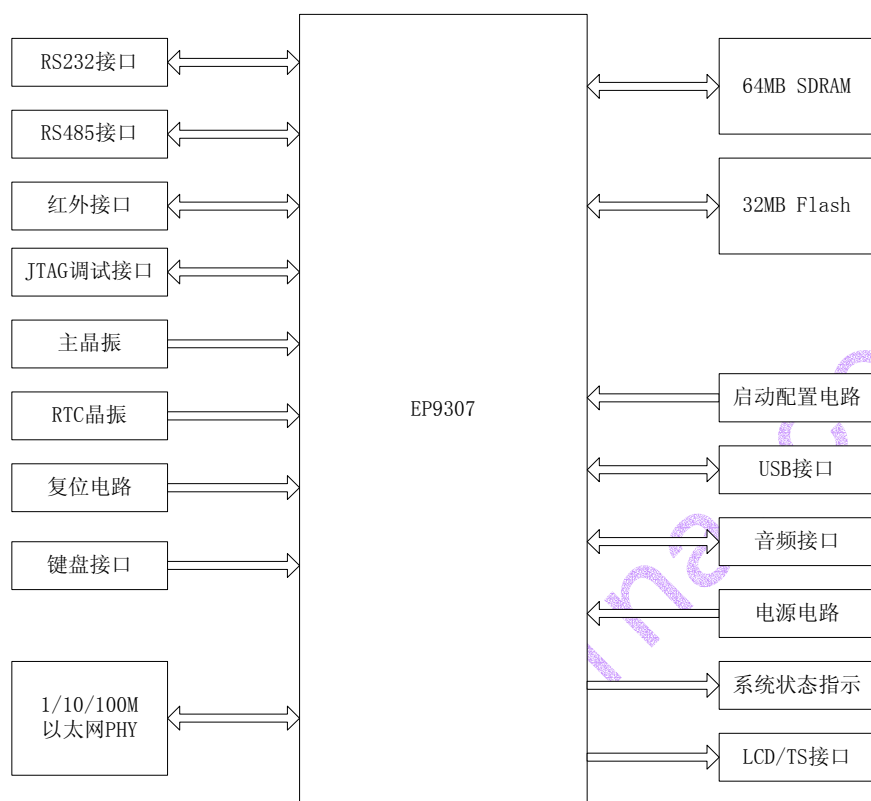


图 3-22 HDB9307 开发系统硬件结构框图

各部分基本功能描述如下：

- λ RS232 接口电路用于与其他应用系统的 RS232 通讯；
- λ RS485 接口电路用于与其他应用系统的 RS485 通讯；
- λ 红外接口电路用于与其他应用系统的红外通讯；
- λ 复位电路可完成系统上电复位和在系统工作时用户按键复位；
- λ 电源电路为 5V 到 3.3V 和 5V 到 1.8V 的转换器，用于给芯片内核及外围电路的供电；
- λ JTAG 调试接口可对芯片内部的所有部件进行访问，通过该接口可对系统进行软、硬件调试、编程等；
- λ 主晶振为系统提供工作时钟，通过片内 PLL 电路倍频为微处理器核和其它功能部件的工作时钟；
- λ RTC 晶振专为片内的 RTC 电路提供工作时钟；
- λ Flash 存储器可用于存放用户应用程序、嵌入式操作系统及其他在系统掉电后需要保存的用户数据等；
- λ SDRAM 存储器为系统程序的运行空间，系统及用户数据、堆栈均位于 SDRAM 存储器中；

- λ 1/10/100M 以太网物理层接口为系统提供以太网接入的物理通道，通过该接口，系统可以 1/10/100Mbps 的速率接入以太网；
- λ USB 接口支持与 USB 从设备的数据通信；
- λ 键盘接口支持用户外接矩阵键盘作为输入设备；
- λ LCD/TS 接口提供系统的 LCD 显示和触摸输入功能；
- λ 音频接口提供系统的音频输入/输出功能；
- λ 系统状态指示电路由芯片引脚外接的两个 LED 组成，用于指示系统的工作状态。

由于 EP93XX 系列嵌入式微处理器在结构上具有很好的一致性，基于 EP93XX 系列微处理器的应用系统电路有部分相同的，因此，在进行 HDB9307 的系统设计时，我们只需要说明其中不同的部分。

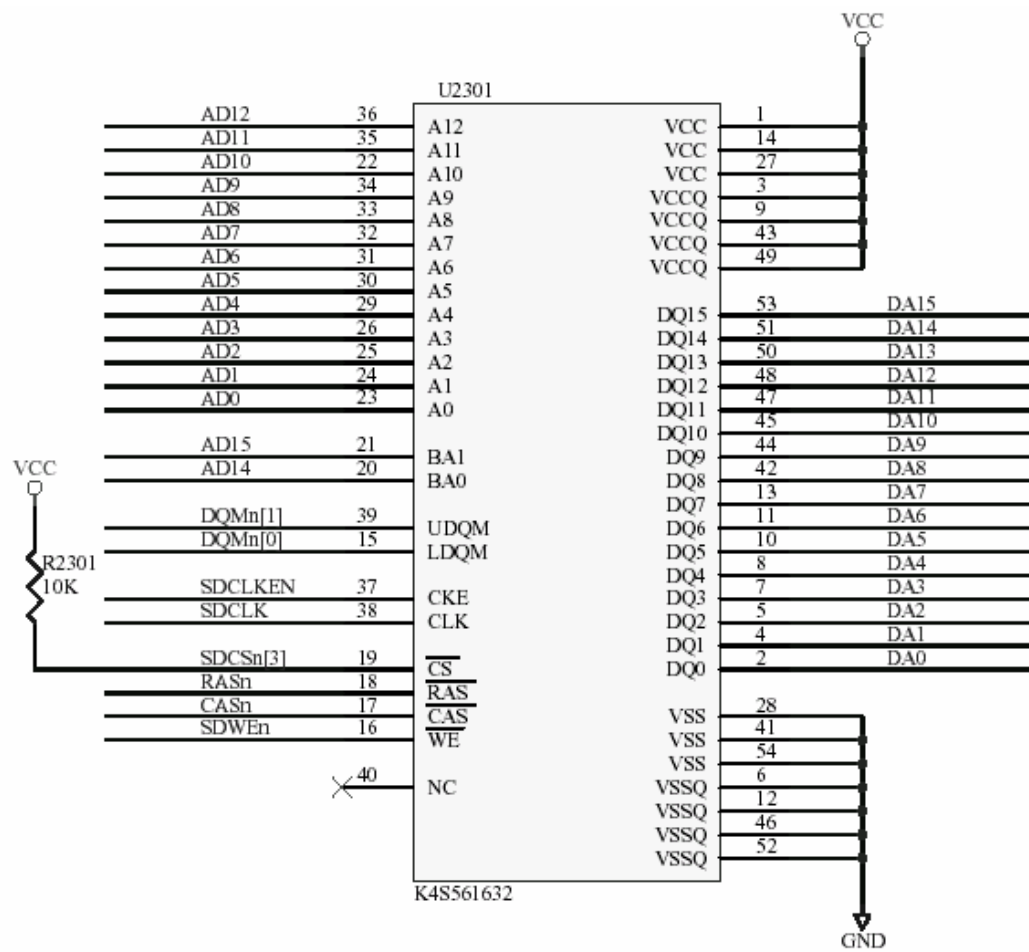
HDB9301/02 系统的 RS232 接口电路、RS485 接口电路、复位电路、晶振电路、电源电路、USB 接口电路、以太网接口电路、JTAG 调试接口电路、音频接口电路和系统状态指示电路，均可直接在 HDB9307 系统中使用，因此不再单独描述。

HDB9307 系统与 HDB9301/02 系统不同的电路包括：SDRAM 接口电路、Flash 接口电路、红外接口电路、启动配置电路、LCD/TS 电路和键盘接口电路，我们在以下部分详细描述。

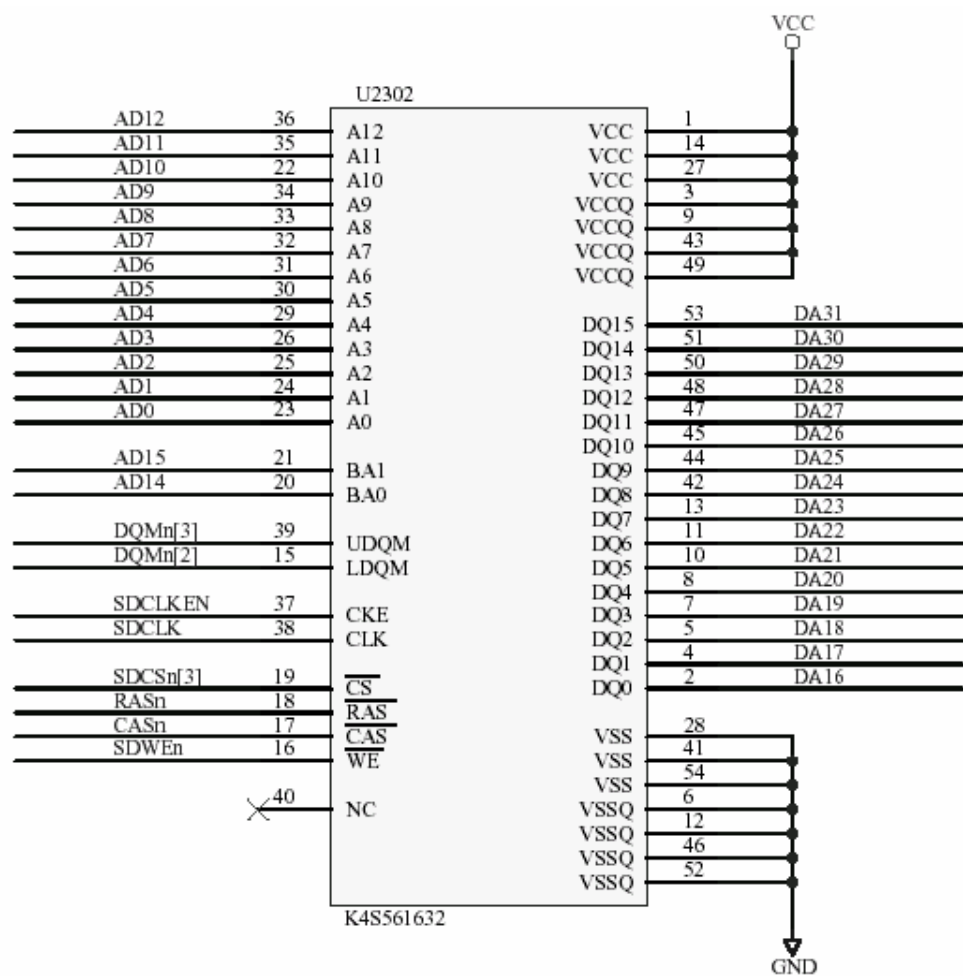
3.3.1 SDRAM 接口电路

基于 EP9307 的系统可以完成比较复杂的应用，因此，系统也配置了更大 SDRAM，为保持系统设计的一致性，仍然采用 SAMSUNG 的 K4S561632 作为系统的 SDRAM。

在 HDB9307 开发系统中，采用了两片 K4S561632 并联，使 SDRAM 容量达到 64MB，接口电路如图 3-23 所示。



a K4S561632 与 EP9307 低 16 位数据总线的接口



b K4S561632 与 EP9307 高 16 位数据总线的接口

图 3—23 HDB9307 开发系统的 SDRAM 接口电路图

两片 K4S561632 并联构建 32 位的 SDRAM 存储器系统，与 EP9307 的 32 位数据总线相连，图 3—23a 为低 16 位，图 3—23b 为高 16 位，两片 K4S561632 作为一个整体配置到 SDCSn[3]。

两片 K4S561632 的 CLK、CKE、/RAS、/CAS、/WE 端都连接到 EP9307 的 SDCLK、SDCLKEN、RASn、CASn 和 SDWEn 端；

两片 K4S561632 的地址总线与 BANK 选择信号 BA1 和 BA0 都连接到 EP9307 的地址总线；

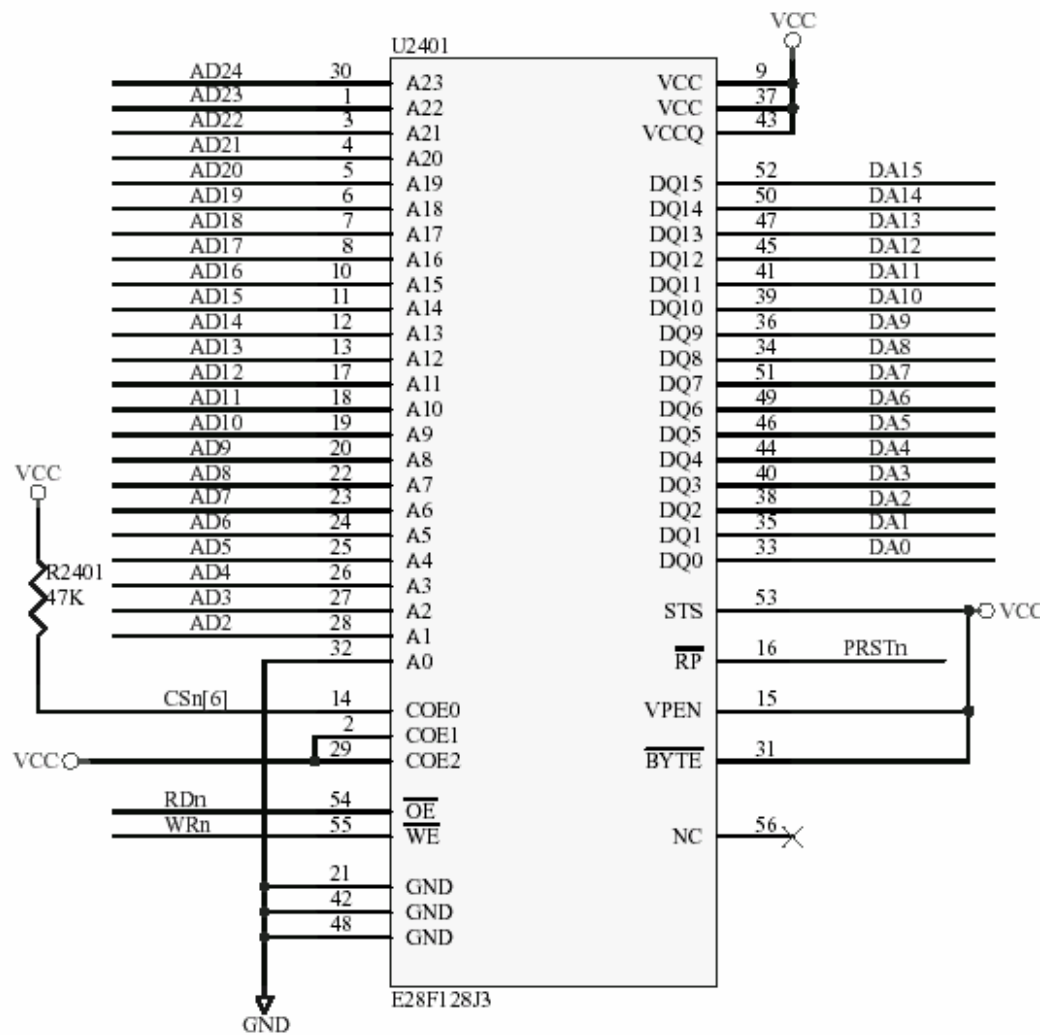
其中，低 16 位片的数据总线 DQ15~DQ0 连接 EP9307 数据总线的低 16 位 DA15~DA0，高 16 位片的数据总线 DQ15~DQ0 连接 EP9307 数据总线的高 16 位 DA31~DA16；

低 16 位片的 UDQM、LDQM 分别连接 EP9307 的 DQMn[1]和 DQMn[0]，高 16 位片的 UDQM、LDQM 分别连接 EP9307 的 DQMn[3]和 DQMn[2]。

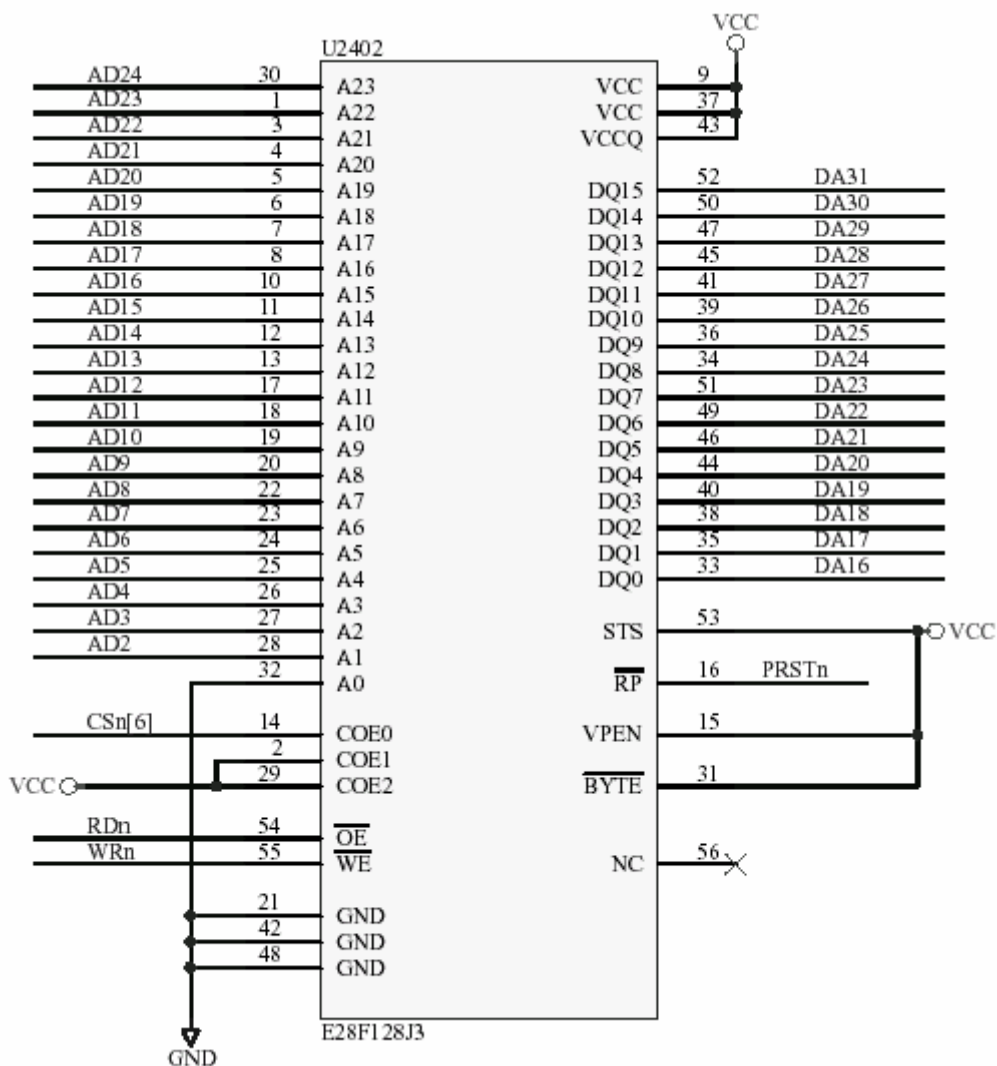
3.3.2 Flash 存储器接口电路

基于 EP9307 的系统也配置了更大 Flash 存储器，仍然采用 Intel 的 E28F128J3 作为系统的 Flash 存储器。

在 HDB9307 开发系统中，采用了两片 E28F128J3 并联，使 Flash 存储器的容量达到 32MB，接口电路如图 3-24 所示。



a 28F128J3 与 EP9307 低 16 位数据总线的接口



b 28F128J3 与 EP9307 高 16 位数据总线的接口

图 3-24 HDB9307 开发系统的 Flash 存储器接口电路图

两片 E28F128J3 并联构建 32 位的 Flash 存储器系统,与 EP9307 的 32 位数据总线相连,图 3-24a 为低 16 位,图 3-24b 为高 16 位,两片 E28F128J3 作为一个整体配置到 CSn[6]。

两片 E28F128J3 的/OE、/WE 端都连接到 EP9307 的 RDn、WEn 端;

两片 E28F128J3 的地址总线连接到 EP9307 的地址总线;

其中,低 16 位片的数据总线 DQ15~DQ0 连接 EP9307 数据总线的低 16 位 DA15~DA0,高 16 位片的数据总线 DQ15~DQ0 连接 EP9307 数据总线的高 16 位 DA31~DA16。

3.3.3 红外接口电路

EP93XX 系列微处理器都内嵌红外模块，通过 UART2 与外部接口，用户只需外接红外收发电路，即可实现系统的红外数据传输。

HDB9307 系统采用 TFDS6501E 作为红外收发器，该器件是一款低功耗快速红外收发器，支持快速红外数据传输 IrDA 标准，可用于多种设备的红外数据传输。

TFDS6501E 为 8 引脚封装，各引脚功能描述如表 3-9 所示。

表 3-9 TFDS6501E 引脚功能描述

引脚	名 称	类型	功 能 描 述
1	IREC 阴极		IREC 阴极，内部连接到驱动三极管。
2	RXD	输出	接收数据输出，
3	VCC		电源。
4	GND		接地。
5	MODE	输入	模式控制：高电平为高速模式、低电平为低速模式。
6	SD/MODE	输入	关断模式控制，高电平有效。
7	TXD	输入	发送数据输入。
8	IREC 阳极		IREC 阳极，需要通过限流电阻连接到 VCC。

HDB9307 的红外接口电路如图 3-25 所示，该电路可以在基于 EP93XX 系列微处理器的系统中通用。

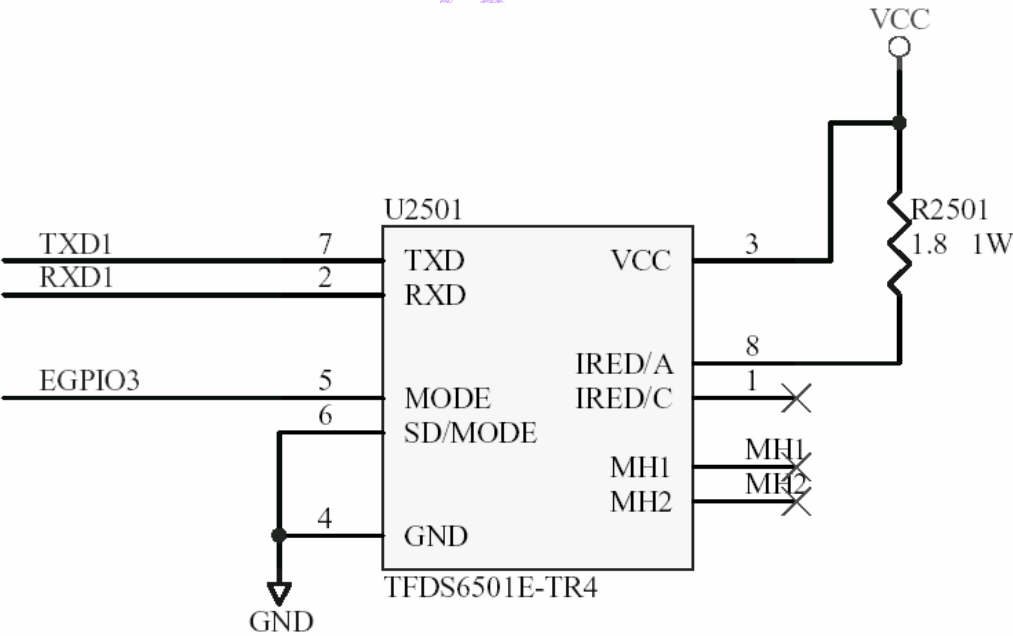


图 3-25 HDB9307 开发系统的红外接口电路图

在图 3-25 中，TFDS6501E 的 TXD、RXD 引脚与 UART2 的发送与接收连接，模式

引脚与 EP9307 的 EGPIO3 连接，由 EGPIO3 的状态控制 TFDS6501E 的工作模式。

3.3.4 启动配置电路

EP9307 的硬件启动配置由芯片的 EECLK、EEDAT、BOOT[1: 0]、ASDO 和 CSn[7: 6]引脚的上电初始化状态决定，如图 3-26 所示。

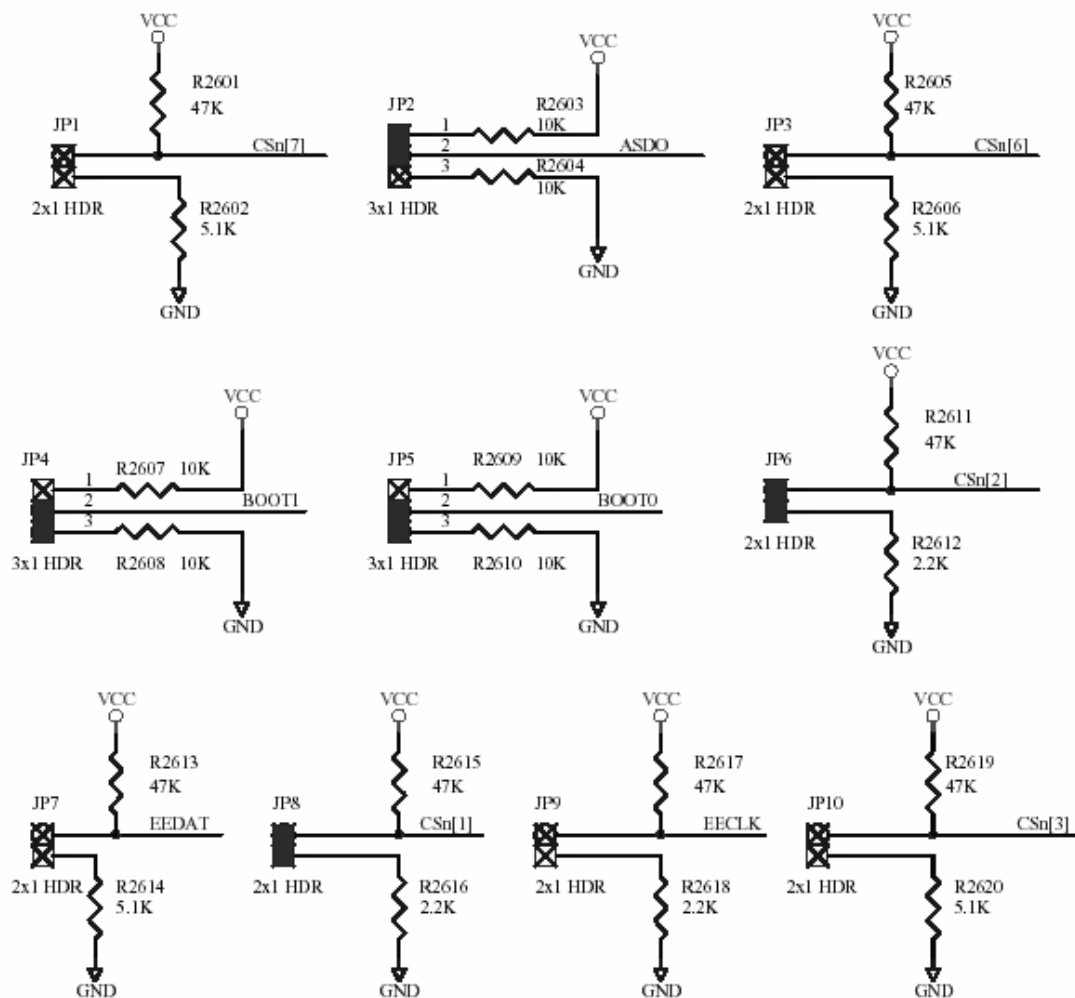


图 3-26 HDB9307 开发系统的启动配置

在图 3-26 的启动配置中，各引脚的配置含义可以参考第二章的启动硬件配置部分的相关内容。

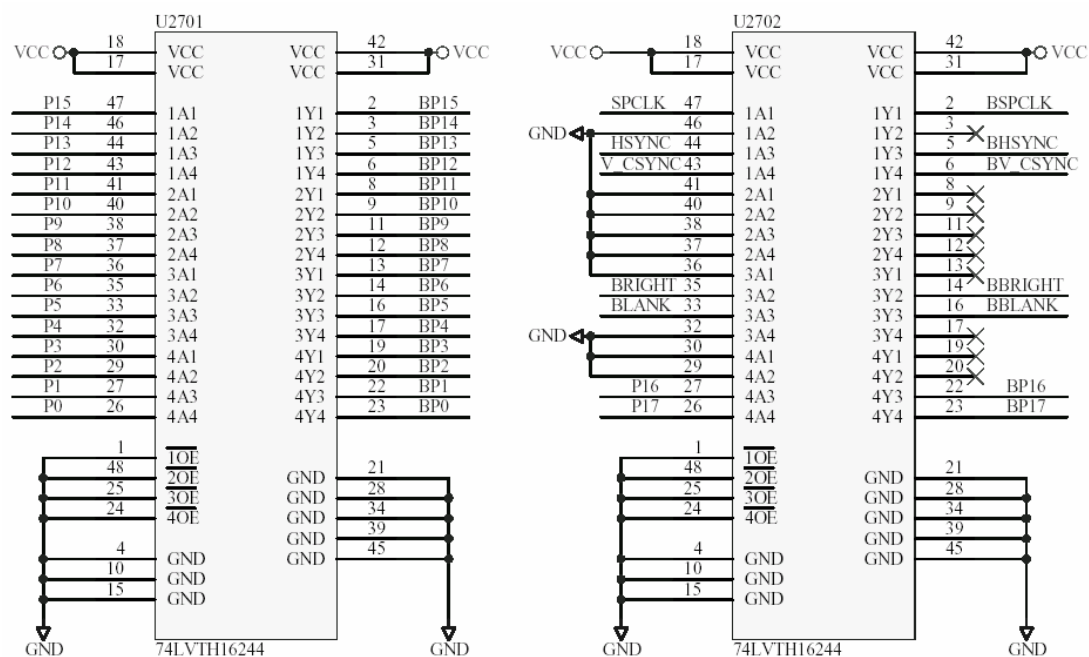
图中的阴影部分表示默认的硬件配置状态。

注意图中的 BOOT0 引脚，它的配置决定系统执行串行下载方式或从 Flash 存储器运行代码，我们在进行系统初始化操作时会改变该引脚的状态。

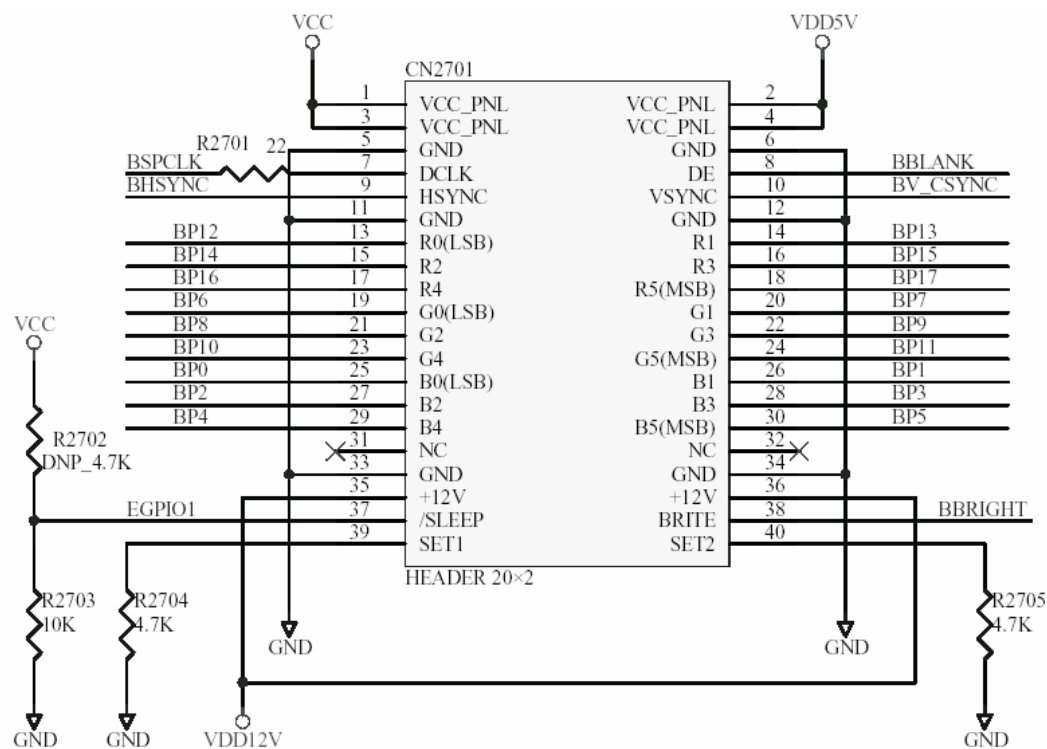
3.3.5 LCD/TS 接口电路

EP9307 内嵌 LCD 控制器，可方便的与各种 LCD 接口，同时，EP9307 还包括触摸屏接口，支持与 4~8 线的电阻式模拟触摸屏的连接。

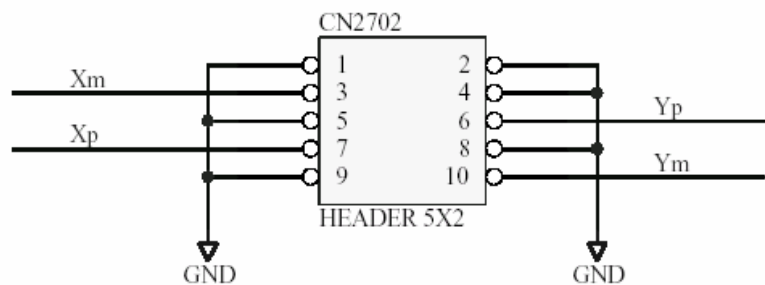
HDB9307 的 LCD/TS 接口电路如图 3-27 所示。



a LCD 信号驱动电路



b LCD 信号接口



c TS 信号接口

图 3—27 HDB9307 开发系统的 LCD/TS 接口电路

图 3—27a 为 CPU 的 LCD 数据信号和控制信号的驱动电路，图 3—27b 为 LCD 的接插口，图 3—27c 为 4 线电阻式模拟触摸屏的接插口。

3.3.6 键盘接口电路

EP9307 内嵌 8×8 键盘扫描电路，用户可以外扩多达 64 个单触点按键，当系统无需外

接键盘时，EP9307 的键盘接口可以作为 GPIO。

HDB9307 的键盘接口电路如图 3-28 所示。

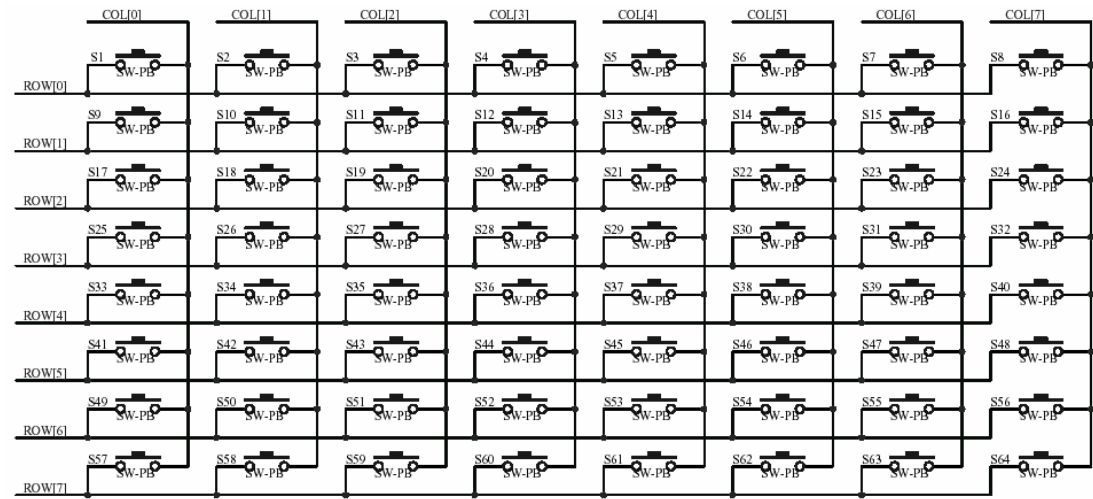


图 3-28 HDB9307 开发系统的键盘接口电路

以上所描述的内容为基于 EP9307 应用系统常用接口电路，读者可以以此为基础，根据自身的特定应用需求，做适当的增减，设计出最佳的应用系统。

3.4 基于 EP9312/15 的硬件系统设计

EP9312 和 EP9515 是 EP93XX 系列微处理器中的旗舰产品，是目前集成度最高、功能最强的 ARM 微处理器之一，可适用于多种应用领域。

中嵌技术有限公司（www.emsyschina.com）基于 EP93012/15 的开发系统为 HDB9312/15。图 3-29 为 HDB9312/15 开发系统的硬件结构框图。

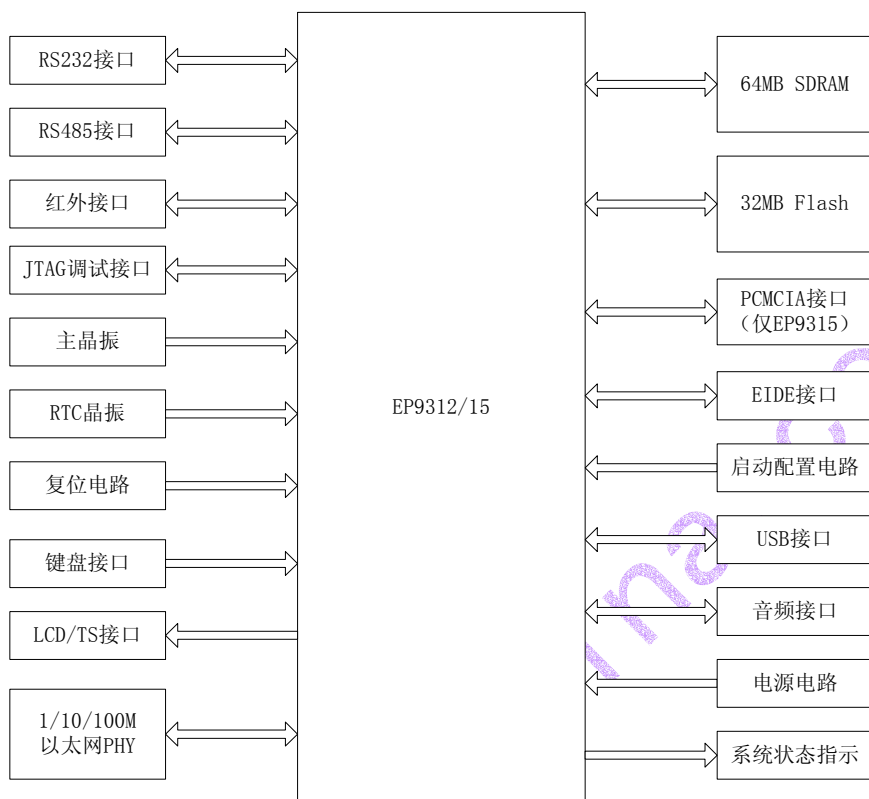


图 3-29 HDB9312/15 开发系统硬件结构框图

各部分基本功能描述如下：

- λ RS232 接口电路用于与其他应用系统的 RS232 通讯；
- λ RS485 接口电路用于与其他应用系统的 RS485 通讯；
- λ 红外接口电路用于与其他应用系统的红外通讯；
- λ 复位电路可完成系统上电复位和在系统工作时用户按键复位；
- λ 电源电路为 5V 到 3.3V 和 5V 到 1.8V 的转换器，用于给芯片内核及外围电路的供电；
- λ JTAG 调试接口可对芯片内部的所有部件进行访问，通过该接口可对系统进行软、硬件调试、编程等；
- λ 主晶振为系统提供工作时钟，通过片内 PLL 电路倍频为微处理器核和其它功能部件的工作时钟；
- λ RTC 晶振专为片内的 RTC 电路提供工作时钟；
- λ Flash 存储器可用于存放用户应用程序、嵌入式操作系统及其他在系统掉电后需要保存的用户数据等；
- λ SDRAM 存储器为系统程序的运行空间，系统及用户数据、堆栈均位于 SDRAM 存储器中；

图 3—30 HDB9312/15 开发系统 IDE 接口电路

3.4.2 PCMCIA 接口电路

EP9315 内嵌 PCMCIA 接口模块，通过简单的外围电路，可支持与 PCMCIA 设备的连接。

在 HDB9315 系统的 PCMCIA 接口中，使用了一个支持串行 PCMCIA 控制器的双口电源接口开关 TPS2202，该器件支持 PCMCIA 标准，带有两个 PCMCIA 接口的电源管理。TPS2202 的引脚封装及功能说明如图 3—31 和表 3—10 所示。

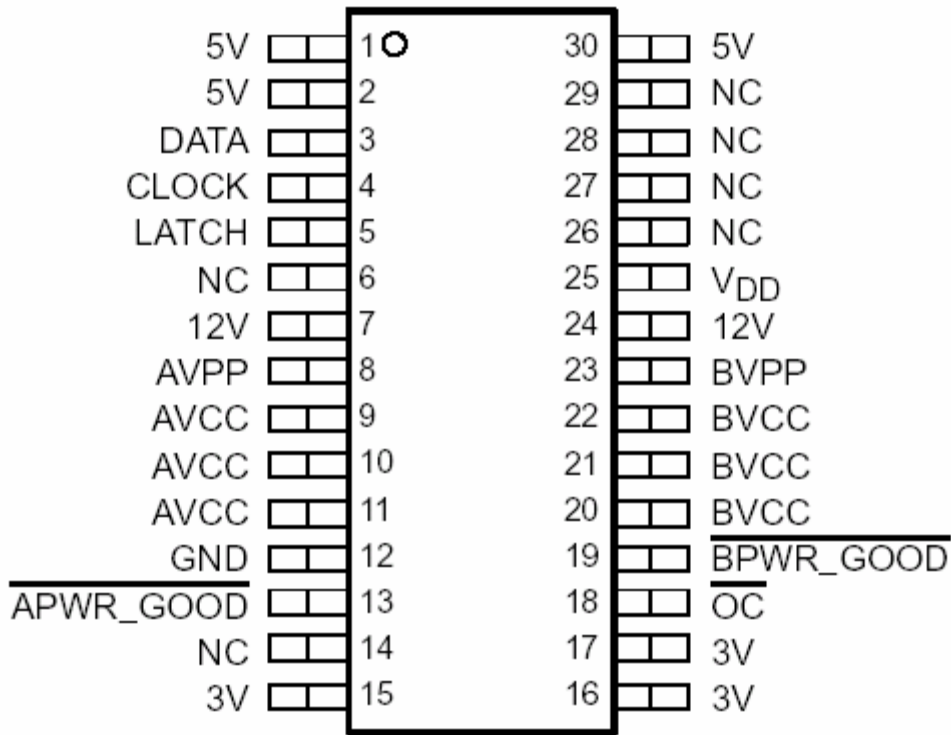
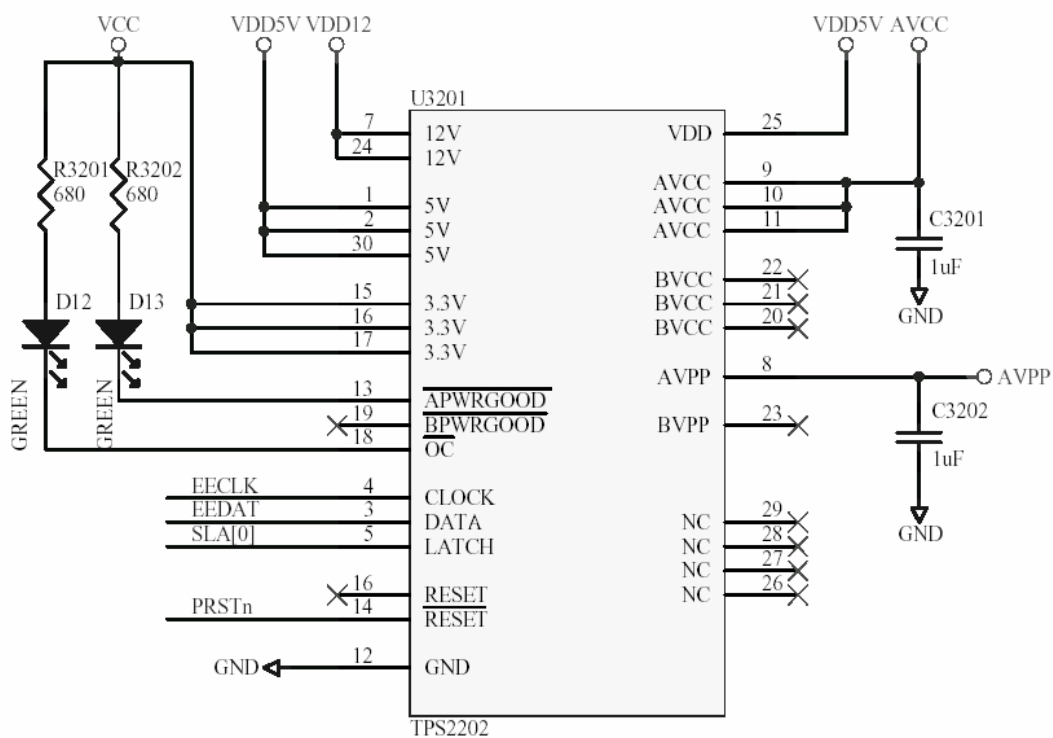


图 3—31 TPS2202 引脚封装图

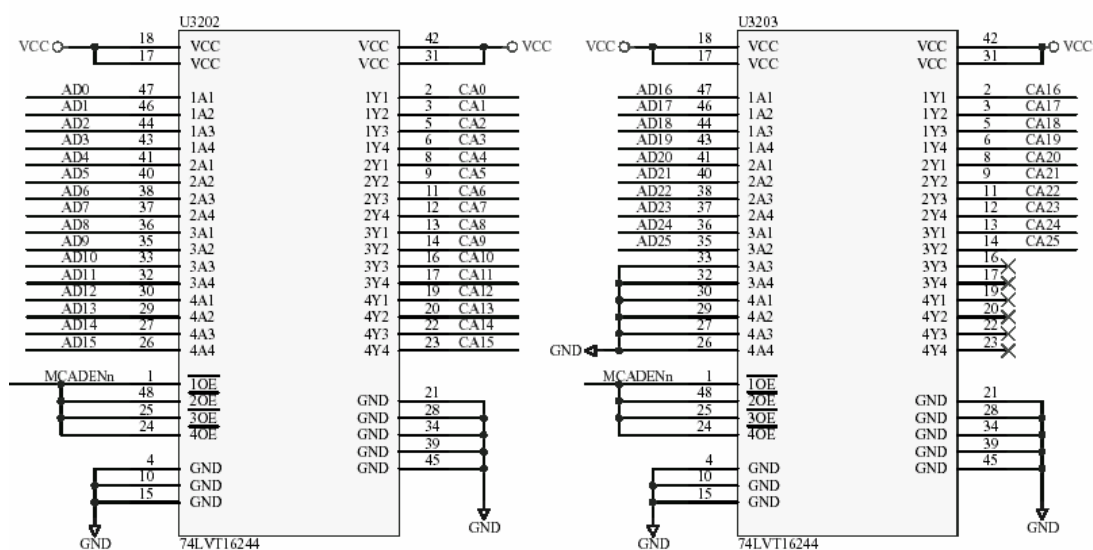
表 3—10 TPS2202 引脚功能描述

名 称	引脚	类型	功 能 描 述
3V	15、16、17	输入	3V 电源输入。
5V	1、2、30	输入	5V 电源输入。
12V	7、24	输入	12V 电源输入。
AVCC	9、10、11	输出	开关电源输出。
AVPP	8	输出	开关电源输出。
/APWR_GOOD	13	输出	电源好信号输出。
BVCC	20、21、22	输出	开关电源输出。
BVPP	23	输出	开关电源输出。

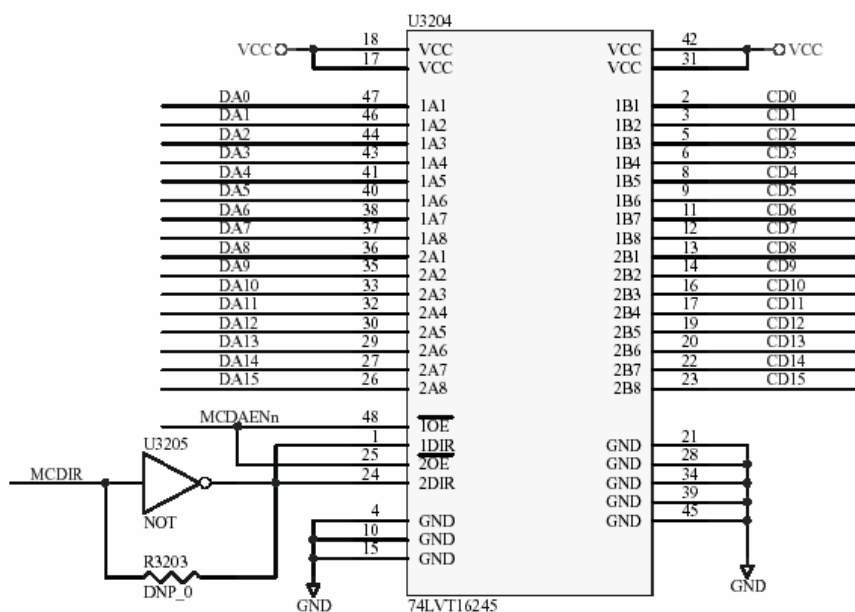
/BPWR_GOOD	19	输出	电源好信号输出。
CLOCK	4	输入	串行时钟信号。
DATA	3	输入	串行数据信号
GND	12		接地。
LATCH	5	输入	数据锁存信号。
/OC	18	输出	过电流信号输出。
VDD	25		5V 电源输入。
NC	6、14、26、27、28、29		未连接。



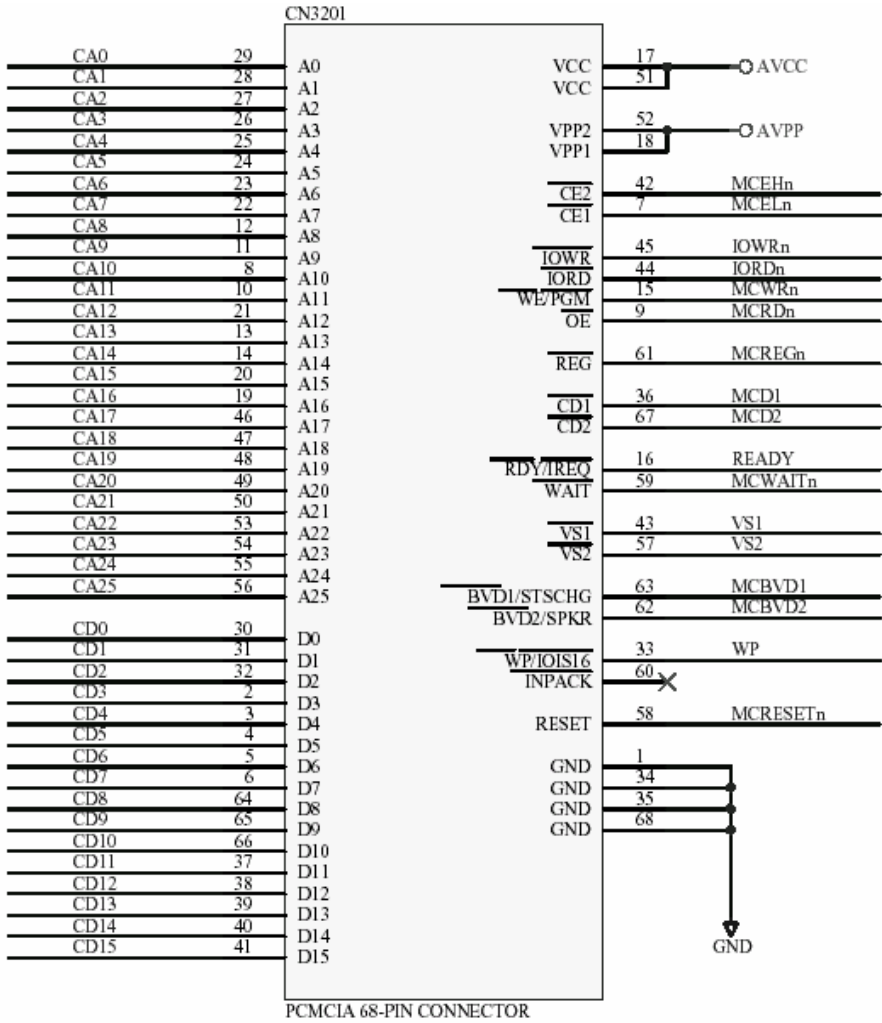
a PCMCIA 电源管理电路



b 地址总线驱动电路



c 数据总线驱动电路



d PCMCIA 连接器电路图

图 3—32 HDB9315 开发系统 PCMCIA 接口电路

图 3—32a 为 PCMCIA 接口的电源管理电路，TPS2202 为双口电源管理器件，我们在此仅使用 A 口；图 3—32b 为 PCMCIA 接口的地址总线驱动电路；图 3—32c 为 PCMCIA 接口的数据总线驱动电路；图 3—32d 为 PCMCIA 连接器电路图。

3.5 本章小结

本章详细介绍了基于 EP93XX 系列微处理器基本应用电路图，这些电路在 EP93XX 系列微处理器的系统设计中会经常使用到，读者也可以根据自身的特殊需求，做必要的修改。由于 EP93XX 系列微处理器在结构上具有很好的一致性，一些功能电路是在整个

EP93XX 系列微处理器系统的设计中可以通用的。

www.ensyschina.com

第 4 章 RedBoot 与 eCos

本章介绍基于 EP93XX 系列嵌入式微处理器系统的初始化过程, 以及 RedBoot 的使用方法, 同时介绍了 eCos 在嵌入式系统上的移植过程。作为一个实例, 本章还介绍了 eCos 在 EP93XX 系列微处理器上的移植及基于 eCos 的应用程序开发与调试过程。

本章主要包括:

- λ BootLoader 的相关概念;
- λ RedBoot 及常用命令的使用方法;
- λ 如何在 EP93XX 系列微处理器系统上运行 Redboot 并引导嵌入式 Linux 内核;
- λ eCos 的基本结构;
- λ eCos 在 EP93XX 系列微处理器上的移植及 Redboot 的编译方法;
- λ 基于 eCos 的应用程序开发与调试。

4.1 嵌入式系统的 BootLoader

4.1.1 BootLoader 的一般概念

对于任何一个计算机系统来说, 系统的启动都有一个初始化的过程, 嵌入式系统也概莫能外。完成系统初始化功能的程序在 PC 机上是由引导加载程序由 BIOS 和位于硬盘 MBR 中的 OS Boot Loader 一起构成的。BIOS 在完成硬件检测和资源分配后, 将硬盘 MBR 中的 Boot Loader 读到系统的 RAM 中, 然后将控制权交给 OS Boot Loader。Boot Loader 的主要任务就是将内核映象从硬盘上读到 RAM 中, 然后跳转到操作系统内核的入口点去运行, 也即开始启动操作系统。

早期的嵌入式系统, 如在基于 8/16 位微处理器的系统中, 由于使用比较简单的处理器, 运行的程序也相对简单, 系统的启动初始化代码一般和应用代码放在一起, 并没有 BootLoader 的概念。随着嵌入式应用系统的逐渐发展, 软硬件系统变得越来越复杂, 特别是在使用嵌入式操作系统的应用中, 系统从上电到嵌入式操作系统的运行, 需要比较多的代码完成初始化工作, 因此, 在这类复杂的嵌入式系统中, 就将系统的初始化代码单独组织, 同时加入一些方便系统设计开发的功能, 形成一段相对独立的代码, 这段代码就称为 Bootloader。

因此, 在比较复杂的嵌入式系统中, 系统的初始化工作就由 BootLoader 完成。BootLoader 初始化硬件设备、建立内存空间的映射, 从而将系统的软硬件环境配置为一个合适的状态, 以便为最终调用操作系统内核和应用程序运行。

不同于 PC 的一致性架构, 嵌入式系统是千差万别的, 而 Boot Loader 则严重地依赖于

硬件而实现的。因此，在嵌入式系统中，对于两个不同的嵌入式系统而言，即使它们是基于同一种嵌入式微处理器构建的，要想让运行在其中一个系统上的 Boot Loader 也能运行在另一个系统上，通常也都需要作一些修改，即不存在一个通用的 BootLoader。但尽管如此，BootLoader 仍然有一些共性的东西。

BootLoader 的存放位置

嵌入式系统上电或复位后，PC 指针通常都从某个特定地址读取第一条指令，例如，基于 ARM 架构的微处理器在复位时都从地址 0x0000_0000 读取，为使系统正常启动，必须将非易失性存储器（如 ROM、EEPROM 或 Flash 等）映射到这个地址上，BootLoader 就固化在这些非易失性存储器里。

BootLoader 的工作模式

大多数的 BootLoader 都支持两种不同的操作模式：即所谓“启动加载”模式和“下载”模式，嵌入式系统开发人员会使用这两种模式，但最终用户一般只使用“启动加载”模式，用来启动加载嵌入式操作系统。

启动加载模式是 BootLoader 的正常工作模式，此时 BootLoader 从嵌入式系统的非易失性存储器上将操作系统加载到系统 RAM 中并运行。

而下载模式主要用于在嵌入式系统的开发过程当中，在这种模式下，运行在嵌入式目标系统上的 BootLoader 将通过串口连接或网络连接等通信手段，从 Linux 服务器或其它主机下载诸如内核映像和根文件系统等文件。从 Linux 服务器或其它主机下载的文件通常首先被 BootLoader 保存到嵌入式目标系统的 RAM 存储器中，然后再由 BootLoader 烧写到嵌入式目标系统的非易失性存储设备中。

BootLoader 的下载模式通常用在嵌入式系统的开发过程和以后的系统更新，工作在这种模式下的 BootLoader 通常都会向用户提供一个简单的命令行接口，以便于用户进行各种系统操作。目前，常见的 BootLoader，如 Uboot 和 RedBoot 等，都支持这两种模式。

BootLoader 的通信协议

当 BootLoader 工作在下载模式时，需要频繁的在嵌入式目标系统和 Linux 服务器或其它主机之间下载文件，因此，BootLoader 必须能够支持常用的通讯协议。

几乎所有的 BootLoader 都支持串行通讯协议，但串行传输的速度毕竟有限，因此通过以太网连接并借助诸如 TFTP 等协议来下载文件是个更好的选择，Uboot 和 RedBoot 都支持这种方式。但由于受到嵌入式目标系统资源的限制，BootLoader 一般只能提供 TFTP 的客户端支持，而主机必须提供 TFTP 服务器支持。

4.1.2 BootLoader 的主要任务

显然，从系统整体的角度看，BootLoader 的总目标就是完成特定嵌入式系统的基本硬件初始化，并正确地调用操作系统内核或应用程序来执行。

为适应嵌入式系统的差异性，BootLoader 在结构上一般都可以分为与硬件相关和与硬件无关的两个部分。与硬件相关的部分完全依赖于嵌入式微处理器的体系结构和系统存储

器配置等，这一部分通常用汇编语言来实现，以达到高效的目的，同时，BootLoader 在不同系统上的移植，也主要是修改该部分。

与硬件无关的部分主要完成通信协议支持、嵌入式操作系统内核引导、以及命令行接口等，通常用 C 语言实现，这样不但可以实现更复杂的功能，而且代码会具有更好的移植性。

与硬件相关部分的任务

与硬件相关部分的任务主要完成嵌入式系统硬件设备的基本初始化工作，为下一步的系统运行准备好基本的硬件环境。它通常包含以下步骤：

1、屏蔽所有的中断

为使系统尽可能顺利启动运行，BootLoader 的执行全过程中一般都不必响应任何中断。中断屏蔽可以通过对嵌入式微处理器与中断控制相关的寄存器操作来完成。

2、设置系统的时钟频率

嵌入式系统在运行之初，一般以较低的时钟运行，当完成必要的系统初始化以后，系统才以较高的时钟运行。

3、初始化系统 RAM

高性能的嵌入式系统一般都包含大容量的 RAM 存储器，作为系统程序的运行空间，因此，需要正确地设置系统的 RAM 控制器的功能寄存器及控制寄存器等。

4、系统工作状态报告

系统工作状态报告的目的是表明系统运行状态如何，可以通过诸如驱动 GPIO 的 LED、向串口打印启动信息等方式完成。

5、启动嵌入式微处理器内部指令 Cache 和数据 Cache

指令 Cache 和数据 Cache 的分离设计是现代 RISC 体系的 CPU 普遍采用的设计方法，配合流水线结构，可以大大的提高指令和数据的访问速度。

需要注意的是，有不少嵌入式微处理器都在片内设计了 BootROM，如 EP93XX 系列、AT 91 系列等，也会完成一些基本的硬件系统初始化工作。

与硬件无关部分的任务

与硬件无关部分的任务主要完成通信协议支持、嵌入式操作系统内核引导、以及命令行接口等。无论是 Uboot 还是 Redboot，当它们运行于下载模式时，都支持以命令行方式与用户进行交互，命令功能从存储器的读写、Flash 的操作，到通信协议的支持等，几乎相当于一个简单的操作系统，而当它们运行于启动加载模式时，则可以引导嵌入式 Linux、VxWorks 和 WinCE 等嵌入式操作系统的运行，因此，BootLoader 与硬件无关的部分，更多的是关于功能方面的实现，并可以在不同的嵌入式系统上运行，即具有很好的移植性。

4.2 Redboot 简介及常用命令

4.2.1 Redboot 简介

本文所描述的基于 EP93XX 系列微处理器的嵌入式系统采用 Redboot 作为系统的 BootLoader。

Redboot (Red Hat Embedded Debug and Bootstrap) 是 Red Hat 公司开发的一个独立运行在嵌入式系统上的 Bootloader 程序, 是目前比较流行的一个功能强大、可移植性好的 BootLoader。

Redboot 是一个采用 eCos 开发环境开发的应用程序, 并采用了 eCos 的硬件抽象层作为基础, 但它完全可以脱离 eCos 环境运行, 可以用来引导任何其他的嵌入式操作系统, 如 Linux、Wince 等。

关于 eCos 更详细的内容, 我们将在本章稍后的部分详细介绍。

Redboot 除了一般 Bootloader 的硬件初始化和引导内核的功能外, 还支持下列功能:

- λ 支持引导脚本, 可方便启动应用程序或嵌入式操作系统内核;
- λ 提供完整的命令行接口, 方便用户进行各种系统操作;
- λ 支持串行通讯协议和网络通性协议;
- λ 支持 GDB 调试, 内嵌 GDB stub;
- λ 支持 Flash 映像文件系统;
- λ 通过 BOOTP 协议支持网络引导, 也可以配置静态 IP。

4.2.2 Redboot 命令集

Redboot 提供了功能完善的命令行接口, 包括对 Flash 的操作、对串口或网络的配置、内存管理以及文件下载等, 在嵌入式系统的开发过程中, 需要经常使用这些命令。

Redboot 所支持的命令也会根据嵌入式系统硬件平台的不同而略有差异, 例如硬件平台没有 Flash, Redboot 就不会支持所有关于 Flash 的操作, 同时, 由于 Redboot 是源代码开放的, 用户甚至可以根据特定系统的需求, 添加新的命令。

以下仅对运行在基于 EP93XX 系列微处理器系统上的 Redboot 命令的使用与功能进行简单说明, 并不完全包含 Redboot 的所有命令, 关于 Redboot 命令集的完整描述, 可以参考相关技术文档。

cache

使用格式: cache [on | off]

功能描述: cache 命令用于管理微处理器的 cache, 当不带参数时, Redboot 将显示系统当前 cache 状态; 有参数时, 则根据参数值来设置 cache 的状态。

使用示例:

RedBoot>cache //显示系统 cache 的当前状态;

channel

使用格式: channel [-l | channel number]

功能描述: 如果不带任何参数, channel 命令会显示当前的控制台通道号, 若参数为 -l 时, 该命令将控制台通道切换到默认的控制台通道, 若参数为硬件平台所支持的其他控制台号, 则 channel 命令就对控制台作相应的切换。

使用示例:

RedBoot>channel // 显示系统的当前控制台通道号;

dump

使用格式: dump {-b location} [-l length] [-s] [-l | -2 | -4]

功能描述: 显示参数指定区域的数据, 显示方式由参数指定。其中:

- b location: 存储器的起始位置;
- l length: 显示的长度;
- s: 使用 Motorola S-reconds 格式显示数据;
- l: 按单字节显示数据;
- 2: 按双字节显示数据;
- 4: 按四字节显示数据;

使用示例:

RedBoot>dump -b 0x0 -l // 显示起始位置为 0x0 的存储器内容, 按单字节显示;

exec

使用格式: exec [-w timeout] [-r ramdisk_address] [-s ramdisk_length] [-b load_address {-l load_length}] [-c kernel_command_line] [entry_point]

功能描述: 执行一个映像文件, 如引导 Linux 内核。

- w timeout: 执行映像文件之前的等待时间;
- r ramdisk_address: 传递给内核的 ramdisk_address 起始地址;
- s ramdisk_length: 传递给内核的 ramdisk_address 长度;
- b load_address: 内核映像文件地址;
- l load_length: 内核映像文件长度;
- c kernel_command_line: 传递给内核的命令行;

fis creat

使用格式: fis create {-b data address} {-l length} [-f flash address] [-e entry] [-r relocation address] [-s data length] [-n] [name]

功能描述: 在 FIS (Flash Image System) 目录中创建一个映像, 将当前 RAM 中的数据写入 Flash 存储器中, 因此, 在使用该命令之前, 映像文件数据必须已经保存在 RAM 中。

-b data address: 待写入 Flash 数据的存放地址;
-l length: 写入数据的长度;
-f flash address: Flash 地址;
-e entry: 可执行映像地址;
-r relocation address: 执行 Fis load 命令时, 可执行映像的重定位地址;
-s data length: 写入 Flash 中的可执行映像实际长度;
-n: 用于更新 FIS 目录;
name: 创建映像的名称。

使用示例:

```
RedBoot>fis create -b 0x1000000 -l 0x1a5179 ramdisk.gz
```

// 在 FIS 目录中创建一个名为 ramdisk.gz 的映像, 并将地址为 0x1000000 的 RAM 的内容写入该映像, 写入长度为 0x1a5179;

fis init

使用格式: fis init [-f]

功能描述: 初始化 FIS 目录。

-f: 将所有 Flash 空间进行初始化;

使用示例:

```
RedBoot>fis init // 初始化 FIS 目录。
```

fis list

使用格式: fis list [-c] [-d]

功能描述: 显示 FIS 中当前的所有映像文件。

-c: 显示映像的校验和;

-d: 显示映像的长度;

使用示例:

```
RedBoot>fis list // 显示 FIS 中当前的所有映像文件。
```

fis free

使用格式: fis free

功能描述: 显示 Flash 当前的空闲空间。

使用示例:

```
RedBoot>fis free // 显示 Flash 当前的空闲空间。
```

fis delete

使用格式: fis delete {name}

功能描述: 删除 FIS 目录中的映像。

name: 需要删除映像的名称;

使用示例:

RedBoot>fis delete ramdisk.gz // 删除 FIS 目录中名为 ramdisk.gz 的映像。

fis lock

使用格式: fis lock {-f flash_address} {-l length}

功能描述: 锁定 Flash 空间。

-f flash_address: 锁定 Flash 空间的起始地址;

-l length: 锁定 Flash 空间的长度;

使用示例:

RedBoot>fis lock -f 0x60000000 -l 0x2000 // 锁定起始地址为 0x60000000, 长度为 0x2000 的 Flash 空间。

fis unlock

使用格式: fis unlock {-f flash_address} {-l length}

功能描述: 解除 Flash 空间的锁定, 使用方式与 fis lock 类似。

fis erase

使用格式: fis erase {-f flash_address} {-l length}

功能功能: 擦除指定的 Flash 空间。

-f flash_address: 指定需要擦除 Flash 空间的起始地址;

-l length: 需要擦除 Flash 空间的长度;

使用示例:

RedBoot>fis erase -f 0x60000000 -l 0x2000 // 擦除起始地址为 0x60000000, 长度为 0x2000 的 Flash 空间。

fis write

使用格式: fis write {-b mem_address} {-l length} {-f flash_address}

功能描述: 将数据由 RAM 写入 flash 中。

-b mem_address: 待写数据在 RAM 中的起始地址;

-l length: 待写数据的长度;

-f flash_address: 写入 Flash 的起始地址;

使用示例:

RedBoot>fis write -b 0x00004000 -l 0x2000 -f 0x60000000 // 将 RAM 中起始地址为 0x00004000, 长度为 0x2000 的数据写入起始地址为 0x60000000 的 Flash 中。

fconfig

使用格式: fconfig

功能描述: 对已保存在 Flash 中的配置选项进行管理和重配置。

使用示例:

RedBoot>fconfig // 对 Flash 中的配置选项进行管理和重配置。

go

使用格式: go [-w <timeout>] [start_address]

功能描述: 执行放在某一位置的可执行代码。

-w <timeout>: 执行代码前的等待时间;

start_address: 可执行代码的起始地址;

使用示例:

RedBoot>go 0x00002000 // 执行位于 0x00002000 的代码。

ip_address

使用格式: ip_address [-l <local_ip_address>] [-h <server_ip_address>] [-d <DNS_server_ip_address>]

功能描述: 设置或改变系统使用的 IP 地址。

-l <local_ip_address>: 本机的 IP 地址;

-h <server_ip_address>: 服务器 IP 地址;

-d <DNS_server_ip_address>: DNS 服务器 IP 地址;

使用示例:

RedBoot>ip_address -l 192.168.67.49 -h 192.168.67.1 // 设置本机 IP 地址为: 192.168.67.49, 服务器 IP 地址为: 192.168.67.1。

load

使用格式: load [-r] [-v] [-h <host>] [-m <varies>] [-c <channel_number>] [-b <base_address>] <file_name>

功能描述: 下载数据到目标系统 RAM 中。

-r: 下载未处理的数据到 RAM;

-v: 下载过程显示进度;

-b <base_address>: 数据下载到 RAM 的地址;

file_name: 下载的文件名;

使用示例:

RedBoot>load -v -r -b 0x1000000 ramdisk.gz // 下载文件 ramdisk.gz 到地址为 0x1000000 的 RAM 中。

mcmp

使用格式: mcmp {-s location} {-d location} {-l length} [-1|-2|-4]

功能描述: 比较两个存储区域的内容。

-s location: 源区域起始地址;

-d location: 目的区域起始地址;

-l length: 需要比较数据的长度;

-1: 单字节读取;

-2: 双字节读取;

-4: 四字节读取;

使用示例:

RedBoot>mcmp -s 0x00000000 -d 0x10000000 -l 0x20 // 比较存储器起始地址为 0x00000000 和 0x10000000 的内容, 共比较 32 个数据。

mcopy

使用格式: mcopy {-s location} {-d location} {-l length} [-1|-2|-4]

功能描述: 将数据从一个存储区域拷贝到另一个存储区域。

-s location: 源区域起始地址;

-d location: 目的区域起始地址;

-l length: 需要拷贝数据的长度;

-1: 单字节读取;

-2: 双字节读取;

-4: 四字节读取;

使用示例:

RedBoot>mcopy -s 0x00000000 -d 0x10000000 -l 0x20 // 将存储器起始地址为 0x00000000 的连续 32 个数据拷贝到起始地址为 0x10000000 位置。

mfill

使用格式: mfill {-b location} {-l length} {-p value} [-1|-2|-4]

功能描述: 将给定的数值填充到指定的存储区域。

-b location: 待填充存储区域的起始地址;

-l length: 需要拷贝数据的长度;

-p value: 填充的数值;

-1: 单字节操作;

-2: 双字节操作;

-4: 四字节操作;

使用示例:

RedBoot>mfill -b 0x50000 -l 0x20 -p 0xaaaaaaaa // 将给定数值 0xaaaaaaaa 填充到起始地址为 0x50000 位置, 共填充 32 个字节。

reset

使用格式: reset

功能描述: 复位系统。

使用示例:

RedBoot>reset // 复位系统。

ping

使用格式: ping [-v] [-n <count>] [-l <length>] [-t <timeout>] [-r <rate>]
[-i <IP_addr>] -h <IP_addr>

功能描述: 该命令向指定主机发送 ICMP 报文, 用于检查网络连接是否正常。

- v: 显示数据包信息;
- n <count>: 发送数据包的数目;
- l <length>: 发送报文的长度;
- t <timeout>: 设置超时时间;
- r <rate>: 发送数据包的时间间隔;
- i <IP_addr>: 本机 IP 地址;
- h <IP_addr>: 远端主机 IP 地址;

使用示例:

RedBoot> ping -v -n 5 -h 192.168.67.1 // 向 IP 地址为: 192.168.67.1 的主机发送 5 个数据包。

help

使用格式: help [<topic>]

功能描述: 显示 redboot 中可用命令的帮助信息。

使用示例:

RedBoot> help // 显示当前 Redboot 可用命令的帮助信息。

version

使用格式: version

功能描述: 显示 Redboot 的当前版本信息。

使用示例:

RedBoot> version // 显示 Redboot 的当前版本信息。

以上介绍了运行在本文所描述的基于 EP93XX 系列微处理器的系统上的 Redboot 的常用命令, 对于移植到不同嵌入式系统的 Redboot, 所支持的命令及命令参数都略有差异, 请读者注意区分。

4.3 在基于 EP93XX 的系统上运行 Redboot 及嵌入式 Linux

4.3.1 运行 Redboot

当用户完成了一个基于 EP93XX 系列微处理器嵌入式系统硬件平台的设计与调试以后,接着就可以考虑将 Redboot 下载烧写到嵌入式系统的 Flash 存储器并运行,当 Redboot 可以在嵌入式系统中运行以后,用户就可以通过 Redboot 所支持的命令,下载嵌入式操作系统内核和应用程序等到嵌入式目标系统,最终形成一个可以独立运行的嵌入式系统。

为方便用户使用, Cirrus Logic 公司已将 Redboot 移植到了 EP93XX 系列微处理器,同时提供了将 Redboot 下载烧写到系统 Flash 的工具,用户只需要使用 Flash 烧写工具,将移植到 EP93XX 系列微处理器上的 Redboot 可执行映像下载烧写到系统的 Flash 即可,如果用户需要深入了解 Redboot,可以阅读 Redboot 的实现代码。

以下以基于 EP9301/02 的嵌入式系统为例,简要说明如何将 Redboot 下载烧写到系统的 Flash 存储器,基于 EP9307、EP9312 和 EP9315 系统的操作方式与之类似。

如前文所述, EP93XX 系列微处理器支持芯片内部的 BootROM 启动方式,系统上电或复位以后, EP93XX 系列微处理器首先从片内 BootROM 启动,片内 BootROM 的物理地址为 0x8009_0000,当系统配置为内部启动时,这个地址就被映射到了 0x0000_0000 处,因此系统首先运行固化在 BootROM 内部的代码。

BootROM 代码执行后,继续选择从以下设备读取程序运行:

- λ SPI 接口的 EEPROM;
- λ FLASH/SyncFLASH 或 SRAM;
- λ 串行接口 UART1;

其具体过程描述可以参考第二章的相关内容。

在系统的初始状态, EEPROM 和 Flash 存储器均为空,因此只能接收从串口 UART1 下载的程序代码运行,但代码长度不能超过 2KB,这些代码被 BootROM 启动程序拷贝到片内的 SRAM 并执行。

而需要下载到嵌入式系统的 Redboot 可执行映像远大于 2KB,因此,需要一个运行在片内 SRAM 中的下载程序,能完成从串口接收大于 2KB 的程序,并将其烧写到系统的 Flash 中。Cirrus Logic 所提供的下载烧写工具就完成这个功能。

以 HDB9301 开发系统为例,具体步骤描述如下:

- 1、设置嵌入式目标系统的启动方式为串行启动模式。

参考第二章相关内容,设置嵌入式目标系统的系统启动配置模式为 16 位串行启动。

- 2、把编译好的 Redboot 可执行映像文件通过串口下载并烧写到系统的 Flash 存储器中。

Redboot 的可执行映像需要在 Linux 服务器上编译生成,我们在本章稍后的部分描述具体编译步骤,在此仅说明如 Redboot 可执行映像的下载烧写过程。

Cirrus Logic 提供的下载烧写工具文件名为 download，将该文件和 Redboot 的可执行映像文件 redboot.bin 拷贝到 Linux 服务器的某个目录。

将 Linux 服务器的 COM1 与 HDB9301 的串口 UART1 连接，然后在 Linux 服务器的当前目录执行如下命令，运行下载烧写程序下载 Redboot 的可执行映像文件 redboot.bin：

```
#./download redboot.bin
```

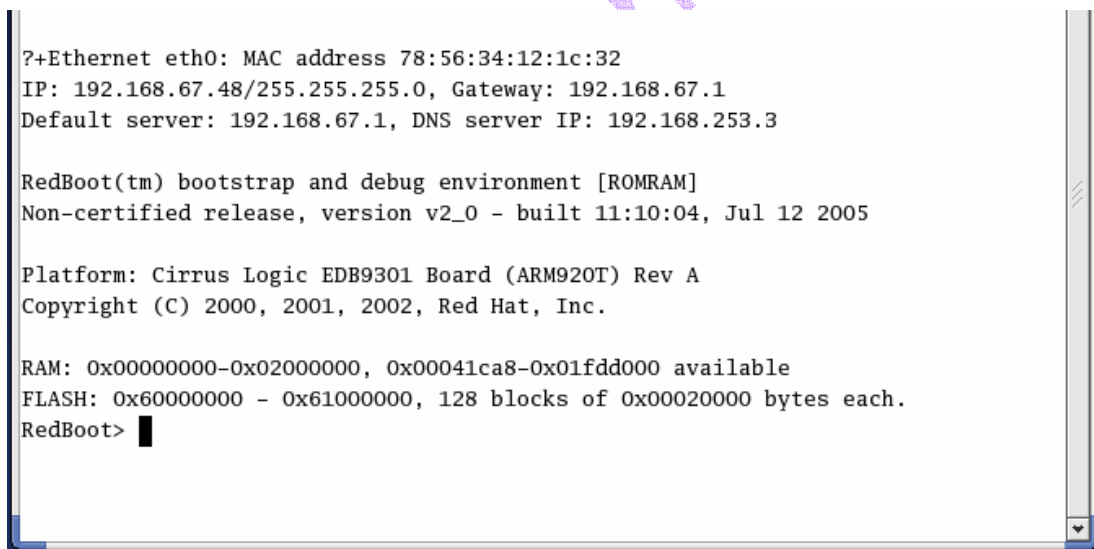
程序运行，提示“Waiting for the board to wake up...”，等待与 HDB9301 开发系统进行通信，此时给 HDB9301 开发系统上电，片内 BootROM 代码运行，开始 redboot.bin 从 Linux 服务器下载并烧写到 HDB9301 开发系统 Flash 的过程。

当烧写完毕以后，显示如下信息：“Successfully programmed 'redboot.bin'”，表示下载烧写过程正确完成。

3、启动运行 Redboot。

在 Linux 服务器上使用 minicom，或在其他 Windows 平台上使用超级终端，配置串行通信协议为波特率：57600，数据位：8，无校验，停止位：1。并将 HDB9301 的 UART1 与 PC 对应的串口连接，同时将系统的启动模式配置为从系统 Flash 启动的方式。

重新启动 HDB9301 系统，烧写在 Flash 中的 Redboot 开始运行，出现命令行接口，支持与用户的交互，如图 4-1 所示。



```
?+Ethernet eth0: MAC address 78:56:34:12:1c:32
IP: 192.168.67.48/255.255.255.0, Gateway: 192.168.67.1
Default server: 192.168.67.1, DNS server IP: 192.168.253.3

RedBoot(tm) bootstrap and debug environment [ROMRAM]
Non-certified release, version v2_0 - built 11:10:04, Jul 12 2005

Platform: Cirrus Logic EDB9301 Board (ARM920T) Rev A
Copyright (C) 2000, 2001, 2002, Red Hat, Inc.

RAM: 0x00000000-0x02000000, 0x00041ca8-0x01fdd000 available
FLASH: 0x60000000 - 0x61000000, 128 blocks of 0x00020000 bytes each.
RedBoot> █
```

图 4-1 Redboot 的命令行接口

用户接着需要做的工作是通过使用 Redboot 提供的命令，完成诸如 Flash 文件系统的初始化、嵌入式 Linux 内核及根文件系统的下载等工作，完成最终支持嵌入式 Linux 的运行。

4、初始化 HDB9301 的 Flash 文件系统。

使用 Redboot 所支持的命令，对 HDB9301 的 Flash 文件系统进行初始化。在 Redboot

的命令提示符下输入如下命令：

```
Redboot> fis init -f
```

命令执行以后，显示如下信息，Redboot 对系统 Flash 的扇区进行擦除，并初始化 Flash 文件系统。

```
*** Initialize FLASH Image System
... Erase from 0x60040000-0x60fc0000: ...
... Erase from 0x60fe0000-0x60fe0000: ...
... Erase from 0x61000000-0x61000000: ...
... Erase from 0x60fe0000-0x61000000: ...
... Program from 0x01fdf000-0x01fff000 at 0x60fe0000: ...
```

5、网络配置和测试。

使用 Redboot 所支持的命令，进行网络配置与测试。在 Redboot 的命令提示符下输入如下命令配置 HDB9301 的 IP 地址和远程主机的 IP 地址：

```
Redboot>ip_address -l 192.168.67.49 -h 192.168.67.1
```

命令中的 IP 地址分别为 HDB9301 的 IP 地址（192.168.67.49）和远程主机的 IP 地址（192.168.67.1）

将 HDB9301 连接到局域网中，测试系统的网络连接。

```
Redboot>ping -v -n 5 -h 192.168.67.1
```

若系统网络工作正常，可以收到远程主机的应答信息。

4.3.2 使用 Redboot 加载运行嵌入式 Linux

当用户完成了 Redboot 下载到 HDB9301 系统并能正确运行，同时完成了 Flash 文件系统的建立及网络测试，就可以下载嵌入式 Linux 操作系统到 HDB9301 运行了。

在嵌入式系统上运行的 Linux 操作系统由两部分构成：内核可执行映像和根文件系统。关于它们更详细的内容以及生成方法，在第五章有专门章节描述，在此仅说明如何将已生成的内核可执行映像和根文件系统下载到 HDB9301 系统运行的过程。

运行在 HDB9301 系统上的内核可执行映像文件和根文件系统的压缩文件分别叫做 zImage 和 ramdisk.gz，我们采用如下步骤将它们下载到 HDB9301 系统。

1、运行 TFTP 服务。

由于 zImage 和 ramdisk.gz 文件都比较大，一般采用 TFTP 方式通过网络下载，Redboot 支持 TFTP 客户端功能，需要在主机上运行 TFTP 服务器，有很多可以运行在 Windows 平台上的 TFTP 服务器软件，如 TFTPDSRV 等。

将 TFTPDSRV 和 zImage、ramdisk.gz 拷贝到主机的同一个目录下，然后运行 TFTP 服务器，等待 HDB9301 系统的 TFTP 发送请求。

2、下载根文件系统 ramdisk.gz 到 HDB9301 系统 SDRAM 并烧写到 Flash 中。

启动 HDB9301 系统的 Redboot，执行如下命令下载 ramdisk.gz：

```
Redboot>load -v -r -b 0x1000000 ramdisk.gz
```

其中 0x1000000 为 HDB9301 系统 SDRAM 的地址，文件下载完毕以后提示信息如下：

```
Raw file loaded 0x01000000-0x011a5179, assumed entry at 0x01000000.
```

系统显示接收到的文件存放在存储器 0x01000000~0x011a5179 的区域，文件大小为 0x1a5179 字节。

执行如下命令将存放在 SDRAM 中的文件拷贝到 HDB9301 的 Flash 文件系统中：

```
Redboot>fis create -b 0x1000000 -l 0x1a5179 ramdisk.gz
```

3、以同样的操作下载Linux内核文件zImage到HDB9301系统SDRAM并烧写到Flash中。

```
Redboot>load -v -r -b 0x218000 zImage
```

其中 0x218000 为 HDB9301 系统 SDRAM 的地址，文件下载完毕以后提示信息如下：

```
Raw file loaded 0x00218000-0x002ca7ef, assumed entry at 0x00218000.
```

将其保存在 Flash 文件系统中：

```
Redboot>fis create -b 0x218000 -l 0xB27EF zImage
```

4、用fis list命令列出FIS目录，可以看到ramdisk.gz和zImage在Flash中的存储位置。

```
RedBoot> fis list
```

Name	FLASH addr	Mem addr	Length	Entry point
RedBoot	0x60000000	0x60000000	0x00040000	0x00000000
RedBoot config	0x60FC0000	0x60FC0000	0x00001000	0x00000000
FIS directory	0x60FE0000	0x60FE0000	0x00020000	0x00000000
ramdisk.gz	0x60040000	0x01000000	0x001C0000	0x01000000
zImage	0x60200000	0x00218000	0x000C0000	0x00218000

5、启动 Linux 系统。

当 zImage 和 ramdisk.gz 下载到 HDB9301 系统的 Flash 中后，就可以使用如下命令运行嵌入式 Linux：

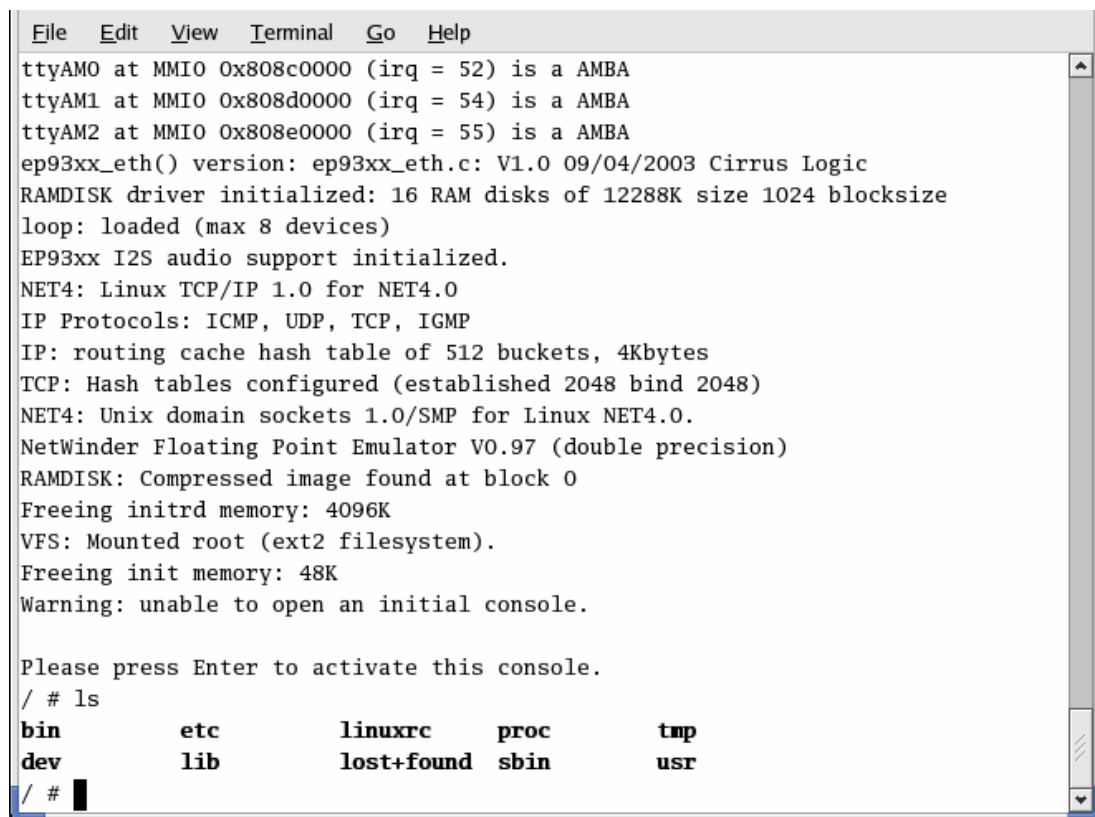
```
Redboot>fis load ramdisk.gz // 把 ramdisk.gz 从 Flash 拷贝到 SDRAM 中；
```

```
Redboot>fis load zImage // 把 zImage 从 Flash 拷贝到 SDRAM 中；
```

```
Redboot>exec -r 0x1000000 0x218000 // 运行 Linux 操作系统；
```

当命令执行完毕以后，HDB9301 系统中的嵌入式 Linux 操作系统开始运行，如图 4-2 所示。

以上命令也可以写成命令行脚本，由 Redboot 自动执行。



```
File Edit View Terminal Go Help
ttyAM0 at MMIO 0x808c0000 (irq = 52) is a AMBA
ttyAM1 at MMIO 0x808d0000 (irq = 54) is a AMBA
ttyAM2 at MMIO 0x808e0000 (irq = 55) is a AMBA
ep93xx_eth() version: ep93xx_eth.c: V1.0 09/04/2003 Cirrus Logic
RAMDISK driver initialized: 16 RAM disks of 12288K size 1024 blocksize
loop: loaded (max 8 devices)
EP93xx I2S audio support initialized.
NET4: Linux TCP/IP 1.0 for NET4.0
IP Protocols: ICMP, UDP, TCP, IGMP
IP: routing cache hash table of 512 buckets, 4Kbytes
TCP: Hash tables configured (established 2048 bind 2048)
NET4: Unix domain sockets 1.0/SMP for Linux NET4.0.
NetWinder Floating Point Emulator V0.97 (double precision)
RAMDISK: Compressed image found at block 0
Freeing initrd memory: 4096K
VFS: Mounted root (ext2 filesystem).
Freeing init memory: 48K
Warning: unable to open an initial console.

Please press Enter to activate this console.
/ # ls
bin          etc          linuxrc      proc          tmp
dev          lib          lost+found  /sbin         usr
/ #
```

图 4-2 Redboot 启动嵌入式 Linux 运行

我们在以上的内容描述了 Redboot 的基本特性，以及通过 Redboot 引导嵌入式 Linux 启动的方法，关于嵌入式 Linux 内核可以执行映像的编译和根文件系统的制作方法，将在第五章讲述。

Redboot 作为一个源代码开放、功能强大的 BootLoader，已经在很多嵌入式系统上运行，同时，Redboot 与嵌入式实时操作系统 eCos 有着密不可分的联系，我们将在随后的部分简单介绍 Redboot 的实现方法、eCos 嵌入式实时操作系统和基于 eCos 的应用程序开发流程。

4.4 eCos 概述与特性

4.4.1 eCos 概述

自 Ready System 在 1981 年推出世界上第一个商业嵌入式实时内核（VRTX32）以来，嵌入式实时操作系统的开发已经成为操作系统开发领域一个非常重要和极具活力的分支，迄今市场上已面世的商业嵌入式实时操作系统已有上百种。

与传统桌面 PC 操作系统市场不同,没有那种嵌入式实时操作系统在这一市场占据明显优势,这也就为许多高效、高质量的新型嵌入式操作系统解决方案的诞生和发展创造了有利的条件。

eCos (Embedded Configurable Operating System), 嵌入式可配置操作系统,就是 Cygnus 公司 1998 年推出的 RTOS 解决方案。Cygnus 公司设计 eCos 的目的,是希望向用户提供一种具有高度适应性的嵌入式操作系统,即用户可以利用系统提供的配置工具,对操作系统进行源代码级别的裁减配置,通过实现不同的配置组合来建立起各种需要的特定系统,它们共享一个操作系统源码库。

eCos 自 1998 年推出以来,其发展也颇经历了一番曲折,Cygnus 公司于 1998 年 11 月发布了第一个 eCos 版本 eCos1.1,当时只支持有限的几种处理器结构。1999 年 11 月,Cygnus 公司被 Red Hat 公司收购,作为 Red Hat 的嵌入式部门继续 eCos 的开发,在此后的几年里,eCos 得到了迅速的发展,如在 eCos 中加入了对嵌入式调试和对 Redboot 的支持,成为了 Red Hat 公司嵌入式领域的关键产品。2002 年,RedHat 公司由于财务方面的原因,裁剪了 eCos 开发队伍,但并没有停止 eCos 的发展。而原 eCos 主要开发人员又组建了 eCos Centric 公司,继续 eCos 的开发和技术支持工作,并于 2003 年 5 月正式发布了 eCos2.0。2004 年 1 月,Red Hat 公司宣布将其对于 eCos 的权利转移给 FSF (Free Software Foundation),但其将继续支持 eCos 的发展。

不过这一切,并未减弱 eCos 拥护者对于 eCos 的信心和动力,而且其优良的性能已获得了越来越多用户的认可和肯定,并在数字通信、无线系统和消费类电子产品等领域内有了许多成功的商业应用。

4.4.2 eCos 特性分析

与绝大多数嵌入式实时操作系统相比较,eCos 在系统可配置性、源码开放性以及功能方面的某些特性上具有较为明显的优势,这也是 eCos 取得成功的主要原因。

eCos 的主要特性可以描述如下:

可配置性

eCos 最为核心的特性在于它的完全可配置性。所谓的可配置性就是用户可以通过选择系统的不同组件来形成应用于不同嵌入式设备的特定系统。以前的嵌入式系统开发一般都是开发人员通过调整应用程序来适用底层的操作系统,而 eCos 的出现,让这一常规模式有所改变。

在嵌入式应用开发中,一个最应遵循的原则就是在满足所需功能和可靠性的基础上,系统应尽可能简洁,这一原则是由嵌入式系统的应用特性所决定的。目前的许多嵌入式操作系统,有大量的代码利用率极低,对于大多数嵌入式应用来说,这些额外代码的存在不仅增加了系统开销,同时降低了系统的可靠性。

eCos 系统则通过提供大范围的可配置选项,实现了系统源代码级的可配置,使用户通过选择系统所提供的可配置组件,来实现对特定应用的重构。eCos 同时提供了强大的配置

工具，用户可以方便的使用该配置工具，针对特定应用进行配置，即可以通过调整操作系统来适应应用需求。

eCos 操作系统源码库是一个可重用的组件库，用户不仅可以利用配置工具方便地建立自定义系统，还可以利用配置工具对组件库进行管理，如对组件库中的组件包进行添加，删减等操作。此外，用户还可以建立自己的配置包，当这些配置包发行后，其他用户只要将这些配置包添加到自己的组件库中后，就可以方便地使用，很好的提高了系统的可重用性。

源码开放

eCos 操作系统遵循与 GPL 兼容的 GPL-compatible Free Software License 认证，用户可以在产品开发中免费的，自由的使用 eCos 操作系统，包括其开发工具；除涉及到对 eCos 系统本身的修改以外，用户的应用程序可以不公开，这一点既显著减少了开发者的开发成本，又最大限度的保护了开发者的利益，因而使得越来越多的用户在产品开发中选择了 eCos 操作系统。

功能特性

eCos 系统除了具有上面所说完全可配置性和源码开放特性，对嵌入式系统开发中涉及到许多特定技术需求也有很好的支持。

- λ eCos 的调度内核是一个嵌入式硬实时内核，并且内核设计时在调度策略选择、防止优先级反转、中断与异常的处理模式、中断处理的最短延迟以及内存分配方式等方面都做了许多精巧的设计，使其在系统的实时性能上有了良好的表现。
- λ eCos 的硬件抽象层 HAL (Hardware Abstraction Layer) 通过向其他组件提供标准的 API 函数的机制，屏蔽了底层的 CPU 架构与平台的特定信息。这样用户只需编写少量代码就能方便地将 eCos 移植到自己的特定平台上，因此提高了 eCos 的移植能力，迄今为止 eCos 已经成功地移植到了 ARM、Fujitsu FR-V、Hitachi H8/300、Intel x86、Matsushita AM3x、MIPS、NEC V8xx、PowerPC、Samsung CalmRISC16/32、SPARC、SPARClite、SuperH 等平台上。最新的硬件支持列表可参考如下站点：<http://sources.redhat.com/ecos/hardware.html>。
- λ ISO 标准 C 库和 math 库支持，兼容各种常见 C 函数和数学函数调用。
- λ 支持 POSIX 和 μ ITRON3.0 两种类型的兼容层，使许多在不同平台和系统上开发的应用程序可以比较容易的移植到 eCos 系统上。如在嵌入式 Linux 上开发的应用程序几乎可以不加改动移植到 eCos 系统运行，大大方便了 eCos 系统上的应用程序开发。
- λ 支持完整的 TCP/IP 协议栈，以及上层的各种常用标准协议，便于嵌入式设备的网络连接。
- λ 支持多种外部设备，包括标准串行设备、以太网控制器、Flash 存储器、USB 设备、LCD 显示设备以及 PCI 设备等。
- λ 支持 GDB 调试。eCos 系统中内嵌了 GDB stub，通过使用 GDB stub 与 GDB 主机进行通信，用户可以在不使用仿真器的情况下，通过串口或网络对嵌入式目标系统

上应用程序进行调试。

λ 支持多种文件系统，尤其对 JFFS2 文件系统支持良好。

基于 eCos 嵌入式实时操作系统系统的以上特性，将会在 RTOS 领域得到更广泛的应用。

4.5 eCos 开发环境的建立

为实现 eCos 系统的自身配置及基于 eCos 系统应用系统的开发，建立和拥有一个完备的 eCos 开发环境是必需的。

一个简单的嵌入式系统的应用开发环境一般是由嵌入式目标系统和 PC 主机构成，用户在 PC 主机进行嵌入式操作系统和应用程序的开发，然后将应用程序下载到目标系统调试和运行。eCos 系统的开发环境也采用这种方式，PC 主机既可以运行 Linux 操作系统，也可以运行 Windows 操作系统。在 Linux 系统下，建立 eCos 开发环境比较简单，但对于大多数用户来说，对 Windows 环境无疑是更为熟悉的，

我们以在 Windows 操作系统建立 eCos 开发环境为例，说明开发环境建立的具体过程，同时说明在 Linux 操作系统建立 eCos 开发环境的不同之处。

4.5.1 Cygwin 软件的安装

Cygwin 也是 Cygnus 公司开发的一个自由软件，它是一个在 Windows 平台上运行的 Unix 模拟环境。Cygnus 在开发 eCos 之初，就把它自己开发的 GNUPro 工具包作为 eCos 开发的必要工具链，而 GNUPro 工具包一般都是运行在 Unix 环境下的，因此如果在 Windows 下进行 eCos 系统的开发，需要安装 Cygwin，模拟出一个 Unix 环境。但如果在 Linux 系统下构造 eCos 开发环境，则不需要安装 Cygwin。

可以从以下站点获得相关的 Cygwin 软件包：<http://cygwin.com/>。

下载 Cygwin，并安装到 Windows 主机。

4.5.2 eCos 源码的安装

eCos2.0 系统提供的安装工具可以让用户自由选择安装相关的开发工具包，并自动进行下载安装。

我们约定将 eCos 源码安装到 Cygwin 目录体系中的 opt/ecos 下，如在 Linux 环境下，则直接安装在 /opt/ecos 目录，以便 ecos 安装目录可以被 PC 上所有用户访问和使用。

通过 Cygwin 的操作终端或 Linux 系统终端进入 /opt/ecos 目录，键入如下命令：

```
$ wget --passive-ftp ftp://ecos.sourceforge.org/pub/ecos/ecos-install.tcl
```

该命令自动下载 eCos 的安装工具到当前目录，然后输入下面的命令启动安装工具：

```
$ sh ecos-install.tcl
```

安装过程开始后，会首先给用户显示一个提供 eCos 代码下载的镜像站点的列表，选择站点后回车；然后选择 eCos 的安装目录，建议回车选择默认；接着会显示一个列表让用户

选择安装相应的交叉开发工具包，对于本文所描述的 ARM 开发来说，选择 arm-elf 即可，当选择好工具包后，键入 q，安装工具就会自动开始下载安装直到安装完成。

4.5.3 相关工具的安装

以下两个工具对于建立 eCos 的开发环境并不是必须的，但它们有助于 eCos 的开发。

configtool-2.11-setup.exe

configtool-2.11-setup.exe 是一个独立运行在 Windows 平台上 eCos 的图形配置工具，虽然在 eCos 安装源码中已经包括了图形配置工具 configtool，但建议安装并使用该工具。

Insight

Insight 是一个图形化的 GDB 调试器，它继承了 Windows 的操作风格，用户使用起来非常方便，我们在随后的系统调试会使用该工具。

4.5.4 运行 eCos 开发环境

eCos 开发环境在 Linux 平台下的运行方式与在 Windows 平台略有差异。

在 Linux 平台运行 eCos 开发环境

切换到在 Linux 主机上我们所约定的安装目录：/opt/ecos，键入如下命令：

```
$ ecosenv.sh
```

```
$ export PATH=/opt/ecos/ecos_2.0/tools/bin:$PATH
```

以上命令对 eCos 组件库的位置及交叉编译工具链的相关路径进行配置，如果在安装 eCos 时选择了不同的安装位置，需要对路径名作相应调整。

再键入 configtool 命令，运行图形化的 eCos 配置工具，如图 4-3 所示。

```
$ configtool
```

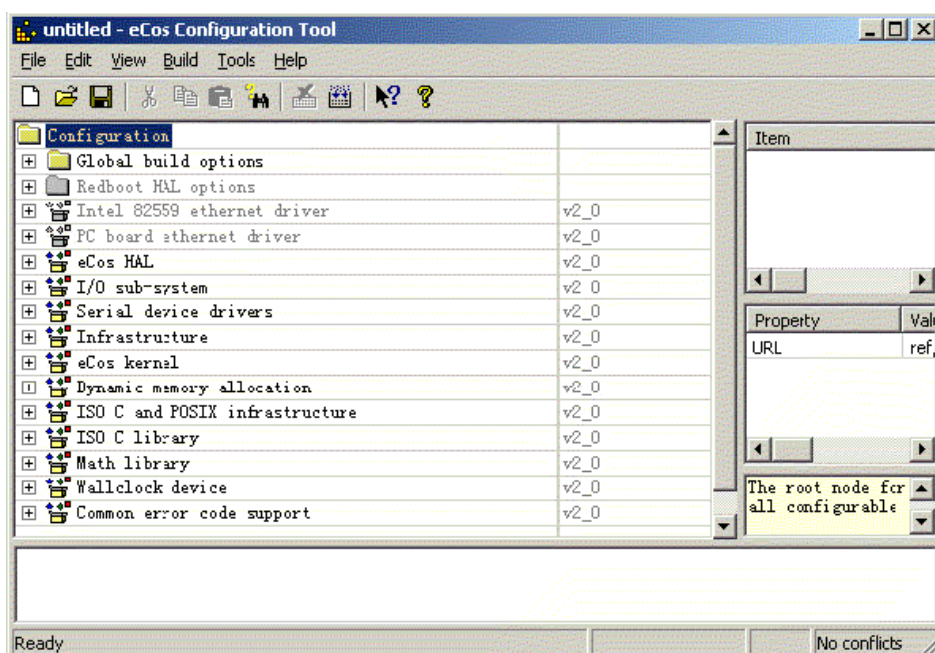


图 4-3 图形化的 eCos 配置工具

在 Windows 平台运行 eCos 开发环境

与在 Linux 环境相似，在 Windows 下运行 eCos 开发环境之前，也需要配置 eCos 开发环境的环境变量。

打开 Windows 操作系统“控制面板”→“系统”→“高级”→“环境变量”设置窗口，在设置窗口中新建环境变量（选择用户变量或系统变量均可）ECOS_REPOSITORY，其值设置为 eCos 组件库的位置，即 C:\cygwin\opt\ecos\ecos-2.0\packages，如果安装位置不同，则调整相应路径名。

设置好 ECOS_REPOSITORY 环境变量后，点击 eCos Config Tool 快捷方式，就可以运行图形化的 eCos 配置工具，如图 4-3 所示。

4.6 eCos 在特定嵌入式系统的移植

eCos 实时操作系统和应用程序最终都需运行在特定的嵌入式系统，不同的嵌入式系统，eCos 的配置也会有所不同，本节以在 EP93XX 系列微处理器上实现 Redboot 为例，对 eCos 在 ARM 平台上的基本移植步骤作简单说明。

首先介绍 eCos 的基本体系结构。

4.6.1 eCos 的基本体系结构

eCos 实现系统可配置机制的核心就在于它是由许多个的组件（Component）构成的，

包括调度内核组件、硬件抽象层、文件系统组件、网络协议栈组件等，而且这些组件都是可重用的，因此，用户可以根据不同的应用需求选择所需组件，同时对一些特定的配置选项进行设置，就可以构造出满足特定应用需求的系统。显然，采用这种方式构建出来系统不仅仅可以完全满足应用系统的需求，同时也非常高效和可靠。

所有的 eCos 系统组件都在组件库（Component repository）中，组件库其实是一个包含了 eCos 安装的所有系统组件的文件目录，对应于 /opt/ecos/ecos_2.0/packages 安装目录。该目录的内容如表 4-1 所示。

表 4-1 packages 目录内容描述

目 录	描 述
compat	包含支持与 POSIX 和 μ ITRON3.0 标准兼容的组件文件包。
cygmon	生成 Cygmon 调试监视器的配置文件包。
devs	eCos 支持的所有外部设备驱动，如串口、以太网等。
error	包含各种常见的错误和状态描述代码，便于系统调试时报告错误和状态。
fs	包含 ROM 和 RAM 文件系统包。
hal	包含的所有硬件抽象层的目标硬件配置文件包。
infra	包含系统定义的基本结构，如数据类型、宏定义、系统启动方式选择等。
io	I/O 子系统文件包，它建立在外部设备驱动上层而与特定硬件无关。
isoinfra	包含的文件包是提供支持 ISO 标准 C 库和兼容 POSIX 标准的接口。
kernel	所有提供 eCos 内核功能（如调度器、信号量、线程）的配置文件包。
Language	包含 ISO C 库和 math 库，使应用程序能使用一些标准 C 函数和数学函数。
net	提供 TCP/IP 协议栈支持的文件包。
redboot	包含生成 Redboot 调试监视器的配置文件包。
services	包含提供动态内存分配和文件压缩与解压功能的库文件。

从表 4-1 可以看出，eCos 中所有硬件抽象层的目标硬件配置包都包含在 hal 子目录。硬件抽象层（HAL，Hardware Abstraction Layer）是 eCos 的设计过程中为了方便其在不同硬件平台间的移植而定义的。硬件抽象层是内核与实际硬件之间的转换层，它通过向内核或其他组件提供标准的 API 函数，屏蔽了底层的 CPU 架构与平台的特定信息，因此，eCos 的移植就是根据不同的目标硬件特征，对硬件抽象层作出相应修改。

eCos 系统的硬件抽象层是用 C 和汇编语言来实现的，保证了 HAL 有较好的移植性能。硬件抽象层的接口都是用宏来实现的，并且在宏实现中采用了内联汇编、内联 C 代码或直接调用外部 C 函数和汇编函数等手段。

以下两段代码对 HAL 的实现机制作了比较直观的说明。

```
#define HAL_ENABLE_INTERRUPTS()
asm volatile (
    "mrs r3,cpsr;"
    "bic r3,r3,#0xC0;"
    "msr cpsr,r3"
    :
    :
```

```

        : "r3"
    );
#define HAL_ENABLE_INTERRUPTS()
    CYG_MACRO_START
        cyg_uint32 tmp1, tmp2;
        asm volatile (
            "mfmsr %0;"
            "ori    %1,%1,0x8000;"
            "rlwimi %0,%1,0,16,16;"
            "mtmsr %0;"
            : "=r" (tmp1), "=r" (tmp2)
        );
    CYG_MACRO_END

```

第一段代码是在 ARM 体系结构下，开启硬件中断宏 HAL_ENABLE_INTERRUPTS() 的实现代码，第二段代码是在 PowerPC 平台下实现同样功能的源代码。当内核或者其他组件要开启硬件中断功能时，只需要直接调用宏 HAL_ENABLE_INTERRUPTS() 或相应的 API 函数即可，而宏 HAL_ENABLE_INTERRUPTS() 或相应的 API 函数的具体实现机制对它是透明的，因此是与平台无关的。

当用户需要将 eCos 移植到某一特定硬件平台时，需要完成的工作就是要根据目标系统的硬件特征，修改 HAL 的相应数据结构和各个接口宏的具体实现。

在硬件抽象层中，根据不同硬件之间的相似程度，定义了三个 HAL 级别，但是每个级别之间的界限可能是比较模糊的。三个级别的 HAL 描述如下：

- λ Architecture HAL：它抽象了 CPU 内核架构的基本特征，包括中断的传递、上下文切换以及 CPU 的启动等。
- λ Variant HAL：它抽象了不同 CPU 内核的共同特征，如 Cache、MMU 和 FPU 等，同时它还包括片上的其他外围接口，如存储控制器和中断控制器等。
- λ Platform HAL：它抽象了不同硬件平台的属性，包括硬件平台的启动、定时器、IO 寄存器的访问等。

4.6.2 eCos 的移植方法

对 eCos 的体系结构，尤其是硬件抽象层有了一定了解，就可以将 eCos 移植到特定的嵌入式系统了。本节以在 HDB9301/02 系统上实现 Redboot 为例，说明 eCos 在基于 ARM 架构的嵌入式系统中移植方法。

eCos 所采用的层次结构，使其在不同嵌入式系统的移植过程变得相对简单，移植的主要工作就是根据嵌入式目标系统的特征，对硬件抽象层作相应的修改，包括数据类型的定义、接口宏的实现、内存布局的调整，以及添加必要的外部设备驱动等。

最简单的着手方法是从 eCos 的硬件配置包中选择一个与嵌入式目标系统类似硬件平台的 HAL 实现，然后根据自己的目标系统的硬件特征，进行相应的修改。选择的硬件平台与

目标系统硬件的相似程度决定了移植工作量的大小，相似程度越高，所需的修改也就越小。

Redboot 实质上就是建立在 eCos 系统上的一个独立的最小系统，它使用了 eCos 系统的硬件抽象层，但不包含 eCos 提供的与操作系统内核功能相关的组件模块，同时，还根据需要配置了一些其他的相关组件。

因此，只要能顺利实现 Redboot 在特定嵌入式系统的移植，也就完成 eCos 移植的主体工作，同时，Redboot 内嵌了 GDB stub，能对运行在嵌入式目标系统上的应用程序进行调试，这对于后续嵌入式系统开发是非常有用的。

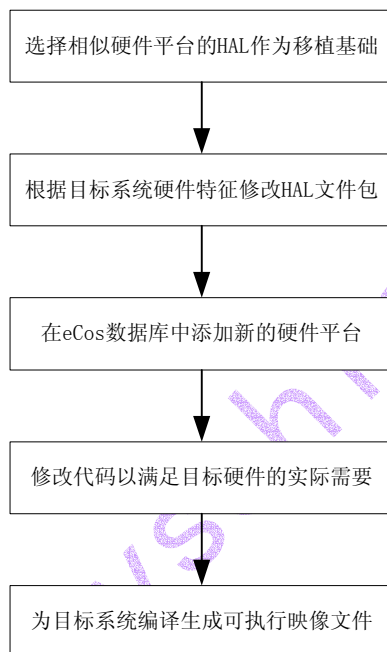


图 4-4 移植 eCos 的基本流程

图 4-4 说明了 eCos 移植的基本流程，每个步骤的具体描述如下：

选择相似硬件平台的 HAL 作为移植基础

选择与需要移植的目标系统最相似硬件平台的 HAL，作为移植的基础。eCos 是一个开源软件，在 eCos 源码的 `/opt/ecos/ecos_2.0/packages/hal` 目录下有很多已成功移植的硬件配置包，尤其是对于 ARM 系列微处理器。若已有相似的移植，无疑会对自己的工作带来很大的帮助。

针对基于 EP93XX 系列微处理器的嵌入式目标系统，可以选择 `hal/arm/arm9` 目录中已有的 arm9 开发系统的 HAL 实现包，作为移植工作的基础。可以复制其中任意一个文件夹，将新复制的文件夹命名为 EP93XX，并仍然存放在 `hal/arm/arm9` 目录下。

关于 eCos 系统移植的最新情况还可以到 eCos 的讨论组查询：

<http://source.redboot.com/ml/ecos-discuss>。

根据目标系统硬件特征修改 HAL 文件包

根据嵌入式目标系统的硬件特征对复制的 HAL 实现文件作相应修改，涉及到的修改主

要是对各配置包内文件名的修改和对配置包内.cdl 文件修改。cdl 文件是一个用组件描述语言（CDL, Component Description Language）编写的脚本文件，eCos 的每一个配置包中，都至少存在一个 CDL 脚本文件来对该配置包进行描述，配置工具也是通过该文件来与配置包联系起来。因此，对于 cdl 文件的修改也主要是对配置包的名称和文件名进行修改，使之与目标系统硬件相联系。

CDL 是 tcl 语言的一个扩展集合，关于它的详细说明可以参考相关文档。

在 eCos 数据库中添加新的目标平台

需要在/opt/ecos/ecos_2.0/packages 目录的数据库文件 ecos.db 中添加新的目标平台，新平台才能被添加到配置工具中，并进行配置和编译处理。

数据库文件 ecos.db 也是使用 CDL 语言编写的，在 ecos.db 中需要添加两部分内容，可以根据相似硬件平台在 ecos.db 的内容进行修改。

以下是在 ecos.db 添加基于 EP9301 硬件平台的示例代码：

```
package CYGPKG_HAL_ARM_ARM9_EP93XX {
    alias      { "Cirrus Logic EP93XX based systems" hal_arm_ep93xx
arm_ep93xx_hal }
    directory  hal/arm/arm9/ep93xx
    script     hal_arm_arm9_ep93xx.cdl
    hardware
    description "The EP93XX HAL package provides the support needed to run
eCos on Cirrus Logic development boards."
}

target edb9301 {
    alias      { "Cirrus Logic EDB9301 evaluation board" ep9301 }
    packages   { CYGPKG_HAL_ARM
CYGPKG_HAL_ARM_ARM9
CYGPKG_HAL_ARM_ARM9_EP93XX
CYGPKG_DEVS_FLASH_STRATA
CYGPKG_DEVS_FLASH_EDB9301
CYGPKG_DEVS_ETH_ARM_EP93XX
}
    description "The HDB9301 target provides the packages needed to run
eCos on the Cirrus Logic EP9301 based development boards."
    set_value CYGHWL_HAL_ARM_EDB93XX_VARIANT "EP9301"
}
```

在以上的示例中，第一部分用于对新的硬件配置包进行描述，主要包括配置包的名称、功能描述、配置包的路径名以及配置包的 CDL 脚本文件名等；第二部分则是添加目标系统硬件的模板入口，这部分的主要内容是指明了与该平台相关的其他配置包，即当选择该硬件平台时，这些相关的组件包也会自动加载到应用中。

修改代码以满足目标硬件的实际需要

根据目标系统硬件的实际情况对代码进行相应修改。这部分的工作量是移植工作中最大的。

首先需要对目标系统平台的 CPU 时钟、所支持的控制台通道等进行设置,通过对配置包的脚本文件.cd1 文件进行修改完成。

然后根据目标系统的配置调整内存布局。对内存布局的调整通过对配置包 include/pkgconf 目录中的*_rom.h/*_romram.h 和*_rom.ldi/*_romram.ldi (rom/romram 代表的是 redboot 的不同启动方式,启动方式不同,相应的内存布局文件也不同)两个文件的进行修改完成,主要是调整目标系统的 ROM 和 RAM 的起始位置及容量大小。

最后是根据目标系统的需要,添加相关的外部设备驱动,如串行接口、Flash 设备及以太网驱动等。

为目标系统编译生成可执行映像文件

完成以上修改后,就可以为目标系统编译生成可执行映像文件了。eCos 所提供的配置编译工具有命令行式和图形方式,均可方便完成可执行映像文件的编译生成。

当可执行映像文件正确生成以后,就可以下载到相应嵌入式目标系统运行了。

4.6.3 eCos 在 EP93XX 系列微处理器上的移植

作为 eCos 的一个移植实例,以下将详细说明在 Windows 平台利用 eCos 图形化配置工具,配置生成基于 HDB9301/02 开发系统的 Redboot 可执行映像文件的过程。对于 Linux 平台,其基本过程也是一致的。

将 Redboot 在 EP93XX 系列微处理器上移植的 patch 文件 patch-ecos-2.0-cirrus.bz2 拷贝到 eCos 的安装目录/opt/ecos,运行 Cygwin,输入如下指令安装该补丁文件:

```
$ bzip2 -dc patch-ecos-2.0-cirrus.bz2 | (cd ecos-2.0; patch -p1)
```

补丁安装结束后,按前文所述方式启动 eCos 的图形配置工具,进行如下操作:

1、选择菜单 Build → Templates,显示出如图 4-5 所示的模板选择对话框。在对话框中,点击 Hardware 下拉框,并在下拉列表中选择“Cirrus Logic EDB9301 evaluation board”选项。然后在 Packages 下拉框选择 redboot 选项,这表示在应用中只包含与 Redboot 功能相关的配置包。

点击 OK 按钮确认选项,此时可能会弹出 Resolve conflicts 的提示窗口,点击 Continue 按钮继续即可,配置工具一般会解决相关的冲突问题。

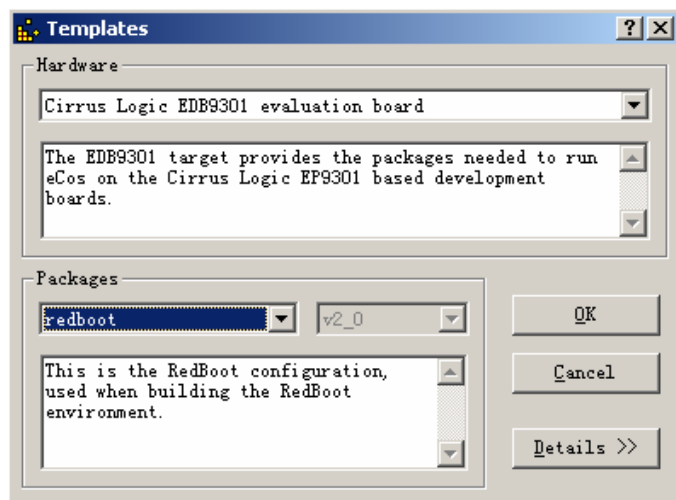


图 4-5 模板选择对话框

2、选择目标系统硬件的 HAL 配置包中提供的 eCos 的最小配置文件。选择菜单 File→Import，HDB9301/02 开发系统的 HAL 配置包提供的最小配置文件在目录 /opt/ecos/ecos_2.0/packages/hal/arm9/ep93xx/misc 中，在本例中，我们设置 Redboot 的启动方式为 ROMRAM 方式，所以选择 redboot_ROMRAM.ecm 文件。如果弹出了 Resolve conflicts 窗口，点击 Continue 按钮继续即可。

3、对目标系统的 CPU 类型，时钟、启动方式以及控制台通道号等进行设置。

4、保存配置文件。我们约定建立目录 /opt/ecos/redboot，配置文件就保存在该目录中。选择菜单 File→Save As，将配置文件命名为 redboot.ecc，点击 Save 按钮保存。

保存完毕后，打开 /opt/ecos/redboot 文件夹，可以看到配置工具除在其中保存了配置文件外，还生成了另外三个文件夹，其中，redboot_build 文件夹包含配置过程中使用和生成的一些临时文件，如 makefile 文件和目标文件等；redboot_install 文件夹包含编译过程将使用到一些头文件，最终生成的二进制可执行映像文件也存放该文件夹中；redboot_mlt 文件夹包含配置工具所使用的内存配置文件。

5、编译生成 redboot 可执行映像文件。

在编译前，需要看看配置工具右下角的冲突提示，如果提示有冲突存在，选择菜单 View→Conflicts 打开冲突显示窗口，在冲突窗口右击鼠标，选择 Resolve 选项来清除冲突项。

选择菜单 Build→Library 启动编译过程，如果是第一次启动编译过程，配置工具会提示需要选择 Build Tools 和 User Tools 的路径，本例中，编译工具路径名是 C:\cygwin\opt\ecos\gnutools\arm-elf\bin 和 C:\cygwin\bin。编译结束后，生成的可执行映像文件 redboot.bin 存放在 redboot_install\bin 目录中。

如果编译过程发生错误，则需要返回来对相应错误进行修改，然后重新保存配置文件，再启动编译过程。

Redboot 的映像文件生成后，就可以将其烧写到目标系统运行，具体的烧写方法可参考 4.3 节所描述的步骤。

4.7 基于 eCos 的简单应用程序开发

作为一个优秀的嵌入式实时操作系统，eCos 及应用开发受到越来越多的关注，本节以一个简单的应用例程，说明基于 eCos 的应用程序开发过程。

基于 eCos 系统的应用程序开发分为两步：第一步需要配置和编译 eCos 系统，得到一个 eCos 库文件；第二步才是编写上层的应用程序，应用程序在链接时要用到第一步所生成 eCos 库文件，这个库文件为应用程序提供操作系统支持。

4.7.1 生成 eCos 的支持库文件

eCos 库文件的生成需要对 eCos 进行配置和编译，其步骤与前文所述编译生成 Redboot 的步骤基本一致：

1、配置 eCos

在配置工具中通过菜单“Build-->Templates”来选择模板，在“Hardware”下拉框中仍然选择“EDB9301”平台，但在“Packages”下拉框中就不能再只选择“redboot”，需要选择“default”或“all”选项，因为 redboot 中不包含提高 eCos 内核功能的配置包。选定“Template”后，就可以根据应用程序的需求来对 eCos 进行相应配置了，在本例中，我们选择 eCos 的启动方式为 RAM 方式，这样使后面生成的应用程序的可执行文件既可以写入 RAM 中运行，也可以通过 Redboot 来调试运行。

由于本章随后的内容是讲述如何使用 Redboot 对该例程进行调试，因此需要去掉全局编译标志（Global compiler flags）和链接标志（Global linker flags）中的优化选项-O2。

2、保存配置

我们约定建立工作目录/opt/ecos/works，配置文件名为 ecos.ecc，保存在该工作目录中。

3、编译 eCos

选择菜单 Build-->Library 编译 eCos。编译结束后，在 ecos_install/lib 目录下生成 ecos 库文件 libtarget.a，该库文件即为应用程序运行所需要的支持库文件。

4.7.2 应用程序开发与调试

我们约定在/opt/ecos/works 目录下建立 test 文件夹，然后在该文件夹编辑源文件 test.c，代码如下：

```
/* this is a simple calculate program */
#include <stdio.h>
int main(void)
{
    int i, sum=0, temp_sum=0;
    for(i=1; i<=100; i++)
```

```
{
    sum=sum+i;
    temp_sum=sum;
    printf("temp_sum = %d\n",temp_sum);
}
printf("Calculate finished! sum = %d\n",sum);
return 0;
}
```

这是计算从 1 到 100 累加和的简单例程，程序运行时会分别打印出临时结果和最终结果。

使用以下命令对 test.c 文件进行编译和链接：

```
$ /opt/ecos/gnutools/arm-elf/bin/arm-elf-gcc -g
-I/opt/ecos/works/ecos_install/include test.c
-L/opt/ecos/works/ecos_install/lib -Ttarget.ld -nostdlib
```

编译链接结束后，在 test 目录下会生成可执行文件 a.out。

接着使用 Insight 工具调试程序：将嵌入式目标系统与主机通过串口或网络（我们在此使用网络）连接起来后，目标系统上电运行 Redboot。

运行 Insight，在 Insight 主操作界面中，通过菜单“File→Open”，选择需要调试的可执行文件，在 Insight 的“Source Windows”中将会显示 test.c 例程所包含的源代码。

设置目标系统与主机的通信方式，选择菜单“File→Target Settings”打开目标选择对话框如图 4-6 所示。

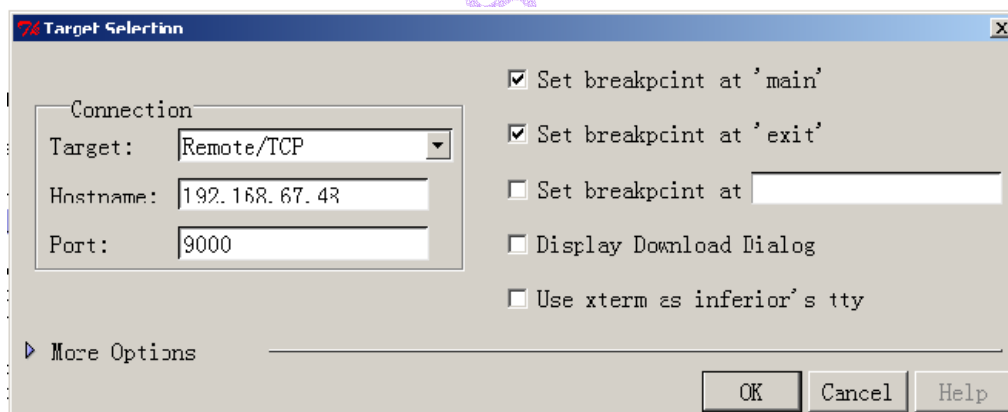


图 4-6 目标选择对话框

选择通过网络进行调试（Remote/TCP），在“Hostname”中设置目标系统的 IP 地址，端口一般设置为 9000。

软件设置完后，选择菜单“Run→Connect to target”连接目标系统，连接成功后会有“Successfully connected”的提示，就可以将代码下载到目标系统了。

选择菜单“Run→Download”启动下载过程，下载结束后，选择菜单“View→console”打开控制台窗口。然后就可以根据实际调试需要，使用菜单或在控制台窗口中输入相应命

令，来实现控制程序单步运行或添加断点等调试方式了。

调试过程中，程序的相应运行状态也会显示在控制台窗口中。Insight 源码窗口中的“Current file name”和“Current function name”下拉框可以方便对被调试文件的文件名和函数名进行查找。图 4-7 为 Insight 的程序调试窗口。

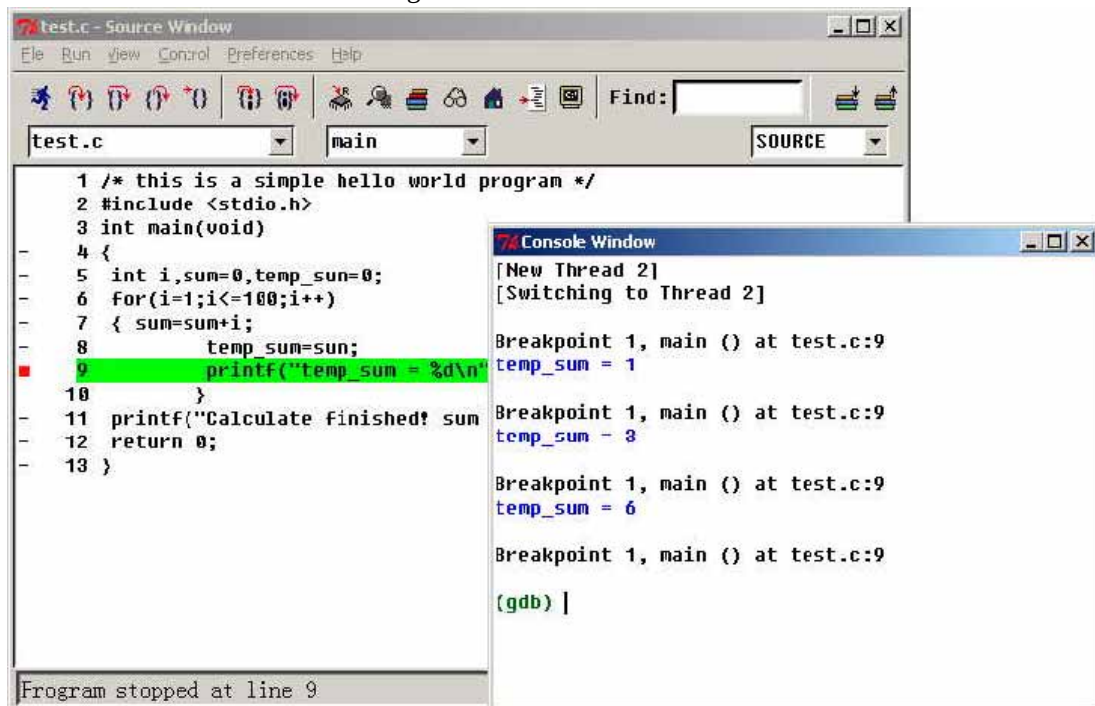


图 4-7 Insight 的程序调试窗口图

Insight 的 View 菜单中还提供了查看目标系统硬件的 CPU 寄存器、内存、函数调用栈结构、全局变量和局部变量等功能。

该调试过程实际上是通过运行在目标系统 Redboot 的 GDB stub 支持，与运行在主机上的 GDB Server，通过网络进行交互，完成程序的调试功能。

关于 Insight 的详细使用说明可参考相关技术文档。

当程序调试结束后，去掉所有断点，然后选择菜单“Control->continue”或直接在控制台中输入命令“continue”，控制台窗口显示程序的运行结果。

为支持调试功能，程序在编译时包含了大量与调试相关的信息，当程序调试完毕以后，就可以去掉这些信息，使可执行程序变小。

可以使用下面的命令将可执行文件 test 转化为代码尺寸更小 srec 格式。

```
$/opt/ecos/gnutools/arm-elf/bin/arm-elf-objcopy -O srec a.out test.srec
```

此时生成的可执行文件 test.srec 就可以通过前文所描述的方式，下载到嵌入式目标系统运行了。

4.8 本章小节

本章介绍了基于 EP93XX 系列嵌入式微处理器系统的初始化过程，以及 RedBoot 的命令使用方法。作为一个优秀的嵌入式实时操作系统，eCos 正在受到越来越多的关注，同时，Redboot 与 eCos 也有着天然的联系，因此，本章还介绍了 eCos 在嵌入式系统的基本结构，以及在嵌入式系统的移植方法。

作为一个实例，本章给出了 eCos 在 EP93XX 系列微处理器上的移植过程、基于 eCos 的应用程序开发，以及通过 GDB 方式对应用程序进行在线调试的过程。

用户如果对 eCos 的相关内容不感兴趣，尽可忽略本部分，直接进入下一章关于嵌入式 Linux 的描述。

第 5 章 嵌入式 Linux 开发基础

本章介绍嵌入式 Linux 操作系统在 EP93XX 系列微处理器的移植与开发过程，包括嵌入式 Linux 开发环境的建立、Linux 内核配置与编译，根文件系统的制作、网络文件系统的建立、简单应用程序的开发，以及基于嵌入式 Linux 应用程序的在线调试方法。希望读者通过对本章的阅读，能够步入嵌入式 Linux 系统开发的殿堂。

本章主要内容：

- 嵌入式 Linux 概述；
- 嵌入式 Linux 开发环境的建立；
- 嵌入式 Linux 内核的配置与编译；
- 根文件系统的制作；
- 网络文件系统 NFS 的建立；
- 基于嵌入式 Linux 的简单应用程序开发；
- 基于嵌入式 Linux 的应用程序的在线调试方法。

5.1 嵌入式 Linux 概述

Linux 是由 UNIX 操作系统发展而来的，具备现代一切功能完整的 UNIX 系统所具备的全部特征，其中包括真正的多任务、虚拟内存、共享库、优秀的内存管理以及 TCP/IP 网络支持等。

Linux 起初为基于 X86 架构的 PC 机开发，随着 Linux 的不断发展和完善，它已经可以很好的支持诸如 ARM、DEC Alpha、SUN Sparc、M68000、MIPS 和 PowerPC 等主流处理器架构，目前，Linux 已经成为嵌入式应用的主流操作系统之一。

Linux 之所以能成为一种流行的嵌入式操作系统，除具有功能强大、高性能、稳定性好以及源代码开发等优势以外，其最大的特点是 Linux 内核具有非常好的结构，即可由用户根据特定的系统需求，对内核进行配置或裁减，而这一特点恰恰满足了嵌入式应用的差异性需求。许多统计数字表明，在嵌入式系统中使用或者打算使用 Linux 作为操作系统的比例都在强劲增长，在某些领域，Linux 实际上已经处于领导地位，Linux 已经成为最为流行的嵌入式操作系统之一。

与运行于 PC 平台的标准 Linux 相比较，嵌入式 Linux 并不是一个新的东西，它实际上是对标准 Linux 内核做针对嵌入式应用的修改，例如 uClinux 就是针对无 MMU 的处理器而开发的一个变种；而 RTLinux 则增加了内核调度的实时性，可用于实时性要求较高的场合。但所有这些嵌入式 Linux 基本的东西是一致的，都很好的保持了 Linux 内核的特性。

值得注意的是，我们通常所说的嵌入式 Linux 指的是 Linux 内核，并不包括各种基于内核的应用，比如 Linux 的商业发行版 RedHat 等，就在内核的基础上加入了许多的应用。因此，为区别这些商业的 Linux，我们可以简单的定义嵌入式 Linux：嵌入式 Linux 是在标准 Linux 基础上，经过适当的简化（裁剪），然后加入一些特定的功能，形成的一个精巧的、高效的、满足特定应用需求的专用（定制）的操作系统，它具有用户可裁减，可配置的特点。

下面我们简单介绍 Linux 系统基本结构以每个模块的功能。

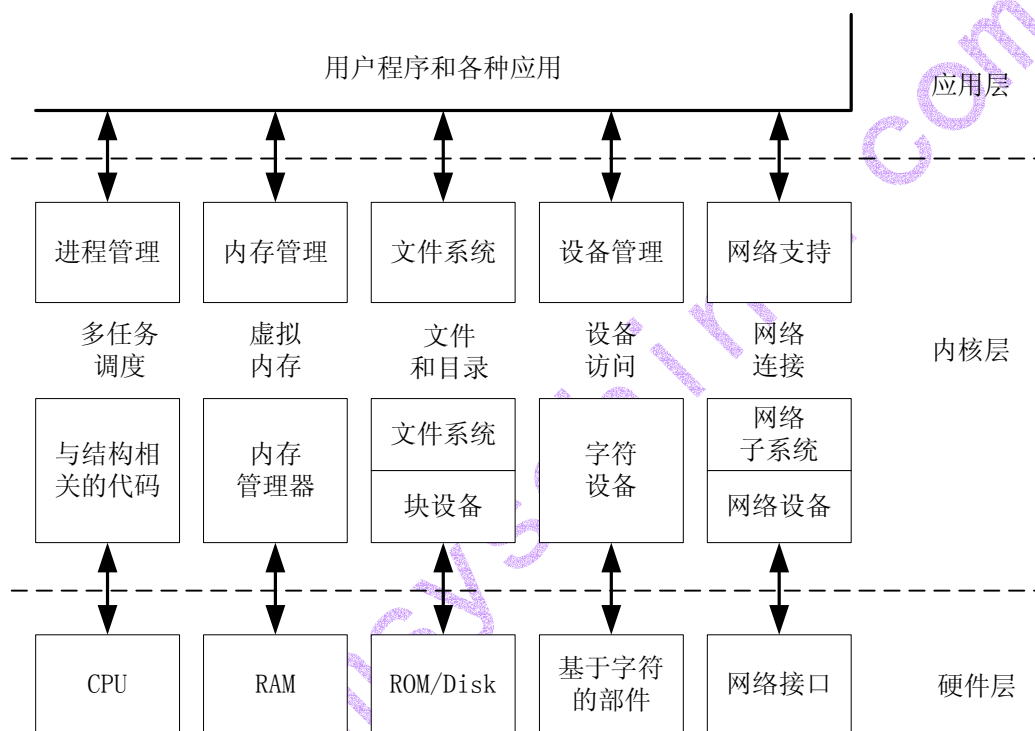


图 5-1 Linux 系统结构

尽管 Linux 应用系统结构复杂，但按照大的任务划分，整个系统可以分成如图 5-1 的各个部分，其中 Linux 内核的各个功能模块描述如下。

进程管理

进程管理主要完成进程的创建、中止、进程间的通讯及任务调度，这个是 Linux 内核最为核心的地方。进程管理的相关文件是在 Linux 内核源码目录的 kernel 中实现，在嵌入式系统的开发过程中，一般不需要改动该目录下的文件，以免出现致命的错误。

最新的 Linux 内核版本中增加了基于抢占式的任务调度，从而使得 Linux 内核的实时性得到了很大的提高，更适合于嵌入式应用。

内存管理

内存管理的主要任务是为每个进程在有限的内存资源上建立虚拟地址空间。内核不同的部分通过一组函数与内存管理子系统之间实现交互。内存管理的相关文件是在内核源码

目录下的 `mm` 当中实现的。

文件系统

文件系统实际上就是在物理存储介质上实现的对数据的一种组织结构，或者叫数据结构。Linux 系统是建立在文件系统这个概念之上的，几乎所有的系统资源都可以抽象成文件，例如设备文件就是对特定硬件设备的抽象，然后对这些设备的访问就可以通过操作设备文件来完成。

Linux 支持多种不同的文件系统，这些文件系统适合不同的物理介质，各有其特点，常见的文件系统有 `ext2`、`romfs`、`nfs`、`jffs` 等。

文件系统的开发和应用在嵌入式系统中十分重要，设计者要根据系统的硬件设计来选择不同的文件系统支持。

文件系统的相关文件在内核源码目录下的 `fs` 中，每一个子目录下就是一种文件系统。

设备控制

除了处理器，内存和少数其它实体外，几乎所有的设备控制操作都要由设备相关的代码来实现，这些代码实际上就是设备驱动程序。Linux 内核包含了大量的标准以及非标准设备的驱动，开发者可以根据系统的需求进行选择，但由于嵌入式设备面向应用的特点，往往需要根据系统特有的硬件来设计专门的驱动程序，因此在嵌入式 Linux 的系统开发中，设备驱动的开发占据了相当的份量。

网络系统

网络设备属于较为特殊的一类设备，对它的访问是通过协议来实现的，这一部分也是由内核来管理和实现的。

Linux 是一个优秀的网络操作系统，支持完整的 TCP/IP 协议栈和许多其它的网络协议栈，用户可以通过协议栈开发各种基于网络的应用，如 Socket 编程、嵌入式 web 服务、ftp 服务和 telnet 服务等。

Linux 操作系统中和网络相关的代码在源码目录下的 `net` 中。

事实上，Linux 是一个很复杂的操作系统，若要较为深入的了解每个模块的基本机理和工作方式，需要较长时间的积累，有很多的书讲解 Linux 系统原理和源代码分析，但在此我们并不提倡读者过早的花很多时间去阅读系统源代码，希望读者能以比较简单的方式，尽快开始 Linux 的系统开发，以实际应用为导向，逐步深入理解 Linux 的精髓，从而最终达到熟练应用的目的。

5.2 嵌入式 Linux 开发环境的建立

5.2.1 嵌入式 Linux 开发环境的基本结构

“工欲善其事，必先利其器”，为提高嵌入式 Linux 的开发效率，一个完善的嵌入式 Linux 开发环境是必不可少的。嵌入式 Linux 开发环境一般由如下几部分构成：Linux 服务

器（宿主机）、工作站、嵌入式目标系统和将它们连接在一起的网络环境，其具体结构如图 5-2 所示。

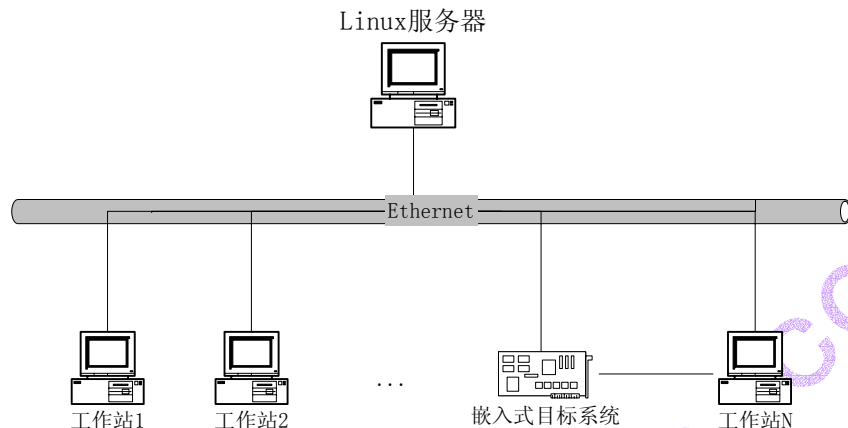


图 5-2 常见嵌入式 Linux 开发环境的结构

在图 5-2 所示的嵌入式 Linux 开发环境中，Linux 服务器作为嵌入式 Linux 内核编译、应用程序编译的公共平台，由单独的一台 PC 机充当，安装常用的桌面标准 Linux 操作系统，如 RedHat Linux 等。

工作站即为普通的局域网络计算机，可以是一台或多台，以支持小组项目开发，工作站一般安装常用的 Windows 操作系统，仍然可以完成各种日常工作，当需要使用 Linux 服务器资源时，就从工作站远程登录到 Linux 服务器，以完成各项需要的操作。

嵌入式目标系统是我们需要开发的最终产品，可以根据开发需要与工作站连接（通常是通过串行接口或 USB 接口），或连接到局域网络中。

工作站作为开发人员的操作终端，可能需要经常登录 Linux 服务器使用相关资源，并在 Linux 服务器和工作站之间进行文件的传输，因此，一般需要在工作站上安装 FTP 客户端程序（如 Cuteftp 等）和 Telnet 客户端程序（如 SecureCRT 等），当 Linux 服务器的 FTP 服务和 Telnet 服务运行时，开发人员就可以方便的在工作站和 Linux 服务器之间进行文件传输，并可以通过 Telnet 方式登陆到 Linux 服务器，并对其进行相关操作。

为便于读者的理解，我们约定以后的操作方式如下：开发人员在其中的一台工作站进行操作，通过远程登陆的方式操作 Linux 服务器，并使用 FTP 方式在 Linux 服务器和工作站之间进行文件传输，同时嵌入式目标系统需要与网络连接，嵌入式目标系统的串行接口与该工作站的 RS232 接口连接，使用工作站上的超级终端作为嵌入式目标系统输入输出终端。

5.2.2 在 Linux 服务器安装交叉编译工具

5.2.2.1 获取交叉编译工具

开发基于 Linux 的应用程序，一般的步骤是先编写程序源码，然后编译调试，最终生

成可执行程序。

目前,已有各种基于 X86 架构的 GUN 开发工具集,如 C 编译器 GCC,C++编译器 G++、连接器、调试器等,这些工具一般都作为标准 Linux 的一部分存在。但使用这些工具编译出来的可执行程序只能在基于 X86 架构的 CPU 上运行,因为编译器最终编译出来的是基于 X86 架构的机器码。

为使编译出的可执行代码能在诸如 ARM 或其它体系架构的 CPU 上运行,必须有一个编译工具,这个工具应该能运行在基于 X86 架构的 Linux 服务器上,但通过它编译生成的可执行代码能够支持在其他体系结构的 CPU 上运行,这样的一个编译工具就叫交叉编译工具。为什么要使用交叉编译呢?这是由于嵌入式设备没有足够的内存以及存储资源来完成其编译过程,所以人们想了个办法,就在主机上完成针对目标机的代码编译生成,这个过程称为交叉编译。这也目前大多数嵌入式设备开发所使用的编译方式。

我们如何得到支持特定微处理器架构的交叉编译工具呢?交叉编译工具一般由专门的机构负责维护,可以从他们的网站上免费取得,下面我们描述交叉编译环境的建立过程:

我们首先在 Linux 服务器上新建一个用于嵌入式 Linux 开发的工作目录: /home/work,以后所有的开发工作都在这个目录下进行。

然后取得基于 ARM 微处理器架构的交叉编译工具。基于 ARM 架构的交叉编译工具集可以从网站: <http://www.armlinux.org.uk> 下载。这个网站专门维护基于 ARM 架构的 Linux 内核,在这里,有专门针对不同的 ARM 处理器的 Linux 内核移植补丁,有大量的驱动以及工具集,我们在这里下载交叉编译工具集。

在 Linux 服务器上(或工作站的 Linux 终端)以 FTP 方式登陆该站点,执行如下命令:

```
# ftp ftp.arm.linux.org.uk
```

此时系统提示输入用户名和密码,任意输入一个用户名和密码(如均输入 FTP),可以成功登陆到以上站点,用 ls 或 dir 可以看到与如下类似的目录结构。

```
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
227 Entering Passive Mode (195.92.253.2,251,158)
150 Here comes the directory listing.
drwxr-xr-x  19 0      0      4096 Jun 15  2000 LDP
drwxr-xr-x   2 0      1      4096 May 22  1998 bin
drwxr-xr-x   2 0      4      4096 Oct 20  1996 dev
-rw-r--r--   1 0      0           42 Jan 25  2000 ftpmon.test
dr-xr-xr-x   2 0      1      4096 Oct 20  1996 lib
drwxr-xr-x   6 0      0      4096 Sep 23  2002 mirrors
drwxr-xr-x  14 19904  19904  4096 Mar 20 23:14 pub
-rw-r--r--   1 0      0           0 Apr 17  2001 robots.txt
drwxr-xr-x   2 0      0      4096 Apr 27  2003 rsmith
drwxr-xr-x   2 0     512      4096 Aug 03  1998 ukcrypt
drwxr-xr-x   4 0      1      4096 Oct 06  1993 usr
226 Directory send OK.
ftp> █
```

图 5-3 站点的目录结构

执行如下命令进入/pub/armlinux/toolchain/目录:

```
# cd pub/armlinux/toolchain/
```

在该目录下存在如下文件:

```
README
cross-2.95.3.tar.bz2
cross-3.0.tar.bz2
cross-3.2.tar.bz2
cross-3.2.tar.gz
src-2.95.3
src-3.2
```

其中, cross-x.x.tar.bz 为交叉编译工具集合, 当前最稳定的版本是 2.95.3, linux-2.4.xx 及以下的内核源码一般就用该版本的交叉编译器来编译, 而对于 linux-2.6-xx 的内核源码, 一般需要 cross-3.x 以上的版本来编译, 因为 linux-2.6-xx 的内核增加了诸如抢占式任务调度等许多完全不同于 linux-2.4.xx 的内容, 相应的交叉编译器 cross-3.x 的库文件也做了一些修改, 因此, 如果需要编译 linux-2.6-xx 的内核, 请使用 cross-3.x 的交叉编译器, 否则, 即使能够用 cross-2.95.3 版本的交叉编译编译器编译通过, 通常也无法在嵌入式目标系统上运行起来。

在以后的示例中, 我们需要编译 linux-2.4.21 的内核, 因此, 我们下载 cross-2.95.3 版本的交叉编译工具。

输入如下命令切换到二进制方式传输文件:

```
# bin
```

然后通过 FTP 方式下载文件 cross-2.95.3.tar.bz2:

```
# get cross-2.95.3.tar.bz2
```

此时, 系统开始将文件 cross-2.95.3.tar.bz2 下载到 Linux 服务器的当前工作目录 /home/work, 下载完毕以后退出 ftp。

```
# bye
```

5.2.2.2 安装交叉编译工具

下载的交叉编译工具以压缩包的方式存在, 需要将其安装到 Linux 服务器上。我们把交叉编译工具安装在 Linux 服务器的/usr/local/目录下, 执行如下操作, 建立/usr/local/arm 目录, 并将下载的交叉编译工具拷贝到该目录下:

```
# cd /usr/local
# mkdir arm
# cp /home/work/cross-2.95.3.tar.bz2 ./
```

键入如下命令解压安装交叉编译工具:

```
# tar Ixvf cross-2.95.3.tar.bz2
```

当以上命令执行完毕以后, 会生成/usr/local/arm/2.95.3 子目录, 表明交叉编译器就已经成功安装到了/usr/local/arm 下, 我们来看一下交叉编译工具集所包含的内容及各部分的基

本功能。

进入/usr/local/arm/2.95.3 子目录，列表查看该目录所包含的内容：

```
# cd /usr/local/arm/2.95.3/  
# ls  
arm-linux bin include info lib man share test-if-write
```

其中，bin 目录下是交叉编译器和一些工具，如编译器 arm-linux-gcc 是 c 交叉编译器，arm-linux-g++ 是 c++ 编译器，arm-linux-as 是汇编器，arm-linux-ld 是链接器。还有一些常用的工具，如 arm-linux-objcopy 是二进制转换工具，arm-linux-strip 用来去掉影像文件的调试信息，使得文件变小等。对于一般的应用这几个工具已经足够了。

lib 目录下存放的是库文件，包括 gcc 库 lib-gcc，g++ 库 lib-g++等。

arm-linux/include 目录存放编译器的头文件，包含了标准 c 的头文件，linux 的头文件，特定处理器相关的 linux 头文件等。

在以上的工具中，arm-linux-gcc 作为基于 ARM 架构的 c 编译器，在我们的系统开发过程中是最为常用的，因此，在下一节的内容，我们将详细介绍如何使用 arm-linux-gcc 来编译文件，因为嵌入式 Linux 内核和应用程序的编译都需要使用该编译器。

需要读者注意的是，以上所描述的内容可能会随着 Linux 内核的不断升级而出现一些变化，但基本的操作步骤应该是相似的。

5.2.3 arm-linux-gcc

arm-linux-gcc 是 arm-linux 系统中 c 语言的编译器，它兼容 ANSI C 的标准。

编译文件的过程可以分为下面 4 个阶段，每个阶段分别调用不同的工具进行处理，具体过程如图 5-4 所示。

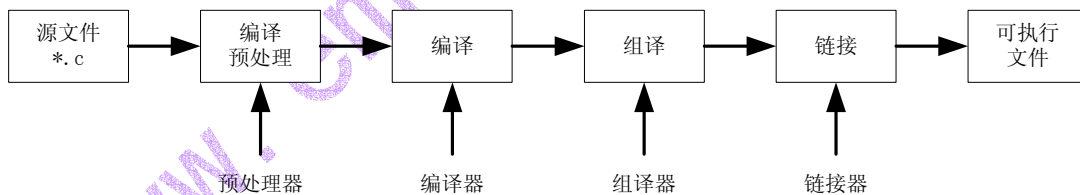


图 5-4 嵌入式 Linux 的文件编译流程

各个环节的功能简述如下：

- λ 预处理：预处理是指 arm-linux-gcc 调用预处理器（cpp）将预处理指令如#include、#define 等所包含的文件内容插入程序代码中。
- λ 编译：编译是调用编译器将预处理后的文件进行处理，并生成汇编文件。
- λ 组译：或者叫汇编（as），其主要工作是处理汇编语言的文件，并生成扩展名为.o 的目标文件。
- λ 链接：这个阶段由 arm-linux-gcc 调用链接器（arm-linux-ld）来处理。链接器的主要作用是把程序中用到的函数库放到程序中，最后生成可执行文件。

arm-linux-gcc 的语法与 gcc 相似，在此不作详细描述，读者如需要进一步了解，可以查阅在线帮助手册。

5.3 嵌入式 Linux 内核的配置与编译

如前文所述，Linux 之所以能成为一种流行的嵌入式操作系统，除具有功能强大、高性能、稳定性好以及源代码开发等优势以外，其最大的特点是 Linux 内核具有非常好的结构，可由用户根据特定的系统需求，对内核进行配置或裁减，而这一特点恰恰满足了嵌入式应用的差异性需求。

Linux 操作系统除了基本内核以外，还集成了丰富的设备驱动程序，这些程序从广义上看包含了设备驱动、文件系统、协议栈等。如果每一个系统都全部包含这些源代码，那么编译以后的内核将是一个比较庞大的文件，这样显然不能满足不同的嵌入式应用需求，因此，如何合理的配置 Linux 内核，使之能够在功能和大小之间取得一个优化的结果，是嵌入式应用的一个重要问题，以下我们将结合实际操作来说明如何裁减、编译一个 Linux 内核。

5.3.1 获取 Linux 内核源代码

嵌入式系统最大的特征是差异性，几乎每一个嵌入式应用都是唯一的，这种差异性体现在硬件方面，就表现为系统可以采用不同的微处理器架构，如 X86、ARM、PowerPC、MIPS 等，同时，即使采用同一种微处理器架构，不同生产商生产的微处理器也会有不小的差异，运行在这些嵌入式微处理器上的操作系统也会有所相同，因此，如何得到一个适合在某个特定的嵌入式系统上运行的 Linux 内核，是我们首先要解决的问题。

Linux 内核由专门的机构维护，我们可以从 Linux 内核的官方发布网站：<http://www.kernel.org>，获取功能完善的最新内核，但由于不同的嵌入式硬件平台会有一些的差异，因此，从这里获取的 Linux 内核还需要根据不同系统的硬件情况做相应的修改，而这个过程对绝大多数用户来说是比较困难的。

目前，各个芯片生产商在推出特定的嵌入式微处理器时，生产商或者第三方合作伙伴都会针对自己的微处理器发布基于 Linux 的内核补丁，同时，国内外的开发系统生产商也会做相关的工作，因此，适合于特定嵌入式系统的 Linux 内核源代码可以从芯片生产商或开发系统生产商获得。本文所使用的内核版本为 Linux-2.4.21，Cirrus Logic 公司已为该版本的内核写好了针对 EP93xx 系列微处理器的补丁，因此，加上这个补丁的 Linux-2.4.21 内核就是一个“适合”运行在基于 EP93XX 系列微处理器系统上的版本。

将得到的 Linux 内核源代码包拷贝到 Linux 服务器上我们约定的工作目录：`/home/work`，然后执行如下命令解压该内核源代码包：

```
# tar xzvf linux-2.4.21.tar.gz
```

当解压完毕以后，会在当前目录下生成一个内核源码目录：`linux-2.4.21`，Linux 内

核的所有源文件都在这个目录下。

根据 Cirrus Logic 公司提供的内核补丁 patch-2.4.21-rmk1-cirrus.bz2, 需要先去 armlinux 站点 <http://www.armlinux.org.uk> 下载相应的 arm 补丁 patch-2.4.21-rmk1.gz (<ftp://ftp.arm.linux.org.uk/pub/armlinux/source/kernel-patches/v2.4/patch-2.4.21-rmk1.gz>), 然后再为内核打上两个补丁。运行以下命令:

```
# gzip -dc patch-2.4.21-rmk1.gz | (cd linux-2.4.21; patch -p1)
# bzip2 -dc patch-2.4.21-rmk1-cirrus.bz2 | (cd linux-2.4.21; patch -p1)
```

接下来我们就可以在此基础上进行内核的配置与编译了。

5.3.2 Linux 内核配置

Linux 内核的配置(或裁剪)是与嵌入式系统的应用需求相适应的。尽管 Linux 内核功能强大, 支持的设备众多, 但对于一个特定的嵌入式应用来说, 可能只会使用到其中的部分功能, 而对于其它不使用的部分, 如果让它驻留在系统中, 不但耗费系统资源, 同时还会增加系统安全隐患, 因此, 内核配置的目的, 就是保留系统需要的功能, 去掉不必要的部分, 使操作系统以最精简、最优化的状态运行。

Linux 内核支持三种配置方式, 第一种方式是基于命令行的问答方式, 针对每一个内核配置选项会有一个提问, 回答“y”包含该项功能, 回答“n”则不包含该项功能。通过执行 make config 可以开始第一种方式。

第二种是菜单式的, 用户可以在 Linux 服务器或网络中的某个工作站进行操作, 执行 make menuconfig 命令以后, 会出现一个配置菜单, 通过该菜单可以很方便的进行内核的配置。

第三种方法也是采用菜单方式进行配置的, 但必须在 Linux 服务器上执行。通过执行 make xconfig 可以开始第三种方式。

显然, 三种方式的实质是相同的, 我们通常使用第二种方式进行配置, 这种方式简单明了, 受条件制约小。执行如下命令:

```
# make menuconfig
```

系统出现图 5-5 的内核配置界面。

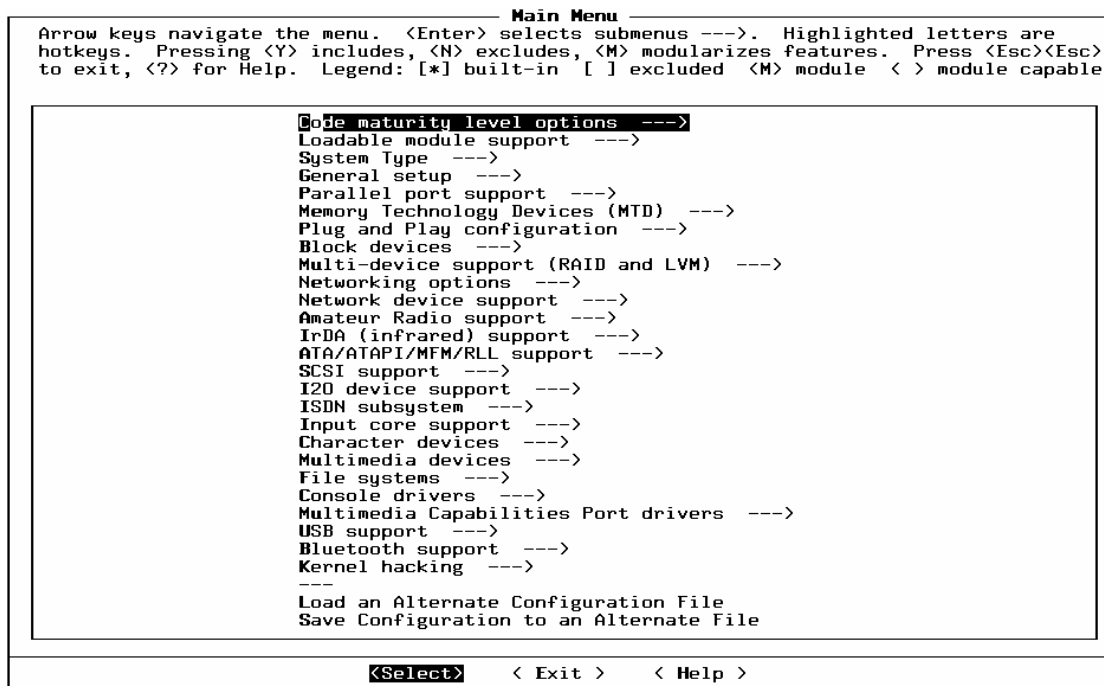


图 5-5 嵌入式 Linux 内核配置界面

图 5-5 所显示的内容为 Linux-2.4.XX 版本的主要配置选项，常用配置选项功能描述如下：

◆ Code maturity level options--->：代码成熟度选择，包含如下两个选项。

└ “Prompt for development and/or incomplete code/drivers” 选项。如果选中这个选项，每个模块中处于测试中的子模块会出现在模块的配置选项中，并且在配置选项后面注明（EXPERIMENTAL），表面这个模块还没有经过成熟的测试，正在测试中，不保证试验模块的可靠性。

└ “Prompt for obsolete code/drivers” 选项。这个选项表明在后面的模块配置选项中只出现成熟的模块选项。试验性的、测试中的模块将不被包含进来。如果选中该选项，则后面模块的配置中将没有试验模块出现，所有的模块都经过了可靠性的测试。

Code maturity level options 选项根据系统的特定需求来配置，如果某些模块是系统必需的，但该模块处于测试状态，我们也需要配置到内核中，因为处于测试状态的模块总比完全没有要强。

因此，我们一般选择第一项，允许对试验性模块的配置。

◆ Loadable module support --->：可加载模块支持。

模块是一小段代码，编译后可在系统内核运行时被动态地加入内核，从而为内核增加一些特性或对某种硬件进行支持。一般一些不常用到的驱动或特性可以编译为独立的模块，以缩小内核的体积，当 Linux 系统运行时，可以根据需要把模块加载到内核中去，在不需要时还可以卸载它，显然，这种方式增加了系统配置的灵活性。

- └ “Enable loadable module support” 选项，该选项可让内核支持模块挂载与卸载功能；
- └ “set version information on all symbols for modules” 选项，该选项是有关内核模块版本控制的选项。如果用户打算开发内核模块代码，就开启这个设置。一般的用户可以关闭。

◆ System Type --->：系统类型。

该选项用于配置 CPU 类型以及对某些特定评估板的支持，即对系统基本的硬件类型的支持。

◆ General setup --->：通用配置。

用户在此可设置内核对虚拟内存的支持和为 System V 的进程提供通信机制等；

其后的内核配置选项都是针对设备驱动的。设备驱动的配置需要根据用户所设计的嵌入式系统的软、硬件配置情况进行选择。

在 Linux 中，每个设备驱动可以选择作为模块（Module）动态链接到内核，或者选择作为内核的一部分静态编译进内核中，前一种方式可以减小编译出来的内核文件大小。因此，每一个设备驱动可以有三种状态，即不选择、以动态链接方式选择或以静态方式选择。

当用户不选择某个模块时，其模块前端的括号为空，即<>；

当用户以动态链接方式选择模块时，其模块前端的括号标识“M”，即<M>；

当用户以静态编译方式选择模块时，其模块前端的括号标识“*”，即<*>。

用户可以通过按空格键在以上三种方式间进行切换。

◆ Memory Technology Devices (MTD) --->：存储技术设备。

MTD 实际上就是嵌入式系统 Flash 设备的驱动，它抽象了对一类 Flash，包括 NOR Flash 和 NAND Flash 的驱动。

◆ Block devices --->：块设备。

块设备包括了对磁盘、ramdisk 等的驱动支持。嵌入式系统一般多为无盘系统，因此，把 ramdisk 作为根文件系统加载是很常见的，ramdisk 实际上就是在系统内存（ram）上虚拟了一个文件系统设备或盘（disk）。

通常，我们需要选择支持 ramdisk，同时指定大小，并初始化 ramdisk 等。

◆ Networking options --->：网络协议栈支持。

对于大部分的嵌入式应用而言，支持 TCP/IP 协议栈几乎是必须的。

选项中还有一些别的协议栈，如果你的系统用到了它们，在这里就需要选上。

◆ Network device support --->：网络设备支持。

网络设备即网络接口硬件，对于大部分的嵌入式应用，Ethernet（10 or 100Mbit）是最为常见的选择，这其中包含了对特定网卡的驱动，比如 EP93XX Ethernet support。

用户需要选择与硬件系统对应的网络设备支持模块。

◆ Character devices --->: 字符设备。

字符设备包括了一类设备，比如 spi 接口设备，串口设备。对于嵌入式设备，常把串口作为控制台设备，即作为人机交互设备，因此通常要支持串口设备。

◆ File systems --->: 文件系统。

文件系统选项所涉及的内容较多，建议在缺省配置的基础上进行修改。对于大多数嵌入式应用来说，需要 DOS FAT(FAT16、FAT32)、VFAT,EXT2、NFS 支持。对于 Flash 设备，则需要 JFFS 文件系统。

当用户在根据自己的系统需求配置好内核，退出配置菜单时，需要保存修改后的内核配置，如图 5-6 所示。

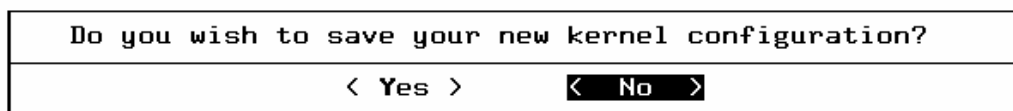


图 5-6 嵌入式 Linux 内核配置保存界面

若用户选择不保存，则进行的所有配置操作无效，内核配置仍然为原来的状态，若选择保存，系统会在当前目录下生成一个.config 文件，其后要进行的内核编译就是根据这个.config 文件来进行条件编译以生成相应的可执行文件的。

以上所描述的内容为嵌入式 Linux 内核的常见配置选项，不同的系统可能会有一些差异，但配置方式是基本一致的。

对某些选项所代表的意义，读者如果一时不能完全理解，可以采用内核的默认配置，暂时不作修改。

5.3.3 Linux 内核编译

当完成对 Linux 内核的配置以后，内核仍然以源代码的方式存在，不能直接下载到嵌入式系统运行，因此，需要对内核进行编译，生成最终可以在嵌入式系统上运行的可执行代码。

基于 EP93XX 系列微处理器的内核编译可按照下面的步骤进行，其它的嵌入式系统 Linux 内核编译步骤也大致相似：

第一步，执行如下命令，删除过时的文件：

```
# make clean
```

make clean 会删除掉原来的编译结果以及一些旧的数据文件。

第二步，执行如下命令进行依赖性编译：

```
# make dep
```

依赖性编译的目的可以描述如下：例如，如果你使用了程序 A，而 A 需用到程序 B（比如 B 是 A 的一个子程序），B 需用到程序 C。如果当你进行内核配置时，添加或删除了某个功能，这个功能刚好会对程序 C 造成影响，显然就需要重新编译程序 C、B 和 A，如果程序数量非常多，是很难手工完成的，而采用依赖性编译则可以自动完成该项工作。

第三步，执行如下命令，生成可执行内核映象文件：

```
# make zImage
```

这一步是实际的编译过程，最终会生成可运行在嵌入式系统上的内核映象文件。执行完这一步后，会在 arch/arm/boot/下生成一个内核映象文件 zImage。

但是，对于一个实际的嵌入式应用系统来说，仅仅包含 Linux 内核是不够的，还必需有文件系统的支持。在嵌入式 Linux 内核启动后，需要加载根文件系统，以支持系统与用户交互，我们将在以下的内容描述如何制作简单的根文件系统。

5.4 根文件系统及制作

每一台运行 Linux 操作系统的计算机都有根文件系统，嵌入式设备也不例外，对于 PC 来说，根文件系统一般在本地磁盘中，而对于嵌入式系统来说，则一般以 RAMDISK 或其他形式存在。

在 Linux 操作系统中，根文件系统包含系统引导文件和使其他文件系统得以挂载所必要的文件。在嵌入式 Linux 中，可以作为根文件的设备有很多，如在 RAM 中虚拟的文件系统 RAMDISK，Flash 上的只读文件系统 Romfs，可以通过网络加载到本机的网络文件系统 NFS，还有在 Flash 上实现的可读写文件系统 JFFS 等，都可以作为根文件系统被加载。但无论哪一种作为根文件系统，都至少应该包含以下几项：

- λ 基本文件系统结构；
- λ 至少应该包含目录：/dev, /proc, /bin, /etc, /lib, /usr, /tmp 等；
- λ 一些最基本的应用程序和命令支持，如 ls、mv、cp 命令等；
- λ 最低限度的配置文件支持，如 rc、inittab、fstab 等；
- λ 基本的设备支持：如/dev/hd*, /dev/tty*, /dev/fd0 等；
- λ 程序运行所需的基本库函数。

对于嵌入式 Linux 来说，最简单、最常用的根文件系统是 RAMDISK。

那么 RAMDISK 到底是什么含义呢？RAMDISK 就是指使用系统的一部分内存空间来模拟一个用户所熟悉的磁盘分区，用户在 RAMDISK 上进行的所有操作都与传统的磁盘文件操作一样。因此，在内存性价比越来越高的今天，对于无盘系统或嵌入式系统，选择 RAMDISK 作为根系统是很合适的。

目前 Linux 内核可以支持两种文件格式的 RAMDISK，即 minix 和 ext2，其中 ext2 性能更好。

如果从头开始制作一个 RAMDISK，其过程是比较繁琐的，在实际的应用过程中，通常采用的方式是从开发系统提供商获得一个已经做好的 RAMDISK，然后由用户根据自身系统的实际需求，对已有的 RAMDISK 进行增减，最终形成一个适合于自身应用的 RAMDISK。

以下我们说明如何以已有的 RAMDISK 为基础，制作一个新的 RAMDISK。

第一步：从开发系统提供商获得一个 RAMDISK，该 RAMDISK 一般以压缩文件的方式存在。拷贝 RAMDISK 的压缩文件 `ramdisk.gz` 到 Linux 服务器的工作目录：`/home/work`。

执行如下命令解压压缩文件 `ramdisk.gz`：

```
# gzip -d ramdisk.gz
```

解压完毕以后，文件 `ramdisk` 出现在当前目录。

第二步：建立一个临时目录，如 `ram_disk`，将文件 `ramdisk` 挂载到该目录。执行如下命令：

```
# mkdir ram_disk
# mount -o loop ramdisk ram_disk
# cd ram_disk
# ls
```

用户可以在 `ram_disk` 目录下看到文件 `ramdisk` 包含的所有子文件系统内容，各子文件系统的含义如下：

`/bin` 和 `/sbin`：该目录中包含有必不可少的应用程序，如 `ls`、`mv`、`cat` 等，用户可以根据自己的需要进行选择，不过一定要包括以下程序：`init`、`getty`、`login`、`mount`，运行 `rc` 的 `shell` 等。

`/etc`：这个目录中含有一些必不可少的系统配置文件。

`/lib`：该目录中包含启动过程中所需要的共享函数库，如果缺少必须的函数库，系统将不能正常启动。

`/dev`：该目录中含有系统不可缺少的设备文件。

`/tmp`：该目录用于存放临时文件，一般为空。

第三步：用户可以根据特定系统的需求，对相关文件进行增减操作，即可以往该目录拷贝或删除一些文件，操作完毕以后卸载目录 `ram_disk` 与文件 `ramdisk` 的连接，并对新的 `ramdisk` 文件进行压缩。

执行如下命令：

```
# umount /ram_disk
# gzip ramdisk
```

即可在当前目录下生成新的压缩文件 `ramdisk.gz`。

有了嵌入式 Linux 内核和根文件系统，一个完整的嵌入式 Linux 系统就编译完成了，我们就可以采用第 4 章所描述的方法，将生成的内核映像文件 `zImage` 和 `ramdisk.gz` 下载到系统的 Flash 中，最终形成一个完整的嵌入式系统。

事实上，在大多数情况下，开发商提供的 RAMDISK 已经可以满足绝大多数的应用需求，用户可以直接使用开发商所提供的 RAMDISK 压缩文件。

5.5 网络文件系统（NFS）及配置

网络文件系统（NFS，Network File System）是一种将远程主机上的分区（目录）经网络挂载到本地的主机的一种机制，通过对网络文件系统的支持，用户可以在本地主机上像操作本地分区一样来对远程主机的共享分区（目录）进行操作。

在嵌入式 Linux 的开发过程中，开发者需要在 Linux 服务器上进行所有的软件开发，交叉编译后，通用 FTP 方式将可执行文件下载到嵌入式系统运行，显然，这种方式不但效率低下，且无法实现在线的调试。

因此，可以通过建立 NFS，把 Linux 服务器上的特定分区共享到待调试的嵌入式目标系统上，就可以直接在嵌入式目标系统上操作 Linux 服务器，同时可以在线对程序进行调试和修改，大大的方便了软件的开发。因此，NFS 是嵌入式 Linux 开发的一个重要的组成部分，本部分内容将详细说明如何配置嵌入式 Linux 的 NFS 开发环境。

嵌入式 Linux 的 NFS 开发环境的包括两个方面：一是 Linux 服务器端的 NFS 服务器支持；二是嵌入式目标系统的 NFS 客户端的支持。因此，NFS 开发环境的建立需要配置 Linux 服务器端和嵌入式目标系统端。

Linux 服务器端 NFS 服务器的配置

以 root 身份登陆 Linux 服务器，编辑共享目录配置文件 exports，指定共享目录及权限等。

执行如下命令编辑文件/etc/exports：

```
# vi /etc/exports
```

在该文件里添加如下内容：

```
/home/work 192.168.67.* (rw, sync, no_root_squash)
```

然后保存退出。

添加的内容表示：允许 ip 地址范围在 192.168.67.* 的计算机以读写的权限来访问 /home/work 目录。/home/work 也称为服务器输出共享目录。

括号内的参数意义描述如下：

rw：读/写权限，只读权限的参数为 ro；

sync：数据同步写入内存和硬盘，也可以使用 async，此时数据会先暂存于内存中，而不立即写入硬盘。

no_root_squash：NFS 服务器共享目录用户的属性，如果用户是 root，那么对于这个共享目录来说就具有 root 的权限。

接着执行如下命令，启动端口映射：

```
# /etc/rc.d/init.d/portmap start
```

最后执行如下命令启动 NFS 服务，此时 NFS 会激活守护进程，然后就开始监听 Client 端的请求：

```
# /etc/rc.d/init.d/nfs start
```

在 NFS 服务器启动后，还需要检查 Linux 服务器的防火墙等设置，确保没有屏蔽掉 NFS

使用的端口和允许通信的主机，主要是检查 Linux 服务器 iptables, ipchains 等选项的设置，以及/etc/hosts.deny, /etc/hosts.allow 文件。

我们首先在 Linux 服务器上进行 NFS 服务器的回环测试，验证共享目录是否能够被访问。在 Linux 服务器上运行如下命令：

```
# mount -t nfs 192.168.67.1:/home/work /mnt
# ls /mnt
```

命令将 Linux 服务器的 NFS 输出共享目录挂载到/mnt 目录下，因此，如果 NFS 正常工作，应该能够在/mnt 目录看到/home/work 共享目录中的内容。

嵌入式目标系统 NFS 客户端的配置

在 Linux 服务器设置好后，接下来需要进行客户端的设置。

首先需要运行在嵌入式目标系统的 Linux 内核支持 NFS 客户端，在进行内核配置时，必须选择如下选项：

```
Code maturity level options --->
Loadable module support --->
System Type --->
General setup --->
Parallel port support --->
Plug and Play configuration --->
Block devices --->
Multi-device support (RAID and LVM) --->
Networking options --->
Network device support --->
Amateur Radio support --->
IrDA (infrared) support --->
ATA/ATAPI/MFM/RLL support --->
SCSI support --->
I2O device support --->
ISDN subsystem --->
Input core support --->
Character devices --->
Multimedia devices --->
File systems --->
Console drivers --->
Sound --->
Multimedia Capabilities Port drivers --->
USB support --->
Bluetooth support --->
Kernel hacking --->
Library routines --->
---
Load an Alternate Configuration File
Save Configuration to an Alternate File
```

图 5-7 NFS 的客户端配置（一）

选中 File System --->并按回车进入下级菜单

```

[ ] Quota support
<*> Kernel automounter support
<> Kernel automounter version 4 support (also supports v3)
<> Reiserfs support
<> ADFS file system support (EXPERIMENTAL)
<> Amiga FFS file system support (EXPERIMENTAL)
<> Apple Macintosh file system support (EXPERIMENTAL)
<> BeOS file systemv(BeFS) support (read only) (EXPERIMENTAL)
<> BFS file system support (EXPERIMENTAL)
<> Ext3 journalling file system support
<M> DOS FAT fs support
<M> MSDOS fs support
<>   UMSDOS: Unix-like file system on top of standard MSDOS fs
<M> VFAT (Windows-95) fs support
<> EFS file system support (read only) (EXPERIMENTAL)
<> Compressed ROM file system support
[ ] Virtual memory file system support (former shm fs)
<> ISO 9660 CDROM file system support
<> JFS filesystem support
<> Minix fs support
<> FreeVxFS file system support (VERITAS VxFS(TM) compatible)
<> NTFS file system support (read only)
<> OS/2 HPFS file system support
[*] /proc file system support
[ ] /dev file system support (EXPERIMENTAL)
<> QNX4 file system support (read only)
<> ROM file system support
<*> Second extended fs support
<> System V/Xenix/V7/Coherent file system support
<> UDF file system support (read only)
<> UFS file system support (read only)
Network File Systems --->
Partition Types --->
Native Language Support --->

```

图 5-8 NFS 的客户端配置 (二)

选中 Network File Systems --->并按回车进入下级菜单:

```

<> Coda file system support (advanced network fs)
<> InterMezzo file system support (replicating fs) (EXPERIMENTAL)
<*> NFS file system support
[*] Provide NFSv3 client support
[ ] Root file system on NFS
<> NFS server support
<> SMB file system support (to mount Windows shares etc.)
<> NCP file system support (to mount NetWare volumes)

```

图 5-9 NFS 的客户端配置 (三)

选中 NFS file system support 和 Provide NFSv3 client support, 保存退出, 然后根据 5.3.3 所描述的 Linux 内核编译一节的步骤重新编译内核, 并采用第 4 章所描述的方法, 将生成的内核映像文件 zImage 重新下载到系统中。

在嵌入式目标系统的 Linux Shell 下, 执行如下命令来进行 NFS 共享目录挂载:

```

# mkdir /mnt/nfs           //建立 Linux 服务器输出共享目录的挂载点;
# mount -o nolock -t nfs 192.168.67.1:/home/work /mnt/nfs
# cd /mnt/nfs
# ls

```

此时, 嵌入式目标系统端所显示的内容即为 Linux 服务器的输出目录的内容, 即 Linux 服务器的输出目录/home/work 通过 NFS 映射到了嵌入式目标系统的/mnt/nfs 目录。可以用增/删/修改文件的方式来验证实际效果。

mount 命令中的 192.168.67.1 为 Linux 服务器的 IP 地址，/home/work 为 Linux 服务器端所配置的共享输出目录，/mnt/nfs 为嵌入式设备上的本地目录。由于很多嵌入式设备的根文件系统中不带 portmap，所以一般都使用-o nolock 选项，不需要使用 NFS 文件锁来避免使用 portmap。

5.6 嵌入式 Linux 应用程序的开发

5.6.1 应用程序的开发流程

基于嵌入式 Linux 的应用程序开发流程，与基于 Windows 的应用程序开发有很大的不同。在 Windows 环境中，开发者习惯使用各种集成编译开发环境，完成程序编辑、编译和运行。

而在 Linux 下开发应用程序，目前还缺乏比较简单、高效的开发工具和手段，同时，由于应用程序的最终运行平台是嵌入式目标系统，但程序开发仍然需要在 PC+Linux 平台完成，因此，在程序的开发与调试过程中，需要频繁的在 Linux 服务器和嵌入式目标平台交换信息。由于以上这些原因，基于嵌入式 Linux 的应用程序开发，还是一个相对比较复杂的过程，但随着嵌入式 Linux 的推广使用，基于嵌入式 Linux 应用程序的开发会变得越来越方便和容易。

有几种开发模式可以用于基于嵌入式 Linux 应用程序的开发，但最常用的方式有如下两种，即 FTP 方式和 NFS 方式。

FTP 方式：

FTP 方式是首先在 Linux 服务器上编辑源文件，然后交叉编译，最后生成可执行文件，并将生成的可执行文件下载到嵌入式目标系统上运行。

因此，当开发者在 Linux 服务器上完成了应用程序的编辑和编译，生成可执行文件以后，需要在嵌入式目标系统上通过 FTP 方式，下载编译好的可执行文件到嵌入式目标系统上运行，如果程序运行错误，则到 Linux 服务器上修改源文件并重新编译、下载运行，直到程序正确运行为止。

FTP 方式需要 Linux 服务器运行 FTP 服务，同时要求嵌入式目标系统端运行 FTP 客户程序。

FTP 方式是一种常用的嵌入式 Linux 应用程序开发方式，其流程如图 5-10 所示。

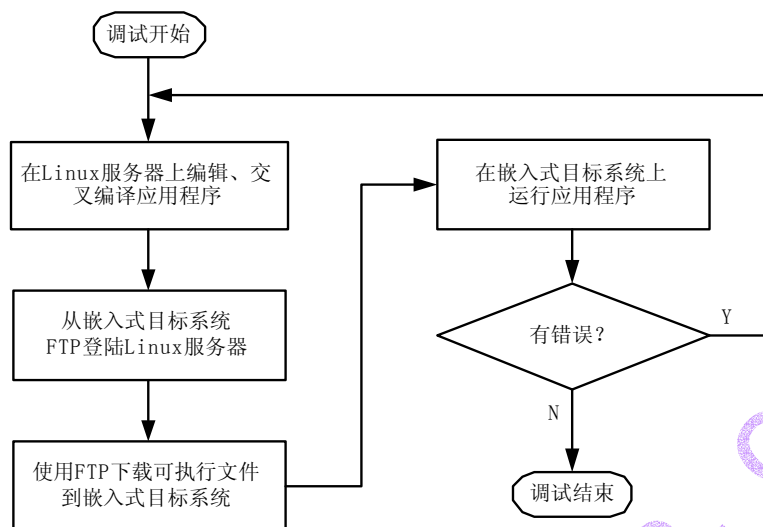


图 5-10 FTP 方式的应用程序开发流程

NFS 方式：

NFS 方式也是首先在 Linux 服务器上编辑源文件，然后交叉编译，最后生成可执行文件，但生成的可执行文件不再通过 FTP 等方式下载到嵌入式目标系统，而是在嵌入式目标系统端通过 NFS 方式挂载 Linux 服务器的共享分区，让应用程序直接运行在嵌入式目标系统，并进行调试。

NFS 方式需要 Linux 服务器端和嵌入式目标系统端均能支持 NFS 文件系统。

与其他方式相比较，NFS 方式具有较高的调试效率，是一种经常采用的方法。NFS 方式的开发流程如图 5-11 所示。

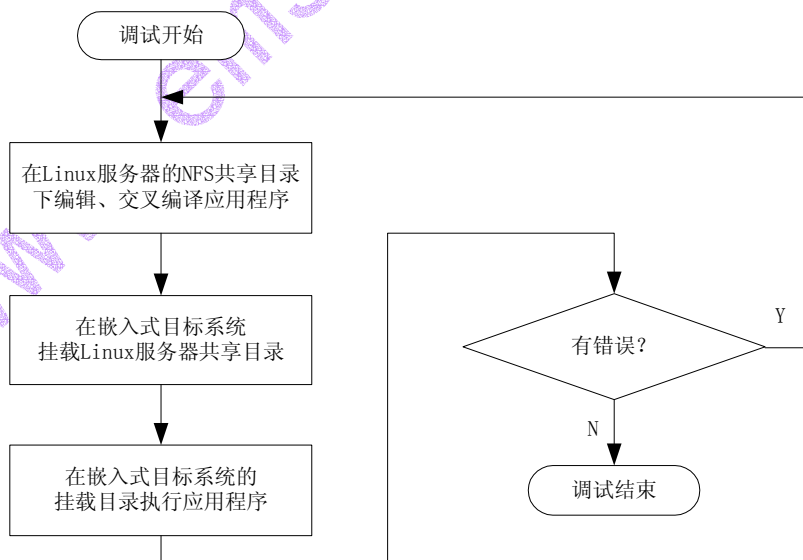


图 5-11 NFS 方式的应用程序开发流程

当开发人员完成了应用程序的调试与开发，可以将调试好的应用程序下载到嵌入式目标系统的 Flash 文件系统，或直接编译到嵌入式 Linux 内核并烧写到系统的 Flash，从而最终形成一个独立的嵌入式应用系统。

在以下的部分，我们将以一个简单的示例程序来说明基于嵌入式 Linux 应用程序的开发过程。

5.6.2 一个最简单的应用程序

我们使用一个最简单的示例，来说明基于嵌入式 Linux 的应用程序开发过程。示例程序完成 1~100 的累加求和，运行时输出每一步的累加结果和最后的总和。

FTP 方式：

第一步：编辑应用程序。

在 Linux 服务器的工作目录/home/work，使用 VI 文本编辑器编辑应用程序，应用程序名为 test.c：

```
# vi test.c
```

程序代码如下：

```
/* this is a simple calculate program */
#include <stdio.h>
int main(void)
{
    int i,sum=0,temp_sum=0;
    for(i=1;i<=10;i++)
    {
        sum=sum+i;
        temp_sum=sum;
        printf("temp_sum = %d\n",temp_sum);
    }
    printf("Calculate finished! sum = %d\n",sum);
    return 0;
}
```

编辑完源代码后，保存文件并退出 VI。

第二步：编译应用程序。

键入如下命令编译应用程序：

```
# arm-linux-gcc -o test test.c
```

当命令执行完毕以后，会在当前目录下生成可执行文件 test。

第三步：通过 FTP 方式将可执行文件 test 下载到嵌入式目标系统运行。

启动嵌入式目标系统的 Linux，使用超级终端作为其用户操作界面，切换到 Linux 的可写目录，如 `\var`，键入如下命令使用 FTP 方式登陆 Linux 服务器：

```
# ftp 192.168.67.1
```

192.168.67.1 为 Linux 服务器的 IP 地址。输入合法的用户名和密码，登陆到 Linux 服务器，执行如下命令将可执行文件 `test` 下载到嵌入式目标系统的当前目录。

```
# bin          // 以二进制方式传输文件；
# get test     // 将文件 test 下载到嵌入式目标系统的当前目录；
# bye
```

当以上命令执行完毕以后，可执行文件 `test` 下载到嵌入式目标系统的当前目录。

第四步：修改文件属性，运行应用程序。

当可执行文件 `test` 下载到嵌入式目标系统后，可能不具有可执行属性，键入如下命令修改文件属性，并运行应用程序：

```
# chmod 777 test // 修改文件属性为可执行；
# ./test        // 运行应用程序；
```

此时，用户可以看到应用程序的运行效果，输出每一步的求和结果和总的累加和。

NFS 方式：

第一步：编辑应用程序。

在 Linux 服务器的工作目录 `/home/work`，使用 VI 文本编辑器编辑应用程序，应用程序名为 `test.c`。

注意该目录必须是 Linux 服务器的输出共享目录。

```
# vi test.c
```

程序代码如下：

```
/* this is a simple calculate program */
#include <stdio.h>
int main(void)
{
    int i, sum=0, temp_sum=0;
    for(i=1; i<=10; i++)
    {
        sum=sum+i;
        temp_sum=sum;
        printf("temp_sum = %d\n", temp_sum);
    }
    printf("Calculate finished! sum = %d\n", sum);
    return 0;
}
```

编辑完源代码后，保存文件并退出 VI。

第二步：编译应用程序。

键入如下命令编译应用程序：

```
# arm-linux-gcc -o test test.c
```

当命令执行完毕以后，会在当前目录下生成可执行文件 `test`。

第三步：在嵌入式目标系统端挂载 Linux 服务器的输出共享目录，并运行应用程序。

启动嵌入式目标系统的 Linux，使用超级终端作为其用户操作界面，键入如下命令挂载 Linux 服务器的输出共享目录：

```
# mkdir /mnt/nfs           //建立 Linux 服务器输出共享目录的挂载点；
# mount -o nolock -t nfs 192.168.67.1:/home/work /mnt/nfs
# cd /mnt/nfs
# ls
```

此时，嵌入式目标系统端所显示的内容即为 Linux 服务器的输出目录的内容，可执行文件 `test` 就在该共享目录下，运行该程序：

```
# ./test           // 运行应用程序；
```

此时，用户可以看到应用程序的运行效果，输出每一步的求和结果和总的累加和。

显然，由于 NFS 方式省去可执行文件从 Linux 服务器传输到嵌入式目标系统端的过程，当开发的应用程序比较复杂时，采用该方式可以大大提高应用程序的开发效率。

5.7 嵌入式 Linux 应用程序的调试方法

GNU 工具链的调试器 `gdb` 可以对 Linux 下开发的应用程序进行调试，它能够控制程序执行，查看和改变程序变量，分析程序的内在结构，研究崩溃程序的核心文件等。

对于在 Linux 服务器上如何使用 `gdb` 调试应用程序，绝大多数使用过 Linux 操作系统的读者都应该略知一二，但如何使用 `gdb` 工具对运行在特定嵌入式设备上的应用程序进行调试，其过程却有一定的差异。因此，本节首先简述在 linux 服务器上使用 `gdb` 调试应用程序的过程，然后对比说明如何通过嵌入式 Linux 开发环境，使用 `gdb` 工具调试运行在特定嵌入式目标系统的应用程序。

在进行嵌入式 Linux 应用程序的调试之前，我们先看看如何使用 `gdb` 在 Linux 服务器上调试。

5.7.1 在 Linux 服务器上使用 GDB 调试程序

在 Linux 服务器上使用 `gdb` 工具调试应用程序，由于待调试的应用程序和 `gdb` 调试器均运行在 Linux 服务器上，其过程相对简单，具体调试方法描述如下。

将前文的示例程序 `test.c` 在 Linux 服务器上编译，并用 `-g` 编译选项让生成的可执行程序集成调试信息。命令如下：


```
# gcc -g -o test test.c
编译完毕以后，就可以使用 gdb 对生成的可执行程序 test 进行调试。命令如下：
# gdb test
此时进入 gdb 调试界面，如图 5—12 所示。
GNU gdb Red Hat Linux (5.3post-0,20021129,18rh)
Copyright 2003 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "i386-redhat-linux-gnu"...
(gdb) █
```

图 5—12 运行在 linux 服务器上的 gdb 调试工具

可以在 gdb 提示符下键入 help 来查看 gdb 命令，gdb 的常用命令及功能描述如表 5—1 所示。

表 5—1 常用 gdb 命令及功能描述

命 令	缩 写	功能描述
help	h	帮助；
file		装入待调试的可执行文件；
list	l	列出程序源码；
run	r	运行被调试的程序；
break	b	设置一个断点；
delete	d	删除断点或其他表达式；
continue	c	继续调试程序；
next	n	单步执行但不进入函数内部；
step		单步执行进入函数内部；
print	p	打印变量值；
set variable		修改变量值；
watch	wa	监视变量值；
quit	q	终止 gdb；

说明：

- 1、gdb 命令只要没有二意性就可以被缩写。如果不能确定命令缩写，可以用缩写作为 help 的参量看看是否对应于某个命令。
- 2、gdb 支持命令历史，可以用上、下键进行浏览。如果没有给出命令就直接回车，会自动使用上次的命令。
- 3、gdb 支持按 tab 键自动补齐命令的功能。

在 gdb 命令行下键入“l (list)”可以列出 test.c 的源码，缺省一次只显示 10 行，可以直接回车让其继续显示，如图 5—13 所示。

```
(gdb) l
1      #include <stdio.h>
2      int main(void)
3      {
4          int i,sum=0,temp_sum=0;
5          for(i=1;i<=10;i++)
6          {
7              sum=sum+i;
8              temp_sum=sum;
9              printf("temp_sum = %d\n",temp_sum);
10         }
(gdb)
11         printf("Calculate finished! sum = %d\n",sum);
12         return 0;
13     }
```

图 5-13 使用 gdb 调试应用程序（一）

可以先键入“r (run)”运行程序，其结果如图 5-14 所示。

```
Starting program: /home/zg/test
temp_sum = 1
temp_sum = 3
temp_sum = 6
temp_sum = 10
temp_sum = 15
temp_sum = 21
temp_sum = 28
temp_sum = 36
temp_sum = 45
temp_sum = 55
Calculate finished! sum = 55
```

Program exited normally.

图 5-14 使用 gdb 调试应用程序（二）

由于当前没有设置断点，程序立刻执行完毕，打印出计算结果。

我们用“b (break)”命令来在第 9 行设置一个断点，如图 5-15 所示。

```
(gdb) b 9
Breakpoint 1 at 0x8048363: file test.c, line 9.
```

图 5-15 使用 gdb 调试应用程序（三）

然后使用“r”命令再次启动被调试程序，如图 5-16 所示。

```
(gdb) r
Starting program: /home/zg/test

Breakpoint 1, main () at test.c:9
9              printf("temp_sum = %d\n",temp_sum);
```

图 5-16 使用 gdb 调试应用程序（四）

此时，程序运行到第 9 行断点处停止执行，可以使用“n (next)”来单步执行程序，如图 5-17 所示。

```
(gdb) n
temp_sum = 1
5          for(i=1;i<=10;i++)
```

图 5-17 使用 gdb 调试应用程序（五）

程序执行第 9 行，打印出“temp_sum = 1”，然后返回到第 5 行，用“c (continue)”命令继续执行，程序将再次于第 9 行暂停，如图 5-18 所示。

```
(gdb) c
Continuing.

Breakpoint 1, main () at test.c:9
9          printf("temp_sum = %d\n",temp_sum);
```

图 5-18 使用 gdb 调试应用程序（六）

再用“n”单步执行，并使用“p (print)”命令查看变量 i 的值，如图 5-19 所示。

```
(gdb) n
temp_sum = 3
5          for(i=1;i<=10;i++)
(gdb) p i
$2 = 2
```

图 5-19 使用 gdb 调试应用程序（七）

当前 i 值为 2，因为是第 2 次循环刚结束。可以这样一次次的继续下去，但也可以通过修改变量 i 的值，加速循环。

使用命令“set variable”来设置 i 的值为 9，然后使用“c”命令来继续执行程序，如图 5-20 所示。

```
(gdb) set variable i=9
(gdb) c
Continuing.

Breakpoint 1, main () at test.c:9
9          printf("temp_sum = %d\n",temp_sum);
(gdb) c
Continuing.
temp_sum = 12

Breakpoint 1, main () at test.c:9
9          printf("temp_sum = %d\n",temp_sum);
(gdb) c
Continuing.
temp_sum = 22
Calculate finished! sum = 22

Program exited normally.
```

图 5-20 使用 gdb 调试应用程序（八）

最后执行了两次循环，对应的 i 为 9 和 10，最后不满足循环条件，程序执行完毕，用“q (quit)”命令退出。

我们在 Linux 服务器上完成了对 test 程序的基本调试过程，接下来我们看看调试运行

在嵌入式目标系统上的应用程序的过程有何不同？

5.7.2 在嵌入式目标系统上使用 GDB 调试程序

在嵌入式目标系统的程序调试中，由于应用程序是在嵌入式目标系统上运行，而 gdb 调试器则运行在 Linux 服务器上，因此需要采用远程调试的方式。

采用这种远程调试的原因就和交叉编译的原因是一样的，均是由于受到嵌入式设备的资源限制，因此需要利用 Linux 服务器的部分资源，完成系统的调试。

远程调试需要运行在嵌入式目标系统上的调试服务器 gdbserver 和运行在 Linux 服务器上的特殊版本的 gdb 调试器两部分共同完成，特殊版本的 gdb 调试器是专门用于调试运行在嵌入式目标系统上的应用程序的。在整个调试过程个，gdb 和 gdbserver 需要进行通信，如果嵌入式目标系统支持网络通信，就可以采用 TCP/IP 方式，否则可以采用串口通信方式进行调试。

我们在已建立的 Linux 环境调试程序，采用 TCP/IP 通信方式，具体硬件连接如图 5-21 所示。

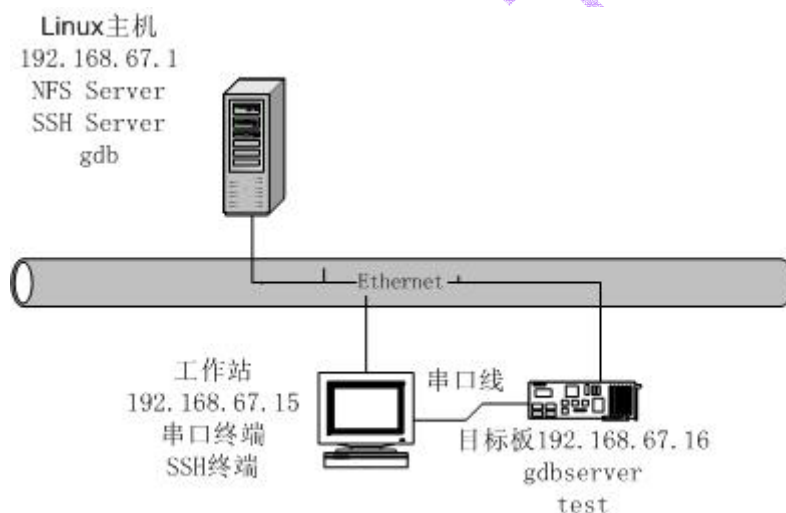


图 5-21 远程调试的系统硬件连接

将前文所述的示例程序 test.c 拷贝到 Linux 服务器我们所约定的工作目录:/home/work，并在该目录下交叉编译生成一个带调试信息的可执行程序，命令如下：

```
# arm-linux-gcc -g -o test test.c
```

采用前文所述的 NFS 方式，将工作目录与嵌入式目标系统建立连接，然后将 gdbserver 程序下载到嵌入式目标系统运行。在嵌入式目标系统的交互终端输入如下命令启动 gdbserver：

```
# gdbserver 192.168.67.1:2345 test
```

该命令表示 gdbserver 接收来自 Linux 服务器（192.168.67.1）的调试命令，gdbserver 监听 2345 端口，然后启动 test 程序，会看到“Process test created; pid = 51”等类似信息。

回到 Linux 服务器终端，运行一个特殊版本的 gdb 调试器，虽然该调试器是在 Linux

服务器上运行，但能够调试运行在嵌入式目标系统上的应用程序。

这种gdb和以及gdbserver可以在一些开发工具包里得到，比如MIZI Linux 2.0 开发包 (http://www.mizi.com/developer/mz20/tools_list.htm)。

键入如下命令运行 gdb 调试器：

```
# /opt/host/armv4l/bin/armv4l-redhat-linux-gdb test
```

可以看到如图 5-22 所示信息。

```
GNU gdb 5.2.1
Copyright 2002 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "--host=i686-pc-linux-gnu --target=armv4l-redhat-linux"...
(gdb) █
```

图 5-22 远程调试嵌入式系统的应用程序（一）

在输出信息的最后一行可以看到该 gdb 是在 x86 的主机上执行，但调试目标是 ARM 代码。

在 gdb 命令行输入命令“target extended-remote <目标系统 IP>:监听端口”：

```
# target extended-remote 192.168.67.16:2345
```

可以看到如图 5-23 所示信息。

```
(gdb) target extended-remote 192.168.67.16:2345
Remote debugging using 192.168.67.16:2345
0x40001c60 in ?? ()
```

图 5-23 远程调试嵌入式系统的应用程序（二）

该信息表明运行 gdb 调试器的 Linux 服务器已经和运行 gdbserver 的嵌入式目标系统建立了连接，可以进行应用程序的远程调试了。

切换到嵌入式目标系统的交互终端，也可以看到“Remote debugging from host 192.168.67.1”信息。

类似于在 Linux 服务器上调试应用程序，可以在 Linux 服务器的 gdb 命令行下输入“l”来列出程序源码，如图 5-24 所示。

```
(gdb) l
1      #include <stdio.h>
2      int main(void)
3      {
4          int i,sum=0,temp_sum=0;
5          for(i=1;i<=10;i++)
6          {
7              sum=sum+i;
8              temp_sum=sum;
9              printf("temp_sum = %d\n",temp_sum);
10         }
(gdb)
11         printf("Calculate finished! sum = %d\n",sum);
12         return 0;
13     }
```

图 5-24 远程调试嵌入式系统的应用程序（三）

然后用“c”命令来继续执行，注意不需要用 r 命令来执行程序，因为程序已经在目标板上由 gdbserver 启动了。此时在主机上可以看到如下信息，表明程序已经执行完毕，如图 5-25 所示。

```
(gdb) c
Continuing.

Program exited normally.
```

图 5-25 远程调试嵌入式系统的应用程序（四）

由于应用程序是嵌入式目标系统上执行，输出信息当然也是在目标系统的交互终端。切换到目标系统的交互终端，可以看到如图 5-26 所示信息。

```
Remote debugging from host 192.168.67.1
temp_sum = 1
temp_sum = 3
temp_sum = 6
temp_sum = 10
temp_sum = 15
temp_sum = 21
temp_sum = 28
temp_sum = 36
temp_sum = 45
temp_sum = 55
Calculate finished! sum = 55

Child exited with retcode = 0

Child exited with status 1
Killing inferior
GDBserver restarting
Process test created; pid = 52
```

图 5-26 远程调试嵌入式系统的应用程序（五）

gdbserver 又会重新启动程序，以便进行下一次的调试。同样的，我们可以在 Linux 服务器的 gdb 命令行下设置断点、单步执行、查看设置变量值等，方法和在 Linux 服务器上调试一样，需要注意的只是应用程序在嵌入式目标系统上运行，输入和输出都是在目标系统的交互终端。

事实上，采用这种方式调试运行在嵌入式系统上的应用程序，其过程仍然显得有一些复杂，同时对调试环境的依赖性也比较大，因此，在实际的软件调试过程中，还需要经常结合其他手段，如使用 printf（）函数在合适的位置输出变量值和其他用户自定义信息等，以提高程序的调试效率。

5.8 本章小节

本章介绍了嵌入式 Linux 的基本概念、嵌入式 Linux 开发环境的建立、嵌入式 Linux 内核的配置与编译、NFS 的建立，以及与应用程序开发调试等相关问题。

本章所涉及的内容，都是嵌入式 Linux 系统开发所面临的共同问题，同时也是必须解

决的基本问题，读者若有兴趣从事这方面的开发工作，希望能提供一些参考。

www.emsyschina.com

第 6 章 嵌入式 Linux 的应用开发

本章介绍了嵌入式 Linux 的常用应用开发,包括设备驱动程序的开发、网络应用开发、基于 Web 的应用开发以及图形应用开发等。

本章主要内容:

- Linux 设备驱动概述;
- Linux 字符设备驱动的开发;
- 网络应用的开发;
- 串口应用的开发;
- 基于 Web 的应用开发;
- 图形应用开发;

6.1 Linux 设备驱动程序开发

6.1.1 Linux 设备驱动概述

在基于 PC 的应用程序中,由于 PC 架构的一致性,绝大多数底层的硬件驱动都由设备制造商完成,用户在进行应用程序的开发过程中,一般都感觉不到硬件的存在,因此不需要涉及更多的硬件细节,然而,在嵌入式系统的开发过程中,情况就完全不同了,嵌入式系统的差异性决定了系统开发者不但需要进行特定硬件的设计,同时需要编写相关的设备驱动程序。

设备驱动程序在 Linux 内核中占有极其重要的位置,它是内核用于完成对物理设备的控制操作的功能模块。除了 CPU,内存以及其他很少的几个部分之外,所有的设备控制操作都必须由与被控设备相关的代码,也就是驱动程序来完成。内核必须包含与系统中的每个外部设备对应的驱动程序,否则设备就无法在 Linux 下正常工作。这也是驱动程序开发成为 Linux 内核开发的主要工作的原因,从内核源码的代码分布可以看出,设备驱动源码大都在 drivers 目录下,至少占据了一半的内核源码量。

由于实际的外部设备千差万别,在 Linux 下采用了一些与设备无关的模型,通过驱动程序定义了一些统一的编程接口,完全隐藏了具体设备的工作细节,当应用程序需要对该设备进行操作时,就通过标准化的与设备无关的系统调用完成,由驱动程序完成这些系统调用与实际的设备操作的映射。

在 Linux 中,将各种设备大致分成三种基本类型:字符设备、块设备及网络设备。

其中,字符设备是指采用字符流方式访问的设备,如字符终端,串口,一般只能按顺

序访问，但也可以前后移动访问指针。对应的字符设备驱动的接口是标准的文件系统调用接口。

块设备是采用数据块方式访问的设备，如磁盘等。块设备的存取是通过缓冲方式来进行的，并且可以进行随机访问，即不管块位于设备中何处都可以对其进行读写。对应的块设备驱动接口除了和字符设备驱动一样的文件系统调用接口之外，还提供了专门的面向块设备的接口，但这个接口对用户来说是不可见的。由于块设备通常都提供文件系统支持，因此常用的设备访问方式是通过文件来进行。

字符设备和块设备在 Linux 中都被看成是一种特殊的文件，称为设备文件，例如系统中串口一的设备文件通常就是 `/dev/ttyS0`。由于每个设备都至少由文件系统中的文件来代表，这样的设备文件在系统中就唯一确定了对应的设备，而对于设备的访问，则是由用户通过对设备文件进行标准的文件系统调用来操作的，如文件的打开、关闭、以及读/写等。设备文件一般是通过 `mknod` 系统调用来创建。

网络设备是采用数据包传输方式访问的设备。和前两者不同，由于不是面向流的设备，因此将网络设备映射为文件系统中的设备文件就比较困难，Linux 采用的方法是给它们分配一个唯一的名字（比如 `eth0`）。同时，内核与网络设备驱动间的通信完全不同于与字符设备以及块设备驱动程序之间的通信，而是采用了一套和数据包传输相关的函数，如最常见的套接字（`socket`）接口。网络设备一般通过 `ifconfig` 系统调用来创建和配置。

在 Linux 中，还有一些驱动程序，不属于以上的三种类型。这种驱动程序最常见的就是各种总线驱动，如 USB、PCI、SCSI 等，它们一般是作为内核提供的公共服务，和其他驱动联合起来使用。

尽管 Linux 内核中存在许多不同的设备驱动，但它们都具有如下一些共性：

内核代码

设备驱动是内核的一部分，和内核中其它代码一样，出错将导致系统的严重损伤。一个编写不好的设备驱动甚至能使系统崩溃并导致文件系统的破坏和数据丢失。

内核接口

设备驱动必须为 Linux 内核或其从属子系统提供一个标准接口，该接口根据驱动类型确定，字符设备驱动和块设备驱动一般采用文件接口，而网络设备驱动一般采用套接字接口，其他驱动程序采用自定义接口。

内核机制与服务

设备驱动可以使用标准的内核服务如内存分配、中断发送和等待队列等。

动态可加载

多数 Linux 设备驱动可以作为模块动态的链接到内核中，同时可在不再使用时卸载，这样可以灵活的配置内核的功能和大小，从而使内核能有效地利用系统资源。

可配置

Linux 设备驱动可以连接到内核中。当内核被编译时，哪些驱动程序被连入内核是可配置的。

动态性

当系统启动及设备驱动初始化时将查找它所控制的硬件设备。如果某个设备的驱动为一个空过程并不会有什么问题，此时该设备驱动仅仅是一个冗余的程序，它除了会占用少量系统内存外不会对系统造成什么危害。

以下我们介绍 Linux 系统的三大类设备和内核的接口，理解设备驱动与内核的接口对于编写 Linux 设备驱动是非常关键的。

6.1.2 字符设备

字符设备是 Linux 设备中最简单和最常见的一种，应用程序可以同存取文件相同的系统调用来打开、读写及关闭它。

字符设备初始化时，它的设备驱动通过在内核的 device_struct 结构的 chrdevs 数组中添加一个入口来将其注册到 Linux 内核中，当设备注册成功以后，就由内核分配一个设备号和一个用户自定义的设备文件名，对该设备进行标识。

chrdevs 数组每个入口中的 device_struct 数据结构包含两个元素：一个指向已注册的设备驱动名称，另一个则是指向一组文件操作指针，它们是位于此字符设备驱动内部的文件操作函数的地址指针，用来处理相关的文件操作如打开、读写与关闭等。

当打开代表字符设备的设备文件时，内核必须作好准备以便调用相应字符设备驱动的文件操作函数。



图 6-1 字符设备和内核的接口

当应用程序打开字符设备文件时，就使用由内核分配的设备号来索引 chrdevs 数组，以便得到相关文件操作函数指针，同时建立起描述此字符设备文件的 file 结构，使其文件操作指向此设备驱动中的文件操作函数集合，这样，所有应用程序对它进行的文件操作都被映射到此字符设备的文件操作函数集合上。

字符设备和内核的接口如图 6-1 所示。

我们简单描述一下字符设备驱动接口中关键的 file_operations 结构，基于 Linux2.4.21

的<linux/fs.h>的定义如下:

```
struct file_operations {
    struct module *owner;
    /*指向拥有本结构的 module, 用于内核维护模块的引用计数。*/
    loff_t (*llseek) (struct file *, loff_t, int);
    /*改变当前文件的读/写位置并返回新位置, 错误返回负数。*/
    ssize_t (*read) (struct file *, char *, size_t, loff_t *);
    /*用来从设备接收数据, 返回的非负数表示读取的字节数。*/
    ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
    /*用来往设备发数据, 返回的非负数表示成功写入的字节数。*/
    int (*readdir) (struct file *, void *, filldir_t);
    /*仅用于文件系统的目录读取, 不用于设备驱动。*/
    unsigned int (*poll) (struct file *, struct poll_table_struct *);
    /*是两个系统调用 poll 和 select 的背后支撑。如果驱动未定义, 则假设设备既可读又可写。返回值是表示设备状态的掩码。*/
    int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
    /*用于给设备发送命令的接口函数。如果驱动不提供, 则所有调用失败, 返回 ENOTTY。*/
    int (*mmap) (struct file *, struct vm_area_struct *);
    /*用于请求将设备内存映射到进程空间。如果驱动不提供, 则所有调用失败, 返回 ENODEV。*/
    int (*open) (struct inode *, struct file *);
    /*打开设备, 通常是对设备的第一个操作函数。*/
    int (*flush) (struct file *);
    /*当一个进程关闭掉设备文件时, 该函数将被系统调用。它应该执行并等待设备上未完成操作。*/
    int (*release) (struct inode *, struct file *);
    /*关闭设备。只有当设备文件的所有拷贝都被释放时, 才进行 release 调用, 而不是每次调用 close 时都执行。*/
    int (*fsync) (struct file *, struct dentry *, int datasync);
    /*是系统调用 fsync 的背后支撑, 用户可调用 fsync 来刷新缓存数据。*/
    int (*fasync) (int, struct file *, int);
    /*用于通知设备的 FASYNC 标志被改变。*/
    int (*lock) (struct file *, int, struct file_lock *);
    /*锁住文件。一般设备不支持。*/
    ssize_t (*readv) (struct file *, const struct iovec *, unsigned long, loff_t *);
    /*适合于一次读设备需要使用多个缓存区的情况。其他同 read。*/
    ssize_t (*writev) (struct file *, const struct iovec *, unsigned long, loff_t *);
    /*适合于一次写设备需要使用多个缓存区的情况。其他同 write。*/
};
```

```
ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t
*, int);
    /*将内存页发给文件，通常用于文件拷贝的情况，一般设备不支持。*/
    unsigned long (*get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
    /*得到设备未经过内存映射的设备内存地址。*/
};
```

在以上描述的 `file_operations` 结构所包含的文件操作函数中，一些常用的文件操作函数的使用方法，我们将在其后章节的字符设备驱动程序示例中说明。

6.1.2 块设备

块设备也支持以文件方式访问，系统对块设备提供了非常类似于字符设备的文件操作机制，但字符设备驱动的接口相对清晰且易于使用，而块设备驱动接口则要复杂一些。

与 `chrdevs` 数组一样，Linux 在 `blkdevs` 数组中维护所有已注册的块设备。`blkdevs` 也使用设备的设备号进行索引，其入口也是 `device_struct` 结构。和字符设备不同的是系统通常有几类块设备，例如 SCSI 设备是一类，而 IDE 设备则是另外一类。它们将以各自类别登记到 Linux 内核中并为内核提供文件操作功能。

由于块设备的注册与请求处理相对复杂，我们在此不作详述，如果用户需要深入了解，可以参考相关书籍。

6.1.3 网络设备

在 Linux 中，网络是一个单独的子系统。网络子系统是一个用于发送与接收数据包的部件，一般由一个如以太网卡之类的物理设备支持，但有些网络设备，如 loopback 设备，则仅仅是一个用来向自身发送数据的软件。

每个网络设备都用一个 `device` 结构来表示。网络设备驱动在内核启动初始化网络时将这些受控设备注册到 Linux 中。`device` 数据结构中包含有关设备的信息以及用来支持各种网络协议的函数指针，这些函数主要用来使用网络设备传输数据。

关于网络设备更详细的内容及编程方法，我们在以后的相关章节描述。

6.2 Linux 字符设备驱动的开发

以下我们举例说明如何在 Linux 下开发字符设备驱动。

示例中驱动的设备是 HDB93XX 开发系统的 GPIO：EGPIO8～EGPIO15。通过设备驱动的方式对 GPIO 设置高、低电平。

示例由设备驱动程序和用户应用程序两部分构成。

6.2.1 设备驱动程序

我们首先编写如下设备驱动程序

```

/*****
 * File Name: GPIOdrv.c
 * Description: drive GPIO on HDB93XX system.
 * Author: Wei An.
 *****/

#ifdef MODULE
#define __NO_VERSION__
#include <linux/module.h>
#include <linux/version.h>
#endif

#define __KERNEL__
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/sched.h>
#include <linux/fs.h>
#include <linux/mm.h>
#include <linux/poll.h>
#include <linux/slab.h>
#include <linux/ioport.h>
#include <asm/uaccess.h>
#include <asm/io.h>
#include <linux/fcntl.h>
#include <asm-arm/arch-ep93xx/regmap.h>

/* 定义写设备函数 */
static int GPIO_write(struct file *,const char *,int,loff_t *);

/* 静态函数和变量声明 */
static void Delay(int);
static unsigned int major =0;

/* 定义文件操作指针 file_operations GPIO_fops 结构并赋值 */
static struct file_operations GPIO_fops=
{
    write      :(void(*) )GPIO_write,
};

```

```
/* 定义设备名称 */
char GPIO_name[]="GPIOdrv";

/* 模块初始化 */
static int __init GPIOdrv_init_module(void)
{
    int retv;
    /* 注册设备 */
    retv=register_chrdev(major,GPIO_name,&GPIO_fops);
    if(retv<0)
    {
        printk("<1>Register Fail!\n");
        return retv;
    }
    if(major==0)
        major=retv;

    printk("<1>GPIO device OK!\n");
    return 0;
}

/* 注销模块 */
static void __exit GPIOdrv_cleanup(void)
{
    int retv;
    /* 注销设备 */
    retv=unregister_chrdev(major,GPIO_name);
    if(retv<0)
    {
        printk("<1>UnRegister Fail!\n");
        return;
    }
    printk("<1>GPIODRV:GOOD-bye!\n");
}

/* 写设备函数的具体实现 */
static int GPIO_write(struct file *GPIO_file, const char *buf,int
len,loff_t *loff)
{
    unsigned int iopdata;
    volatile unsigned int *pbddr;
```



```

volatile unsigned int *pbdr;
/* 申请虚拟地址 */
pbddr=(volatile unsigned int *) IO_ADDRESS(GPIO_PBDDR);
pbdr=(volatile unsigned int *) IO_ADDRESS(GPIO_PBDR);
/* 设置 EGPI08~EGPI015 为输出 */
*pbddr=0xff;
/* 从用户空间读取数据 */
if(copy_from_user((char*)&iopdata,buf,len))
    return -EFAULT;
/* 输出用户数据到 EGPI08~EGPI015 */
*pbdr=iopdata;
return len;
}

/* 软件延时函数 */
static void Delay(int x)
{
    int i,j,k;
    for(i=0;i<x;i++)
        for(j=0;j<0xff;j++)
            for(k=0;k<0xff;k++)
                ;
}

module_init(GPIOdrv_init_module);
module_exit(GPIOdrv_cleanup);
/*end of GPIOdrv.c*/

```

我们来看看上面这个字符设备驱动程序都包含了那些主要内容，以此说明编写这类设备驱动程序的一般方法。

注册设备

在 Linux 中，任何设备都有一个主设备号作为操作系统区别不同驱动程序的标识。在本例程中为 GPIO 设备动态的申请了一个设备号，由如下语句实现：

```
retv=register_chrdev(major,GPIO_name,&GPIO_fops);
```

函数 register_chrdev() 在 <linux/fs.h> 中定义，用于向内核注册字符设备。其中，major 参数为向内核请求分配的主设备号，如果 major 为 0，则 register_chrdev() 函数返回内核为设备动态分配的设备号。

如果需要指定分配一个设备号，则由 major 参数传递，如果申请成功，register_chrdev() 函数返回 0，否则返回负值。

内核源码目录下的 /Documentation/devices.txt 记录了内核已经分配的主设备号，许

多数字已经被分配了，要选择一个不重复的设备号比较困难，因此可以选择给设备申请一个动态设备号，但这样带来的问题是创建设备文件节点的时候，每次可能使用的设备号并不相同。

注销设备

注销设备是和注册设备相对应的操作，当卸载模块的时候，系统将注销模块对应的设备，释放主设备号，由如下函数实现：

```
unregister_chrdev(major, GPIO_name)
```

函数 `unregister_chrdev()` 用于注销设备，释放设备号和设备文件名。

文件操作

定义系统调用的结构 `file_operations` 访问驱动程序的函数。`file_operations` 结构是定义在 `linux/fs.h` 中的函数指针数组。每个文件都与它自己的函数集相关联，这些操作主要负责系统调用的实现。如本例程中定义的 `GPIO_write()` 函数就是系统调用 `write` 指向的驱动函数，还要很多的字符设备系统调用方法，如 `read`, `ioctl` 等，如果需要，都可以在这个结构体中指向实际的驱动函数。

```
static struct file_operations GPIO_fops=
{
    write      :(void(*) )GPIO_write,
};
```

系统调用的实现方法

用户可以通过文件操作，使得系统调用和设备驱动的实现方法之间对应起来，具体的实现函数就是 `GPIO_write()`，该函数实现向特定硬件设备的数据传输。

设备驱动程序作为 Linux 内核的一部分，运行在内核空间，而应用程序则运行在用户空间，因此，如何在内核空间与用户空间进行数据传输，是我们需要解决的问题。

内核函数 `copy_from_user()` 可以实现由用户空间到内核空间的的数据拷贝。与此相对应的从内核空间拷贝数据到用户空间的操作由内核函数 `copy_to_user()` 实现。一般 `read` 系统调用要用到内核函数 `copy_to_user()`。同时，这两个函数的作用还并不限于在内核空间和用户空间之间拷贝数据，它们还会检查用户空间的指针的有效性，如果指针无效，就不会进行拷贝；另一方面，如果在拷贝过程中遇到无效地址，则仅仅会复制部分数据。在这两种情况下，返回值是还未拷贝完的内存的数量值。

从用户空间拷贝内核空间的数据最终会对硬件设备进行操作，而从内核空间拷贝到用户空间的数据则来自于实际的硬件设备。

需要注意的是，在 Linux 下，系统不允许对设备的物理地址直接访问，而必须转换为虚拟地址后才可访问。在移植到 EP93XX 系列嵌入式微处理器的 Linux 的源码目录 `asm-arm/arch-ep93xx/regmap.h` 中，定义了 `IO_ADDRESS` 宏完成物理地址到虚拟地址的映射，`IO_ADDRESS(GPIO_PBDDR)` 可以完成把 `GPIO_PBDDR` 这个寄存器地址转换成虚拟地址，这个转换的过程如下：

```
#define IO_ADDRESS(pa) (((unsigned int)pa) - IO_BASE_PHYS + IO_BASE_VIRT)
```

可以看出, I/O 对应的虚拟地址实际上就是 I/O 的物理地址偏移量加虚拟基地址形成的。

顺便指出, 在标准 Linux 中对硬件的物理地址的访问和 uClinux 是完全不同的, 在 uClinux 下可以直接引用硬件的物理地址, 因为 uClinux 是针对无 MMU 的处理器的, 因此对硬件的物理地址采用的是实际地址。与此相反, 标准 Linux 下, 硬件的实际物理地址是直接访问不到的, 必须映射到 Linux 的虚拟地址空间统一管理。通常, 完成物理地址到虚拟地址转换的函数或者宏位于 linux 源码的/include/asm/arch-xxx/下, 如本例程中的 asm-arm/arch-ep93xx/regmap.h 文件。

在驱动程序中程序的开发中, 要注意驱动程序的作用在于提供机制 (mechanism), 而不是提供策略 (policy)。机制在于需要提供什么功能, 而策略在于如何使用这些功能。因此, 在驱动程序中不涉及对设备使用的具体方法, 如本例程中实现的是可以驱动 GPIO, 但并不关心 GPIO 为高电平或低电平, 这些问题要留给应用程序来处理。

6.2.2 把设备驱动程序编译到 Linux 内核

把驱动程序编译进内核的步骤如下:

1、把 GPIOdrv.c 拷贝到 Linux-2.4.21/drivers/char 下, 并修改该目录下的 Configs.in 文件, 添加如下内容:

```
bool 'GPIO Driver' CONFIG_GPIODRV
if [ "$CONFIG_GPIODRV" = "y" ]; then
bool 'Support for GPIO on EP93XX board' CONFIG_GPIODRV
fi
```

Config.in 是每个模块的配置脚本, 在这个文件当中定义配置那些模块, 怎么样配置等, 因此当添加了 GPIO 的配置后, 会在内核配置的时候出现这个模块的配置选项。

2、修改当前目录下的 Makefile, 添加如下内容:

```
obj-$(CONFIG_GPIODRV) += GPIOdrv.o
```

在每个模块的 Makefile 中包含该模块所包含的子模块, 在 char devices 中, 要包含 GPIO 设备, 就要告诉 Makefile 编译 GPIOdrv.c 并包含编译出的 GPIOdrv.o 文件, 这样最后内核做链接的时候, 才会链接进这个模块。

3、重新配置内核, 选中 GPIO 模块

执行命令 “make menuconfig”, 进入内核配置菜单, 选中如下选项:

```
Character devices -->
[*] GPIO Driver
[*] Support for GPIO on EP93XX board
```

4、重新编译内核, 并更新嵌入式目标系统的内核。

5、创建设备文件。

重新启动嵌入式目标系统的 Linux，执行如下命令：

```
# cd /proc
```

进入目录/proc，查看 devices 文件：

```
# cat devices
```

Character devices:

```
1 mem
2 pty
3 tty
4 vc/0
5 console
7 vcs
10 misc
13 input
14 sound
162 raw
180 usb
204 ttyAM
205 cuaam
254 GPIOdrv
```

Block devices:

```
1 ramdisk
7 loop
```

devices 文件列出了当前系统所有的字符设备和块设备，包括设备号和设备名称，在 Character devices 的最后一行的设备 254 GPIOdrv，即我们在驱动程序中定义的设备文件名，以及内核为这个设备动态分配的设备号，说明 GPIOdrv 设备已经正确的加载到了内核。

6、添加设备文件节点。

执行如下命令，添加设备文件节点：

```
# cd /dev
```

```
# mknod /dev/GPIO c 254 0
```

mknod 命令用于创建设备文件节点，命令格式为：

```
mknod [设备文件名] [设备类型] [主设备号] [次设备号]
```

在此创建的设备名称用于在应用程序中访问对应的设备；如果为字符类型设备，设备类型用 c 表示，块设备则用 b 表示；主设备号就是/proc/devices 中的 GPIOdrv 的设备号，由于该类设备只有一个，因此从设备号为 0，如果还有该类型的其它设备，则依次为 1、2、3 等。

到此为止，GPIO 设备的驱动工作全部完成，以下就可以通过编写应用程序访问并测试特定设备。

6.2.3 通过应用程序访问设备

通过应用程序访问特定设备的步骤如下：

1、编写访问设备的应用程序。

访问设备的应用程序示例代码如下：

```
/* *****  
 * File Name: GPIOdrv_test.c  
 * Description: Test GPIO driver on EP93XX board.  
 * Author: Wei An.  
 * *****  
#include <stdio.h>  
#include <string.h>  
#include <malloc.h>  
#include <sys/types.h>  
#include <sys/stat.h>  
#include <fcntl.h>  
#include <unistd.h>  
#include <signal.h>  
  
/* 静态函数声明 */  
static void Delay(int);  
  
int main(void)  
{  
    /* 定义文件句柄、返回值 */  
    int fd, count;  
    int i, j;  
  
    /* 定义用户数据 */  
    unsigned char iodata;  
  
    /* 打开 GPIO 设备文件 */  
    if((fd=open("/dev/GPIOdrv", O_RDWR))==-1)  
    {  
        perror("open error");  
        exit(1);  
    }  
  
    /* 用户数据赋值，复位 GPIO */  
    iodata=0x0;  
    /* 向文件写入用户数据 */
```

```
if((count=write(fd, (char *)&iodata, 1))!=1)
{
    perror("write error");
    exit(1);
}
for(j=0; j<5; j++)
{
    /* EGPI08~EGPI015 依次为高电平 */
    for(i=0; i<8; i++)
    {
        iodata|=0x1<<i;
        if((count=write(fd, (char *)&iodata, 1))!=1)
        {
            perror("write error");
            exit(1);
        }
        Delay(25);
    }
    /* EGPI08~EGPI015 依次为低电平*/
    for(i=0; i<8; i++)
    {
        iodata&=0x7F>>i;
        if((count=write(fd, (char *)&iodata, 1))!=1)
        {
            perror("write error");
            exit(1);
        }
        Delay(25);
    }
}

/* 关闭设备文件 */
close(fd);
return 0;
}

static void Delay(int x)
{
    int i, j, k;
    for(i=0; i<x; i++)
        for(j=0; j<0xff; j++)
            for(k=0; k<0xff; k++)
```

```
        ;  
    }  
/*end of GPIOdrv_test.c*/
```

2、编译生成可执行文件。

在 Linux 服务其执行如下命令编译生成可执行文件：

```
# /usr/local/arm/2.95.3/bin/arm-linux-gcc -o GPIOdrv_test GPIOdrv_test.c
```

命令执行完毕以后，在当前目录生成可执行文件 GPIOdrv_test。

3、下载可执行文件到嵌入式目标系统并运行。

通过第 5 章所描述的 FTP 方式下载可执行文件到目标系统或通过 NFS 方式，执行如下命令运行该可执行文件：

```
# ./ GPIOdrv_test
```

应用程序的运行结果为基于 EP93XX 系列微处理器的开发系统的 EGPI08~EGPI015 依次为高电平，然后再改变为低电平，用户可以采用在 EGPI08~EGPI015 外接 LED 的方式，查看程序的运行状况。

从上面的应用程序，可以体会到编写设备驱动程序不提供策略，只提供机制的含义。在应用程序中可以任意的对 GPIO 施加控制，而底层的操作对用户是透明的。

编写设备驱动程序既要掌握一定的设备驱动的知识，更要不断的实践，只有在实际的操作中才能体会到编写设备驱动中的一些规则。同样的一个设备，不同的人去写驱动可能都是不一样的，但彼此之间肯定有好坏之分，一个编写良好的驱动程序不但稳定，而且尽可能的提供给用户多的功能，很好的区分了机制和策略。

6.3 网络应用程序的开发

Linux 提供了强大的网络功能，可以支持多种网络协议，同时，Linux 还提供了许多实用的网络工具，因此，使用 Linux 就不能不涉及网络。

Linux 下的网络编程一般使用套接字(socket)接口，套接字是 IP 地址和相应 TCP/UDP 端口号的组合体，其中，IP 地址标识了网络中的计算机，端口号则标识用于网络通信的特定进程。

套接字接口能够同时向其它进程发送数据或从其它进程接收数据，可以使用三种套接字进行网络通信：流格式、数据报格式和原始格式套接字，每一种类型的套接字都分别代表了一种不同类型的通信服务。

- λ 流格式套接字：提供了可靠的基于连接的通信。基于连接的通信过程中所包含的两个进程必须建立相互间的逻辑连接。然后，字节的信息流在没有错误和副本的情况下被发送出去，在接收时，其顺序与发出时一致。流格式套接字对应于 TCP/IP 协议中的 TCP 协议。

- λ 数据报格式套接字：信息以数据包的形式发送，通过称为数据报的离散信息进行通信。数据报格式套接字是无连接的，也就是说，通信进程相互之间并不存在着逻辑连接。数据报或许会丢失或被复制，或许它们不会按照发送时的顺序被接收。数据报格式套接字对应于 TCP/IP 协议中的 UDP 协议。
- λ 原始格式套接字：直接支持对底层协议所进行的访问，例如，IP 和因特网控制消息协议（ICMP）。

可以通过如下函数使用套接字进行网络通信：

`socket()`：该函数用于创建套接字；

`bind()`：该函数将套接字和网络地址联系在一起；

`connect()`：该函数将套接字和远程网络地址连在一起；

`listen()`：该函数侦听传入的连接意图；

`accept()`：该函数侦听传入的连接意图；

除了以上函数以外，还有 `setsockopt()`、`getsockopt()`、`fcntl()` 和 `ioctl()` 等函数可用于操作套接字接口的属性，而 `select()` 函数可用于识别处于特定状态的套接字接口，`close()` 函数可用于关闭套接字接口的连接。

可用下面的函数向套接字接口写入数据：`write()`、`writew()`、`send()`、`sendto()` 和 `sendmsg()`。

可用下面的函数从套接字接口读出数据：`read()`、`readv()`、`recv()`、`recvfrom()` 和 `recvmsg()`。

基于套接字接口的进程通常用于客户机/服务器（B/S）编程，客户端进程由用户直接或间接操纵，而服务器端进程则驻留在主机上等待连接进入。由于服务器进程能够自动连续地运行，这种进程也通常被叫做端口监控进程。

以下我们通过一个例子说明在嵌入式 Linux 下，如何通过流式套接字编程访问网络资源。在本例中，将 HDB9302 目标系统作为服务器，运行服务器程序，接收来自网内主机的 TCP 包信息，并显示到超级终端上。

第一步：编辑服务器端 `server.c` 文件

```

/*****
 * File Name: server.c
 * Description: Run as Server and Receive packages from client
 * Author: Wei an
 *****/

#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <sys/socket.h>

```

```
#include <sys/wait.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <unistd.h>

/*服务器监听的端口号 */
#define SERVPOR 2222
/* 最大同时连接请求数 */
#define BACKLOG 8

int main()
{
    /* 定义临时缓冲区 */
    char tmpBuff[20];
    int sin_size=0;
    /*sock_fd: 监听 socket; client_fd: 数据传输 socket */
    int sockfd,client_fd;
    /* 本地地址信息 */
    struct sockaddr_in my_addr;
    /* 客户端地址信息 */
    struct sockaddr_in remote_addr;
    /* 创建 socket */
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1)
    {
        perror("socket 创建错误!");
        exit(1);
    }
    /* Ipv4 */
    my_addr.sin_family=AF_INET;
    /* 服务器端口 */
    my_addr.sin_port=htons(SERVPOR);
    /* 内核指定服务器 IP 地址 */
    my_addr.sin_addr.s_addr = INADDR_ANY;
    bzero(&(my_addr.sin_zero),8);
    /* 绑定端口 */
    if (bind(sockfd, (struct sockaddr *)&my_addr, sizeof(struct
sockaddr)) == -1)
    {
        perror("bind 错误! ");
        exit(1);
    }
    /* 监听连接 */
```

```
if (listen(sockfd, BACKLOG) == -1)
{
    perror("listen 错误!");
    exit(1);
}
while(1)
{
    sin_size = sizeof(struct sockaddr_in);
    /* 建立连接 */
    if ((client_fd = accept(sockfd, (struct sockaddr *)
        &remote_addr, (socklen_t *)&sin_size)) == -1)
    {
        perror("accept 错误");
        continue;
    }
    /* 连接成功建立 */
    printf("received a connection from %s\n",
        inet_ntoa(remote_addr.sin_addr));
    while(1)
    {
        /* 接收数据 */
        if (0 == recv(client_fd, tmpBuff, 20, 0))
            break;
        tmpBuff[20] = '\0';
        printf("recieve data : %s \n", tmpBuff);
        /* 回发数据 */
        if (0 == send(client_fd, tmpBuff, 20, 0))
            break;
        /* 清除缓冲区 */
        memset(tmpBuff, 0, 20);
    }
    /* 关闭套接字 */
    close(client_fd);
    break;
}
}
/* end of server.c */
```

第二步：编译 server.c

在宿主机上执行

```
# /usr/local/arm/2.95.3/bin/arm-linux-gcc -o server server.c
```

如果程序没有错误，则在当前目录下生成名为 server 的可执行文件。

第三步：执行 server

执行如下命令配置目标板 IP 地址

```
#ifconfig eth0 [局域网内部 IP 地址]
```

在此我们配置目标板的 IP 地址为：

```
#ifconfig eth0 192.168.67.39
```

然后通过 FTP 或者 NFS 方式执行 server

```
# ./server
```

第四步：配置客户端主机的超级终端

在 Windows 操作系统下单击开始→程序→附件→通讯→超级终端，打开超级终端，打开文件→属性，如图 6-2 配置。



图 6-2 超级终端配置界面（一）

在随后弹出的属性页中如图 6-3 配置。



图 6-3 超级终端配置界面（二）

配置完毕以后点击超级终端工具栏上的连接图标，如果服务器端正确并且超级终端配置正确，则超级终端应该建立连接。同时服务器端的超级终端应该显示 received a connection from [客户端主机 IP 地址]，表明客户端和服务器的 TCP 链接已经建立。此时，在客户端的超级终端中任意输入字符，在服务器端的超级终端中就会显示出接收到的字符。

6.4 基于串行通信的应用程序开发

串行通信在嵌入式设备中具有非常重要的地位，几乎所有的嵌入式设备都具有串行接口，并能够通过串行接口与外部进行通信。串行接口可以充当嵌入式设备的输入输出接口，因此，在嵌入式设备的开发中，第一步就是使用串行接口，很多嵌入式程序的下载，调试等都是通过串行接口进行，针对串行接口进行的开发也是嵌入式 Linux 开发中的一个重要部分。

串行接口标准包括 RS-232、RS-422 和 RS-485，最初都是都是由电子工业协会（EIA）制订并发布的，RS-232 在 1962 年发布，命名为 EIA-232-E，作为工业标准，以保证不同厂家产品之间的兼容。RS-422 由 RS-232 发展而来，它是为弥补 RS-232 之不足而提出的。为改进 RS-232 通信距离短、速率低的缺点，RS-422 定义了一种平衡通信接口，将传输速率提高到 10Mb/s，传输距离延长到 4000 英尺（速率低于 100kb/s 时），并允许在一条平衡总线上连接最多 10 个接收器。RS-422 是一种单机发送、多机接收的单向、平衡传输规范，被命名为 TIA/EIA-422-A 标准。为扩展应用范围，EIA 又于 1983 年在 RS-422 基础上制定了 RS-485 标准，增加了多点、双向通信能力，即允许多个发送器连接到同一条总线上，同时增加了发送器的驱动能力和冲突保护特性，扩展了总线共模范围，后命名为 TIA/EIA-485-A 标准。由于 EIA 提出的建议标准都是以“RS”作为前缀，所以在通讯工业领域，仍然习惯将上述标准以 RS 作前缀称谓。这三种接口在嵌入式设备中都有应用，但最为常用的是 RS-232 接口。最早的标准是定义了 25 个针脚的 DB25 连接器，但在嵌入式设备中一般是使用 9 个针脚定义的 DB9 连接器。每种连接器又可以分成孔型（阴头）和针型（阳头）两种。针型接口如图 6—4 所示，各引脚定义由表 6—1 说明。孔型接口的 2 和 3 互换，即 2 为 TXD，3 为 RXD，7 和 8 互换，即 7 为 CTS，8 为 RTS，其余针脚都和针型接口相同。

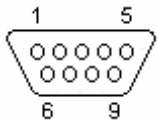


图 6—4 RS-232 DB9 接口

表 6—1 RS-232 DB9 各引脚定义说明

序号	名字	流向	说明
1	CD	←	载波检测（Carrier Detect），表示 DCE 接收到远程载波
2	RXD	←	接收数据（Receive Data）
3	TXD	→	发送数据（Transmit Data）
4	DTR	→	DTE 准备好（Data Terminal Ready），表示 DTE 可以进行通信
5	GND		系统地（System Ground）
6	DSR	←	DCE 准备好（Data Set Ready），表示 DCE 可以进行通信
7	RTS	→	请求发送（Request to Send），DTE 请求 DCE 将线路切换到发送方式

8	CTS	←	允许发送（Clear to Send），DCE 告诉 DTE 线路已接通可以发送数据
9	RI	←	振铃指示（Ring Indicator），表示 DCE 与线路接通，出现振铃

如果嵌入式设备提供的串行接口能完全提供这 9 针的功能，可以称为全串口，但更常见的是设备提供的串行接口只支持 3 针（RXD，TXD，GND）或 5 针（RXD，TXD，RTS，CTS，GND）。这在我们以后的编程中需要注意。

在进行串口通信编程之前，需要了解串口通信的一些基本概念和术语。串口通信包括异步通讯和同步通讯两种通讯方式，但最常用的是异步通讯。异步通讯可以随时进行数据通信，需要通信双方知道如何判别数据的起始和终止，所以发送的数据帧由起始位，数据位，校验位，和停止位构成，各部分的单位是位(bit)。起始位总是 space(0)。

串口通信的一些常用术语解释如下：

波特率：串口通信的传输速度。对于 RS-232/422/485 来讲，波特率的单位是标准 bps(位每秒)。

数据位：一次串口数据传输的有效数据位，有 5，6，7，8 等。

数据校验：如果无校验，则不传输校验位，如果有校验，则传输一位校验位。具体取值根据校验类型来定，分奇校验，偶校验，Space 校验和 Mark 校验四种。奇校验保证数据和校验位的 1 个数为奇数。偶校验保证数据和校验位的 1 个数为偶数。Space 校验就是校验位总是 0。Mark 校验就是校验位总是 1。

停止位：一次串口数据传输的停止位，有 1，1.5，2 等。

流量控制：由于数据在两个串行接口之间进行传输时，如果通信双方的处理能力不相匹配，可能会出现数据丢失的现象。例如接收端处理速度较慢，数据缓冲区已满，而发送端还在不断发送数据，则此时发送来的数据或者丢失，或者覆盖掉数据缓冲区中未经过处理的数据，出现数据丢失。而流量控制（简称流控）就是来解决这个问题的，当接收端数据处理不过来时，就发出“不再接收”的信号，发送端就停止发送，直到收到“可以继续发送”的信号再发送数据。因此流控可以控制数据传输的过程，防止数据丢失。串口通信中常用的流控方式有硬件流控（包括 RTS/CTS、DTR/CTS 等）和软件流控 XON/XOFF（继续/停止）。

在熟悉串口通信的概念后，我们先来看一下 Linux 下串口通信的例程，第一个例程为一个简单的串口发送程序，它需要两个参数，一个为发送次数，另一个为发送数据。第二个例程为串口接收程序。两个例程都是使用目标系统的第二个串口，设备名为 ttyS1，设置为波特率 57600，8 位数据位，无校验，1 位停止位，无流控。

第一个例程的具体实现如下：

```
/* Description: send data to COM
 * Author: Zheng Geng <gzheng@hyesco.com>
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <termios.h>
#include <errno.h>
#include <string.h>

typedef unsigned char U8;
typedef unsigned short U16;
typedef unsigned long U32;

/* 串口设置函数 */
void setTermios(struct termios * pNewtio, U16 uBaudRate)
{
    bzero(pNewtio, sizeof(struct termios)); /* clear struct for new port
settings */

    pNewtio->c_cflag = uBaudRate | CS8 | CREAD | CLOCAL;
    pNewtio->c_iflag = IGNPAR;
    pNewtio->c_oflag = 0;
    pNewtio->c_lflag = 0; //non ICANON
    /*
    initialize all control characters
    we don't need them here
    */
    pNewtio->c_cc[VINTR] = 0; /* Ctrl-c */
    pNewtio->c_cc[VQUIT] = 0; /* Ctrl-\ */
    pNewtio->c_cc[VERASE] = 0; /* del */
    pNewtio->c_cc[VKILL] = 0; /* @ */
    pNewtio->c_cc[VEOF] = 4; /* Ctrl-d */
    pNewtio->c_cc[VTIME] = 5; /* inter-character timer, timeout
VTIME*0.1 */
    pNewtio->c_cc[VMIN] = 0; /* blocking read until VMIN character
arrives */
    pNewtio->c_cc[VSWTC] = 0; /* '\0' */
    pNewtio->c_cc[VSTART] = 0; /* Ctrl-q */
    pNewtio->c_cc[VSTOP] = 0; /* Ctrl-s */
    pNewtio->c_cc[VSUSP] = 0; /* Ctrl-z */
    pNewtio->c_cc[VEOL] = 0; /* '\0' */
    pNewtio->c_cc[VREPRINT] = 0; /* Ctrl-r */
    pNewtio->c_cc[VDISCARD] = 0; /* Ctrl-u */
    pNewtio->c_cc[VWERASE] = 0; /* Ctrl-w */
```



```
pNewtio->c_cc[VLNEXT] = 0;    /* Ctrl-v */
pNewtio->c_cc[VEOL2]   = 0;    /* '\0' */
}

int main(int argc, char **argv)
{
    int fd;
    int nCount,nTotal,i;
    struct termios oldtio,newtio;
    char *dev = "/dev/ttyS1";

    if ((argc!=3) || (sscanf(argv[1], "%d", &nTotal) != 1))
    {
        printf("Usage: COMsend count data!\n");
        return -1;
    }

    /*1.打开串口*/
    if ( (fd = open(dev,O_RDWR | O_NOCTTY ))<0)
    {
        printf("Can't Open Serial Port!\n");
        return -1;
    }

    /*2.设置串口*/
    tcgetattr(fd,&oldtio); /* save current serial port settings */
    setTermios(&newtio,B57600);

    tcflush(fd, TCIFLUSH);
    tcsetattr(fd,TCSANOW,&newtio);

    /*3.串口通信，发送数据*/
    for (i=0;i<nTotal;i++)
    {
        nCount=write(fd, argv[2], strlen(argv[2]));
        printf("send data OK! count=%d\n",nCount);
        sleep(1);
    }

    /*4.关闭串口*/
    tcsetattr(fd,TCSANOW,&oldtio);
    close(fd);
}
```

```
    return 0;
}
```

第二个例程的具体实现如下：

```
/*Description: recieve data from COM and printf
 * Author: Zheng Geng <gzheng@hyesco.com>
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <termios.h>
#include <errno.h>
#include <string.h>

typedef unsigned char U8;
typedef unsigned short U16;
typedef unsigned long U32;
/*串口设置函数*/
void setTermios(struct termios * pNewtio, U16 uBaudRate)
{
    bzero(pNewtio, sizeof(struct termios)); /* clear struct for new port
settings */

    pNewtio->c_cflag = uBaudRate | CS8 | CREAD | CLOCAL;
    pNewtio->c_iflag = IGNPAR;
    pNewtio->c_oflag = 0;
    pNewtio->c_lflag = 0; //non ICANON
    /*
    initialize all control characters
    we don't need them here
    */
    pNewtio->c_cc[VINTR] = 0; /* Ctrl-c */
    pNewtio->c_cc[VQUIT] = 0; /* Ctrl-\ */
    pNewtio->c_cc[VERASE] = 0; /* del */
    pNewtio->c_cc[VKILL] = 0; /* @ */
    pNewtio->c_cc[VEOF] = 4; /* Ctrl-d */
    pNewtio->c_cc[VTIME] = 5; /* inter-character timer, timeout
VTIME*0.1 */
    pNewtio->c_cc[VMIN] = 0; /* blocking read until VMIN character
```

```
arrives */
pNewtio->c_cc[VSWTC] = 0; /* '\0' */
pNewtio->c_cc[VSTART] = 0; /* Ctrl-q */
pNewtio->c_cc[VSTOP] = 0; /* Ctrl-s */
pNewtio->c_cc[VSUSP] = 0; /* Ctrl-z */
pNewtio->c_cc[VEOL] = 0; /* '\0' */
pNewtio->c_cc[VREPRINT] = 0; /* Ctrl-r */
pNewtio->c_cc[VDISCARD] = 0; /* Ctrl-u */
pNewtio->c_cc[VWERASE] = 0; /* Ctrl-w */
pNewtio->c_cc[VLNEXT] = 0; /* Ctrl-v */
pNewtio->c_cc[VEOL2] = 0; /* '\0' */
}

#define BUFSIZE 512
int main(int argc, char **argv)
{
    int fd;
    int nread;
    char buff[BUFSIZE];
    struct termios oldtio,newtio;
    struct timeval tv;
    char *dev = "/dev/ttyS1";
    fd_set rfds;

    /*1.打开串口*/
    if ( (fd = open(dev,O_RDWR | O_NOCTTY ))<0)
    {
        printf("Can't Open Serial Port!\n");
        return -1;
    }

    /*2.设置串口*/
    tcgetattr(fd,&oldtio); /* save current serial port settings */
    setTermios(&newtio,B57600);

    tcflush(fd, TCIFLUSH);
    tcsetattr(fd,TCSANOW,&newtio);

    /*3.串口通信,接收数据*/
    tv.tv_sec=30;
    tv.tv_usec=0;
    while(1)
```

```

{
    FD_ZERO(&rfd);
    FD_SET(fd,&rfd);
    if (select(1+fd,&rfd,NULL,NULL,&tv)>0)
    {
        if(FD_ISSET(fd,&rfd))
        {
            nread=read(fd, buff, BUFSIZE);
            printf("readlength=%d\n",nread);
            buff[nread]='\0';
            printf("%s\n",buff);
        }
    }
}

/*4.关闭串口*/
tcsetattr(fd,TCSANOW,&oldtio);
close(fd);
}

```

我们通过如下方式测试这两个例程：将嵌入式目标系统的串口二通过串口线和主机某个串口相连，打开主机下的串口工具程序，如超级终端程序，设置主机串口属性为：波特率 57600，8 位数据位，无校验，1 位停止位，无流控。然后就可以进行嵌入式目标系统与主机之间的串行通信测试了。

测试第一个例程时，嵌入式目标系统充当串口数据发送端，主机串口程序充当数据接收端，测试第二个例程时相反。

我们结合例程看一下 Linux 下的串口编程。例程的流程非常简单，依次为打开串口，设置串口，串口通信（发送或接收），关闭串口退出。

（1） 打开串口

程序通过 `open` 系统调用来打开要使用的串口，第二个串口对应的设备名为 `/dev/ttyS1`。我们先了解一下串口的命名机制：由于 Linux 和 Unix 的传统关系，在 Linux 下对串口的控制基本上等同于 Unix 对串口的控制，也就是说将串口作为系统的串行终端来使用。所以在 Linux 下，串口设备的主设备号为终端设备的主设备号 4，次设备号默认是从 64 开始。也就是说第一个串口对应的设备文件为 `ttyS0`，主设备号是 4，次设备号是 64，第二个串口对应的设备文件为 `ttyS1`，主设备号是 4，次设备号是 65，以次类推。其实串口设备的名字 `ttySx` 也不是固定不变的，要根据串口驱动中的命名确定，只是较为常见的命名方式为 `ttySx`。

`Open` 的第二个参数是串口设备打开的标志，一般为可读写，能发送和接受数据 `O_RDWR`，以及非控制台 `O_NOCTTY`，不响应 `Ctrl-C` 等特殊字符。具体取值可见 `open` 系统调用说明。

（2） 设置串口

在串口设备打开成功后,需要对串口进行设置,这也是串口编程的主要内容。在 Linux 下,每个串口对应于一个结构 `termios`,具体定义在 `include/asm/termbits.h` 中。

```
struct termios {
    tcflag_t c_iflag;          /* 输入模式标志 */
    tcflag_t c_oflag;          /* 输出模式标志 */
    tcflag_t c_cflag;          /* 控制模式标志 */
    tcflag_t c_lflag;          /* 本地模式标志 */
    cc_t c_line;               /* 行规程 */
    cc_t c_cc[NCCS];           /* 控制字符 */
}
```

这个结构中成员的设置是比较复杂的,我们来看一下具体含义和取值情况。

输入模式标志 `c_iflag`

该标志用于设置串口的输入处理特性,也就是说串口从其它设备得到数据,在被 `read` 函数读入之前,会先按照标志进行预处理。它的取值为:

IGNBRK: 忽略 **BREAK**;

BRKINT: 如果 **IGNBRK** 没有设置,那么输入的 **BREAK** 将会导致输入和输出缓存刷新,如果终端是控制终端的话,将会发送 **SIGINT** 信号给前台应用程序。如果 **IGNBRK** 和 **BRKINT** 都没有设置,则 **BREAK** 作为 **NUL** 处理,除非设置了 **PARMRK**,此时作为 `\377 \0`;

IGNPAR: 忽略奇偶校验出错的字节;

PARMRK: 如果 **IGNPAR** 没有设置,奇偶校验出错的字节前加上前缀 `\377 \0`。如果两者都没有设置,则奇偶校验出错的字节为 `\0`;

INPCK: 允许输入奇偶校验检查;

ISTRIP: 去掉第 8 位;

INLCR: 将新行 **NL** 转化成回车 **CR**;

IGNCR: 忽略回车 **CR**;

ICRNL: 把 **CR** 转化成新行,除非 **IGNCR** 设置;

IUCLC: 将大写字符转化成小写;

IXON: 支持输出 **XON/XOFF** 流控;

IXANY: 支持任何字符来重新开始输出;

IXOFF: 支持输入 **XON/XOFF** 流控;

IMAXBEL: 当输入队列满后会响铃。Linux 没有实现该位的控制,当作该位为永远被设定上。

在嵌入式设备的串口编程中,一般只关注 **IGNPAR/INPCK**,用来控制串口的奇偶校验,以及 **IXON/IXOFF**,用来控制串口的 **XON/XOFF** 流控。在本例程中,

```
pNewtio->c_iflag = IGNPAR;
```

表示串口为无奇偶校验,不进行输入校验,无 **XON/XOFF** 流控。

输出模式标志 c_oflag

该标志用于设置串口的输出处理特性，也就是说串口在 `write` 函数调用之后，将数据传给其它设备之前，会先按照标志进行预处理。它的取值为：

OPOST: 允许实现定义的输出处理；
OLCUC: 将小写字符转化成大写；
ONLCR: 将 NL 映射成 NL-CR；
OCRNL: 将 CR 映射成 NL；
ONOCR: 不要在第 0 列输出 CR；
ONLRET: 不要输出 CR；
OFILL: 时延时发送填充字符；
OFDEL: 填充字符为 ASCII DEL (0177)。如果没有设置，填充字符为 ASCII NUL；
NLDLY: 新行时延掩码，值为 NL0, NL1；
CRDLY: 回车时延掩码，值为 CR0, CR1, CR2, CR3；
TABDLY: 水平 TAB 时延掩码，值为 TAB0, TAB1, TAB2, TAB3, XTABS；
BSDLY: BackSpace 时延掩码，值为 BS0, BS1；
VTDLY: 垂直 TAB 时延掩码，值为 VT0, VT1；
FFDLY: 换页时延掩码，值为 FF0, FF1。

在嵌入式系统的串口编程中，一般将该成员设置为 0，表示是原始输出，串口得到什么数据就怎么传输，不进行任何处理。如在本例程中，就设置输出模式标志为 0

```
pNewtio->c_oflag = 0;
```

控制模式标志 c_cflag

该标志用于设置串口的主要通信特性，包括波特率，数据位，停止位等。取值为：

CBAUD, CBAUDEX: 串口波特率，取值从 B0 开始，到 B4000000。也可以使用 `cfsetispeed` 和 `cfsetospeed` 函数来设置；

CSIZE: 数据位掩码，值为 CS5, CS6, CS7, CS8；
CSTOPB: 设置串口停止位为 2，不设置为 1；
CREAD: 允许接收数据；
PARENB: 允许奇偶校验；
PARODD: 奇校验，不设置为偶校验；
HUPCL: 在程序关闭设备后拉低调制解调器控制线信号；
CLOCAL: 本地连接，无调制解调器控制；
CIBAUD: 输入波特率，未用；
CMSPAR: MARK 校验，不设置为 SPACE 校验；
CRTSCTS: 支持硬件流控。

在嵌入式系统的串口编程中，需要根据具体情况，设置串口的波特率，数据位，停止位，奇偶校验等。如果允许串口接收数据，CREAD 是肯定要设定的。对于一般的串口数据

通信，不需要进行调制解调器通信，CLOCAL 也需要设定上。如果嵌入式系统的串口硬件支持硬件流控，即提供 RTS/CTS 两针，则可以进行 CRTSCTS 设定来支持硬件流控。在例程中，

```
pNewtio->c_cflag = uBaudRate | CS8 | CREAD | CLOCAL;
```

表示串口的波特率由 uBaudRate 变量传入，主函数调用 setTermios 时取值 B57600，表示波特率为 57600bps，数据位为 8，无奇偶校验，停止位为 1，无硬件流控。串口允许接收数据，忽略调制解调器通信线信号。

本地模式标志 c_lflag

该标志用于设置串口的本地工作方式，如字符是否回显，信号是否生成等。取值为：

ISIG：收到 INTR, QUIT, SUSP, DSUSP 字符时，生成相应信号；

ICANON：启动标准输入模式，EOF, EOL, EOL2, ERASE, KILL, LNEXT, REPRINT, STATUS, WERASE 等字符都需要处理；

XCASE：如果 ICANON 设置上，终端只处理大写字符。所有输入转化成小写，除非遇到带头的字符。在输出时，大写字符用\带头，小写字符将会转化成大写；

ECHO：回写输入字符；

ECHOE：如果 ICANON 设置，则 ERASE 字符删除前面字符，WERASE 删除前面字；

ECHOK：如果 ICANON 设置，则 KILL 字符删除当前行；

ECHONL：如果 ICANON 设置，则即使 ECHO 没有设置也要回显 NL 字符；

NOFLSH：当生成 SIGINT, SIGQUIT, SIGSUSP 信号时停止刷新输入输出队列；

TOSTOP：发送 SIGTTOU 信号给准备写它们的控制终端的后台进程组；

ECHOCTL：如果 ECHO 设置，则 ASCII 码控制信号中，除了 TAB, NL, START, STOP 之外，都回显成^X，X 为控制信号字符加上 0x40，如 0x08 (BS)回显成^H；

ECHOPRT：如果 ICANON 和 ECHO 都设置，则字符显示成好象它们正在被删除的样子；

ECHOKE：如果 ICANON 设置，则 KILL 回显成删除当前行的字符；

FLUSHO：输出正在被刷新。该设置等同于敲 DISCARD 字符；

PENDIN：当下一个字符读到的时候，在输入队列中的所有字符都再次打印出；

IEXTEN：允许采用自定义的输入处理进程。

本地模式标志适用于串口终端的处理，接收的数据都是以行为单位的，但对于串口通信来说，一般都无需设置，即将串口输入模式设为 0，指定串口工作在非标准输入处理模式。如在例程中，就设置输出模式标志为 0：

```
pNewtio->c_lflag = 0;
```

在这种非标准输入处理模式下，输入的数据并不组合成行，也不会进行 erase、kill、delete 等输入处理。此时需要和控制字符中的 VTIME 和 VMIN 结合起来，来控制这种模式下串口的输入行为。VTIME 设定字符输入间隔时间的计时器，而 VMIN 设置满足读取函数的最少字节数。此时串口对于输入的处理分类如下：

当 $VMIN > 0$, $VTIME = 0$ 时, 读取函数在读到了 $VMIN$ 个字符数后返回。

当 $VMIN = 0$, $VTIME > 0$ 时, $VTIME$ 决定了读取的超时值, 以 0.1 秒为单位。读取函数在读到一个字节的字符, 或者等待读取时间超过 $VTIME * 0.1$ 秒以后返回, 也就是说, 即使没有从串口中读到数据, 读取函数也会在 $VTIME * 0.1$ 秒后返回。

当 $VMIN > 0$, $VTIME > 0$ 时, 读取函数会在收到了 $VMIN$ 个字符数后, 或者等待读取时间超过 $VTIME * 0.1$ 秒后返回。计时器会在每次收到字符的时候重新计时, 也只会收到第一个字节后才启动。

当 $VMIN = 0$, $VTIME = 0$, 读取函数会立即返回。实际读取到的字符数, 或者要读到的字符数, 会作为返回值返回。

如在例程中, 就设置:

```
pNewtio->c_cc[VTIME]    = 5;  
pNewtio->c_cc[VMIN]     = 0;
```

表示串口 read 系统调用时的超时为 0.5 秒, 即使串口没有收到数据, 也必须返回。

行规程 c_line

用于设定串口的行规程 (line discipline)。在 Linux 下, 串口通信模块可在逻辑上分为三层: TTY 层、line discipline 层和底层驱动层。TTY 层是用户空间和内核空间的桥梁, 用户程序和内核需要通过 TTY 层来交换数据; 底层驱动层则负责硬件的交互, 它对硬件进行控制和读写操作; 而 line discipline 层是整个串行通信模块中最灵活、设计最巧妙的一层, 它要为一个串口的使用定下数据交互的“规程”。在 Linux 内核中已经存在了许多行规程, 例如 PPP、SLIP、TTY 等, 缺省使用 TTY 行规程。该成员变量只用于内核中与串口相关模块的设计中, 应用程序无需考虑。

控制字符 c_cc

该数组定义了各特殊控制字符, 包括中断字符 $VINTR$, 退出字符 $VQUIT$, 删除字符 $VERASE$, 文件结束字符 $VEOF$, 等等。在非标准输入处理模式下, 各特殊控制字符都可以不设置, 只需要设置 $VTIME$ 和 $VMIN$ 的数值即可, 两者的具体含义见本地模式标志一节说明。

在了解了 `termios` 结构中成员的含义与取值后, 我们就可以根据自己应用情况对 `termios` 结构进行赋值来设置串口了。在进行新的串口设置前, 一般我们需要先通过

```
tcgetattr(fd, &oldtio);
```

来保存原来的串口设置, 以便在使用完串口后将其恢复到原来设置, 同时还需要调用

```
tcflush(fd, TCIFLUSH);
```

来将串口缓存清空。最后调用

```
tcsetattr(fd, TCSANOW, &newtio);
```

使新的设置生效。

（3） 串口通信

串口通信实际上就是对串口设备文件进行读写调用，从串口读数据使用 `read` 系统调用，往串口发数据则使用 `write` 系统调用。

注意每次 `read/write` 调用的返回值可能会小于需要传输的数据量，因此需要进行相关处理以便完整的将数据进行传输。

（4） 关闭串口

在关闭串口之前，需要通过

```
tcsetattr(fd, TCSANOW, &oldtio);
```

将串口设置恢复为原来的设置，最后通过 `close` 系统调用来关闭串口。串口通信过程就此结束。

本节所描述的 Linux 下串口通信编程的知识，对于一般性的嵌入式设备串口通信编程来说也已经足够，需要更进一步学习和掌握的读者也可以参见《Serial Programming Guide for POSIX Operating Systems》，《Linux Serial HOWTO》，《Serial Programming HOWTO》等资料，这些都可以在网上得到。

6.5 基于 Web 的应用程序开发

随着 Internet 技术的兴起，在嵌入式设备的管理与交互中，基于 Web 方式的应用成为目前的主流，这种程序结构也就是大家非常熟悉的 B/S 结构，即在嵌入式设备上运行一个支持脚本或 CGI 功能的 Web 服务器，能够生成动态页面，在用户端只需要通过 Web 浏览器就可以对嵌入式设备进行管理和监控，非常方便实用。本节主要介绍这种应用的开发和移植工作。

用户首先需要在嵌入式设备上成功移植支持脚本或 CGI 功能的 Web 服务器，然后才能进行应用程序的开发。

6.5.1 嵌入式 Web 服务器移植

由于嵌入式设备资源一般都比较有限，并且也不需要能同时处理很多用户的请求，因此不会使用 Linux 下最常用的如 Apache 等服务器，而需要使用一些专门为嵌入式设备设计的 Web 服务器，这些 Web 服务器在存储空间和运行时所占有的内存空间上都会非常适合于嵌入式应用场合。

典型的嵌入式 Web 服务器有 Boa（www.boa.org）和 tthttpd（<http://www.acme.com/software/tthttpd/>）等，它们和 Apache 等高性能的 Web 服务器主要的区别在于它们一般是单进程服务器，只有在完成一个用户请求后才能响应另一个用户的请求，而无法并发响应，但这在嵌入式设备的应用场合里已经足够了。

本节介绍比较常用的 Boa 服务器的移植。

Boa 是一个非常小巧的 Web 服务器，可执行代码只有约 60KB。它是一个单任务 Web 服务器，只能依次完成用户的请求，而不会 fork 出新的进程来处理并发连接请求。但 Boa 支持 CGI，能够为 CGI 程序 fork 出一个进程来执行。Boa 的设计目标是速度和安全，在其站点公布的性能测试中，Boa 的性能要好于 Apache 服务器。

第一步完成Boa程序的移植。从www.boa.org下载Boa源码，当前最新版本为 0.94.13，将其解压并进入源码目录的src子目录

```
# tar xzf boa-0.94.13.tar.gz
# cd boa-0.94.13/src
```

生成 Makefile 文件

```
# ./configure
```

修改 Makefile 文件，找到 CC=gcc，将其改成 CC = arm-linux-gcc，再找到 CPP = gcc -E，将其改成 CPP = arm-linux-gcc -E，并保存退出。

然后运行 make 进行编译，得到的可执行程序为 boa，将调试信息剥去，得到的最后程序只有约 60KB 大小。

```
# make
# arm-linux-strip boa
```

第二步完成 Boa 的配置，使其能够支持 CGI 程序的执行。Boa 需要在/etc 目录下建立一个 boa 目录，里面放入 Boa 的主要配置文件 boa.conf。在 Boa 源码目录下已有一个示例 boa.conf，可以在其基础上进行修改，下面解释一下该文件的含义：

#监听的端口号，缺省都是 80，一般无需修改

Port 80

bind 调用的 IP 地址，一般注释掉，表明绑定到 INADDR_ANY，通配于服务器的所有 IP 地址

#Listen 192.68.0.5

#作为哪个用户运行，即它拥有该用户的权限，一般都是 nobody，需要/etc/passwd 中有

#nobody 用户

User nobody

#作为哪个用户组运行，即它拥有该用户组的权限，一般都是 nogroup，需要在/etc/group 文件中

#件中有 nogroup 组

Group nogroup

#当服务器发生问题时发送报警的 email 地址，目前未用，注释掉

#ServerAdmin root@localhost

#错误日志文件。如果没有以/开始，则表示从服务器的根路径开始。如果不需要错误日志，则用 #/dev/null。在下面设置时，注意一定要建立/var/log/boa 目录

ErrorLog /var/log/boa/error_log

#访问日志文件。如果没有以/开始，则表示从服务器的根路径开始。如果不需要错误日志，则用 #/dev/null 或直接注释掉。在下面设置时，注意一定要建立/var/log/boa 目录

#AccessLog /var/log/boa/access_log

#是否使用本地时间。如果没有注释掉，则使用本地时间。注释掉则使用 UTC 时间

```
#UseLocaltime
#是否记录 CGI 运行信息，如果没有注释掉，则记录，注释掉则不记录
#VerboseCGILogs
#服务器名字
    ServerName www.hyesco.com
#是否启动虚拟主机功能，即设备可以有多个网络接口，每个接口都可以拥有一个虚拟的 Web 服务器。一般注释掉，即不需要启动
#VirtualHost
#非常重要，HTML 文档的主目录。如果没有以/开始，则表示从服务器的根路径开始。
    DocumentRoot /var/www
#如果收到一个用户请求的话，在用户主目录后再增加的目录名
    UserDir public_html
#HTML 目录索引的文件名，也是没有用户只指明访问目录时返回的文件名
    DirectoryIndex index.html
#当 HTML 目录没有索引文件时，用户只指明访问目录时，boa 会调用该程序生成索引文件然后返回给用户，因为该过程比较慢最好不执行，可以注释掉或者给每个 HTML 目录加上
#DirectoryIndex 指明的文件
#DirectoryMaker /usr/lib/boa/boa_indexer
#如果 DirectoryIndex 不存在，并且 DirectoryMaker 被注释，那么就用 Boa 自带的索引生成程序来生成目录的索引文件并输出到下面目录，该目录必须是 Boa 能读写
# DirectoryCache /var/spool/boa/dircache
#一个连接所允许的 HTTP 持续作用请求最大数目，注释或设为 0 都将关闭 HTTP 持续作用
    KeepAliveMax 1000
#HTTP 持续作用中服务器在两次请求之间等待的时间数，以秒为单位，超时将关闭连接
    KeepAliveTimeout 10
#指明 mime.types 文件位置。如果没有以/开始，则表示从服务器的根路径开始。可以注释掉
#避免使用 mime.types 文件，此时需要用 AddType 在本文件里指明
    MimeTypes /etc/mime.types
#文件扩展名没有或未知的话，使用的缺省 MIME 类型
    DefaultType text/plain
#提供 CGI 程序的 PATH 环境变量值
    CGIPath /bin:/usr/bin:/usr/local/bin
#将文件扩展名和 MIME 类型关联起来，和 mime.types 文件作用一样。如果用 mime.types 文件，则注释掉，如果不使用 mime.types 文件，则必须使用
#AddType application/x-httpd-cgi cgi
#指明文档重定向路径
#Redirect /bar http://elsewhere/feh/bar
#为路径加上别名
    Alias /doc /usr/doc
#非常重要，指明 CGI 脚本的虚拟路径对应的实际路径。一般所有的 CGI 脚本都要放在实际路径里，用户访问执行时输入站点+虚拟路径+CGI 脚本名
    ScriptAlias /cgi-bin/ /var/www/cgi-bin/
```

用户可以根据自己需要,对 `boa.conf` 进行修改,但必须要保证其他的辅助文件和设置必须和 `boa.conf` 里的配置相符,不然 `Boa` 就不能正常工作。在上面的例子中,我们还需要创建日志文件所在目录 `/var/log/boa`,创建 HTML 文档的主目录 `/var/www`,将 `mime.types` 文件拷贝到 `/etc` 目录,创建 CGI 脚本所在目录 `/var/www/cgi-bin/`。`mime.types` 文件用来指明不同文件扩展名对应的 MIME 类型,一般可以直接从 Linux 主机上拷贝一个,大部分也都是在主机的 `/etc` 目录下。

第三步就是测试 `Boa` 能否正常工作,静态 HTML 页面能否正常访问,CGI 脚本能否正常运行,一般采用 NFS 方式来进行测试工作,如图 6-5 所示。

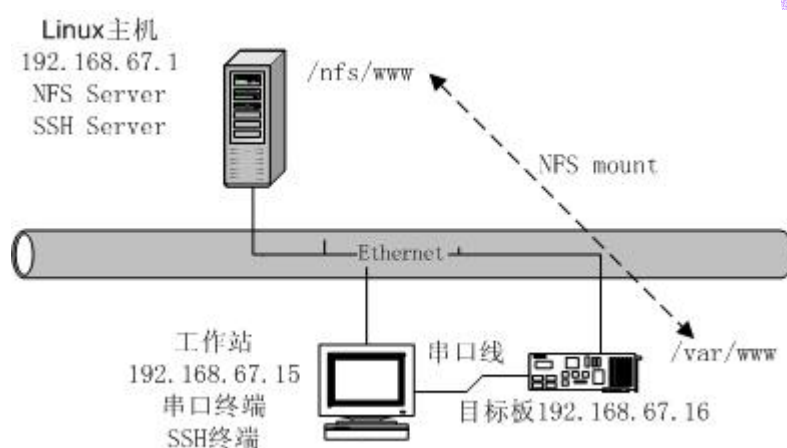


图 6-5 Boa 测试开发环境

可以将嵌入式目标系统上的 `/etc` 目录拷贝到主机的 NFS 共享目录下,然后将 NFS 共享目录下的 `etc` 目录重新 NFS mount 为目标系统上的 `/etc` 目录。这样就可以在主机上对 `etc` 目录下的各种配置文件,如进行修改而立刻在目标系统上生效。

假设主机 `/nfs` 目录为共享目录,在其下面建立一个 `www` 子目录作为 Web 站点的主目录,其内容如下:

```
[root@Linuxsrv www]# ls -l
总用量 20
drwxr-xr-x  2 root    root      4096  7月 30 15:12 cgi-bin
drwxr-xr-x  3 root    root      4096  7月 30 15:12 images
-rw-r--r--  1 root    root     10715  7月 30 15:11 index.html
```

`index.html` 为测试主页面, `images` 为存放各种图片的子目录, `cgi-bin` 为 CGI 脚本的存放目录。根据示例 `boa.conf` 的配置,目前 HTML 文档的主目录为 `/var/www`,CGI 脚本目录为 `/var/www/cgi-bin`,则运行以下命令将主机的 `/nfs/www` 目录 mount 成目标板上的 `/var/www` 目录。然后就可以运行 `boa` 了:

```
# mount -t nfs 192.168.67.1:/nfs/www /var/www -o nolock
# boa
```


在工作站上运行浏览器进行测试，在地址栏输入目标系统 IP，即 `http://192.168.67.16`，可以看到如下页面，表示静态 HTML 页面测试通过，如图 6-6 所示。



图 6-6 静态 HTML 页面测试例子

接下来进行 CGI 脚本的测试，我们需要一个测试用的 CGI 脚本。可以写个最简单的 Hello World 程序，示例代码如下

```
#include <stdio.h>
void main() {
    printf("Content-type: text/html\n\n");
    printf("<html>\n");
    printf("<head><title>CGI Output</title></head>\n");
    printf("<body>\n");
    printf("<h1>Hello, world.</h1>\n");
    printf("</body>\n");
    printf("</html>\n");
    exit(0);
}
```

然后进行交叉编译，将得到的 `helloworld.cgi` 拷贝到主机的 `/nfs/www/cgi-bin` 目录下。

```
#arm-linux-gcc -o helloworld.cgi helloworld.c
#cp helloworld.cgi /nfs/www/cgi-bin
```

在浏览器地址栏输入 `http://192.168.67.16/cgi-bin/helloworld.cgi`，可以看到如图 6-7 所示页面，表示 CGI 脚本测试通过。



图 6-7 CGI 脚本测试例子

现在我们已经可以让 Boa 在嵌入式目标系统上正常工作了，嵌入式 Web 服务器移植成功。

在以上的移植过程中，最好设定 `boa.conf` 中的错误日志文件 `ErrorLog`，允许 Boa 记录错误信息；

测试静态 HTML 页面和 CGI 脚本时，不管结果如何，最好都查看错误日志文件；

CGI 脚本测试很容易发生权限不够的错误，要保证 Boa 访问的主目录、CGI 脚本目录以及临时文件目录（如果没有设置 `TMP` 环境变量时，缺省是 `/tmp` 目录），都必须能被 Boa 运行时所代表的用户完全访问，该用户由 `boa.conf` 中的 `User` 指出。

6.5.2 Web 方式应用开发

目前 Web 技术中生成动态 Web 页面的方法有 CGI 和服务端脚本，如 JSP, ASP 等，但后者需要 Web 服务器具有这些脚本的运行支持模块。在嵌入式 Web 服务器中，考虑到资源限制问题，一般都只提供 CGI 支持，因此在嵌入式设备中 Web 方式应用实际上就是基于 CGI 的程序开发。

CGI (Common Gate Intergace) 是一段运行在 Web 服务器上的程序，提供同客户端 Html 页面的接口。我们看一个实际例子：常见的个人主页上大都有一个留言本，留言本的工作方式是这样的：先由用户输入一些信息，如名字之类的东西，接着用户按一下“留言”（到目前为止工作都在客户端），浏览器就把这些信息传送到服务器的 CGI 程序中，于是 CGI 程序在服务器上按照预定的方法进行处理，在本例中就是把用户提交的信息存入指定的文件中，最后 CGI 程序给客户端发回一个“留言结束”字样的页面，用户可以在浏览器里看到。

在进行 CGI 编程之前，我们先了解 HTML 的一些知识。

CGI 可以使用多种编程语言来实现，包括 C、C++、Perl 等，但在嵌入式设备的开发中，一般都不会采用 Perl 等解释性语言，因为这种语言还需要有解释执行的支撑模块，会占用存储空间和内存，最常用的方法当然是用 C 来编写，但 C 并不是很适合开发象 CGI 这种需要大量进行字符串操作的程序，编程比较烦琐，因此，对于一个专业的开发人员来

说，首先想到的应该是有没有可复用的库来支持快速高效的开发 CGI 程序。幸运的是目前就有不少开放源码的支持 CGI 开发的 C 库。我们在此只介绍 CGIC，有兴趣的朋友可以自己在 Internet 上搜索其他的 C 库。

λ CGIC 库的移植

CGIC 是一个支持 CGI 开发的开放源码的标准 C 库，可以免费使用，只需要在开发的站点和程序文档中有个公开声明即可，表明程序使用了 CGIC 库，用户也可以购买商业授权而无需公开声明。

CGIC 能够提供以下功能：

- | 分析数据，并自动校正一些有缺陷的浏览器发来的数据；
- | 透明接收用 GET 或 POST 方法发来的 From 数据；
- | 能接受上传文件；
- | 能够设置和接收 cookies；
- | 用一致的方式处理 From 元素里的回车；
- | 提供字符串，整数，浮点数，单选或多选功能来接收数据；
- | 提供数字字段的边界检查；
- | 能够将 CGI 环境变量转化成 C 中的非空字符串；
- | 提供 CGI 程序的调试手段，能够回放 CGI 程序执行时的 CGI 状态；

总之，CGIC 是一个功能比较强大的支持 CGI 开发的标准 C 库，并支持 Linux, Unix 和 Windows 等多操作系统。

以下描述 CGIC 的移植过程。

从 CGIC 的主站点<http://www.boutell.com/cgiic/>下载源码，当前最新版本是 2.05 版。将其解压并进入源码目录

```
# tar xzf cgic205.tar.gz
# cd cgic205
```

修改 Makefile 文件，找到 CC=gcc，将其改成 CC=arm-linux-gcc，找到 AR=ar，将其改成 AR=arm-linux-ar，找到 RANLIB=ranlib，将其改成 RANLIB=arm-linux-ranlib。找到 gcc cgictest.o -o cgictest.cgi \${LIBS}，将其改成\$(CC) \$(CFLAGS) cgictest.o -o cgictest.cgi \${LIBS}，找到 gcc capture.o -o capture \${LIBS}，将其改成\$(CC) \$(CFLAGS) capture.o -o capture \${LIBS}，并保存退出。

然后运行 make 进行编译，得到的 CGIC 库 libcgic.a，我们通过调试辅助程序 capture 和测试程序 cgictest.cgi，来验证生成 CGIC 库的正确性。

将 capture 和 cgictest.cgi 拷贝到主机的/nfs/www/cgi-bin 目录下。

在工作站的浏览器地址栏输入<http://192.168.67.16/cgi-bin/cgictest.cgi>，可以看到如图 6-8 所示页面，表示 CGIC 库和测试脚本都移植成功。cgictest.cgi 比较完整的展现了 CGIC 库的功能，在开发基于 CGIC 库的 CGI 程序前最好先掌握 cgictest.cgi 程序，也是用户开发特定应用程序时的参考范例。

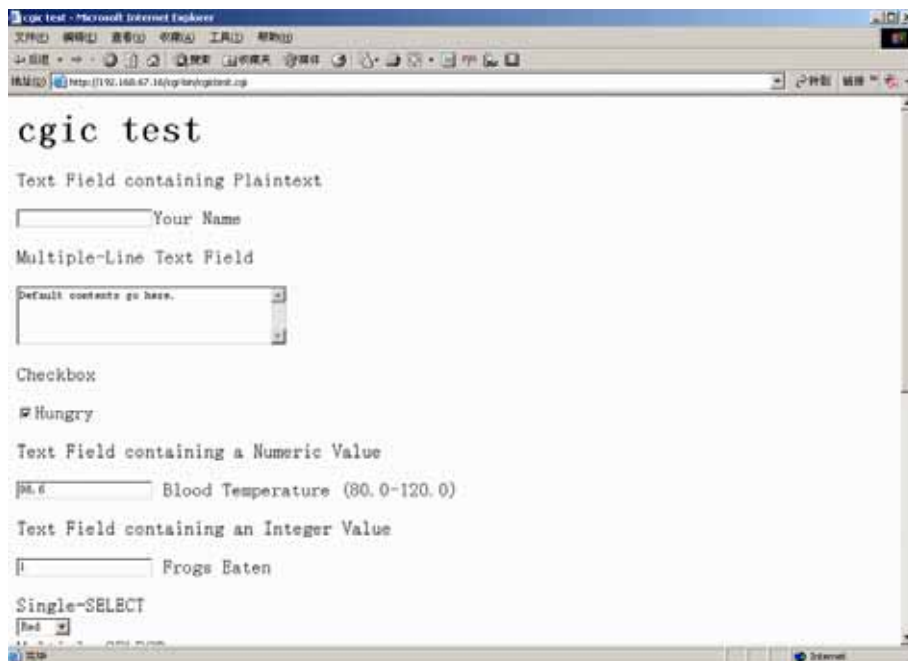


图 6-8 CGIC 库测试例子

λ HTML 模板的制作

Web 方式的应用开发一般都会将界面和程序逻辑脱离开来, 允许在一定程度下更改界面, 如改变界面文本的属性, 建立多语言版本等, 而无需改动程序逻辑。界面一般由美工来进行制作, 而程序员负责具体功能的实现。在 HTML 中, 表单 (FORM) 是最主要的传递信息的手段, 它适用于任何浏览器。表单中有很多元素, 包括输入文本框, 单选框, 多选框, 按钮, 等等, 可以提供信息的交互。具体对象说明和语法请参见其他 HTML 书籍, 在这里不作介绍。根据应用需求, 美工或其他设计人员将最后的 Web 页面设计出来, 作为程序员进行开发的模板。

CGI 程序的工作一般就是接收表单数据, 进行数据处理, 最后根据处理结果生成新的页面返回给浏览器。表单数据一般是以 POST 方法提交给服务器, 由 CGI 程序获得, 程序必须要将界面数据和内部数据对应起来才能够进行下一步的处理。CGI 程序从页面获取数据就根据元素名字/值中的元素名字来进行区分。但 CGI 返回页面就比较麻烦。由于界面在程序开发完成后还有可能会改变, 而且有些需要程序处理的地方可能没有表单元素, 因此对程序来说, 不能以表单元素名作为区分的基础, 一般方法是采用 HTML 中的注释 `<!--xxx-->` 来标记。

程序员需要在模板中为每一个表单元素以及其他任何需要程序处理的地方, 按照一定规则, 如注释的下一行就是表单元素行, 建立其注释标记。CGI 程序就可以根据注释标记来判断表单元素信息并进行处理。程序逐行读取模板文件, 检查有无注释标记, 如有的话,

则下一行需要进行处理，给表单元素赋上数据，最后就可以返回带数据的页面给浏览器。

HTML 模板还需要关注的是输入的检查。根据输入检查越早越好的原则，需要在用户界面上就对用户提交的数据进行检查。目前一般是采用 javascript 脚本的方式。当用户提交数据时，表单对象的 onSubmit 方法就会被调用，在该方法里就可以对用户的输入进行检查。常用的检查有是否必需、最大/小长度、是否字符、是否数字、email 地址、IP 地址是否正确、是否匹配一个正则表达式等。

λ CGI 程序的开发

CGI 程序的工作一般就是接收表单数据，根据应用需求进行数据处理，最后根据处理结果生成新的页面返回给浏览器。表单数据一般是以 POST 方法提交给服务器，由 CGI 程序获得，程序根据元素名字/值中的元素名字来区分数据，完成数据处理后，再读取相应的模板文件，根据注释标记将对应的数据填充到 HTML 文本中去，生成最后的页面返回给浏览器。

程序一般逻辑为：

1. 安全性检查，是否允许运行脚本；
2. 处理用户提交的数据，根据元素名字/值中的元素名字来区分数据，然后根据应用需求进行数据处理；
3. 将处理结果填充表单，根据注释标记将对应的数据填充到 HTML 文本中去，生成最后的页面返回给浏览器。

下面以一个简单的 Web 应用为例，说明编程方法。该 CGI 功能用于列出目标系统上指明目录下的文件，主要逻辑是接受用户输入的目录，用 shell 命令 ls 将结果重定向到一个临时文件中，然后再读取临时文件中的内容并将它嵌入到 Web 页面中返回。当然也可以直接调用系统调用来获取目录下的文件列表。

我们首先制作 HTML 模板页面，如图 6-9 所示：


```

        <p>
<!--files-->
<textarea name="files" rows="16" cols="73">
</textarea>
</TD>

        </TR>
    </TABLE>
    <p>
<input name="Submit" type="submit" value="确定">
    </TD>
</TR>
</TABLE>
</form>
</BODY>
</HTML>

```

模板文件制作完成后，将其拷贝到 Web 站点的主目录的某个子目录下，假设为 /var/www/cgi-bin/template/cn/list.htm。

然后编写一个 CGI 程序 list.c，示例代码如下：

```

/*
 * Description: list files
 * Author: Zheng Geng <gzheng@hyesco.com>
 */
#include "cgic.h"
#ifndef TRUE
#define TRUE 1
#define FALSE 0
#endif
typedef unsigned char U8;
typedef unsigned short U16;
typedef unsigned long U32;
//保存当前的目录
char strDir[128];
#define BUFSIZE 1024
char buf[BUFSIZE];
//处理用户提交的数据，使用 CGIC 库中的函数根据元素名字获得数据，然后执行动作
void HandleSubmit()
{
    <!--Dir-->
    cgiFormStringNoNewlines("Dir", buf, BUFSIZE);
    strcpy(strDir, buf);
}

```

```
//用 ls 命令显示目录下文件到临时文件中，当然也可以直接调用系统 API 来做
sprintf(buf, "ls -l %s >/var/1", strDir);
system(buf);
}
//下面三个函数用于逐行读取文件
#define bufferFileSize 4096
static char bufferFile[bufferFileSize];
static U16 ubufferActualSize=bufferFileSize;
static int nEOB=FALSE;
static U32 uStrHead=0, uStrTail=-1; //str pos in bufferFile
static int FillBufferFile(FILE * fp)
{
    int nCount=0;
    ubufferActualSize=0;
    while (ubufferActualSize<bufferFileSize) {
        nCount=fread(bufferFile+ubufferActualSize, sizeof(char),
bufferFileSize-ubufferActualSize, fp);
        if(nCount<0) {
            printf("FillBufferFile: read file failed!\n");
            return -1;
        }
        else {
            ubufferActualSize+=nCount;
            if (feof(fp))
                break;
        }
    }
    return ubufferActualSize;
}
int InitReadString(FILE * fp)
{
    int n=0;
    if (NULL==fp)
        return -1;
    nEOB=FALSE;
    //init buffer
    n=FillBufferFile(fp);
    if (n<0)
        return -1;
    else
        return 0;
}
```

```

/*
Description:
    read string from fd to buffer starting at buf when encount a '\n'
Parameters:
    fp:          file point to read
    buf:          buffer to write
    nMax:         buffer length
Return Vvalue:
    >0  Succeed, the number of bytes read are returned
    0   End of BufferF
    <0  Fail
*/
int ReadString(FILE * fp, char * buf, U32 uMax)
{
    U32 nCount=0,n=0;
    if ( (NULL==fp) || (NULL==buf) || (0==uMax))
        return -1;
    if (nEOB==TRUE)
        return 0;
    uStrHead=uStrTail+1;
    if (0x0a==bufferFile[uStrHead])
        uStrHead++;
    while(++uStrTail<ubufferActualSize){
        if (0x0d==bufferFile[uStrTail]){
            //'\\n'==0d0a in windows, only check 0d
            nCount=uStrTail-uStrHead;
            if (nCount>uMax)
                return -1;
            memcpy(buf,bufferFile+uStrHead,nCount);
            return nCount;
        }
    }
    nCount=ubufferActualSize-uStrHead;
    if (nCount>uMax)
        return -1;
    memcpy(buf,bufferFile+uStrHead,nCount);
    while (1){
        if (feof(fp)){
            nEOB=TRUE;
            return nCount;
        }
        n=FillBufferFile(fp);
    }
}

```



```

        if (n<0)
            return -1;
        uStrHead=0;
        uStrTail=-1;
        while(++uStrTail<n) {
            if ( 0x0d==bufferFile[uStrTail] ) {
                //'\\n'==0d0a in windows, only check 0d
                if (nCount+uStrTail-uStrHead>uMax)
                    return -1;
                memcpy(buf+nCount,bufferFile+uStrHead,uStrTail-uStrHead);
                nCount+=uStrTail-uStrHead;
                return nCount;
            }
        }
        if (nCount+n-uStrHead>uMax)
            return -1;
        memcpy(buf+nCount,bufferFile+uStrHead,n-uStrHead);
        nCount+=n-uStrHead;
    }
}
//基于 CGIC 库的主函数
int cgiMain()
{
    char strTemplateFileName[128];
    char TemplateDir[]="/var/www/cgi-bin/template/cn/";
    int nCount=0;
    FILE * fp=NULL;
    FILE * fp1=NULL;
    cgiHeaderContentType("text/html");
    //处理提交
    if (cgiFormSubmitClicked("Submit") == cgiFormSuccess)
        HandleSubmit();
    //将处理结果填充表单
    strcpy(strTemplateFileName,TemplateDir);
    strcat(strTemplateFileName,"list.htm");
    if ((fp=fopen(strTemplateFileName, "r"))==NULL){
        fprintf(cgiOut, "<HTML><HEAD>\\n");
        fprintf(cgiOut, "<TITLE>ERROR</TITLE></HEAD>\\n");
        fprintf(cgiOut, "<BODY><H1>ERROR: Can't open template file :
%s</H1>\\n", strTemplateFileName);
        fprintf(cgiOut, "</BODY></HTML>\\n");
        return -1;
    }
}

```

```
}
else {
    if (InitReadString(fp)<0)
        goto ReadStringError;
    //根据注释标记处理
    //<!--Dir-->
    while (1) {
        nCount=ReadString(fp,buf,BUFSIZE);
        if (nCount<0)
            goto ReadStringError;
        else if (nCount>0) {
            buf[nCount]='\0';
            if (strcmp(buf,"<!--Dir-->")==0) {
                nCount=ReadString(fp,buf,BUFSIZE);
                if (nCount<0)
                    goto ReadStringError;
                else if (nCount>0) {
                    fwrite(buf,sizeof(char),nCount-1,cgiOut);
                    fprintf(cgiOut," value=\"");
                    fwrite(strDir,sizeof(char),strlen(strDir),cgiOut);
                    fprintf(cgiOut,"\">\n");
                    break;
                }
            }
            else
                break;
        }
        else {
            fwrite(buf,sizeof(char),nCount,cgiOut);
            fprintf(cgiOut,"\\n");
        }
    }
    else
        break;
}
//显示文件列表
//<!--files-->
while (1)
{
    nCount=ReadString(fp,buf,BUFSIZE);
    if (nCount<0)
        goto ReadStringError;
    else if (nCount>0) {
```

```
buf[nCount]='\0';
if (strcmp(buf,"<!--files-->")==0) {
    nCount=ReadString(fp,buf,BUFSIZE);
    if (nCount<0)
        goto ReadStringError;
    else if (nCount>0) {
        fwrite(buf,sizeof(char),nCount,cgiOut);
        fprintf(cgiOut,"\n");
    }
    else
        break;
    if ((fp1=fopen("/var/1", "r"))==NULL)
        goto ReadStringError;
    else {
        while (1) {
            nCount=fread(buf,sizeof(char), BUFSIZE,fp1);
            if (nCount<0){
                fprintf(cgiOut, "ERROR: read/var/1 \n");
                fclose(fp1);
            }
            else {
                fwrite(buf,sizeof(char),nCount,cgiOut);
                if (feof(fp1))
                    break;
            }
        }
        fclose(fp1);
        fprintf(cgiOut,"\n");
        break;
    }
    else {
        fwrite(buf,sizeof(char),nCount,cgiOut);
        fprintf(cgiOut,"\n");
    }
}
else
    break;
}
//others
while (1) {
    nCount=ReadString(fp,buf,BUFSIZE);
```

```
        if (nCount<0)
            goto ReadStringError;
        else if (nCount>0) {
            fwrite(buf,sizeof(char),nCount,cgiOut);
            fprintf(cgiOut,"\n");
        }
        else
            break;
    }
    fclose(fp);
}
return 0;
ReadStringError:
    fclose(fp);
    fprintf(cgiOut, "<HTML><HEAD>\n");
    fprintf(cgiOut, "<TITLE>ERROR</TITLE></HEAD>\n");
    fprintf(cgiOut, "<BODY><H1>ERROR: ReadString Error!");
    fprintf(cgiOut, "</BODY></HTML>\n");
    return -1;
}
```

程序完成后，同样需要进行交叉编译。将 list.c 拷贝到 CGIC 库同目录下，然后运行

```
# arm-linux-gcc -o list.cgi list.c -L./ -lcgic
```

将编译得到的 list.cgi 拷贝到 CGI 脚本目录下，再进行测试。在浏览器地址栏输入 <http://192.168.67.16/cgi-bin/list.cgi>，可以类似于图 6-10 的页面，在查看目录输入框中输入目标系统上的某个目录，按确定即可看到该目录下的文件列表。

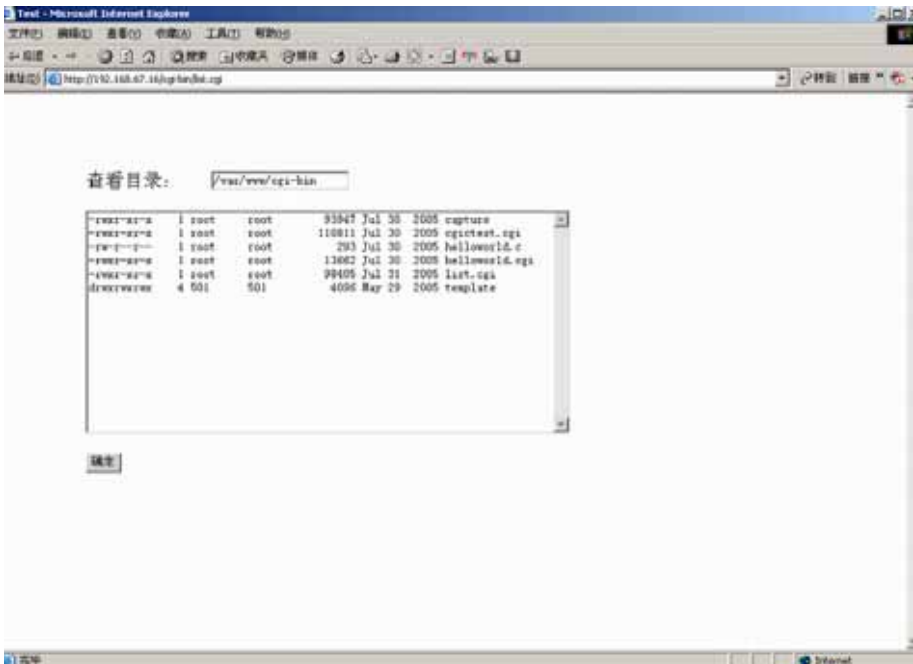


图 6-10 CGI 演示例子

交叉编译得到的 CGI 程序也是目标系统上的可执行程序，可以直接在目标系统上执行，此时会在控制台输出 HTML 文本，很适合发现字符串处理出错的问题。

由于 CGI 程序的调试是无法采用交互式方式调试的，只能采用输出调试信息的方式进行。

6.6 图形应用开发

6.6.1 嵌入式图形系统概述

图形用户界面 (GUI) 在很多需要进行人机交互的嵌入式设备中是非常重要的，如 PDA 等掌上设备、信息家电、可视电话、娱乐设备、POS 终端等，都要求能提供良好的图形界面，才能够吸引用户以及方便用户的使用操作。

在 Linux 下，最经典的图形系统架构就是 X Window 系统，在此基础上使用 GNOME 或 KDE 等作为窗口环境，为用户提供了非常美观而又强大的图形环境。但这种架构却无法适合嵌入式设备的要求，原因就在于嵌入式设备资源的限制。

我们来看一下它们的资源要求：

- λ X Window System: 5MB RAM, 16MB disk
- λ GNOME: 14MB RAM, 95MB disk
- λ KDE: 11MB RAM, 96MB disk

显然，这已经大大超出了很多嵌入式设备的能力，大多嵌入式设备只能提供 2-16M Flash 和 4-32M RAM，对于这些设备来说，首先是要求图形界面必须精简，占用资源少，而且还需要可配置和高可靠性。为了满足快速增长的嵌入式设备以及嵌入式 Linux 市场的需求，目前也已经有不少开放源码的图形系统项目和产品。

常见的嵌入式 GUI 如下：

- λ DirectFB (<http://directfb.org/>)，是专注于 Linux FrameBuffer 加速的一个图形库，并试图建立一个兼容 GTK 的嵌入式 GUI 系统。它提供硬件图形加速、输入设备处理和抽象、集成窗口系统并支持透明窗口以及多显示层功能。图 6-11 就是一个采用 DirectFB 的可视电话。



图 6-11 DirectFB 应用例子

- λ Familiar (<http://familiar.handhelds.org/>)，是一个专门为 PDA 发布的 Linux 版本，提供了完整的基于 Linux 的 PDA 计算平台。它包括基于 Tiny-X 的图形系统，PIM 应用，嵌入式 Python 实现，OpenSSH 服务器和客户，系统配置程序等。它的 Linux 内核是基于 Debian 的 ARM 发布包，提供了三种图形环境，分别是：
 - | GPE (GPE 掌上电脑环境，即 GPE Palmtop Environment)，基于 GTK+，最类似于桌面领域中的 GNOME。
 - | OPIE (开放掌上电脑集成环境，即 Open Palmtop Integrated Environment)，基于 Qtopia，最类似于桌面领域中的 KDE。
 - | pyraq，基于 Python 和 GTK。

图 6-12 是一个典型的 GPE 界面，图 6-13 是一个典型的 OPIE 界面。



图 6-12 GPE 主界面



图 6-13 OPIE 主界面

- λ GPE (<http://gpe.handhelds.org/>): 为手持设备提供了一个免费的 GUI 环境。它是基于 X Window 系统和 GTK+。GPE 不是一个程序，而是集成了手持设备必需功能的整个环境，包括 PIM 程序，系统设置，USB，无线，蓝牙等通信程序等，以及为进一步开发提供了共享库和 GPE 开发标准。GPE 典型界面可见图 6-12。
- λ GtkFB (<http://developer.gnome.org/>): 就是基于 Linux FrameBuffer 的 GTK+。在 GTK+ 2.0 版本中，将支持直接基于 FrameBuffer 来画图而不需要依赖于 X Window 系统，因此可以减少 X Window 的资源占用，很适合于嵌入式设备。GtkFB 保持了 GTK+ 的强大功能，并且可以很容易的移植已有的基于 GTK+ 的应用程序，如桌面领域中的 GNOME 上的应用程序。但 GtkFB 也有其局限性，主要的局限是运行于单进程模式，系统的所有代码必须在一个二进制文件中，并且运行在一个进程中。图 6-14 就是一个 GtkFB 应用例子界面。

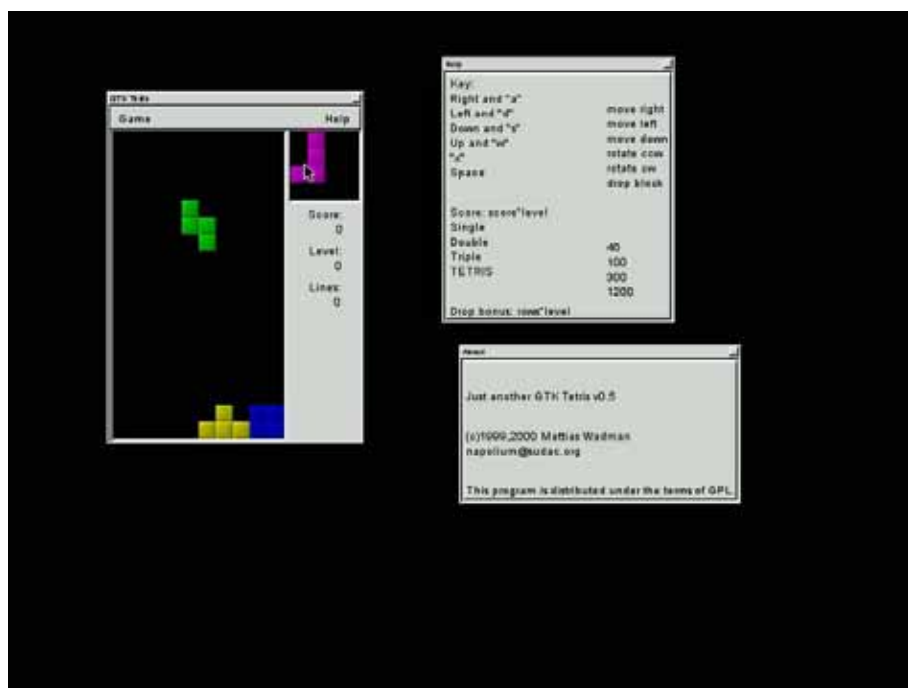


图 6-14 GtkFB 应用例子

- λ Microwindows (<http://microwindows.org/>): 是一个著名的开放式源码的嵌入式GUI软件，目的是把现代图形视窗环境引入到运行Linux的小型设备和平台上。作为X Window的替代品，Microwindows可以使用更少的RAM和文件存储空间（100K-600K）提供与X Window相似的功能。该体系结构允许设计者轻松加入各种显示设备、鼠标、触摸屏和键盘等。它提供了比较完善的图形功能，包括一些高级的功能，比如 Alpha 混合，三维支持，TrueType 字体支持等。它的图形引擎被设计成能够运行在任何支持readpixel、writepixel、drawhorzline、drawvertline和setpalette 的系统之上。Microwindows支持Linux FrameBuffer，目前提供每像素1、2、4、8、16、24和32位的支持，另外还支持彩色显示和灰度显示，对于Linux平台的嵌入式系统来说，Microwindows无疑是合理的选择之一。

Microwindows 的可移植性非常好，基本上用 C 语言实现，只有某些关键代码使用了汇编以提高速度，并能支持 Intel 16 位、32 位 CPU，MIPS R4000 以及 ARM 架构的处理器。

Microwindows 系统以图形方式支持在主机平台上仿真目标平台。这样，为 Linux 设计的 Microwindows 应用程序就可以在台式机上编写和开发而不用进行交叉编译就可测试和运行，并且直接在目标平台上运行。台式主机上运行的目标应用程序在 X window 中显示，这一点通过由 Microwindows 提供的 X window 屏幕驱动来完成，而不是通过帧缓存驱动。该驱动可以在像素位素和颜色深浅方

面准确的模拟目标平台上的显示器，这样即使桌面系统是 24 位颜色，它也可以预览显示 2bpp 灰度级的目标应用程序。

Microwindows 采用分层设计方法。在最底层，屏幕，鼠标/触摸屏以及键盘驱动程序提供了对物理设备访问的能力。在中间层，实现了一个可移植的图形引擎，支持行绘制，区域填充，剪切以及颜色模型等。在上层，实现多种 API 以适应不同的应用环境。

Microwindows 使用两种流行的图形编程接口：Microsoft Windows Win32/WinCE 图形显示接口 (GDI) 和 Xlib-like 接口。前者应用于所有的 Windows CE 和 Win32 应用程序，后者就象 Nano-X，应用于所有 Linux X 插件集的最底层。这样做可以让大量的 Windows 程序员能快速上手开发 Microwindows 图形应用程序，类似地也可以让 Linux 图形程序员用 X 接口开发图形应用程序。图 6—15 就是一个采用 Nano-X 服务器的 PIXIL PDA's 上的 WorldClock 程序界面。



图 6—15 Microwindows 应用例子



图 6—16 MiniGUI 应用例子

- λ MiniGUI (<http://www.minigui.org/>): 是一种面向嵌入式系统或者实时系统的图形用户界面支持系统，是国内做得较好的自由软件之一。它的主要特点为：
- | 跨操作系统支持，包括 Linux, uClinux, eCos, UC/OS-II, VxWorks 等；
 - | 支持多种运行模式；
 - | 内建资源支持，适合于无文件系统的小型嵌入式操作系统；

- | 完备的多窗口机制和消息传递机制;
- | 提供对话框和常用的的控件类 (按钮、单行和多行编辑框、列表框、进度条、工具栏等);
- | 界面皮肤支持, 可获得华丽的图形界面;
- | 多字符集和多字体支持, 包括 ISO8859-1、GB2312、Big5 等字符集, 并且支持各种光栅字体和 TrueType、Type 1 等矢量字体;
- | 多汉字输入法支持, 包括全拼、五笔、智能拼音等;
- | BMP、GIF、JPEG、PCX、TGA 等常见图像文件的支持;
- | Windows 的资源文件支持, 如位图、图标、光标等;
- | 针对嵌入式系统的特殊支持, 包括一般性的 I/O 流操作, 字节序相关函数等。

与其它针对嵌入式产品的图形系统相比, MiniGUI 具有以下一些技术优势:

1、轻型、占用资源少: MiniGUI 本身的占用空间非常小, 以嵌入式 Linux 操作系统为例, MiniGUI 的典型存储空间占用情况如下:

Linux 内核: 300K~500K(由系统需求决定);

文件系统: 500K~2MB(由系统需求决定);

MiniGUI 支持库: 500K~700K(由编译选项确定);

MiniGUI 字体、位图等资源: 400K(由应用程序确定, 可缩小到 200K 以内);

GB2312 输入法码表: 200K(不是必需的, 由应用程序确定);

应用程序: 1M~2M(由系统决定)。

总体的系统占有空间应该在 2MB 到 4MB 左右。在某些系统上, 功能完备的 MiniGUI 系统本身所占用的空间可进一步缩小到 1MB 以内。

最新的资料表明, MiniGUI 能够在 CPU 主频为 30MHz, 仅有 4M RAM 的系统上正常运行(使用 uClinux 操作系统), 这是其它图形系统, 如 MicroWindows 或者 Qt/Embedded 所无法达到的。

2、高性能、高可靠性

MiniGUI 良好的体系结构及优化的图形接口, 可确保最快的图形绘制速度, 它在设计之初就充分考虑到了实时嵌入式系统的特点, 针对多窗口环境下的图形绘制开展了大量的研究及开发, 优化了 MiniGUI 的图形绘制性能及资源占有。

目前, MiniGUI 已在大量实际系统中成功应用, 尤其在工业控制系统的应用, 证明 MiniGUI 具有非常好的性能。

3、可配置性

为满足嵌入式系统的差异性需求, 必须要求 GUI 系统是可配置的。和 Linux 内核类似, MiniGUI 也实现了大量的编译配置选项, 通过这些选项可指定 MiniGUI 库中包括哪些功能而同时不包括哪些功能。大体说来, 可以在如下几个方面对 MiniGUI 进行定制配置:

指定 MiniGUI 要运行的操作系统;

指定生成基于线程的 MiniGUI-Threads 运行模式还是基于进程的 MiniGUI-Lite 运行模式, 或者只是最简单的 MiniGUI-Standalone 运行模式;

指定要采用老的 GAL/GDI 接口 (低端显示设备) 还是新的 GAL/GDI 接口 (高端显示设备);

指定需要支持的 GAL 引擎和 IAL 引擎, 以及引擎相关选项;

指定需要支持的字体类型;

指定需要支持的字符集;

指定需要支持的图像文件格式;

指定需要支持的控件类;

指定控件的整体风格, 是三维风格、平面风格还是手持终端风格。

4、可伸缩性强

MiniGUI 丰富的功能和可配置特性, 使得它既可运行于低端产品中, 亦可运行于高端产品中, 并使用 MiniGUI 的高级控件风格及皮肤界面等技术, 创建华丽的用户界面。

5、跨操作系统支持

MiniGUI 理论上可支持任意一个多任务嵌入式操作系统, 目前实际已支持 Linux/uClinux、eCos、uC/OS-II、VxWorks、pSOS、ThreadX 等嵌入式操作系统, 同时还可以在 Win32 平台上运行, 并在不同操作系统上的 MiniGUI, 提供完全兼容的 API 接口。从最初的数控系统到目前流行的智能手持终端设备, MiniGUI 已经在高端手机、PDA 类产品、数字媒体及机顶盒类产品、工业仪表及控制系统等大量产品中得到了广泛的应用。

MiniGUI 参照了 Microsoft Windows 图形显示接口, 其概念和 API 都和 Windows 的非常相似, 可以让大量的 Windows 程序员能快速上手开发 MiniGUI 图形应用程序。同时, MiniGUI 技术文档和手册都有中文版本, 很适合国内用户开发。图 6-16 就是一个 MiniGUI 应用例子。

λ OpenGUI (<http://www.tutok.sk/fastgl/>): 在 Linux 系统上存在已经很长时间了。最初的名字叫 FastGL, 只支持 256 色的线性显存模式, 但目前也支持其他显示模式, 并且支持多种操作系统平台, 比如 MS-DOS、QNX 和 Linux 等等, 不过由于其使用了大量 x86 汇编语言, 目前只支持 x86 硬件平台, 可移植性稍差。OpenGUI 可以为软件开发提供简单的 2D 制图和事件驱动窗口 API。它支持 BMP 图形格式, 功能非常强大而使用简单。你可以在老的 Borland BGI 下编写应用程序, 也可以在像 QT 那样的窗口环境下使用。OpenGUI 支持像鼠标、键盘这样的事件源, 而且可以使用 8、15、16 和 32 位颜色模式。OpenGUI 采用 LGPL 条款发布。

OpenGUI 也分为三层。最低层是由汇编编写的快速图形引擎; 中间层提供了

图形绘制 API，包括线条、矩形、圆弧等，并且兼容于 Borland 的 BGI API。第三层用 C++ 编写，提供了完整的 GUI 对象集。图 6-17 就是一个 OpenGUI 应用例子。

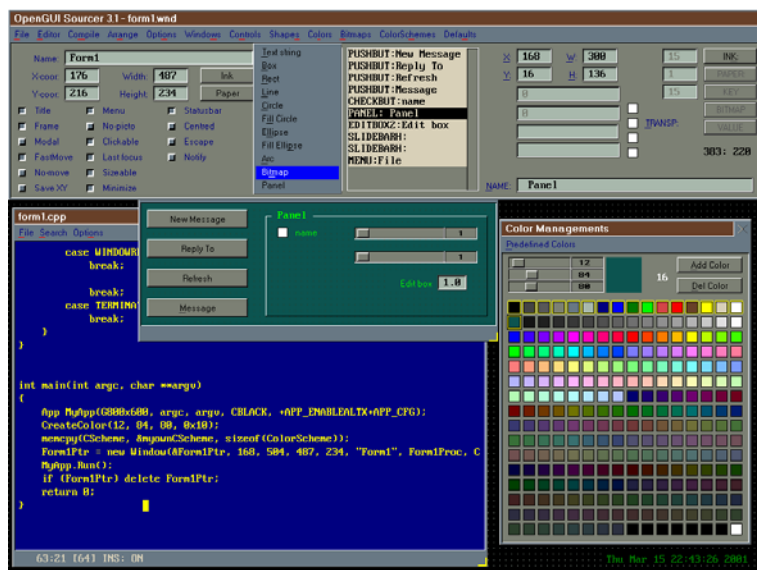


图 6-17 OpenGUI 应用例子

- λ OPIE (<http://opie.handhelds.org>): 可以说是Trolltech公司的Qtopia的一个翻版，目的也是为基于Linux的PDA、智能电话和其他移动设备设计一个完整的图形应用环境。它是一个完全开放源码的项目，和作为商业版本的Qtopia有很大区别，但它同样也是在Qt/Embedded的基础上开发的。OPIE具有综合个人信息管理功能的框架，并且带有其他功能比如扩展的多媒体功能，文档模板，网络和通信工具以及对多种语言的支持。典型界面如图 6-13 所示。
- λ PicoGUI (<http://picogui.org/>): 是一个可以工作在包括手持式设备等各种硬件上的小型的、可移植的、基于客户/服务器结构的GUI。同X Window系统一样，它具有客户—服务器结构的灵活性，但又不同于X Window系统，它将字体、BMP文件、控件以及一些应用程序所需要的其它的一些资源直接集成在服务器。虽然减少了系统的灵活性，但在速度上有了很大的提高，并且减小了程序大小。PicoGUI和别的GUI最大的不同，是它具有Themes（即主题），就象WIN98 或XP一样，可以更换不同的界面。它的目标是集成X Window的灵活性和Palm OS的简单和快速。PicoGUI自身非常小，一般在 100k左右，包括了服务器，字体以及其他必需组件。主题的大小就要看包括的位图文件的大小了，自带典型的Aqua主题在 20k左右。客户端使用的PicoGUI库一般在 6k左右。PicoGUI运行时自身用了 5k内存，自带典型的Aqua主题和Demo应用用了 42k左右内存。总之，PicoGUI是一个非常精巧的

小型GUI环境。图6-18就是一个典型的PicoGUI应用例子。



图 6.18 PicoGUI 应用例子

- λ Qt/Embedded (<http://www.trolltech.com/products/qt3/embedded/index.html>): 是著名的Qt库开发商Trolltech公司开发的面向嵌入式系统的Qt版本, 开发人员多为KDE项目的核心开发人员。许多基于Qt的X Window程序可以非常方便地移植到Qt/Embedded上, 与X11版本的Qt在最大程度上接口兼容, 延续了在X上的强大功能, 并在底层彻底摒弃了X lib, 仅采用framebuffer作为底层图形接口。Qt/Embedded类库完全采用C++封装。丰富的控件资源和较好的可移植性是Qt/Embedded最为优秀的一方面, 使用X下的开发工具Qt Designer可以直接开发基于Qt/Embedded的UI (用户操作接口) 界面。越来越多的第三方软件公司也开始采用Qt/Embedded开发嵌入式Linux下的应用软件。

在 Qt/Embedded 的基础上, Trolltech 推出了 Qt Palmtop Environment (Qtopia) 一个面向基于 Linux 的 PDA, 智能电话和其他移动设备设计的一个全面的、可以用户化的应用程序平台和用户界面, 并已经成功应用于多款高档 PDA。Qtopia 包含完整的应用层、灵活的用户界面、窗口操作系统、应用程序启动程序以及开发框架。图 6-19 就是 Qtopia 的典型界面。

- λ Tiny-X (<http://www.xfree86.org>): 是X服务器在嵌入式系统的小巧实现, 它由开放源码的X Window项目Xfree86 核心团队的Keith Packard开发。它的目标是运行于小内存系统环境。典型的运行于X86 CPU 上的Tiny-X Server 尺寸接近(小于)1MB。

由于Tiny-X是以Xfree86为基础进行的精简,因此它最大的优点就是开发速度,很多以Qt、GTK+、FLTK等等所开发的软件可以很容易的移植上来。但虽然移植应用程序速度快,但由于很多软件本来是针对桌面环境所开发,无形之中功能比较多样,有些并不适用于嵌入式环境,因此软件精简成为采用此架构最大的课题,有时候重新改写都可能比精简所需的时间还短。图6-20就是一个典型的Tiny-X应用例子。

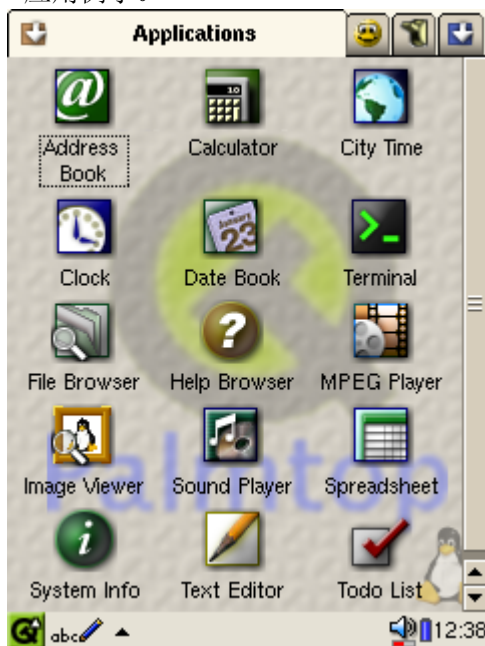


图 6-19 Qtopia 界面



图 6-20 Tiny-X 应用例子

6.6.2 MiniGUI 的移植和应用开发

在上节,我们对当前可用的Linux下嵌入式图形系统作了一个综述,这些系统各有其优势和特点,如何在开发自己的图形应用时进行选择,还需要根据自己应用的特点来确定。由于MiniGUI是唯一主要在国内开发的嵌入式图形系统,国内的开发者也比较多,因此在本节我们只介绍MiniGUI在EP93XX系列嵌入式设备上的移植和应用开发。

关于MiniGUI的更多内容,可以到MiniGUI站点(<http://www.minigui.org/>)去下载查看详细的技术资料。

λ 带 TrueType 字体支持的 MiniGUI 的移植

1. 环境建立

交叉编译工具链的安装以及NFS环境的建立可以参考第五章的相关内容。在此需要再安装一个MIZI Linux 2.0 开发包,可以从http://www.mizi.com/developer/mz20/tools_list.htm下载。它是一个开发工具包,包括了很多已经为ARM处理器交叉编译好的库文件,省去了自

已寻找库源码并且进行交叉编译移植以及测试的工作。

下载完全部包后，运行：

```
#rpm -Uvh *.rpm
```

来进行安装。如果安装过程顺利，会在/opt下生成host和target两个目录。其中，host是用于在主机上开发嵌入式应用的交叉编译开发环境目录，包括armv4l-redhat-linux-gcc交叉编译器工具链和Mizi开发包带的移植好的库。而Target为需要拷贝到目标系统上的程序和库。

2. 下载 MiniGUI 源码

可以到<http://www.minigui.com/download/cindex.shtml>下载，包括：

libminigui-1.3.3.tar.gz: MiniGUI 库源代码，其中包括 libminigui、libmywins、libmgext 和 libvcongui。

minigui-res-1.3.3.tar.gz: MiniGUI 库所使用的资源，包括基本字体，图标，位图和鼠标光标等

mde-1.3.0.tar.gz: MiniGUI 的综合示例程序

mg-samples-1.3.0.tar.gz: 是《MiniGUI 编程指南》的配套示例程序包。

由于要移植的MiniGUI需要支持TrueType字体，因此还需要下载一个依赖库FreeType，地址为 <http://www.minigui.com/downloads/dep-libs/freetype-1.3.1.tar.gz>。

3. FreeType 库移植

在 MiniGUI 移植前，先要完成它所依赖库的移植工作。将 freetype-1.3.1.tar.gz 解开，

```
#tar xzf freetype-1.3.1.tar.gz
# cd freetype-1.3.1
```

然后进行配置

```
#!/configure --prefix=/usr/local/arm/2.95.3/arm-linux
```

接着需要修改一些 Makefile 文件中的编译器设置，使之能够进行交叉编译。具体需要修改 lib/arch/unix/Makefile 文件，将 CC = gcc 改成 CC = arm-linux-gcc，将 CPP = gcc -E 改成 CPP = arm-linux-gcc -E。然后就可以进行编译了

```
#make
```

将会进行编译，但是到 test 目录下会出错，表示测试程序还没有为 ARM 移植做好准备，但在这之前，我们需要移植的 libtff 库已经制作完成。我们来将该移植好的库和头文件安装到交叉编译器库和头文件目录下。

```
#make install
```

同样到 test 目录下会出错，但在这之前，libtff 库文件已经安装到 /usr/local/arm/2.95.3/arm-linux/lib，头文件会安装在 /usr/local/arm/2.95.3/arm-linux/include/freetype 下。

由于 Makefile 文件的小错误，我们还需要手工处理一下

```
#cd /usr/local/arm/2.95.3/arm-linux/include/
```

```
#mkdir freetype1
#mv freetype freetype1
```

该操作将头文件转移到/usr/local/arm/2.95.3/arm-linux/include/freetype1/freetype 下。至此，FreeType 库的移植全部完成。接下去我们要进行 MiniGUI 的移植了。

4. 配置和编译 MiniGUI 库

到 libminigui-1.3.3.tar.gz 所在目录，将其解开

```
#tar xzvf libminigui-1.3.3.tar.gz
#cd libminigui-1.3.3
```

在 libminigui-1.3.3 中，可以支持菜单配置，同 Linux 的内核配置相似，运行

```
#make menuconfig
```

就可以看到比较熟悉的配置菜单界面了

```
System wide options --->
GAL engine options --->
IAL engine options --->
Font options --->
Charset options --->
Keyboard layout specific options --->
Image options --->
Input method options --->
Appearance options --->
Misc options --->
Controls options --->
Ext library options --->
---
```

```
Development environment options --->
---
```

```
Load an Alternate Configuration File
Save Configuration to an Alternate File
```

选中 Development environment options --->，进入下级菜单

```
(Linux) Platform
(arm-linux-gcc) Compiler
(glibc) Libc
--- Installation options
    Path prefix: "/usr/local/arm/2.95.3/arm-linux"
--- Additional Compiler Flags
    CFLAGS: ""
    LDFLAGS: "-L/opt/host/armv4l/armv4l-redhat-linux/lib"
```

在 Compiler 一栏，敲回车进入，选中 arm-linux-gcc，表示需要 arm-linux-gcc 交叉编译器，确定退出。下移到 Path prefix 一栏，敲回车进入，输入/usr/local/arm/2.95.3/arm-linux，表示移植好的库需要安装到交叉编译器的库和头文件目录下，确定退出。下移到 LDFLAGS 一栏，敲回车进入，输入-L/opt/host/armv4l/armv4l-redhat-linux/lib，表示编译需要使用位于/opt/host/armv4l/armv4l-redhat-linux/lib 下的一些移植好的库，确定退出。最后结果如上所示。

然后退回上级菜单，再进入 GAL engine options ---->，

```

[NEWGAL] GAL and its engines
[*] NEWGAL engine on Linux FrameBuffer console
[ ] NEWGAL engine on Qt Virtual FrameBuffer
[ ] NEWGAL engine on eCos LCD interface
[ ] Dummy NEWGAL engine
[*] Have console on Linux FrameBuffer

```

使用 NEWGAL 引擎, 并选中 NEWGAL engine on Linux FrameBuffer console 以及 Have console on Linux FrameBuffer, 表示 NEWGAL 引擎基于 Linux FrameBuffer。然后退回上级菜单, 再进入 IAL engine options --->

```

[ ] EP7211-based board
[ ] ADS Graphics Client
[ ] iPAQ H3600 (also H3800)
[ ] MPC823
[ ] PX255B
[ ] NEC VR4181
[ ] Helio Touch Panel
[ ] MT T800
[ ] SMDK2410 Touch Screen
[ ] uClinux Touch Screen Palm/MC68EZ328
[ ] Dummy IAL engine
[*] Native (console) input engine
--- Native IAL engine subdriver options
[*] PS2 mouse
[*] IntelligentMouse (IMPS/2) mouse
[ ] MS mouse
[ ] MS3 mouse
[ ] GPM daemon

```

如果有适合自己系统的 IAL 引擎或者自己实现了某个 IAL 引擎, 则将其选中。一般还会选中 Native (console) input engine, 可以使用鼠标作为输入设备。然后退回上级菜单, 再进入 Font options --->

```

[*] Raw bitmap font
[ ] Var bitmap font
[ ] Incore font sansserif
[ ] Incore font courier
[ ] Incore font symbol
[ ] Incore font vgas
[*] Qt Prerendered Font
[*] TrueType font
[ ] Adobe Type1 font

```

选中 TrueType font。并且不能选中 Var bitmap font, 否则 MiniGUI 库交叉编译将会出错。退回上级菜单, 其他的配置可以根据自己需要来定。在配置完成后, 存盘退出。

在 MiniGUI 库配置好后, 还需要手工来修改一些 Makefile 文件中的编译器设置, 使之能够进行交叉编译。具体需要修改 src 目录和 src/mybmp 目录下的 Makefile 文件, 找到 INCLUDES=一栏, 在等号右边增加 -I/opt/host/armv4l/armv4l-redhat-linux/include, 找到 LDFLAGS=一栏, 在等号右边增加 -L/opt/host/armv4l/armv4l-redhat-linux/lib, 告诉交叉编译器所使用的库和头文件的搜索路径。然后就可以进行编译了

```
#make
```

进行编译工作。编译完成后需要运行

```
#make install
```

来进行 MiniGUI 库和头文件的安装，将会安装到交叉编译器的库和头文件目录下。

注意在 MiniGUI 库的交叉编译过程中有可能会出现问题类似于 “/usr/lib/libtff.so: could not read symbols: Invalid operation” 的错误，这是由于连接器先去连接了主机的 /usr/lib/libtff.so，而该代码是基于 x86 的，不是针对 arm 的。所以我们需要先移除主机上的 libtff 库，将它用 mv 命令移到其他非库文件搜索目录下，如

```
#mv /usr/lib/libtff.* /usr/
```

然后再重新运行 make 进行编译，编译完后不要忘了将刚才移掉的 libtff 库移回 /usr/lib 目录。

5. 安装 MiniGUI 的资源文件

到 minigui-res-1.3.3.tar.gz 所在目录，将其解开

```
#tar xzvf minigui-res-1.3.3.tar.gz
#cd minigui-res-1.3.3
```

修改 config.linux，找到 prefix = \$(TOPDIR)/usr/local，将其改成 prefix = /usr/local/arm/2.95.3/arm-linux，找到 CC= gcc，将其改成 CC= arm-linux-gcc，然后运行

```
#make install
```

将资源安装到 /usr/local/arm/2.95.3/arm-linux/lib/minigui/res。

6. 移植配套示例程序包

到 mg-samples-1.3.0.tar.gz 所在目录，将其解开

```
# tar xzvf mg-samples-1.3.0.tar.gz
# cd mg-samples-1.3.0
```

进行配置

```
#!/configure --target=arm-linux --host=arm-linux
--prefix=/usr/local/arm/2.95.3/arm-linux --build=i686-linux
```

修改 src/Makefile 文件，找到 LIBS = 一栏，最后增加 -lttf，表明需要 libtff 库。如果 MiniGUI 支持 TrueType 字体，则所有基于该 MiniGUI 的应用程序都必须在编译选项上加上 -lttf。然后进行编译

```
#make
```

最后生成的例子程序都在 src 目录下

7. 安装 MiniGUI 库，资源，libtff 库以及示例程序到目标系统

将 MiniGUI 库（libmgext，libminigui，libvcongui）拷贝到目标板的 /usr/local/lib 目录，并将资源目录 /usr/local/arm/2.95.3/arm-linux/lib/minigui 整个目录拷贝到目标系统的 /usr/local/lib 目录。libtff 库可以拷贝到目标系统的 /lib 目录。示例程序可以根据自己需要拷贝到目标系统的某个目录。

8. 制作目标板上的 MiniGUI 配置文件 MiniGUI.cfg

由于 MiniGUI 库执行时需要寻找一个配置文件路径 MiniGUI.cfg, 寻找顺序依次为用户主目录、usr/local/etc、/etc, 所以只要在这三个目录中生成一个 MiniGUI.cfg 即可。支持 TrueType 字体的示例文件如下, 该设置使用的 TrueType 字体是从 Windows 下拷贝了 simsun.ttf, 将其拷贝到目标板的/usr/local/lib/minigui/res/font/目录下。

```
[system]
# GAL engine
gal_engine=fbcon

# IAL engine, 可以换成自己的 IAL 引擎
ial_engine=console

mdev=/dev/input/mice
mtype=PS2

[fbcon]
#根据显示的大小和颜色来设置
defaultmode=640x480-16bpp

#下面是字体设置, 字体 1 为 ttf-simsun 字体, 被大多数
[systemfont]
font_number=2
font0=rbf-fixed-rrncnn-8-16-IS08859-1
font1=ttf-simsun-rrncnn-16-16-GB2312

default=0
wchar_def=1
fixed=1
caption=1
menu=1
control=1

[rawbitmapfonts]
font_number=1
name0=rbf-fixed-rrncnn-8-16-IS08859-1
fontfile0=/usr/local/lib/minigui/res/font/8x16-iso8859-1.bin

[varbitmapfonts]
font_number=0

[qpf]
```

```
font_number=0

[truetypefonts]
font_number=2
name0=ttf-arial-rrncnn-0-0-IS08859-1
fontfile0=/usr/local/lib/minigui/res/font/arial.ttf
name1=ttf-simsun-rrncnn-0-0-GB2312
fontfile1=/usr/local/lib/minigui/res/font/simsun.ttf

[type1fonts]
font_number=0
```

9. 可以运行示例程序包中的一些程序来进行测试，如 button 等。

λ 基于 MiniGUI 的应用开发

嵌入式图形界面程序的开发一般都是先在主机上完成界面的设置及相关开发，然后再到实际嵌入式设备上看运行效果来进行修正。因此需要在主机上建立 MiniGUI 的本地开发环境。关于在主机上安装和运行 MiniGUI 的步骤请参见 MiniGUI 用户手册，这里也不再赘述。

在主机上安装 MiniGUI 成功后，我们可以尝试来开发一个简单的例子。很多带图形显示的嵌入式设备都会有触摸屏，而触摸屏在使用一段时间后将会不可避免的产生定位偏差，特别是对于一些大尺寸的触摸屏，偏差相应也会大些，造成定位不准，用户使用不便。因此在产品中一般都需要能进行触摸屏校正，在定位不准后需要运行校正程序来进行校正。该功能的实现需要支持校正功能的触摸屏驱动以及一个图形化的校正程序来协同完成。本节就以触摸屏校正程序为例，介绍一下如何进行基于 MiniGUI 的应用开发。触摸屏校正程序的思路就非常简单，在 LCD 的几个固定点处显示标记，这些点会有对应的 LCD 逻辑坐标，让用户去触摸这些点，触摸屏驱动就可以得到这些点的位置以及计算出触摸屏的逻辑坐标来，根据两种逻辑坐标的偏差修正触摸屏的逻辑坐标计算参数，使得触摸屏计算出来的逻辑坐标等同于 LCD 逻辑坐标。

由于主机上一般不带有触摸屏，所以程序无法进行触摸屏设备操作，无法完成整个程序的功能。但在主机上的开发主要是为了确定基于 MiniGUI 的图形界面，所以并不受影响。在主机上完成图形界面以及事件响应编程后，再加上具体触摸屏设备操作的代码，然后经过交叉编译下载到具体板子上执行测试。示例源码如下，程序中的注释部分是对触摸屏的操作部分。

```
/*
 * Description: Calibrate touch screen
 * Copyright (C) 2004- Hyesco Technology Co.,Ltd (www.hyesco.com)
 * Author: Zheng Geng <gzheng@hyesco.com>
 */
#define TS_POINT
```

```
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#include <minigui/common.h>
#include <minigui/minigui.h>
#include <minigui/gdi.h>
#include <minigui/window.h>

int iNumber=0,iWidth,iHeight;
int fd=0;
int fdSaveFile=0;

typedef struct CalInfo{
    POINT screenSample[3];
    POINT displaySample[3];
}CALINFO;
CALINFO calInfo;
typedef struct ts_event {
    unsigned short pressure;
    unsigned short x;
    unsigned short y;
    unsigned short pad;
} TS_EVENT;

#define TS_GET_CAL          0
#define TS_SET_CAL          1
#define TS_GET_RAWPOINT    2

int drawPoint(HDC hdc,int iNum)
{
    switch(iNum)
    {
        case 0:
            MoveTo(hdc,iWidth*0.10,iHeight*0.15);
            LineTo(hdc,iWidth*0.20,iHeight*0.15);
            MoveTo(hdc,iWidth*0.15,iHeight*0.10);
            LineTo(hdc,iWidth*0.15,iHeight*0.20);
            break;
        case 1:
```



```

        MoveTo(hdc, iWidth*0.80, iHeight*0.50);
        LineTo(hdc, iWidth*0.90, iHeight*0.50);
        MoveTo(hdc, iWidth*0.85, iHeight*0.45);
        LineTo(hdc, iWidth*0.85, iHeight*0.55);
        break;
    case 2:
        MoveTo(hdc, iWidth*0.45, iHeight*0.85);
        LineTo(hdc, iWidth*0.55, iHeight*0.85);
        MoveTo(hdc, iWidth*0.50, iHeight*0.80);
        LineTo(hdc, iWidth*0.50, iHeight*0.90);
        break;
    }
    return 0;
}

static int CalibrateWinProc(HWND hwnd, int message, WPARAM wParam, LPARAM
lParam)
{
    HDC hdc;
    int x,y;
    int k=-1,i;

    switch(message)
    {
    case MSG_PAINT:
        hdc = BeginPaint(hwnd);
        TextOut(hdc, iWidth*0.2, iHeight*0.5, "Touch Screen Calibration
Program.");
        TextOut(hdc, iWidth*0.2, iHeight*0.5+20, "Copyright (C) 2004-
Beijing Hyesco Technology Co.,Ltd");
        TextOut(hdc, iWidth*0.2, iHeight*0.5+40, "please select the '+'
on the screen");
        drawPoint(hdc,iNumber);

        EndPaint(hwnd, hdc);
        return 0;

    case MSG_LBUTTONDOWN:
        if(iNumber<3)
        {
            /*if
(ioctl(fd, TS_GET_RAWPOINT, &(calInfo.screenSample[iNumber]))<0)

```

```

        printf ("ioctl error\n");
        */iNumber++;
    }
    if(3==iNumber)
    {
        k=MessageBox (hWnd, "Do you want to ajust again?","Attention!",
MB_OKCANCEL);
        if(IDOK==k) {
            iNumber=0;
        }
        else {
            //calibate TS
            /*if (ioctl(fd,TS_SET_CAL,&calInfo)<0)
                printf ("ioctl error!\n");
            else
                //save to file
                write(fdSaveFile,&calInfo,sizeof(CALINFO));
            */
            DestroyMainWindow(hWnd);
            PostQuitMessage(hWnd);
            return 0;
        }
    }
    InvalidateRect(hWnd,NULL,TRUE);
    return 0;

case MSG_CLOSE:
    DestroyMainWindow(hWnd);
    PostQuitMessage(hWnd);
    return 0;
}
return DefaultMainWinProc(hWnd, message, wParam, lParam);
}

int MiniGUIMain(int argc, const char * argv[])
{
    MSG Msg;
    HWND hMainWnd;
    MAINWINCREATE CreateInfo;
    char pEtcFile[]="/usr/local/etc/MiniGUI.cfg";
    char pCalEtcFile[]="Calibrate.cfg";
    char cValue[50];

```

```

char cSavFileName[128];
int iNum, iX, i, ch;
char charX[10], charY[10];
TS_EVENT ev;

#ifdef _LITE_VERTION
    SetDesktopRect(0, 0, 800, 600);
#endif

//open save file
/*if (GetValueFromEtcFile
(pCalEtcFile, "savefile", "filename", cSavFileName, 128) != ETC_OK) {
    printf("open Calibrate.cfg error!\n");
    return -1;
}
fdSaveFile=open(cSavFileName, O_CREAT | O_RDWR );
if (-1 == fdSaveFile) {
    printf("open calibrate save data file error!\n");
    return -1;
}
memset(&ev, 0, sizeof(struct ts_event));

fd = open("/dev/ts", O_RDWR);
if (-1 == fd){
    printf("open device error!\n");
    return -1;
}
*/
if ( (2==argc) && (strcmp(argv[1], "-r")==0) ) {
    //only calibrate using save data
    /*if (read(fdSaveFile, &calInfo, sizeof(CALINFO)) != sizeof(CALINFO))
    {
        printf("read calibrate save data error!\n");
        return -1;
    }
    if (ioctl(fd, TS_SET_CAL, &calInfo) < 0) {
        printf ("ioctl error!\n");
        return -1;
    }
    */
}
else {
    //得到当前屏幕大小

```

```

    if (GetValueFromEtcFile
(pEtcFile, "fbcon", "defaultmode", cValue, 50) != ETC_OK) {
        printf("open /usr/local/etc/MiniGUI.cfg error!\n");
        return -1;
    }
    for(i=0; i<50; i++) {
        if('x'==cValue[i]) {
            iX=i;
        }
        if('-'==cValue[i]){
            iNum=i;
            break;
        }
    }

    memcpy(charX, cValue, iX);
    memcpy(charY, cValue+iX+1, iNum-iX-1);

    sscanf(charX, "%d", &iWidth);
    sscanf(charY, "%d", &iHeight);
    //计算校正标记点坐标
    calInfo.displaySample[0].x=calInfo.screenSample[0].x=iWidth*0.15;
    calInfo.displaySample[0].y=calInfo.screenSample[0].y=iHeight*0.15;
    calInfo.displaySample[1].x=calInfo.screenSample[1].x=iWidth*0.85;
    calInfo.displaySample[1].y=calInfo.screenSample[1].y=iHeight*0.50;
    calInfo.displaySample[2].x=calInfo.screenSample[2].x=iWidth*0.50;
    calInfo.displaySample[2].y=calInfo.screenSample[2].y=iHeight*0.85;
    //创建主窗口
    CreateInfo.dwStyle = WS_VISIBLE;
    CreateInfo.dwExStyle = WS_EX_NONE;
    CreateInfo.spCaption = "Calibrate TouchScreen";
    CreateInfo.hMenu = 0;
    CreateInfo.hCursor = GetSystemCursor(0);
    CreateInfo.hIcon = 0;
    CreateInfo.MainWindowProc = CalibrateWinProc;
    CreateInfo.lx = 0;
    CreateInfo.ty = 0;
    CreateInfo.rx = iWidth;
    CreateInfo.by = iHeight;
    CreateInfo.iBkColor = COLOR_lightwhite;
    CreateInfo.dwAddData = 0;
    CreateInfo.hHosting = HWND_DESKTOP;

```

```

hMainWnd = CreateMainWindow(&CreateInfo);
if(hMainWnd ==HWND_INVALID)
    return -1;

ShowWindow(hMainWnd, SW_SHOWNORMAL);
while (GetMessage(&Msg,hMainWnd)){
    TranslateMessage(&Msg);
    DispatchMessage(&Msg);
}

MainWindowThreadCleanup(hMainWnd);
}
close(fd);
close(fdSaveFile);
PostQuitMessage (HWND_DESKTOP);
ExitGUISafely(0);
}
#ifdef _LITE_VERTION
#include <minigui/dti.c>
#endif

```

简单解释一下以上的程序：**MiniGUIMain** 是主函数，我们可以看到程序首先是对一些配置文件进行操作，完成存储的校正信息的读取；然后从 **MiniGUI** 配置文件中得到当前屏幕大小，用于计算标记点坐标，接着创建主窗口并显示，然后进入消息循环。由 `CreateInfo.MainWindowProc = CalibrateWinProc;` 指明了消息处理主函数为 **CalibrateWinProc**。在 **CalibrateWinProc** 中，程序对更新画面消息 **MSG_PAINT**，鼠标左击消息 **MSG_LBUTTONDOWN** 和窗口关闭消息 **MSG_CLOSE** 进行响应。

熟悉 Windows 下 C 编程的人很容易看出该编程模式基本上等同于 Windows 下 C 编程模式，这也是 **MiniGUI** 的一个特点，基本概念和 API 都和 Windows 的非常相似，可以让大量的 Windows 程序员能快速上手开发 **MiniGUI** 图形应用程序。程序的主要工作就是完成各窗口消息的响应。

在程序完成后，我们在主机上进行程序的本地编译和运行调试。

```

#gcc -o Calibrate Calibrate.c -lminigui -lpthread
#./Calibrate

```

正常的运行结果可见图 6-21。在用户点击鼠标左键后，会在屏幕的其他标记点处画出十字，再由用户鼠标点击，完成三个标记点的鼠标点击后，程序会对触摸屏进行校正。在主机上完成图形界面以及事件响应编程后，我们将程序注释部分再加上，然后经过交叉编译下载到具体板子上执行测试。此时需要设置板子上的 **MiniGUI** 的配置文件中的 **ial_engine** 为相应支持触摸屏的 **IAL** 引擎。

更进一步的编程参考可以参见 MiniGUI 带的编程指南, API 参考手册以及示例程序等。目前 MiniGUI 开发最大的问题在于没有一个好的可视化编程工具支持, 无法完成所见即所得的设计和编程模式, 这也是在很多开放源码的图形库上开发应用的通病。

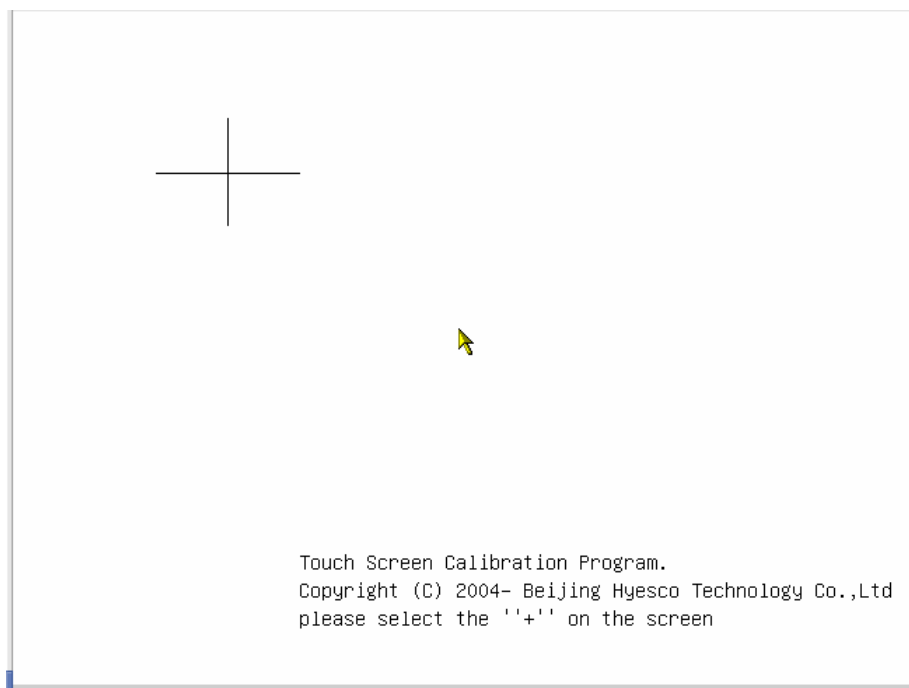


图 6-21 MiniGUI 示例程序运行结果

6.7 本章小结

本章介绍了嵌入式 Linux 的常用应用开发, 包括设备驱动程序的开发、网络应用开发、串行通信应用开发、基于 Web 的应用开发以及图形应用开发等, 希望用户能根据这些编程示例, 完成特定系统的应用开发。

第 7 章 一个完整的应用开发实例

本章介绍一个基于 EP9301/02 的无线数据终端应用系统设计的完整过程，包括从系统需求分析，到硬件平台的设计，软件系统的移植与应用开发的全过程，希望能够使读者对嵌入式系统开发的全过程有所了解。

本章主要包括：

- λ 系统的需求分析；
- λ 硬件平台的选择与设计；
- λ 软件系统的移植与应用开发；

7.1 系统的需求分析

无线数据终端是一种能够通过 GPRS/CDMA 等广域无线网络接入 Internet 的设备。它为用户提供稳定、高速、永远在线、成本低廉的数据传输通道，可广泛应用于金融、保险、电力、监控、交通、气象等行业的如下应用场合：

- λ 电力、自来水、煤气、油井集中监控、远程抄表与收费系统；
- λ 银行无线发卡、ATM 远程监控、POS 等；
- λ 税收、水利、公共安全、公共交通等实时信息查询系统；
- λ 银行、证券、保险等机构的数据网络构建；
- λ 远程监控与安防系统等。

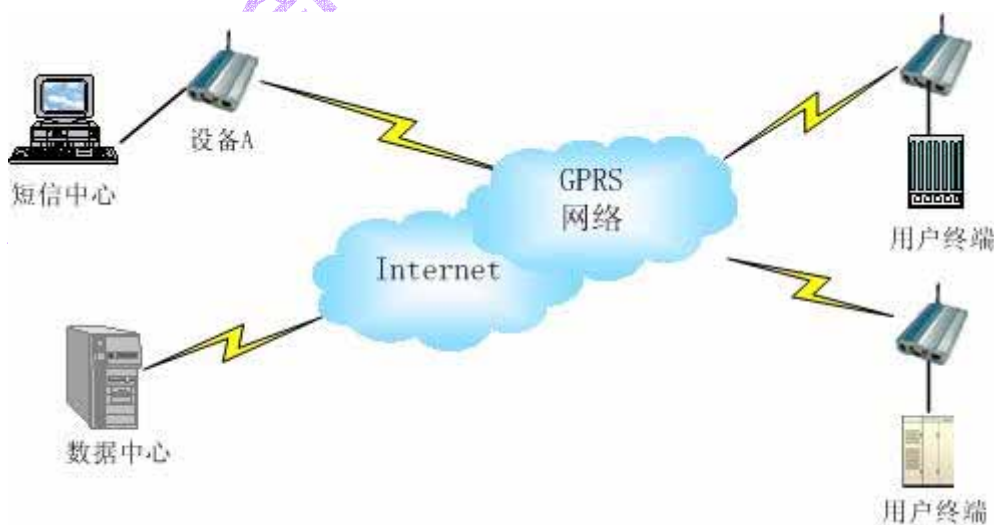


图 7.1 GPRS 数据终端应用示意图

本章以某客户一个 GPRS 数据终端产品委托开发项目为例,介绍应用系统设计开发的完整过程。

客户提出的产品需求如下:

- λ 支持 GPRS800/1900MHz 双频, GPRS Class 2-10, 编码方案: CS1 - CS4, 符合 SMG31bis 技术规范;
- λ 支持接口:
 - v 1 个 RS23 串口, 速率 9600-115200bps, 物理接口 DB9 阴头;
 - v 1 个以太网接口, 速率 10/100M 自适应, 物理接口 RJ45;
 - v SIM/UM 卡接口, 电压 3V, 物理接口抽屉式卡座;
 - v 天线接口, 50Ω/SMA 阴头;
- λ 网络功能:
 - v 支持 INTERNET 互连, 并保持双向数据传输;
 - v 加电自动/手动/短信呼叫拨号;
 - v VPN 隧道技术;
 - v 路由与虚拟地址转换服务 (NAT);
 - v 支持 SNMP 网络监控;
 - v 寻址方式:
 - v IP 直接寻址;
 - v 域名寻址;
 - v 永远在线/网络唤醒;
 - v 重复呼叫频度: 呼叫连接不成功, 重复登录, 超过预定次数, 短信报警;
- λ 数据链路传输方式:
 - v TCP/UDP;
- λ 系统配置:
 - v 串口配置 (通过“超级终端”);
 - v TELNET 配置 (通过命令行/终端软件);
 - v WEB 配置 (通过 IE 或 NetScape);
- λ 自诊断与报警输出, 警示灯状态显示可设置;
- λ 终端内嵌协议栈:
 - v TCP/IP Library;
 - v HTTP, HTML, ICMP, IGMP, TCP, UDP, ARP, SNMP, PPP, IP, FTP, DHCP, DNS, TELNET;
 - v 支持 PAP/CHAP 认证;
- λ 供电电压:
 - v 宽电压直流供电: DC 5~40V;
- λ 环境参数:

- v 工作环境温度：-10~+70°C；
- v 储存温度：-20~+80°C；
- v 相对湿度：95%(无凝)；

7.2 硬件平台的选择与设计

从客户的产品需求可以看出，该无线数据终端对网络功能和系统配置与功能要求较高，综合其它相关需求，并考虑产品的最终性价比和功能扩展，决定选用 EP9301/02 作为系统的核心处理单元，外接 GPRS 模块，作为整个系统的硬件平台。软件以嵌入式 Linux 作为操作系统，并开发相关应用程序，最终满足用户的全部功能需求。

系统硬件平台的结构如图 7-2 所示。

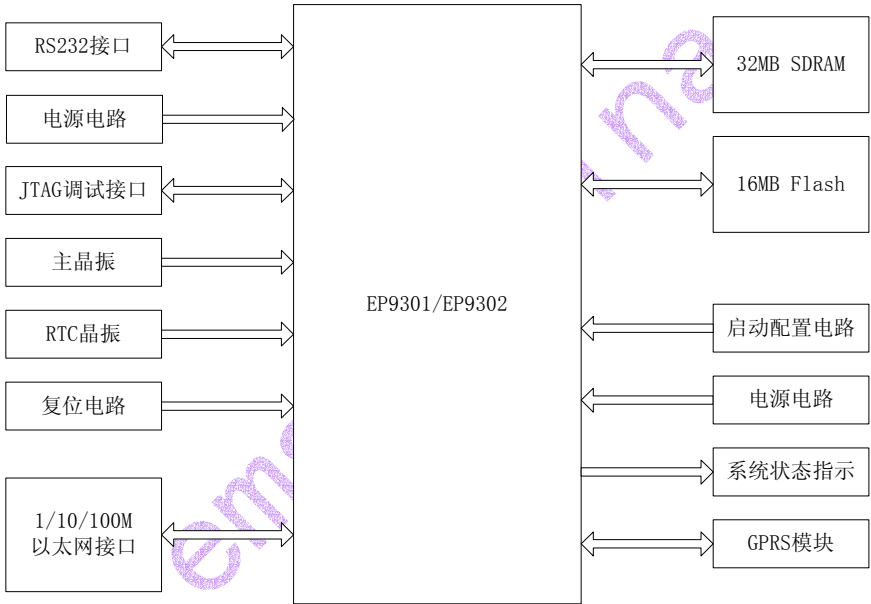


图 7-2 系统的硬件结构

在图 7-2 所示的硬件结构中，RS232 接口和以太网接口实现客户的接口需求，GPRS 模块提供系统的无线数据通信链路，通过 UART 接口与 EP9301/02 通信，实现数据的无线传输，图中的其它功能模块实现系统所需的基本功能。

硬件系统相对比较简单，其具体实现可以参考第三章的相关内容。

7.3 软件系统的移植与应用开发

根据设备的应用需求，需要进行 Linux 内核移植以及应用程序开发。以下从系统角度介绍各部分功能的具体实现方法，可分成：GPRS 联网、网络地址转换（NAT）、Web 配置应

用、串口设备上网以及其他配套程序五大部分。

7.3.1 GPRS 联网

GPRS 联网是通过 PPP（点对点协定）协议来完成的。PPP 是在串行连接的数据链路上实现 IP 以及其它网络协议的一种机制，目前，各种有线/无线 Modem 上网都是采用 PPP 协议。Linux 已经完整实现了 PPP 协议，可以很方便的支持各种有线/无线 Modem 通过拨号访问 Internet。

在此，我们仅说明 PPP 上网的实现方法，关于 PPP 更多的内容可以参考 Linux PPP HOWTO 文档。

PPP 上网的具体实现过程描述如下：

增加 Linux 内核的 PPP 支持

为使嵌入式 Linux 支持 PPP 协议，需要将该功能编译到 Linux 内核。在 Linux 源码目录下运行命令“make menuconfig”进入内核配置菜单，如图 7-3 所示。

```
Code maturity level options --->
Loadable module support --->
System Type --->
General setup --->
Parallel port support --->
Plug and Play configuration --->
Block devices --->
Multi-device support (RAID and LVM) --->
Networking options --->
Network device support --->
Amateur Radio support --->
IrDA (infrared) support --->
ATA/ATAPI/MFM/RLL support --->
SCSI support --->
I2O device support --->
ISDN subsystem --->
Input core support --->
Character devices --->
Multimedia devices --->
File systems --->
Console drivers --->
Sound --->
Multimedia Capabilities Port drivers --->
USB support --->
Bluetooth support --->
Kernel hacking --->
Library routines --->
---
Load an Alternate Configuration File
Save Configuration to an Alternate File
```

图 7-3 配置 PPP 协议到嵌入式 Linux 内核（一）

选中“Network device support”选项，并按回车进入下级菜单，如图 7-4 所示。

```

[*] Network device support
ARCnet devices --->
< > Dummy net driver support
< > Bonding driver support
< > EQL (serial line load balancing) support
< > Universal TUN/TAP device driver support
< > Ethertap network tap (OBSOLETE)
Ethernet (10 or 100Mbit) --->
Ethernet (1000 Mbit) --->
[ ] FDDI driver support
[ ] HIPPI driver support (EXPERIMENTAL)
< * > PPP (point-to-point protocol) support
[*] PPP multilink support (EXPERIMENTAL) (NEW)
< * > PPP support for async serial ports (NEW)
< * > PPP support for sync tty ports (NEW)
< * > PPP Deflate compression (NEW)
< * > PPP BSD-Compress compression (NEW)
< > PPP over Ethernet (EXPERIMENTAL) (NEW)
< > SLIP (serial line) support
Wireless LAN (non-hamradio) --->
Token Ring devices --->
[ ] Fibre Channel driver support
< > Traffic Shaper (EXPERIMENTAL)
Man interfaces --->

```

图 7-4 配置 PPP 协议到嵌入式 Linux 内核 (二)

选中 PPP 各项配置, 其中 PPP multilink support 可选, 其他最好都按图选中, 可以支持 PPP 协议, PPP 异步/同步串口通信以及 PPP 压缩。

退出并保存配置, 然后重新编译内核, 此时生成的 Linux 内核可执行映像文件就支持 PPP 了。

移植 PPP 应用程序 PPPD 和 Chat

PPPD 是 PPP 的一个应用程序, 为 PPP 提供一种在点对点串行线路上传输数据流的方法。如果没有已经交叉编译好的 PPPD 和 chat 程序, 则需要自己从源码开始交叉编译, 生成可执行文件。

可以从站点: <http://www.samba.org/ppp/> 下载源码, 然后根据源码版本选择使用不同版本的交叉编译器来进行编译, 具体编译过程在此忽略。

将编译得到的 PPPD, chat 以及 ppp-on, ppp-off, chat 脚本, pppd 配置文件都拷贝到 NFS 共享目录或更新根文件系统, 来进行下一步的调试, 即保证各个脚本文件和配置文件的正常工作。

在对以上文件的调试过程中, 最好在 Linux 服务器上调通 GPRS 上网全过程, 然后再下载到嵌入式目标系统上调试运行, 因为在 Linux 服务器上的调试手段和调试工具都会比嵌入式目标系统丰富, 而调通后的上网脚本无论是在 Linux 服务器, 还是在嵌入式目标系统上基本都是一致的。

通过 GPRS 联网

现在可以尝试通过 GPRS 联网了。可以开通 Syslogd 日志记录功能, 然后不停的尝试进行 PPP 拨号, 看 /var/log/messages 日志的输出错误, 再调整脚本等配置, 直到成功为止。

PPP 上网需要下面五个脚本文件和配置文件, 分别为: ppp-on、ppp-off、options.gprs、

dialer.gprs、pap-secrets（或 chap-secrets）。

其中：ppp-on 为拨号上网脚本，只要运行 ppp-on，就可以进行 GPRS 拨号上网；

ppp-off 为 PPP 连接断开脚本；

options.gprs 为 pppd 所使用的配置文件；

dialer.gprs 为 chat 所使用的脚本文件；

如果 PPP 服务器是采用 PAP 方式来鉴证用户的话，则需要写一个/etc/ppp/pap-secrets 文件，而如果 PPP 服务器是采用 CHAP 方式来鉴证用户的话，则需要写一个/etc/ppp/chap-secrets 文件。

可以正常工作的脚本文件示例如下。

1、ppp-on 文件示例如下：

```
#!/bin/sh
#
# 需要根据移动运营商提供的参数进行设置，如中国移动为 CMNET；
GPRSCMD='+cgdcont=1,"IP","CMNET"'
# 需要根据移动运营商提供的参数进行设置，如中国移动为*99***1#；
TELEPHONE=*99***1#
# 预留的 AT 初始化命令；
INITCMD=''
# GPRS Modem 所占用的串行接口；
DEVICE=/dev/ttyS1
# 嵌入式目标系统与 GPRS Modem 的串口通信波特率；
SPEED=115200
# 为 pppd 所使用的配置文件；
OPTIONFILE=/etc/ppp/options.gprs
# 为 chat 所使用的脚本文件；
DIALFILE=/etc/ppp/dialer.gprs

export GPRSCMD TELEPHONE INITCMD
exec pppd $DEVICE $SPEED file $OPTIONFILE connect $DIALFILE
```

2、ppp-off 文件示例如下：

```
#!/bin/sh
#####
#
# Determine the device to be terminated.
#
DEV=ppp0
if [ "$1" = "" ]; then
    DEVICE=ttySA0
else
    DEVICE=$1
```

```

fi
#####
#
# If the ppp0 pid file is present then the program is running. Stop it.
if [ -r /var/lock/$DEVICE ]; then
    kill -INT `cat /var/lock/$DEVICE`
#
# If the kill did not work then there is no process running for this
# pid. It may also mean that the lock file will be left. You may wish
# to delete the lock file at the same time.
    if [ ! "$?" = "0" ]; then
        rm -f /var/lock/$DEVICE
    rm -f /var/run/$DEV.pid
        echo "ERROR: Removed stale pid file"
        exit 1
    fi
#
# Success. Let pppd clean up its own junk.
    echo "PPP link to $DEV terminated."
    exit 0
fi
#
# The ppp process is not running for ppp0
echo "ERROR: PPP link is not active on $DEV"
exit 1

```

3、options.gprs 文件示例如下，命令具体含义可参考 pppd 命令说明：

```

-detach
debug
modem
lock
defaultroute
usepeerdns

```

4、dialer.gprs 文件示例如下，命令具体含义可参考 chat 命令说明：

```

#!/bin/sh
#
exec chat -v \
TIMEOUT 3 \
ABORT BUSY \
ABORT NO \
ABORT RINGING \

```

```
' ' AT \
'OK-+++c-OK-AT-OK' 'AT$GPRSCMD' \
'OK' 'AT$INITCMD' \
'OK' 'AT&C0&D0&S0' \
TIMEOUT 30 \
'OK' 'ATD$TELEPHONE' \
CONNECT ' '
```

5、pap-secrets 文件示例如下:

```
# client server secret acceptable local IP addresses
# 这里需要根据移动运营商提供的用户名/密码进行设置, 目前中国移动都为空;
"" * "" *
```

在命令行接口手工运行 ppp-on 脚本文件能够通过 GPRS Modem 上网之后, 就可以在程序里使用: system(“ppp-on”)来调用运行 ppp-on 脚本。

具体逻辑需要根据设置情况, 如果是自动方式, 在设备启动后的设备初始程序里直接调用; 如果是手动方式, 则需要用户在管理界面操作后才可以上网; 而短信呼叫方式则是在收到特殊短信后拨号上网, 需要在短信接收后台程序里调用。

成功上网后, 在命令行使用“ifconfig”命令, 将会看到出现了 ppp0 设备, 如图 7-5 所示, 用户就可以用“ping”命令来测试同其他 Internet 地址的通信了。

```
>/> ifconfig
eth0      Link encap:Ethernet  HWaddr 00:40:95:36:35:34
          inet addr:192.168.1.1  Bcast:192.168.0.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:100
          Interrupt:17

lo         Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0

ppp0      Link encap:Point-to-Point Protocol
          inet addr:220.192.60.118  P-t-P:220.192.56.18  Mask:255.255.255.255
          UP POINTOPOINT RUNNING NOARP MULTICAST  MTU:1500  Metric:1
          RX packets:3 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:100
          ..
```

图 7-5 PPP 协议联网成功后的 PPP0 设备

GPRS 的网络连接是整个应用系统的基础, 由于同时涉及到与 GPRS Modem 的硬件通讯及软件驱动, 因此, 联网过程相对比较复杂, 其完整的联网步骤可以总结如下:

1、首先要保证与 GPRS Modem 的硬件连接无误，可以通过 minicom、tip 或其他超级终端程序使用 AT 指令进行测试，并可直接使用 AT 指令进行网络连接。

2、建议先在 Linux 主机上调通 GPRS PPP 上网全过程，然后再下载到嵌入式目标系统上调试，它们所使用的上网脚本基本上是一致的，上网步骤如下：

- λ 制作支持 PPP 的 Linux 内核；
- λ 获得或移植 PPP 应用程序 PPPD 和 chat 程序；
- λ 进行各个上网脚本文件和配置文件调试，直到成功为止。

7.3.2 网络地址转换

目前的 Internet 主要使用 IPv4 协议，由于 IPv4 只能提供 4 个字节的 IP 地址，远远不能满足设备上网的需求，随着 Internet 应用的逐渐推广，出现了 IP 地址空间即将耗尽的问题。

尽管即将出现的 IPv6 被视作为解决 Internet 不断发展的长期解决方案，但是在过去几年中还提出了其它一些短期解决方案，网络地址转换(NAT, Network Address Translation)就是其中的方案之一。

NAT 是用于将一个地址域（如：专用 Intranet）映射到另一个地址域（如：Internet）的标准方法。通过 NAT 技术，人们就可以将一个内部网，使用私有 IP 地址，如 192.168. x. x，通过网关透明地连接到 Internet。GPRS 数据终端集成 NAT 技术，就可以充当 NAT 网关，使得内部局域网计算机能够透明访问 Internet，其结构如图 7-6 所示。



图 7-6 GPRS 数据终端 NAT 的应用

NAT 技术是通过修改 IP 包的源地址或目的地址来实现的。如果一个节点执行 NAT，它将会修改通过它转发的 IP 包的源地址或目的地址。通常这个节点会记住它怎样修改了这个包，因此当相应的应答包从另一个方向到达时，它知道如何反向修改这个应答包。

NAT 有两种类型：Source NAT (SNAT) 和 Destination NAT (DNAT)。

SNAT 是指改变连接的第一个包的源地址。它是在路由之后做的，即在包到达网线之前处理。SNAT 也可称为 IP 伪装，是最常见的应用，如果仅仅是想让内部网络作为 Internet 应用的客户端，发起对 Internet 的访问，如浏览网页，收发 email 等，就可以采用 IP 伪

装技术。让 GPRS 数据终端作为内部网络的网关，所有内部网络的机器发给 Internet 地址的 IP 包都会转发给它。它会将通过的包的源地址改成是自己通过 PPP 拨号得到的 Internet 地址，好象所有的包都是从它这里发出的，同时记录下修改信息。当相应的应答包从 Internet 返回时，它会修改应答包的目的地地址为对应的内部网络机器地址，返回给对应的机器，从而让该内部网络机器能够透明进行 Internet 访问。

DNAT 是指改变连接的第一个包的目的地地址。它是在路由之前做。端口转发、负载均衡、透明代理都属于目的 NAT。最为常用的是端口转发技术，可以让内部网络的服务器，如 Web 服务器，FTP 服务器等，能够被外部机器通过 Internet 访问。

以下介绍如何让 GPRS 数据终端支持 NAT。

增加 Linux 内核的 NAT 支持

首先仍然需要增加 Linux 内核的 NAT 支持。在 Linux 源码目录下运行“make menuconfig”命令，进入内核配置菜单，如图 7-7 所示。

```
Code maturity level options --->
Loadable module support --->
System type --->
General setup --->
Parallel port support --->
Plug and Play configuration --->
Block devices --->
Multi-device support (RAID and LVM) --->
Networking options --->
Network device support --->
Amateur Radio support --->
IrDA (infrared) support --->
ATA/ATAPI/MFM/RLL support --->
SCSI support --->
I2O device support --->
ISDN subsystem --->
Input core support --->
Character devices --->
Multimedia devices --->
File systems --->
Console drivers --->
Sound --->
Multimedia Capabilities Port drivers --->
USB support --->
Bluetooth support --->
Kernel hacking --->
Library routines --->
---
Load an Alternate Configuration File
Save Configuration to an Alternate File
```

图 7-7 配置 NAT 支持到嵌入式 Linux 内核（一）

选中“General setup”选项，并按回车进入下级菜单，如图 7-8 所示：

```

[ ] Compressed boot loader in ROM/flash
(0) Compressed ROM boot loader base address
(0) Compressed ROM boot loader BSS address
[ ] Support for hot-pluggable devices
[*] Networking support
[*] System V IPC
[ ] BSD Process Accounting
[*] Sysctl support
--- At least one math emulation must be selected
<*> NWFPE math emulation
[ ] Support extended precision
<*> FastFPE math emulation (EXPERIMENTAL)
(ELF) Kernel core (/proc/kcore) format
<*> Kernel support for a.out binaries
<*> Kernel support for ELF binaries
<*> Kernel support for MISC binaries
[ ] Power Management support (experimental)
<*> RISC OS personality
    Default kernel command string: "root=/dev/ram"
[ ] Kernel-mode alignment trap handler

```

图 7-8 配置 NAT 支持到嵌入式 Linux 内核 (二)

选中“Sysctl support”选项，该选项支持通过 /proc/sys 目录下的 sysctl 接口来动态更改内核参数而无须重新编译内核。但需要先选中 File systems 中的 /proc file system support。

退回上级菜单，再进入 Networking options 菜单，如图 7-9 所示。

```

<*> Packet socket
<*> Netlink device emulation
[*] Network packet filtering (replaces ipchains)
[ ] Network packet filtering debugging (NEW)
[ ] Socket Filtering
<*> Unix domain sockets
[*] TCP/IP networking
[*] IP: multicasting
[ ] IP: advanced router
[*] IP: kernel level autoconfiguration
[ ] IP: DHCP support
[ ] IP: BOOTP support
[ ] IP: RARP support
<*> IP: tunneling
<*> IP: GRE tunnels over IP
[ ] IP: multicast routing
[ ] IP: ARP daemon support (EXPERIMENTAL)
[ ] IP: TCP Explicit Congestion Notification support
[ ] IP: TCP syncookie support (disabled per default)
IP: Netfilter Configuration --->
<*> The IPv6 protocol (EXPERIMENTAL)
<*> Kernel httpd acceleration (EXPERIMENTAL)
[ ] Asynchronous Transfer Mode (ATM) (EXPERIMENTAL)
<*> 802.1Q VLAN Support
---

```

图 7-9 配置 NAT 支持到嵌入式 Linux 内核 (三)

选中“Network packet filtering (replaces ipchains)”选项，然后再进入“IP: Netfilter Configuration”菜单，如图 7-10 所示。

```

(*) Connection tracking (required for masq/NAT) (NEW)
(*) FTP protocol support (NEW)
<> Amanda protocol support (NEW)
<> TFTP protocol support (NEW)
<> IRC protocol support (NEW)
<> Userspace queueing via NETLINK (EXPERIMENTAL) (NEW)
(*) IP tables support (required for filtering/masq/NAT) (NEW)
<> Limit match support (NEW)
<> MAC address match support (NEW)
<> Packet type match support (NEW)
<> netfilter MARK match support (NEW)
<> Multiple port match support (NEW)
<> TOS match support (NEW)
<> ECN match support (NEW)
<> IGMP match support (NEW)
<> AH/ESP match support (NEW)
<> LENGTH match support (NEW)
<> TTL match support (NEW)
<> tcpmss match support (NEW)
<> Helper match support (NEW)
<> Connection state match support (NEW)
<> Connection tracking match support (NEW)
<> Unclean match support (EXPERIMENTAL) (NEW)
<> Owner match support (EXPERIMENTAL) (NEW)
(*) Packet filtering (NEW)
(*) REJECT target support (NEW)
(*) MIRROR target support (EXPERIMENTAL) (NEW)
(*) Full NAT (NEW)
(*) MASQUERADE target support (NEW)
(*) REDIRECT target support (NEW)
[*] NAT of local connections (READ HELP) (NEW)
(*) Basic SNMP-ALG support (EXPERIMENTAL) (NEW)
<> Packet mangling (NEW)
<> LOG target support (NEW)
<> ULOG target support (NEW)

```

图 7-10 配置 NAT 支持到嵌入式 Linux 内核 (四)

选中“Connection tracking (required for masq/NAT)”和“IP tables support (required for filtering/masq/NAT)”中的“Packet filtering”和“Full NAT (NEW)”选项，其它选项则根据自己应用需求进行选择。

各选项的具体含义可以参考 IPTables 相关资料。

移植 IPTables 应用程序

IPTables 是用于创建 NAT 规则的应用程序。如果没有已经交叉编译好的 IPTables 程序，则需要自己从源码开始交叉编译，可以从站点：<http://www.netfilter.org/downloads.html> 下载源码，然后根据源码版本选择使用不同版本的交叉编译器来进行编译，具体编译过程在此忽略。

将编译得到的 IPTables 拷贝到 NFS 共享目录或者更新根文件系统，进行下一步的调试。

NAT 应用

我们使用 IPTables 进行 NAT 应用，以下介绍常用的 IP 伪装和端口转发两种情况。

如前文所述，IP 伪装可以用于内部网络的计算机通过 GPRS 数据终端访问 Internet。IP 伪装需要设置内部计算机和 GPRS 数据终端，内部计算机的设置相对简单，只需要将其网络连接的默认网关和 DNS 服务器设置成 GPRS 数据终端的 IP 即可。但 GPRS 数据终端就需要用 IPTables 来做一个 NAT 控制脚本，才能实现相应功能。

NAT 控制脚本示例如下，其命令具体含义可参考 IPTables 相关资料：

```

#!/bin/sh
#

```

```

EXITCODE=1
for x in "1" ; do
    if [ $# -lt 1 ] ; then echo "Usage: $0 {start|stop}" ; break ; fi
    action=$1
    case "$action" in
        start)
            echo "Starting firewall"
            # 开启 IP 转发功能:
            echo 1 > /proc/sys/net/ipv4/ip_forward
            # 清除 NAT 表:
            iptables -F -t nat
            # IP 伪装, 所有通过 ppp0 发出的包的源地址都使用 ppp0 的 IP 地址:
            iptables -t nat -A POSTROUTING -o ppp0 -j MASQUERADE
            EXITCODE=0
            ;;
        stop)
            echo "Shutting down firewall"
            # 关闭 IP 转发功能:
            echo 0 > /proc/sys/net/ipv4/ip_forward
            # 清除 NAT 表:
            iptables -F -t nat
            EXITCODE=0
            ;;
        *)
            echo "Usage: $0 {start|stop}"
            ;;
    esac
done
exit $EXITCODE

```

如将该脚本命名为“firewall”，则运行“firewall start”就启动 IP 伪装功能，所有内部网计算机都可以通过 GPRS 数据终端访问 Internet。运行“firewall stop”关闭 IP 伪装功能。

在某些情况下，内部局域网会有一些服务器希望能被外界访问到，例如一个 Web 服务器，我们可以使用 NAT 的端口转发功能来实现。假如 GPRS 数据终端的内部 IP 为 192.168.0.1，通过拨号上网得到的 IP 为 220.192.60.118，Web 服务器的 IP 为 192.168.0.88，监听常用的 80 端口。那么首先需要在 GPRS 数据终端上进行端口注册，该服务注册为 GPRS 数据终端上的 TCP 10080 端口。

则运行命令如下：

```

iptables -A PREROUTING -t nat -p tcp -i ppp0 --dport 10080 -j DNAT --to
192.168.0.88:80

```

端口注册完成并开启 IP 转发功能后，外部用户通过访问 `http:// 220.192.60.118:10080` 就可以访问到内部的 Web 服务器了。

NAT 还支持很多应用方式，在此不一一叙述，有兴趣的读者可以自己钻研 IPTables 的各项功能，加入到特定的嵌入式应用。

综上所述，NAT 功能支持的实现步骤简述如下：

- 1、制作支持 NAT 的 Linux 内核；
- 2、获得或移植 IPTables 应用程序；
- 3、根据应用需求，制作相应的 NAT 规则，并开启 IP 转发功能。

7.3.3 Web 配置应用

Web 配置应用是指在 GPRS 数据终端上运行一个嵌入式 WEB 服务器，管理端只需要通过网络浏览器（IE 或 NetScape），就可以对其进行配置和管理，这也是目前常用的嵌入式设备的管理方式。

WEB 配置应用的实现过程如下：

移植 WEB 服务器

要实现 WEB 配置应用，首先需要在 GPRS 数据终端上移植能支持脚本或 CGI 功能的 WEB 服务器，我们在本例中使用 BOA 服务器，其具体移植和配置过程可参考第六章的相关内容。

制作 WEB 页面模板

然后需要根据应用需求，制作 Web 页面模板，并在页面的界面之外，增加一些注释标记，以供程序使用，其具体方法可参考第六章的相关内容。

图 7—11 为其中的一个页面示例，可以配置 GPRS Modem 的各项参数。

图 7—11 用于 Modem 配置的 Web 页面

各配置项的具体含义如下：

失败重拨次数：拨号失败后的重试次数；

失败重拨间隔：一次失败后再重新拨号之间的时间间隔，以毫秒为单位，0 表示无延时；

接入点：根据移动运营商提供的参数进行设置，如中国移动为 CMNET；

拨号号码：无线拨号号码，根据移动运营商提供的参数进行设置，如中国移动为 *99***1#；

用户名称：GPRS 上网用户名称，如中国移动为空，不超过 20 字符；

用户密码：GPRS 上网用户密码，如中国移动为空，不超过 20 字符；

初始化 AT 指令：慎重，前面不要加 AT，不超过 50 字符。一般为空；

确定：保存信息；

重启连接：在确定保存信息后，按此键会自动断开当前连接，再重新根据配置参数进行连接。

断开连接：断开当前连接。

以“接入点”项为例，对应的 html 文本如下，注释标记加粗：

```
<TR>
    <TD height="24" valign="middle" >&nbsp;&nbsp;&接入点: &nbsp;&nbsp;&&nbsp;&nbsp;&
    <!--AccessPoint-->
    <input name="AccessPoint" type="text" maxlength="50">
    </TD>
    <TD height="24" valign="middle" >&nbsp;&nbsp;&拨号号码:
    <!--GPRSDialNum-->
    <input name="GPRSDialNum" type="text" maxlength="20">
    </TD>
</TR>
```

在它对应的 CGI 程序里就可以根据注释标记来判断 Form 组件信息并进行处理。

编写 CGI 程序

根据应用需求，编写页面对应的 CGI 程序。程序逻辑可分为两个部分，一部分为原来配置信息的显示，另一部分为用户按“提交”键之后的响应。常用的方式是采用一个数据结构来保存所有配置信息，程序主要工作就是完成该结构和页面的数据双向交换。

关于 WEB 配置应用的详细开发过程，可以参考第六章的相关内容。

对于本例的 GPRS 数据终端的管理来说，其它配置界面还包括：

- v 系统信息：显示系统状态，参数等，包括以太网，GPRS Modem 的连接信息；
- v 网络监控：统计建立无线连接后的网络通信情况，如收到和发送的包数，字节总数和错误包数等。以太网网络通信情况也采用同样的方式进行统计。
- v 系统配置：系统重要配置信息，如以太网参数，系统监控配置等；
- v 应用配置：配置串口/网络应用，可以将设备作为串口服务器，串行设备可以通过 GPRS 数据终端直接上网，同时配置网络地址转换 NAT 功能以及短信报警等；

- v VPN 功能：进行 PPTP VPN 服务器/客户的配置；
- v 高级功能：设置管理用户信息以及系统恢复等功能
- v 系统测试：可以对 GPRS Modem 进行测试以及查看日志等。

综上所述，WEB 应用配置功能支持的实现步骤简述如下：

- 1、移植成功支持脚本或 CGI 功能的 WEB 服务器，如 BOA，并进行配置；
- 2、根据应用需求，制作 Web 页面模板，在页面的界面之外，增加一些注释标记，以供程序使用；
- 3、根据应用需求，编写页面对应的 CGI 程序。

7.3.4 串行设备的网络接入

在很多传统设备中，如数据采集设备，医疗设备，POS 机等，都使用诸如 RS-232/422/485 等串行方式与外界通讯，随着以太网架构逐渐成为通信的主流，迫切需要一种简单、经济的方式，将这些传统设备接入主流网络，同时不需要对这些设备的结构进行改变。

GPRS 数据终端集成了串行通信转网络（TCP/IP）通信的技术，可以很好的完成这项功能。

串行通信转网络（TCP/IP）通信实际上就是在 GPRS 数据终端中运行一个转换网关程序，能够将串行通信数据以 TCP/IP 包的形式发出，将收到的 TCP/IP 包用串行通信的方式传输，完成其透明转换。

由于 TCP/IP 通信可以分成是采用 TCP 或 UDP 协议，设备是作为通信的客户端或服务器端，所以 GPRS 数据终端里的程序需要能处理这四种情况，以下分别进行介绍。

GPRS 数据终端作为服务器端

当网关程序是作为服务器端工作时，需要等待客户端主动建立 TCP 连接或 UDP 通信。在 Linux 下，服务器程序很多是采用由超级服务器 inetd 管理的方式。设计超级服务器 inetd 的动机主要是希望能设计一种有效机制，让系统提供许多服务但不过分使用系统资源。因为系统可能会提供多个 TCP/IP 服务，但有些服务不是经常使用的，如果为每个服务都创建进程将会过多的占用系统资源，因此希望只在服务被请求使用的时候才来创建进程。

超级服务器 inetd 就可以完成这项功能，它能够接受客户请求，根据配置来启动相应的服务进程为客户提供服务，当服务完毕后回收系统资源，达到既能及时提供服务又能节省系统资源的目的。

Inetd 在/etc 下有个配置文件 inetd.conf，由 7 个字段构成。

- λ 服务名：表示提供的服务的名称，该名称必须要在系统的服务数据库 /etc/services 文件里指出；
- λ socket 类型：服务所使用的 socket 类型，最常见的是 stream, dgram, raw 等；
- λ 协议：服务所使用的协议名字，需要在/etc/protocols 文件里指出，最常见的 TCP 和 UDP；

- λ 等待状态: 取值为 `wait/nowait[.max]`。值 `wait` 指明 `inetd` 在处理另一个服务请求前应该等待服务程序处理完一个请求, 而值 `nowait` 则允许并发处理, `max` 参数则定义了 `inetd` 进程在一分钟之内最大产生的服务实例数目;
- λ 用户标识: 表示服务程序在该用户权限下运行;
- λ 服务器程序: 要执行的程序的完整路径。如果是 `inetd` 进程内置的服务, 此处应为 “internally”;
- λ 参数: 提供给程序运行的所需参数, 同样的, 如果是内置服务, 此处也为 “internally”, 该字段可为空。

由于每一个新增加的服务器程序都必须在系统的服务数据库/`etc/services` 里注册, 在此介绍一下/`etc/services` 的格式, 如下所示:

```
# service-name port/protocol [aliases ...] [# comment]
```

service-name: 系统提供的服务名称, 如 `echo`;

port/protocol: 服务使用的端口和协议, 如 `7/tcp`, 表示端口号为 7, 服务使用 TCP 协议。端口号分成三个区段, 预定义端口号可以从 <http://www.iana.org/assignments/port-numbers> 得到。用户最好为自己的服务程序找一个未使用的端口。

aliases 是服务的别名; Comment 是服务的注释信息, 这两项不太重要, 通常都可以忽略。

对一个作为服务器端工作的串口/网络网关程序, 假设其服务名字为 `Serail2Net`, 程序完整路径为 `/bin/Serail2Net`, 使用端口为 18001, 则必须要在/`etc/services` 中注册服务信息如下:

```
Serail2Net 18001/tcp
Serail2Net 18001/udp
```

在/`etc/inetd.conf` 中增加服务信息如下:

```
Serail2Net stream tcp nowait root /bin/Serail2Net -b57600 -p1 -d8 -t1 -f1 7
/dev/ttyS0 /var/lock/LCK.ttyS0
Serail2Net dgram udp wait root /bin/Serail2Net -b57600 -p1 -d8 -t1 -f1 -u 7
/dev/ttyS0 /var/lock/LCK.ttyS0
```

然后重启 `inetd` 进程, 使这些配置生效, 则当有外界客户需要和 GPRS 数据终端相连的串口设备通信时, 只需要和 GPRS 数据终端的 18001 号端口进行通信即可。

GPRS 数据终端作为客户端

当网关程序是作为客户端工作时, 在程序运行之后只需要根据通信端和通信端口的设置与对方进行 TCP 或 UDP 通信, 就可以实现 GPRS 数据终端相连的串口设备的网络连接。

整个过程的实现逻辑为:

1. 根据命令行参数得到串口设置, 如波特率, 数据位, 停止位, 奇偶校验, 流控等, 以及网络通信设置, 是采用 TCP 还是 UDP, 客户还是服务器工作方式;
2. 根据串口设置参数进行串口设置;
3. 根据网络通信设置, 建立 Socket, 如果是 TCP 客户工作方式, 需预先建立连接。

4. 使用 select 机制，不断检查 Socket 和串口是否可以发送和接收。程序维持两个缓存区，一个是 Socket 数据缓存，另一个是串口数据缓存。如果 Socket 有数据获得就放入串口数据缓存，串口有数据获得就放入 Socket 数据缓存。如果 Socket 或串口可以发送，就将相应的缓存数据发送出去。

上述过程的实现程序示例如下：

```
int main(int argc, char *argv[])
{
    /* Input fd set */
    fd_set InFdSet;
    /* Output fd set */
    fd_set OutFdSet;
    /* Char read */
    unsigned char pData[BufferSize];
    unsigned int uDataLen;
    unsigned char C;
    /* Temporary string for logging */
    char LogStr[TmpStrLen];
    /* Actual port settings */
    struct termios PortSettings;
    /* Base timeout for stream reading */
    struct timeval BTimeout;
    /* Timeout for stream reading */
    struct timeval RTimeout;
    /* Pointer to timeout structure to set */
    struct timeval *ETimeout = &RTimeout;
    /* Buffer to Device from Network */
    BufferType ToDevBuf;
    /* Buffer to Network from Device */
    BufferType ToNetBuf;
    /* Socket setup flag */
    int SockParmEnable = 1;
    /* Generic socket parameter */
    int SockParm;
    /* Out buffer clock ticks limit */
    clock_t MaxBTicks;
    /* Optional argument processing indexes */
    int argi = 1;
    int i;
    int nRet;
    int sockfdin, sockfdout;
    Boolean bClient=False;
    Boolean bUDP=False;
```

```
struct sockaddr_in their_addr;
char * p=NULL;
unsigned long speed=57600;
unsigned char parity=1,dataSize=8,stopbits=1,flowcontrol=1;

/* Open the system log */
openlog("Serail2Net", LOG_PID, LOG_USER);

//1、根据命令行参数得到串口设置，如波特率，数据位，停止位，奇偶校验，流控等，以及
//网络通信设置，是采用 TCP 还是 UDP，客户还是服务器工作方式；
if (argc < 4)
    return (Error);

for (argi = 1; argv[argi][0] == '-' && argi < argc; argi++)
{
    i = 1;
    if (argv[argi][i]) {
        switch (argv[argi][i++]) {
            /*serial baudrate*/
            case 'b':
                speed=atoi(argv[argi]+i);
                break;
            /*serial parity*/
            case 'p':
                parity=atoi(argv[argi]+i);
                break;
            /*serial databits*/
            case 'd':
                dataSize=atoi(argv[argi]+i);
                break;
            /*serial stopbits*/
            case 't':
                stopbits=atoi(argv[argi]+i);
                break;
            /*serial flowcontrol*/
            case 'f':
                flowcontrol=atoi(argv[argi]+i);
                break;
            /*-c192.168.0.1:10000*/
            case 'c':
                bClient=True;
                strcpy(LogStr,argv[argi]+i);
        }
    }
}
```

```

        their_addr.sin_family = AF_INET;
        inet_aton(strtok(LogStr, ":"), &their_addr.sin_addr);
        p = strtok(NULL, ":");
        if (p == NULL)
            return -1;
        their_addr.sin_port = htons(atoi(p));
        bzero(&(their_addr.sin_zero), 8);
        break;
    /*UDP socket*/
    case 'u':
        bUDP = True;
        break;
    default:
        break;
    }
}

/* Sets the log level */
MaxLogLevel = atoi(argv[argi++]);
/* Gets device and lock file names */
DeviceName = argv[argi++];
LockFileName = argv[argi++];
/* Retrieve the polling interval */
if (argc == argi + 1) {
    BTimeout.tv_sec = 0;
    BTimeout.tv_usec = atol(argv[4]) * 1000;
    MaxBTicks = (BTimeout.tv_usec * CLOCKS_PER_SEC) / (1000 * 1000);
    if (BTimeout.tv_usec <= 0) {
        ETimeout = NULL;
        MaxBTicks = 0;
    }
}
else {
    BTimeout.tv_sec = 0;
    BTimeout.tv_usec = ModemStatePolling * 1000;
    MaxBTicks = (BTimeout.tv_usec * CLOCKS_PER_SEC) / (1000 * 1000);
}

/* Logs Serail2Net start */
LogMsg(LOG_NOTICE, "Serail2Net started.");
/* Register exit and signal handler functions */
atexit(ExitFunction);

```

```
signal(SIGHUP, SignalFunction);
signal(SIGQUIT, SignalFunction);
signal(SIGABRT, SignalFunction);
signal(SIGPIPE, SignalFunction);
signal(SIGTERM, SignalFunction);
//2、根据串口设置参数进行串口设置;
/* Try to lock the device */
if (HDBLockFile(LockFileName, getpid()) != LockOk) {
    /* Lock failed */
    sprintf(LogStr, "Unable to lock %. Exiting.", LockFileName);
    LogMsg(LOG_NOTICE, LogStr);
    return (Error);
}
else {
    /* Lock succeeded */
    sprintf(LogStr, "Device %s locked.", DeviceName);
    LogMsg(LOG_INFO, LogStr);
}

/* Open the device */
if ((DeviceFd = open(DeviceName, O_RDWR | O_NOCTTY | O_NDELAY, 0))
    == OpenError) {
    /* Open failed */
    sprintf(LogStr, "Device in use. Come back later.\r\n");
    LogMsg(LOG_ERR, LogStr);
    sprintf(LogStr, "Unable to open device %. Exiting.", DeviceName);
    LogMsg(LOG_ERR, LogStr);
    return (Error);
}
else
    DeviceOpened = True;
    /* Get the actual port settings */
    tcgetattr(DeviceFd, &InitialPortSettings);
    InitPortRetrieved = True;
    tcgetattr(DeviceFd, &PortSettings);
    /* Set the serial port to raw mode */
    cfmakeraw(&PortSettings);
    /*Enable HANGUP on close and disable modem control line handling */
    PortSettings.c_cflag = (PortSettings.c_cflag | HUPCL) | CLOCAL;
    /* Write the port settings to device */
    tcsetattr(DeviceFd, TCSANOW, &PortSettings);
    SetPortSpeed(DeviceFd, speed);
```

```

SetPortParity(DeviceFd, parity);
SetPortDataSize(DeviceFd, dataSize);
SetPortStopSize(DeviceFd, stopbits);
SetPortFlowControl(DeviceFd, flowcontrol);

/* Reset the device fd to blocking mode */
if (fcntl(DeviceFd, F_SETFL, fcntl(DeviceFd, F_GETFL)
    & ~(O_NDELAY)) == OpenError)
    LogMsg(LOG_ERR, "Unable to reset device to non blocking mode,
        ignoring.");
/* Initialize the input buffer */
InitBuffer(&ToDevBuf);
InitBuffer(&ToNetBuf);

/* Setup sockets for low latency and automatic keepalive;
 * doesn't check if anything fails because failure doesn't prevent
 * correct functioning but only provides slightly worse behaviour
 */

```

//3、根据网络通信设置，建立 Socket，如果是 TCP 客户工作方式的话，预先建立连接；

```

if (bClient) {
    if (bUDP) {
        //UDP
        if ((sockfdin=socket(AF_INET, SOCK_DGRAM, 0))== -1){
            LogMsg(LOG_ERR, "socket error\n");
            return (Error);
        }
        sockfdout=sockfdin;
    }
    else {
        //TCP
        //connect to server
        if ((sockfdin=socket(AF_INET, SOCK_STREAM, 0))== -1) {
            LogMsg(LOG_ERR, "socket error\n");
            return (Error);
        }
        sprintf(LogStr,
            "IP=%s", inet_ntoa(their_addr.sin_addr));
        LogMsg(LOG_ERR, LogStr);
        sprintf(LogStr,
            "PORT=%d", ntohs(their_addr.sin_port));
        LogMsg(LOG_ERR, LogStr);
        if (connect(sockfdin, (struct sockaddr *)&their_addr,

```



```

        sizeof(struct sockaddr)) == -1) {
            LogMsg(LOG_ERR, "connect error\n");
            return (Error);
        }
        sockfdout=sockfdin;
    }
}
else {
    sockfdin=STDIN_FILENO;
    sockfdout=STDOUT_FILENO;
}

SockParm = IPTOS_LOWDELAY;
setsockopt(sockfdin, SOL_SOCKET, SO_KEEPALIVE,
            &SockParmEnable, sizeof(SockParmEnable));
setsockopt(sockfdin, SOL_IP, IP_TOS, &SockParm,
            sizeof(SockParm));
setsockopt(sockfdin, SOL_SOCKET, SO_OOBINLINE,
            &SockParmEnable, sizeof(SockParmEnable));
setsockopt(sockfdout, SOL_SOCKET, SO_KEEPALIVE,
            &SockParmEnable, sizeof(SockParmEnable));
setsockopt(sockfdout, SOL_IP, IP_TOS, &SockParm,
            sizeof(SockParm));

/* Make reads/writes unblocking */
ioctl(sockfdout, FIONBIO, &SockParmEnable);
ioctl(sockfdin, FIONBIO, &SockParmEnable);
ioctl(DeviceFd, FIONBIO, &SockParmEnable);

```

//4、使用 select 机制，不断检查 Socket 和串口是否可以发送和接收。程序维持两个缓存区，
 //一个是 Socket 数据缓存，另一个是串口数据缓存。如果 Socket 有数据获得就放入串口数据
 //缓存，串口有数据获得就放入 Socket 数据缓存。如果 Socket 或串口可以发送，就将相应的
 //缓存数据发送出去。

```

/* Set up fd sets */
/* Initially we have to read from all, but we only have data to
/*send to the network */
FD_ZERO(&InFdSet);
FD_SET(sockfdin, &InFdSet);
FD_SET(DeviceFd, &InFdSet);
FD_ZERO(&OutFdSet);
FD_SET(DeviceFd, &OutFdSet);
FD_SET(sockfdout, &OutFdSet);

```

```

/* Set up timeout for modem status polling */
if (ETimeout != NULL)
    *ETimeout = BTimeout;

i=MAX(DeviceFd,sockfdout);
/* Main loop with fd's control */
while (True) {
    if (select(i+ 1, &InFdSet, &OutFdSet, NULL, ETimeout)> 0){
        if (FD_ISSET(DeviceFd, &OutFdSet)){
            /* Write to serial port */
            while (!IsBufferEmpty(&ToDevBuf)) {
                GetFromBuffer(&ToDevBuf,&C,1);
                nRet = write(DeviceFd, &C, 1);
                if (nRet < 0 && errno == EWOULDBLOCK){
                    PushToBuffer(&ToDevBuf, &C,1);
                    break;
                }
            }
            else if (nRet < 1) {
                LogMsg(LOG_NOTICE, "Error writing to device.");
                return (NoError);
            }
        }
    }
    if (FD_ISSET(sockfdout, &OutFdSet)) {
        /* Write to network */
        if (bUDP){
            while (!IsBufferEmpty(&ToNetBuf)) {
                uDataLen=GetBufferDataLen(&ToNetBuf);
                if (GetFromBuffer(&ToNetBuf, pData,uDataLen)==0){
                    nRet =sendto(sockfdout,pData,DataLen,0,
                                (struct sockaddr *)&their_addr,
                                sizeof(struct sockaddr));
                    if (nRet < 0){
                        if( errno == EWOULDBLOCK){
                            LogMsg(LOG_NOTICE,"nRet<0 ushToBuffer.");
                            PushToBuffer(&ToNetBuf, pData,uDataLen);
                            break;
                        }
                        else {
                            LogMsg(LOG_NOTICE,"Error write to
                                network.");
                            return (NoError);

```

```

        }
    }
    else if (nRet < uDataLen) {
        PushToBuffer(&ToNetBuf, pData, uDataLen);
        break;
    }
}
}
}
else { //TCP
    while (!IsBufferEmpty(&ToNetBuf)) {
        GetFromBuffer(&ToNetBuf, &C, 1);
        nRet = write(sockfdout, &C, 1);
        if (nRet < 0 && errno == EWOULDBLOCK) {
            PushToBuffer(&ToNetBuf, &C, 1);
            break;
        }
        else if (nRet < 1) {
            LogMsg(LOG_NOTICE, "Error writing to
                network.");
            return (NoError);
        }
    }
}
}
if (FD_ISSET(DeviceFd, &InFdSet)) {
    /* Read from serial port */
    while (!IsBufferFull(&ToNetBuf)) {
        nRet = read(DeviceFd, &C, 1);
        if (nRet < 0 && errno == EWOULDBLOCK)
            break;
        else if (nRet < 1) {
            LogMsg(LOG_NOTICE, "Error reading from device.");
            return (NoError);
        }
        AddToBuffer(&ToNetBuf, &C, 1);
    }
}
if (FD_ISSET(sockfdin, &InFdSet)) {
    /* Read from network */
    if (bUDP) { //UDP
        while (!IsBufferFull(&ToDevBuf)) {

```



```

static int GPRSPPP()
{
    //Succeed return 0, else -1
    FILE * fp=NULL;
    int i=0;
    //1、根据用户配置获得 GPRS 上网连接的配置信息，并生成对应的拨号脚本和配置文件
    //create ppp-on.gprs
    if ((fp=fopen("/etc/ppp/ppp-on.gprs", "w"))==NULL){
        TRACE("can't open /etc/ppp/ppp-on.gprs\n");
        return -1;
    }
    else {
        //GPRSCMD
        fprintf(fp, "#!/bin/sh\n#\nGPRSCMD='+cgdcont=1,\nIP\n,\n");

        fwrite(gSystemInfo.gPRSInfo.strAccessPoint, sizeof(char), strlen(gSystemInfo.gPRSInfo.strAccessPoint), fp);
        //TELEPHONE
        fprintf(fp, "\n\nTELEPHONE=");

        fwrite(gSystemInfo.gPRSInfo.strDialNum, sizeof(char), strlen(gSystemInfo.gPRSInfo.strDialNum), fp);
        //INITCMD
        fprintf(fp, "\nINITCMD=");

        fwrite(gSystemInfo.gPRSInfo.strATInit, sizeof(char), strlen(gSystemInfo.gPRSInfo.strATInit), fp);
        //other
        fprintf(fp, "\n\nDEVICE=/dev/ttyS1\nSPEED=57600\nOPTIONFILE=/etc/ppp/options.gprs\nDIALFILE=/etc/ppp/dialer.gprs\n");
        fprintf(fp, "\n\nexport GPRSCMD TELEPHONE INITCMD\n\n\nexec pppd $DEVICE $SPEED file $OPTIONFILE connect $DIALFILE");
        fclose(fp);

        //create pap-secrets
        if ((fp=fopen("/etc/ppp/pap-secrets", "w"))==NULL){
            TRACE("can't open /etc/ppp/pap-secrets\n");
            return -1;
        }
        else {
            //UserName

```

判断 PPPD 是否拨号成功是通过 IsPPPCConnect 函数来完成，它会检索已有的网络设备信息，检查有没有生成对应设备名的网络设备。

```
int IsPPPConnect(const char * strPPPDeviceName, int nTimeOut, char *
    strIPAddress)
{
    //check if ppp success, return TRUE else return FALSE
    // strIPAddress[out]: return IPAddress if connect
    int sockfd, size = 1;
    struct ifreq * ifr;
    struct ifconf ifc;
    struct sockaddr_in sa;
    int nRet=FALSE;
    int i=0;
```

```

if ( 0 > ( sockfd = socket( AF_INET, SOCK_DGRAM, IPPROTO_IP ) ) ) {
    TRACE("IsPPPConnect: Cannot create socket\n");
    return FALSE;
}
sleep(1); //wait for pppd launch
bIsProcessRunFirst=TRUE;
while (1){
    //check if pppd is not running
    if (IsProcessRun("pppd")==0)
        goto IsPPPConnectEND;

    if (ifc.ifc_req!=NULL){
        free(ifc.ifc_req);
        ifc.ifc_req = NULL;
    }
    size = 1;
    do {
        ++size;
        /* realloc buffer size until no overflow occurs */
        if ( NULL == ( ifc.ifc_req = realloc( ifc.ifc_req, IFRSIZE ) ) ){
            //TRACE("IsPPPConnect: Out of memory.\n");
            goto IsPPPConnectEND;
        }
        ifc.ifc_len = IFRSIZE;
        if ( ioctl( sockfd, SIOCGIFCONF, &ifc ) ) {
            //TRACE("IsPPPConnect: ioctl SIOCGIFCONF\n" );
            goto IsPPPConnectEND;
        }
    } while ( IFRSIZE <= ifc.ifc_len );
    ifr = ifc.ifc_req;
    for ( ; ( char * )ifr < ( char * )ifc.ifc_req + ifc.ifc_len; ++ifr ){
        if ( ifr->ifr_addr.sa_data == ( ifr + 1 )->ifr_addr.sa_data ){
            continue; /* duplicate, skip it */
        }
        if ( ioctl( sockfd, SIOCGIFFLAGS, ifr ) ){
            continue; /* failed to get flags, skip it */
        }
        if ( strcmp(ifr->ifr_name, strPPPDeviceName)==0 ) {
            if ( strIPAddress!=NULL ) {
                strcpy(strIPAddress,
                    inet_ntoa( inaddr( ifr_addr.sa_data ) ));
            }
        }
    }
}

```



```

        }
        nRet=TRUE;
        goto IsPPPConnectEND;
    }
} /* end of for */
if (i++==nTimeOut)
    goto IsPPPConnectEND;
sleep(1); //only sleep 1 sec?
}
IsPPPConnectEND:
    close( sockfd );
    if (ifc.ifc_req!=NULL)
        free(ifc.ifc_req);
    return nRet;
}

```

心跳监控

心跳监控是指 GPRS 数据终端需要定时与监控点进行通信，保证能检测到 GPRS PPP 连接的异常通信情况，并启动重拨等逻辑来保证提供正常工作的无线通信链路。它的程序逻辑主要包括：

- 1、根据用户配置获得监控点 IP 地址和心跳测试周期；
- 2、根据心跳测试周期，定时去 ping 监控点 IP 地址，如果能 ping 通，表示无线通信链路正常，如果多次无法 ping 通，则必须断开连接重新拨号。

心跳监控程序示例如下：

```

void HeartTest()
{
    fd_set set;
    struct timeval tval;
    int retval=0;
    int i=0;
    int nReConnect=TRUE;
    while (1){
        //根据用户配置获得监控点 IP 地址和心跳测试周期;
        if ( ( strlen(gSystemInfo.systemGeneralInfo.strMonitorAddress)>0)
            && (gSystemInfo.systemGeneralInfo.nHeartPeriod>0) ){
            FD_ZERO(&set);
            FD_SET(0, &set);

            tval.tv_sec = gSystemInfo.systemGeneralInfo.nHeartPeriod;
            tval.tv_usec = 0;

```



```

        gSystemInfo.SMSInfo.strAlarmMessage);
    }
    UpdateSystemRunInfo(&gSystemRunInfo);
}
}
}
else
    return 0;
}
}

```

短信收发

短信收发是指 GPRS 数据终端能够完成短信的接收和发送功能。通常，短信收发功能只能在 GPRS 数据终端没有上网的情况下进行，如果 GPRS 数据终端已经上网，则无法进行短信收发。

GPRS 数据终端接收短信主要用于远程接收短信后自动上网，发送短信则主要用于网络断开后的报警等。

短信收发主要是对串口进行编程，结合 GPRS Modem 的 AT 命令集中的短信收发命令来完成的。

以短信发送为例，程序实现逻辑描述如下：

- 1、打开与 GPRS Modem 连接的串口，进行相关设置，如波特率等，保存原有串口设置；
- 2、根据 GPRS Modem 所支持 AT 命令集中的发短信命令，结合配置的短信报警信息，如报警手机号，报警信息等，组成最终的 AT 字符串，并发送给串口，同时需要根据 GPRS Modem 的返回值来判断是否需要进一步操作；
- 3、恢复原有的串口设置。

本系统选用的 GPRS Modem 的短信发送过程为：

```

at+cmgf=1           // 短消息支持格式为 text;
at+cscs="+8613800100500" // 服务中心，根据服务商来确定；
at+cmgs="13611111111" // 给手机号 13611111111 发送短信，将会出现>提示符，然后可以输入短信信息，用 Ctrl+Z 结束短信信息并进行发送。

```

上述过程的示例程序如下：

```

int sendSMS(const char * strSMSServer, const char * strAlarmMobile, const
char * strAlarmMessage)
{
    int fd, nTotal, i, nRet=FALSE;

```

```

struct termios oldtio,newtio;
char strATResult[BUFSIZE];
char strAT[BUFSIZE];
//1、打开 GPRS Modem 连接的串口，进行相关设置，如波特率等，保存原有串口设置；
if ( (fd = open(GPRSTTY,O_RDWR | O_NOCTTY ))<0){
    TRACE("Can't Open Serial Port!GPRSTTY\n");
    return nRet;
}
else{
    tcgetattr(fd,&oldtio); /* save current serial port settings */
    setTermios(&newtio,B9600);

    tcflush(fd, TCIFLUSH);
    tcsetattr(fd,TCSANOW,&newtio);
    //2、根据 GPRS Modem 的 AT 命令集中的发短信命令，结合配置的短信报警信息，如报
    //警手机号，报警信息等，组成最终的 AT 字符串，并发送给串口，并需要根据 GPRS
    //Modem 的返回值来判断是否需要进一步操作；
    sprintf(strAT,"at+cmgf=1%c%c",0x0d,0x0a);
    nTotal=0;
    while (ATComUntilStr(fd,strAT,"OK",strATResult,
        ATREADTIMES)!=TRUE) {
        if ((nTotal++)<ATTRYTIMES)
            continue;
        else
            goto sendSMSSEND;
    }

    sprintf(strAT,"at+csca=\"%s\"%c%c",strSMSServer,0x0d,0x0a);
    nTotal=0;
    while (ATComUntilStr(fd,strAT,"OK",
        strATResult,ATREADTIMES)!=TRUE){
        if ((nTotal++)<ATTRYTIMES)
            continue;
        else
            goto sendSMSSEND;
    }
    //send message
    i=0;
    while (1){
        sprintf(strAT,"at+cmgs=\"%s\"%c%c",
            strAlarmMobile,0x0d,0x0a);
        nTotal=0;

```

```

        while (ATComUntilStr(fd, strAT, ">",
            strATResult, ATREADTIMES) != TRUE) {
            if ((nTotal++) < ATTRYTIMES)
                continue;
            else
                goto sendSMSSEND;
        }
        sprintf(strAT, "%s%c", strAlarmMessage, 0x1a);
        if (ATComUntilStr(fd, strAT, "OK"
            , strATResult, ATREADTIMES*3) != TRUE) {
            if ((i++) < ATTRYTIMES)
                continue;
            else
                goto sendSMSSEND;
        }
        else
            break;
    }
    nRet = TRUE;
sendSMSSEND:
    //3、恢复原有的串口设置。
    tcsetattr(fd, TCSANOW, &oldtio);
    close(fd);
    return nRet;
}
}

```

用于串口读写的主要函数为 ATComUntilStr，它完成写字符串给串口，并检查返回的字符串是否包含了要求的字符，用于判断 Modem 是否正确响应的功能。

具体实现代码如下：

```

int ATComUntilStr(int fd, const char * strATCommand, const char * strPrompt,
char * strResult, int nATReadTimes)
{
    int nRes, ch, nTotal;
    char strAT[BUFSIZE];

    write(fd, strATCommand, strlen(strATCommand));
    nTotal = 0;
    nRes = 0;
    ch = 0;
    while (1)
    {
        nRes = read(fd, strAT, BUFSIZE - nTotal);
    }
}

```

```

        if (nRes>0){
            strAT[nRes]=0;
            memcpy(strResult+nTotal,strAT,nRes);
            nTotal+=nRes;
            ch=0;
        }
        else {
            if (ch++==nATReadTimes)
                break;
        }
    }
    strResult[nTotal]=0;
    //check if OK
    if (strstr(strResult,strPrompt)!=NULL)
        return TRUE;
    else
        return FALSE;
}

```

网络通信统计

网络通信统计主要用于统计网络设备的通信情况，包括收到和发送的数据包数数量，字节总数和错误包数等。

程序通过读取 Linux 下的 /proc/net/dev 文件中的网络设备统计信息，实现网络通信统计的功能。dev 文件是一个文本文件，可以用“cat /proc/net/dev”命令查看，并且每一行的字段都有注释，比较容易读懂。

程序逻辑如下：

- 1、打开 /proc/net/dev 文件；
- 2、读取每一行信息，并进行字符串处理，获得各统计数据。

程序示例代码如下：

```

#define GETDEVICESTAT(VAR) {while (*(pTail++)!=0x20){}
*(pTail-1)=0;(VAR)=strtoul(pHead,NULL,10);*(pTail-1)=0x20;while
(*(pTail++)==0x20){} pHead=pTail-1;}

int getNetDeviceStat(const char * strDeviceName, NetDeviceStat *
pNetDeviceStat)
{
    //get device stat from /proc/net/dev
    //error return -1
    FILE * fpDev=NULL;

```

```

char bufDev[BUFSIZE];
char * pHead=NULL;
char * pTail=NULL;
int nCount=0, nTotal=0;
//1、打开/proc/net/dev 文件;
if ((fpDev=fopen("/proc/net/dev", "r"))==NULL) {
    TRACE("can't open /proc/net/dev!\n");
    return -1;
}
else {
    nCount=0;
    nTotal=0;
    //2、读取每一行信息，并进行字符串处理，获得各统计数据;
    while (nTotal<BUFSIZE) {
        nCount=fread(bufDev+nTotal, sizeof(char),
            BUFSIZE-nTotal, fpDev);
        if(nCount<0) {
            TRACE("read /proc/net/dev error!\n");
            fclose(fpDev);
            return -1;;
        }
        else {
            nTotal+=nCount;
            if (feof(fpDev))
                break;
        }
    }
    fclose(fpDev);

    if (nTotal>0) {
        bufDev[nTotal]=0;
        if ((pHead=strstr(bufDev, strDeviceName))!=NULL){
            while (*(pHead++)!=0x20){}
            while (*(pHead++)==0x20){}
            pHead--;
            pTail=pHead+1;
            //rx_bytes
            GETDEVICESTAT(pNetDeviceStat->rx_bytes);
            /*while (*(pTail++)!=0x20){}
            *(pTail-1)=0;
            pNetDeviceStat->rx_bytes=strtoul(pHead, NULL, 10);
            *(pTail-1)=0x20;

```



```
while (*(pTail++)==0x20){}
pHead=pTail-1;*/

//rx_packets
GETDEVICESTAT(pNetDeviceStat->rx_packets)
//rx_errors
GETDEVICESTAT(pNetDeviceStat->rx_errors)
//rx_dropped
GETDEVICESTAT(pNetDeviceStat->rx_dropped)
//rx_fifo_errors
GETDEVICESTAT(pNetDeviceStat->rx_fifo_errors)
//rx_frame_errors
GETDEVICESTAT(pNetDeviceStat->rx_frame_errors)
//rx_compressed
GETDEVICESTAT(pNetDeviceStat->rx_compressed)
//multicast
GETDEVICESTAT(pNetDeviceStat->multicast)
//tx_bytes
GETDEVICESTAT(pNetDeviceStat->tx_bytes)
//tx_packets
GETDEVICESTAT(pNetDeviceStat->tx_packets)
//tx_errors
GETDEVICESTAT(pNetDeviceStat->tx_errors)
//tx_dropped
GETDEVICESTAT(pNetDeviceStat->tx_dropped)
//tx_fifo_errors
GETDEVICESTAT(pNetDeviceStat->tx_fifo_errors)
//collisions
GETDEVICESTAT(pNetDeviceStat->collisions)
//tx_carrier_errors
GETDEVICESTAT(pNetDeviceStat->tx_carrier_errors)
//tx_compressed
pNetDeviceStat->tx_compressed=strtoul(pHead,NULL,10);
    }
}
else {
    TRACE("can't read /proc/net/dev!\n");
    return -1;;
}
}
return 0;
}
```

7.4 本章小结

本章描述了一个基于 EP9301/02 的无线数据终端的应用系统的设计过程，包括从系统的需求分析，到硬件平台的选择与设计，软件系统的移植与应用开发的全过程，希望能够让读者对嵌入式设备的完整开发过程有一个比较全面的了解。

所有的程序示例代码都经过调试，并在实际系统中使用，但由于嵌入式系统的差异性，读者在参考这些代码时，可能需要根据目标系统的具体情况做相应的修改。

www.emsyschina.com

附录 A 参考文献

- 1、Cirrus logic. EP9301 User Manual
- 2、Cirrus logic. EP9307 User Manual
- 3、Cirrus logic. EP9312 User Manual
- 4、Cirrus logic. EP9315 User Manual
- 5、《Linux 设备驱动程序》，ALESSANDRO RUBINI& JONATHAN CORBET 著，魏永明 骆刚 姜军译 中国电力出版社
- 6、《Linux 上的 C 编程》北京：中国电力出版社，2000.1
- 7、《Linux C 语言实务》北京：机械工业出版社，2002.8